



Menoufia University
Faculty of Engineering
Electrical & Computer Engineering



APEX

A P E X I S A P O R T A L F O R E X A M S

A Classroom Portal for Mixed-Format Exams,
Auto-Grading, and Live Coding

Prepared by:

Hossam Mohamed Samy Abd-Elaziz Rashed
Mohammed Ibrahim Gomaa Ibrahim
Ahmed AbdElfattah Hashem Taha
Adham Basem Saad AbdElghani
Mohamed AbdElfattah Ibrahim Elwa

Project Supervisor:

Prof. Dr. Ayman El-Sayed Omera

Academic year
2025 - 2026

Abstract

Classroom assessment tools force an unhappy choice: coding judges such as HackerRank grade programs but cannot host a written argument, while learning-management platforms such as Moodle host quizzes and essays but cannot run code. A teacher who wants to test a programming concept, a definition, and a short explanation in the same sitting must stitch together separate systems and reconcile the marks by hand. **APEX** closes that gap. It is a self-hostable classroom portal that runs **coding**, **multiple-choice**, and **written** questions inside *one* timed exam, driven by *one* grading pipeline behind all three question types. Teachers build mixed-format exams, assign them to classes, and let the system auto-grade code and MCQs while queueing written answers for manual review; students join a class by invite code, take the exam in an in-browser Monaco editor with live sample-test feedback, and review per-test results afterwards.

The system is deployed as a single Go binary served by Gin, a PostgreSQL 16 store accessed through GORM, and a React and TypeScript single-page frontend, with student code executed out-of-process by **Judge0** inside the `isolate` sandbox under per-problem time and memory limits. Across roughly 6,100 lines of Go and 17,400 of TypeScript, three engineering decisions carry the design. First, a *three-gate authorization* model layers JWT authentication, coarse role checks, and per-handler resource-ownership verification, so a valid token alone never grants access to another user's data. Second, an *idempotent migration pipeline*—born from a real production incident in which a stray GORM column default silently hid every exam from students—separates development auto-migration from a versioned, restart-safe SQL track that is the sole schema authority in production. Third, a *unified grading pipeline* normalizes program output symmetrically before comparison, aggregates per-question scores across an attempt, and finalizes results exactly once. The scope is deliberately honest: two roles only (teacher and student, no admin), fire-and-forget grading goroutines with no durable retry queue, no auth rate limiting, and minimal automated test coverage. These limitations are documented rather than hidden, each paired with the concrete remediation a production hardening pass would apply.

Keywords: automated assessment, online judge, Judge0, `isolate` sandbox, mixed-format examinations, e-learning, Go, Gin, REST API, PostgreSQL, React, JWT authorization.

Acknowledgements

We owe our deepest gratitude to our supervisor, **Prof. Dr. Ayman El-Sayed Omera**, whose guidance, patience, and technical insight shaped this work at every stage. His questions consistently pushed us past easy answers toward the honest engineering trade-offs that define APEX, and his encouragement carried the project through its hardest moments.

We are grateful to the **Department of Electrical and Computer Engineering** and to the **Faculty of Engineering at Menoufia University** for the education, laboratories, and academic environment that made this graduation project possible.

This portal was built by a team of five, and it exists only because each member owned a hard part of it: the grading engine, the backend architecture, the database and data model, DevOps and deployment, and the security review.

Finally, we thank our families for their unwavering support and our peers for the feedback, testing, and camaraderie that kept us going. Any remaining shortcomings are entirely our own.

Contents

Abstract	i
Acknowledgements	ii
List of Abbreviations	xiii
1 Introduction	1
1.1 Context and Background	1
1.2 The Problem	1
1.3 Motivation	2
1.4 The APEX Vision	2
1.5 Project Objectives	3
1.6 Scope and Limitations	4
1.7 Contributions and Novelty	4
1.8 Intended Users	5
1.9 Structure of this Book	5
2 Background and Literature Review	7
2.1 Automated Assessment of Programming	7
2.1.1 A short history	7
2.1.2 Two families of grader: I/O diffing versus unit testing	8
2.1.3 Feedback, partial credit, and resubmission	9
2.2 Online Judges and Sandboxing	9
2.2.1 The online-judge tradition	9
2.2.2 The <code>isolate</code> sandbox	10
2.2.3 Judge0 as execution-as-a-service	10
2.3 Learning-Management Systems and Quiz Tooling	11
2.3.1 The LMS lineage	11
2.3.2 Coding on top of the LMS	11
2.3.3 Manual grading and mixed formats	11
2.4 Code-Similarity and Plagiarism Detection	12
2.5 Gap Analysis and Positioning	12
3 Requirements Engineering	15
3.1 Stakeholders and Actors	15
3.1.1 Stakeholders	15
3.1.2 Actors: two roles, no administrator	16
3.2 User Stories	16
3.3 Use-Case Model	17

3.4	Functional Requirements	19
3.4.1	Teacher requirements	19
3.4.2	Student requirements	19
3.4.3	Platform requirements	21
3.5	Non-Functional Requirements	21
3.5.1	Security	21
3.5.2	Performance and scalability	21
3.5.3	Usability	22
3.5.4	Portability and self-hostability	22
3.5.5	Maintainability and reliability	23
3.6	Requirements Traceability	23
3.7	Constraints and Assumptions	23
4	System Architecture and Design	26
4.1	Architectural Overview	26
4.2	The Four-Layer Request Lifecycle	27
4.2.1	Layer 1 — Router and route groups	28
4.2.2	Layer 2 — Middleware	28
4.2.3	Layer 3 — Handler	28
4.2.4	Layer 4 — Data	29
4.2.5	An honest note on the layering	29
4.3	Three-Gate Authorization	29
4.4	Technology Stack and Rationale	29
4.4.1	Why Go and Gin	29
4.4.2	Why GORM	30
4.4.3	Why PostgreSQL	30
4.4.4	Why React, Vite, and TypeScript	31
4.4.5	Why one binary that also ships <code>migrate</code> and <code>seed</code>	31
4.5	API Design	31
4.5.1	Modular route registration	31
4.5.2	Public, protected, and role subgroups	32
4.5.3	JSON contracts and status codes	33
4.6	Package Layout and Separation of Concerns	33
4.7	Design Patterns and Principles	34
5	Data Model and Persistence	35
5.1	The Entity Landscape	35
5.2	Identity Tables	36
5.3	Class Tables	37
5.4	Assessment Tables and the Polymorphic Problem	38
5.4.1	JSONB Columns	39
5.5	The Attempt Aggregate	39
5.6	Indexes and Soft Constraints	41

5.7	The Migration Pipeline	42
5.7.1	The Development Path: Three Stages, One Direction	42
5.7.2	The Production Path: A Versioned Binary Owns the Schema	43
5.8	Case Study: The <code>is_draft</code> Incident	43
5.9	Design Trade-offs and Honest Gaps	45
6	Implementation	46
6.1	Backend Handler Anatomy	46
6.2	The Exam Builder	48
6.3	Exam Lifecycle and Attempt State Machine	49
6.4	The Timed Exam-Taking Flow	50
6.4.1	Server-Side Autosave	50
6.4.2	Resume and Reconciliation	51
6.5	Question Editors and the Monaco Integration	52
6.6	Notifications and Announcements	53
6.7	The In-Process Reminder Scheduler	53
6.8	Frontend Architecture	55
6.8.1	The API Layer and Data Hooks	55
6.8.2	UI, Styling, and Routing	56
7	Code Execution and the Auto-Grading Engine	57
7.1	Execution Model: Delegating to a Sandbox	57
7.1.1	Why the sandbox is not ours	57
7.1.2	The language map	58
7.1.3	The Judge0 client	59
7.2	Three Entry Points, One That Grades	60
7.2.1	The playground path	61
7.2.2	The sample-test feedback path	61
7.3	The Coding Grader	62
7.3.1	The fire-and-forget dispatch	62
7.3.2	Loading test cases and the empty-set edge case	63
7.3.3	The grading loop	63
7.3.4	Output normalisation: nine lines of fairness	65
7.3.5	Scoring and aggregation into the submission	66
7.3.6	Limits: measured but not yet pushed	66
7.4	The Other Two Formats	67
7.4.1	Multiple-choice: auto-graded by set equality	67
7.4.2	Written answers: the manual-grading queue	67
7.5	Attempt Aggregation	68
7.5.1	The barrier: only the last goroutine proceeds	68
7.5.2	Re-summing over the exam's full problem list	69
7.5.3	The single, deduplicated notification	70
7.6	Sample versus Hidden Test Cases	70

7.7	The Honest Gap: Fire-and-Forget Durability	71
7.8	Summary	71
8	Security	72
8.1	Threat Model	72
8.2	Authentication	74
8.2.1	The fail-closed JWT secret	74
8.2.2	Token issuance and verification	74
8.2.3	Password storage with bcrypt	75
8.2.4	Google sign-in	76
8.2.5	Password reset	76
8.3	Authorization: The Three-Gate Model	76
8.3.1	Why ownership checks are not centralized	78
8.4	Input Handling	78
8.4.1	SQL injection	78
8.4.2	Cross-site scripting	78
8.4.3	CSRF and the localStorage trade-off	79
8.5	Sandbox Isolation for Untrusted Code	79
8.6	Findings and OWASP Mapping	80
8.7	Known Limitations	81
9	Testing and Quality Assurance	82
9.1	Test Strategy	82
9.1.1	The Test Pyramid Applied to APEX	83
9.2	The Continuous Integration Pipeline	83
9.2.1	The Backend Jobs	83
9.2.2	Static Analysis as a Quality Gate	84
9.2.3	The Frontend Jobs	85
9.2.4	The Docker Smoke Build	86
9.3	Validation Against Requirements	86
9.4	Honest Assessment of Current Coverage	86
9.4.1	What Should Be Tested, and How	87
9.5	Manual and Exploratory Testing	88
9.6	Summary	89
10	Deployment and DevOps	90
10.1	One Codebase, Three Binaries, Two Run-Modes	90
10.2	Local Development	90
10.3	Container Images	91
10.3.1	Backend: Multi-Stage to Distroless	91
10.3.2	Frontend: Bun Build to Nginx	92
10.4	The Four-Service Compose Stack	93
10.5	Health Checks in a Shell-Less Image	95
10.6	Twelve-Factor Configuration	95

10.7 Schema as Deployment Artifact	97
10.8 In-Process Background Jobs	97
10.9 Continuous Integration	98
10.10 Operability and Summary	98
11 Results and Evaluation	99
11.1 Scale and Scope of the Delivered System	99
11.2 Functional Completeness Against Requirements	100
11.3 Usability Evaluation	102
11.3.1 Toward an Empirical Study: the SUS Instrument	103
11.4 Performance Characteristics	103
11.4.1 Grading Latency Is Dominated by Judge0	103
11.4.2 Image Size, Startup, and API Latency	104
11.5 Project Timeline	104
11.6 What Works and What Is Limited	105
11.7 Summary	106
12 User Manual	107
12.1 Getting Started: Accounts and Sign-in	107
12.1.1 The landing page	107
12.1.2 Signing up and signing in	108
12.1.3 The two dashboards	109
12.2 Teacher Guide	110
12.2.1 Creating and managing classes	110
12.2.2 Building an exam	111
12.2.3 The question bank	114
12.2.4 Assigning, publishing, and previewing	115
12.3 Student Guide	116
12.3.1 Joining a class	116
12.3.2 Taking a timed exam	117
12.4 Grading and Results	118
12.4.1 The manual-grading queue	118
12.4.2 Reviewing results as a teacher	119
12.4.3 Announcing and reviewing results as a student	120
12.5 Notifications, Profile, and Settings	120
12.6 Summary	121
13 Conclusion and Future Work	122
13.1 Summary of Contributions	122
13.2 Lessons Learned	123
13.2.1 The <code>is_draft</code> Incident and Schema-Change Discipline	123
13.2.2 The Value of Idempotent Migrations	124
13.2.3 Fire-and-Forget Versus Durable Work	124
13.3 Future Work	124

13.3.1	Hardening Authentication: Rate Limiting and a WAF	125
13.3.2	A Durable Grading Work Queue with Retries	125
13.3.3	<code>httpOnly</code> + <code>SameSite</code> Cookie Authentication	126
13.3.4	Real Test Coverage	126
13.3.5	More Execution Languages and Enforced Per-Problem Limits	126
13.3.6	Plagiarism Detection	127
13.3.7	Horizontal Scaling and Observability	127
13.4	Closing Statement	127
A	API Reference	129
A.1	Access model	129
A.2	Liveness	129
A.3	Public	130
A.4	Protected: shared	130
A.5	Protected: teacher	131
A.5.1	Classes	131
A.5.2	Exams	131
A.5.3	Problems and question bank	131
A.5.4	Test cases	132
A.5.5	Grading and dashboard	132
A.5.6	Announcements and folders	132
A.6	Protected: student	133
B	Database Schema (DDL)	134
B.1	Identity and Accounts	134
B.2	Classes and Enrolment	135
B.3	Exams and Assignment	136
B.4	Problems and Test Cases	137
B.5	Attempts, Submissions, and Results	138
B.6	Communication and Organisation	139
B.7	Constraint Summary	140
C	Environment Variables	142
C.1	How configuration is read	142
C.2	Backend variables	142
C.2.1	Notes on individual variables	144
C.3	Frontend variables	145
C.4	Compose-only variable	145
C.5	Minimal working configuration	145
D	Selected Source Listings	147
D.1	The Grading Engine	147
D.1.1	Output Normalisation	147
D.1.2	The Coding Grading Loop	148

D.1.3	Attempt Aggregation	150
D.2	Authentication and Authorisation Middleware	152
D.3	The Reminder Scheduler	154
D.4	Destructive-Edit Reset	155
References		158

List of Figures

3.1	Principal use cases for the Teacher and Student actors.	18
4.1	APEX runtime and deployment architecture.	26
4.2	Four-layer request lifecycle through the Gin backend.	27
5.1	Entity-relationship diagram of the APEX data model.	36
5.2	Schema-migration pipeline and the <code>is_draft</code> incident.	42
6.1	Exam lifecycle and attempt state machine.	49
6.2	Idempotent exam-reminder schedule relative to start time T	54
7.1	Sequence of an auto-graded code submission.	62
8.1	Threat model: four adversary classes and their controls.	73
8.2	Three-gate authorization model.	77
9.1	Quality gates: CI pipeline and the test pyramid.	82
10.1	Container topology of the four-service Docker Compose stack.	93
11.1	Source-code size by language and component.	99
11.2	APEX by the numbers.	100
11.3	Graduation-project timeline (Gantt).	104
12.1	The public landing page at <code>/</code>	108
12.2	The sign-in and sign-up screen at <code>/auth</code> . The role picker is shown for new sign-ups and for first-time Google accounts.	108
12.3	Teacher dashboard: owned classes and recent activity.	109
12.4	Student dashboard: upcoming, active, and completed exams.	110
12.5	The teacher's class list, each card showing its eight-character invite code.	110
12.6	Class detail (teacher view): roster, assigned exams, and announcements.	111
12.7	The exam builder: metadata header, question rail, and editing pane.	112
12.8	Coding-question editor: statement, starter code, limits, and sample/hidden test cases.	113
12.9	Multiple-choice editor: options with per-option <i>Correct</i> checkboxes.	113
12.10	Written-question editor: prompt, rubric, and word-count limit.	114
12.11	The publish toggle, enabled once the exam has questions and a start time.	115
12.12	Exam preview: the student-facing view, rendered read-only for the teacher.	116
12.13	Joining a class with its invite code.	116
12.14	Taking a coding question in the Monaco editor, with the question rail and countdown timer.	117
12.15	The submit confirmation dialog.	118
12.16	The post-submission acknowledgement screen.	118

12.17The pending-grading queue of written answers awaiting a score. 119

12.18Grading a written answer against its rubric, with score and feedback inputs. 119

12.19Per-student results for one exam: score, percentage, and time taken. 119

12.20Per-question breakdown for a single student’s attempt. 120

12.21The student’s review screen: per-question results and teacher feedback. 120

List of Tables

1.1	Three frictions in mixed-format programming assessment that APEX targets.	2
1.2	Project objectives and where each is realised in the system.	3
2.1	APEX against representative alternatives. “Yes/No” is a capability judgement; cost reflects licensing and hosting for a small deployment. Assessments in the mixed-format and simplicity rows are subjective summaries of the discussion above.	13
3.1	Primary actors of the APEX system.	16
3.2	Functional requirements serving the Teacher actor.	19
3.3	Functional requirements serving the Student actor.	20
3.4	Platform-wide functional requirements (cross-cutting both roles).	20
3.5	Functional-requirement traceability to implementing components.	24
4.1	Principal technology choices and their roles.	30
5.1	The four aggregates of the APEX data model.	36
5.2	The three composite unique indexes and their invariants.	41
7.1	Supported execution languages and their Judge0 language identifiers.	59
7.2	Judge0 status identifiers and APEX’s interpretation.	60
7.3	The three code-execution entry points.	61
8.1	Security findings from the pre-ship review, with severity, weakness class, and fix.	80
8.2	APEX posture against the OWASP Top 10 categories.	80
9.1	Continuous-integration jobs and their gates.	84
10.1	Environment variables read by the backend (<code>internal/config/config.go</code>) and their graceful-degradation behaviour.	96
11.1	Functional-requirement coverage of the delivered system.	101
11.2	Heuristic evaluation: Nielsen’s heuristics traced to shipped mechanisms.	102
12.1	The three question types and how each is graded.	114
13.1	Project objectives and the outcome delivered against each.	123
13.2	Future-work roadmap: each remaining gap with its current mitigation and planned fix.	125
C.1	Backend environment variables (from <code>internal/config/config.go</code> and <code>.env.example</code>).	143
C.2	Frontend build-time environment variables.	145

List of Abbreviations

The following abbreviations and acronyms are used throughout this thesis. They are collected here for reference; each is also expanded at its first occurrence in the running text.

Abbreviation	Meaning
API	Application Programming Interface
bcrypt	Blowfish-based adaptive password-hashing function (used at cost 10)
CD	Continuous Deployment
CE	Community Edition (as in Judge0 CE, the self-hostable code sandbox)
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
CWE	Common Weakness Enumeration
DDL	Data Definition Language (SQL schema statements)
GIDS	Google Identity Services
HS256	HMAC-SHA-256, the symmetric JWT signing algorithm used by APEX
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDOR	Insecure Direct Object Reference
IOI	International Olympiad in Informatics
JSON	JavaScript Object Notation
JSONB	Binary JSON column type in PostgreSQL
JWT	JSON Web Token
LMS	Learning Management System
LOC	Lines of Code
MCQ	Multiple-Choice Question
OIDC	OpenID Connect
ORM	Object-Relational Mapping (here, GORM)
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
SQL	Structured Query Language
SUS	System Usability Scale

continued on next page

Abbreviation	Meaning
UI	User Interface
UX	User Experience
WAF	Web Application Firewall
XSS	Cross-Site Scripting

Chapter 1

Introduction

1.1 Context and Background

Programming is now taught in almost every engineering and computer-science curriculum, and the way it is *assessed* has quietly become one of the hardest problems an instructor faces. A well-formed programming assessment is rarely a single kind of question. To measure whether a student has genuinely learned, an instructor typically wants to combine three complementary formats in one sitting: a **coding** task that the student must actually make run against hidden inputs; a set of **multiple-choice** (MCQ) questions that probe conceptual understanding quickly and at scale; and one or more **written** (free-text) questions that ask the student to explain, justify, or design in prose. Each format answers a question the others cannot. Coding proves the student can produce working software; MCQ efficiently covers breadth; written answers expose reasoning that a pass/fail test case never sees.

The difficulty is that these three formats have historically lived in three different families of tools. Learning-management systems (LMSs) such as Moodle [1] and Canvas [2] are excellent at delivering quizzes, collecting essays, and keeping a gradebook, but they do not natively compile and run student code against test cases inside a secure sandbox. Online judges and competitive-programming platforms such as HackerRank [3], and the open-source execution engine Judge0 [4], [5] that many of them build on, are excellent at running untrusted code safely and grading it against inputs, but they are not classroom management systems: they do not model classes, rosters, mixed exams with a single timer, or manual grading of prose. Automated assessment of programming assignments has been an active research area for decades [6], [7], [8], [9], yet the tooling that reaches an ordinary lecturer remains *fragmented*: the coding part, the quiz part, and the essay part are authored separately, timed separately, and graded into separate gradebooks that do not interoperate.

1.2 The Problem

The concrete pain that motivates this project can be stated in one sentence: *setting a single mixed-format programming exam today means orchestrating several tools that were never designed to work together*. We decompose the problem into three specific frictions, summarised in Table 1.1.

None of these three frictions is individually unsolved — LMSs grade quizzes, online judges run code, and sandboxes exist — but no single, small, self-hostable artifact resolves *all three at once* for a classroom. The result is that the instructor becomes the integration layer, copying scores between tools by hand and reconciling three timelines into one exam. That manual integration is exactly where mistakes, unfairness, and wasted time accumulate.

Table 1.1. Three frictions in mixed-format programming assessment that APEX targets.

Friction	What it looks like in practice
Three formats, three tools	A coding task lives in an online judge, the MCQ section in a quiz tool or an LMS, and the written part in a document handed in by e-mail or an upload box. Each has its own login, its own timer, its own submission model, and its own gradebook. The instructor manually reconciles three partial scores into one final grade.
Brittle I/O diffing	Even the auto-gradable coding portion is fragile. The oldest and most common grading method compares a program’s printed output against an expected output byte for byte. A correct program then <i>fails</i> because the student’s language appended a trailing newline, emitted Windows <code>CRLF</code> line endings, or printed one stray trailing space. This false negative is the single most common unfairness on any code-grading platform.
Self-hosting is a project	An instructor who wants to keep student data on their own infrastructure must stand up an execution sandbox, a database, a web server, and the glue between them before a single exam can run. The operational effort dwarfs the teaching task it was meant to support.

1.3 Motivation

The motivation for APEX is both pedagogical and practical. Pedagogically, a mixed-format exam under a single timer is a more faithful measurement instrument than three disconnected sittings: the student experiences one coherent assessment, the instructor sees one weighted score, and the coding, conceptual, and explanatory dimensions of a topic are examined together rather than in isolation. Practically, the project is scoped as a graduation thesis built by a small team, so the design is disciplined by a bias towards *restraint* — the smallest system that still takes mixed-format classroom assessment seriously.

That bias shows up in two deliberate delegations. First, the project does *not* write its own code-execution sandbox. Running untrusted student code safely is one of the most security-critical tasks in computing, so APEX delegates it entirely to Judge0 [4], which executes each submission inside the `isolate` sandbox [10] — the same sandbox originally built for the International Olympiad in Informatics. The most dangerous component in the system is therefore the one component the team did *not* implement. Second, the project delivers itself as a single Go binary [11] rather than a constellation of microservices, so that a department with one server and modest operational appetite can actually run it. These choices are not accidental minimalism; they are the direct answer to the third friction in Table 1.1.

1.4 The APEX Vision

APEX — an *exam, grading, and live-coding portal for classrooms* — is a self-hostable web application that runs **coding, multiple-choice, and written** exams in **one timed sitting**, with **one grading pipeline** behind all three formats, deployed as **one Go binary plus PostgreSQL plus Judge0**. The whole system reduces to three ideas, each a direct rebuttal of one friction above:

One attempt.

A student joins a class by an eight-character invite code, then takes a single timed exam that may interleave coding, MCQ, and written questions. There is one timer, one autosave mechanism, and — backed by a unique database index on the `(user_id, exam_id)` pair — exactly one attempt per

student per exam. Coding is edited in an in-browser Monaco editor [12], the same editor that powers VS Code, but the code never runs in the browser: it is sent to the server and on to Judge0.

One pipeline.

All three formats flow through a single grading path. Coding submissions are compiled and run against hidden test cases inside the sandbox and scored by proportional partial credit; MCQ answers are graded by exact set-equality; written answers are queued for manual grading by the teacher. A single attempt-aggregation step then weights every question’s points into one exam score and fires exactly one “Exam Graded” notification. Fairness is protected at the boundary by output normalisation that strips `CRLF`, trailing whitespace, and edge newlines symmetrically from both the student’s output and the expected output, so a correct answer cannot fail on invisible whitespace.

One artifact.

The backend compiles to a single statically linked Go binary that ships in a `distroless` container [13] alongside PostgreSQL [14]; the React single-page application [15], [16] is served by nginx. A four-service Docker Compose stack (`postgres`, `migrate`, `backend`, `frontend`) brings the whole system up; Judge0 is reached as an external HTTP service and is deliberately *not* part of the stack.

APEX recognises exactly two roles — **teacher** and **student** — and has no administrator role. The reference architecture, layered request lifecycle, and grading sequence are developed in detail in later chapters and illustrated in Fig. 4.1, Fig. 4.2, and Fig. 7.1.

1.5 Project Objectives

The project set out to satisfy the objectives in Table 1.2. Each is stated so that it can be checked against the delivered system, and each maps onto one or more later chapters.

Table 1.2. Project objectives and where each is realised in the system.

#	Objective	Realised in
O1	Deliver one timed exam that interleaves coding, MCQ, and written questions with a single attempt, timer, and autosave.	Exam and student modules
O2	Grade all three formats through one pipeline: sandboxed coding execution, set-equality MCQ, and a manual queue for written answers.	Auto-grading engine
O3	Execute untrusted student code safely by delegating to Judge0 / <code>isolate</code> rather than running it on the application host.	Judge0 client
O4	Make automated grading <i>fair</i> by normalising whitespace symmetrically so a trailing newline cannot fail a correct answer.	<code>normalizeOutput</code>
O5	Enforce authorization with independent, fail-closed gates: identity, role, and per-row resource ownership.	Three-gate model (Fig. 8.2)
O6	Be genuinely self-hostable: one Go binary, PostgreSQL, and a small Docker Compose stack, with no proprietary dependencies.	Deployment topology (Fig. 10.1)
O7	Manage schema evolution safely, with production migrations owned by a versioned migrator rather than by ORM auto-migration.	Migration pipeline (Fig. 5.2)

Quantitatively, the delivered system comprises roughly **6,100** lines of Go and **17,400** lines of TypeScript, organised around **15** first-class data entities, **13** backend route modules, and **47** feature pages in the front end, and it executes submissions in **six** languages — Python, JavaScript, TypeScript, Java, C, and

C++. These counts are revisited and visualised in Fig. 11.1 and Fig. 11.2.

1.6 Scope and Limitations

A graduation thesis is most useful when it is honest about its boundaries, so this section states plainly what APEX does *not* do, and where the delivered system has known rough edges. These limitations are owned openly here and are examined in depth in the security and testing chapters rather than buried.

- **Two roles, no administration.** The system models only `teacher` and `student`. There is no institution-wide administrator, no cross-teacher reporting, and no organisational hierarchy. This keeps the authorization model small and auditable, at the cost of multi-tenant administration features an enterprise LMS would provide.
- **No rate limiting on authentication.** `bcrypt` at cost 10 [17] is the only brake on credential stuffing; a production deployment must be fronted by a WAF or rate limiter. The password minimum is also a modest six characters and should be raised before public exposure.
- **Fire-and-forget grading.** Each submission is graded in a bare background goroutine with no durable work queue and no retry. At classroom concurrency this is acceptable, but a process crash mid-grade can leave a submission stuck in the `running` state so that its attempt never finalises. The roadmap fix is a durable, idempotent work queue.
- **Token stored in `localStorage`.** The single-page application keeps its JSON Web Token [18] in browser `localStorage`. This removes cookie-based CSRF surface but means a cross-site-scripting flaw could lift the token; it is a deliberate trade for a simpler deployment.
- **Grading tolerance is whitespace-fair, not token-fair.** Output normalisation eliminates the most common false negative — trailing whitespace, newlines, and `CRLF` — but it does not collapse internal whitespace runs and has no floating-point tolerance. This is a scoping decision, not a defect.
- **Public Judge0 endpoint is not for production.** The default Judge0 base URL is the public Community Edition instance, which is rate-limited; a real deployment must self-host Judge0.
- **Minimal automated test coverage.** The backend ships without Go unit-test files and the front end ships a single sanity test; the continuous-integration pipeline nonetheless exercises build, vet, and lint on every change.

Naming these limitations up front is deliberate. A classroom tool that is candid about its failure modes is safer to deploy than one that hides them, and several of these items — particularly the grading queue and the authentication rate limiting — define the most valuable future work (Chapter 13).

1.7 Contributions and Novelty

APEX does not claim to invent sandboxed code execution, automated grading, or learning-management systems; each of those exists in mature form. Its contribution is one of *integration and restraint* — assembling the right pieces into the smallest coherent whole for mixed-format classroom assessment. Concretely, the project contributes:

1. **A unified mixed-format exam model.** Coding, MCQ, and written questions coexist in one timed attempt with one autosave and one weighted score, rather than in three separate tools — directly addressing the fragmentation described in Table 1.1.
2. **A single grading pipeline behind three formats.** One attempt-aggregation step reconciles sandboxed

coding results, set-equality MCQ verdicts, and manually graded prose into a single exam score, coordinating concurrent grading goroutines by idempotent re-aggregation rather than locking.

3. **A fairness-first grading boundary.** Symmetric output normalisation — a nine-line function applied identically to student and expected output — removes the trailing-whitespace false negative that plagues naive I/O diffing, and the identical normalisation drives both the live “run sample tests” feedback and the final graded verdict, so students are judged the same way both times.
4. **A defence-in-depth authorization model.** Three independent, fail-closed gates — JWT identity verification (with an explicit HMAC algorithm check to block algorithm-confusion attacks), role checking, and per-row ownership verification inside every handler — are documented and audited (Fig. 8.2, Fig. 8.1).
5. **A genuinely self-hostable single-artifact deployment.** The backend is one statically linked Go binary in a non-root container with a shell-free self-probing health check, brought up by a four-service Compose stack with a one-shot versioned migrator as the sole schema authority in production.
6. **A documented data-integrity discipline.** A real migration incident — an ORM default that silently hid every exam from students — is analysed and distilled into an explicit set of schema-change rules, turning a mistake into transferable engineering guidance (Fig. 5.2).

The novelty is therefore not any single algorithm but the *combination*: a small, self-hostable portal that treats the three exam formats as one assessment and one pipeline, with fairness and authorization designed in rather than bolted on.

1.8 Intended Users

APEX is built for two audiences, mirrored by its two roles.

Teachers author mixed-format exams from a question bank organised into folders, assign them to one or more classes, set a duration and time window, and choose a passing threshold. After a sitting, they auto-grade coding and MCQ questions instantly, work through a manual-grading queue for written answers, and then announce results to a class. A teacher can reuse bank questions across exams, close and reopen exams, and post announcements that fan out as notifications to enrolled students.

Students join a class with an invite code, see their assigned and upcoming exams, and take a timed exam in one page: typing code into the Monaco editor and pressing “Run” for immediate, unlimited, un-graded feedback against the visible sample tests, answering MCQ and written questions in the same attempt, with their progress autosaved. On submission they receive an immediate acknowledgement and, once grading completes, a single “Exam Graded” notification and a per-test breakdown — with hidden test inputs kept hidden, so the tests cannot be reverse-engineered.

The end-to-end workflows for both roles, with annotated screenshots, form the user manual in Chapter 12; the use-case view of these two actors is given in Fig. 3.1.

1.9 Structure of this Book

The remainder of this thesis is organised as follows.

Chapter 1 — Introduction.

The present chapter: the problem, motivation, vision, objectives, scope, and contributions.

Chapter 2 — Background and Literature Review.

Existing learning-management systems [1], [2], online judges and automated code assessment [3], [4], [6], [7], sandboxing [10], and the gap APEX fills.

Chapter 3 — Requirements Engineering.

Functional and non-functional requirements, actors, and use cases (Fig. 3.1).

Chapter 4 — System Architecture and Design.

The four-layer backend, request lifecycle, and design decisions (Fig. 4.1, Fig. 4.2, Fig. 8.2, Fig. 6.1).

Chapter 5 — Data Model and Persistence.

The fifteen entities, their relationships and unique indexes, and the entity–relationship model (Fig. 5.1).

Chapter 6 — Implementation.

How the backend and front end are built, the technology stack, and code organisation.

Chapter 7 — Code Execution and the Auto-Grading Engine.

The Judge0 integration, `isolate` sandbox, output normalisation, scoring, and attempt aggregation (Fig. 7.1).

Chapter 8 — Security.

The threat model, the three-gate authorization model, JWT and bcrypt [17], [18], [19], and the security findings that were fixed before shipping (Fig. 8.1).

Chapter 9 — Testing and Quality Assurance.

The test strategy, continuous integration, and a frank account of coverage (Fig. 9.1).

Chapter 10 — Deployment and DevOps.

The single-binary, `distroless`, four-service Compose deployment and the migration pipeline (Fig. 10.1, Fig. 5.2, Fig. 6.2).

Chapter 11 — Results and Evaluation.

What was delivered, measured against the objectives, including scale metrics (Fig. 11.1, Fig. 11.2) and the project timeline (Fig. 11.3).

Chapter 12 — User Manual.

Step-by-step teacher and student workflows with annotated screenshots.

Chapter 13 — Conclusion and Future Work.

What was learned and the roadmap, including the durable grading queue and the authentication hardening flagged in the scope above.

Appendices provide the API reference, the database schema, the environment-variable reference, and selected source listings. Throughout, the guiding principle is the one stated at the outset: the smallest system that still takes mixed-format classroom assessment seriously, built and documented transparently.

Chapter 2

Background and Literature Review

This chapter establishes the intellectual and engineering context in which APEX was built. It surveys four bodies of work that the system draws on directly: the history and methods of automated assessment of programming (Section 2.1); online judge systems and the sandboxing technology that lets them run untrusted code safely (Section 2.2); learning-management systems and the quiz and coding tooling built on top of them (Section 2.3); and code-similarity detection, which sits adjacent to grading as a fairness concern (Section 2.4). The chapter closes with a gap analysis (Section 2.5) and a comparison table (Table 2.1) that positions APEX against representative alternatives across the dimensions that matter for a single-department classroom deployment.

The review is deliberately selective. Its purpose is not to catalogue every assessment tool ever published, but to show what the mature systems do well, what they trade away, and where a small self-hostable portal that unifies three answer formats behind one grading pipeline earns its place. Throughout, claims about APEX are grounded in the actual implementation rather than aspiration.

2.1 Automated Assessment of Programming

2.1.1 A short history

Automatically grading programs is nearly as old as teaching programming. The earliest graders date to the 1960s, when batch systems compiled a student’s submission and compared its printed output to a reference. The idea has proven remarkably durable: the fundamental loop—feed an input, capture the output, compare it to what was expected—is still the core of every modern coding-assessment platform, APEX included.

The first systematic academic survey of the modern era is Douce, Livingstone and Orwell [6], who in 2005 organised the field into three “generations”: early command-line graders tied to a single machine; web-based systems that decoupled the student from the grading host; and a nascent third generation concerned with pedagogy, feedback quality, and integration with course-management tooling. Their central observation has aged well: the hard problems in automated assessment are not the mechanics of running code but the surrounding concerns—security of execution, quality of feedback, and the integration of grading into an actual course.

Five years later, Ihantola et al. [7] produced the reference survey for the field, reviewing dozens of tools against a common set of features: supported languages, the style of test specification, the resubmission policy, the security model, and—critically—how tightly the grader integrated with a learning-management system. Their finding was that classroom features and grading features consistently lived in different products, and that stitching them together fell to the individual instructor. That observation is precisely the gap APEX targets.

The most cited recent survey, Waśik et al. [9], examines online judges specifically and decomposes them into three recurring components that recur throughout this chapter: (1) the *contest or exam scaffold*

that presents problems and collects submissions; (2) the *language-aware compile-and-run sandbox* that executes untrusted code under resource limits; and (3) the *verdict aggregator* that turns per-test outcomes into a score. APEX maps cleanly onto this decomposition: its scaffold is the Go/Gin backend and React SPA, its sandbox is delegated to Judge0, and its aggregator is the `judge0.Grade / maybeFinalizeAttempt` pipeline analysed in Chapter 7.

2.1.2 Two families of grader: I/O diffing versus unit testing

The literature divides auto-grading into two broad families, and the choice between them is the single most consequential design decision a grader makes.

Input/output diffing. The student writes a complete program that reads from standard input and writes to standard output. The grader supplies an input, captures the program’s output, and compares it—textually—against an expected output. This is the model of the competitive-programming judges and the one Douce and Waśik describe as dominant [6], [9]. Its virtues are that it is entirely language-agnostic (the grader never needs to understand the student’s code, only its output), simple to specify (a test case is just a pair of strings), and robust against students who structure their solution differently than the instructor imagined.

Unit-test execution. The student writes a function or class conforming to a fixed signature, and the grader compiles it against an instructor-authored test harness that asserts on return values. This model, used by tools such as CodeRunner [20] and much of the CS-education tooling surveyed by Ihantola [7], gives finer-grained feedback (“your function returned the wrong value for a negative argument”) and can grade code that never produces console output. Its cost is that the harness is language-specific and the student is constrained to an exact interface.

APEX uses I/O diffing, for the reasons the survey literature endorses: it keeps the grader uniform across all six supported languages (Python, JavaScript, TypeScript, Java, C, and C++) and lets a teacher author a coding problem as nothing more than a list of `(input, expected_output)` pairs. The well-known weakness of I/O diffing is its brittleness: a solution that computes the correct answer can still be rejected on a cosmetic mismatch—a trailing newline, a Windows carriage return, or a stray trailing space. This “trailing-newline unfairness” is the most common false negative on any diff-based platform.

APEX’s response is a nine-line normalisation function, `normalizeOutput`, applied symmetrically to both the student’s actual output and the teacher’s expected output before comparison. It converts CRLF and lone CR line endings to LF, trims trailing spaces and tabs on each line, and trims leading and trailing whitespace of the whole string. Listing 2.1 reproduces it. The design intent, developed fully in Chapter 7, is to make the comparison measure the *correctness of content* rather than the *byte-exactness of whitespace*. It is honest about its scope: it does not collapse internal whitespace runs and applies no floating-point tolerance, so it addresses the most common false negative rather than every conceivable one.

```

1 // normalizeOutput strips CR, trailing whitespace on each line, and trims
2 // edges so Judge0 output (often \r\n with a trailing newline) compares
3 // cleanly to teacher-entered expected output.
4 func normalizeOutput(s string) string {
5     s = strings.ReplaceAll(s, "\r\n", "\n")
6     s = strings.ReplaceAll(s, "\r", "\n")
7     lines := strings.Split(s, "\n")
8     for i, ln := range lines {

```



```
9         lines[i] = strings.TrimRight(ln, " \t")
10     }
11     return strings.TrimSpace(strings.Join(lines, "\n"))
12 }
```

Listing 2.1. The output-normalisation function that mitigates I/O-diff brittleness, applied symmetrically to actual and expected output.

2.1.3 Feedback, partial credit, and resubmission

Beyond the pass/fail verdict, the assessment literature emphasises three pedagogical levers. The first is *feedback*: Ihantola et al. note that the difference between a grader that merely scores and one that teaches lies in what it tells the student about a failure [7]. APEX distinguishes visible *sample* test cases (`is_sample = true`), whose input, expected, and actual output are shown, from hidden tests, for which the student sees only pass/fail and timing—preventing reverse-engineering of the graded tests while still offering pre-submission feedback.

The second lever is *partial credit*. Rewarding a solution that passes seven of ten tests with seventy percent rather than zero is both fairer and more informative. APEX awards proportional partial credit on coding problems (`score = passed / total * 100`) but uses all-or-nothing set-equality on multiple-choice questions, reflecting the different semantics of the two formats.

The third lever is the *resubmission policy*—how freely a student may iterate. APEX separates “run” from “submit”: a student may execute sample tests any number of times during an exam at zero database and grading cost, because only the final `POST /student/exams/:id/submit` persists a graded submission. This mirrors the iterate-freely, grade-once posture that the survey literature associates with good learning outcomes, without the storage and sandbox load of grading every keystroke.

2.2 Online Judges and Sandboxing

2.2.1 The online-judge tradition

Whereas the assessment literature grew out of education research, the online-judge tradition grew out of competitive programming, where the correctness and, above all, the *security* of the execution sandbox dominate every other concern. Systems such as the UVa Online Judge, Codeforces, and HackerRank [3] all share the same shape: accept a source submission, run it against a battery of tests inside an isolation layer, and return a verdict. Waśik et al. [9] survey this lineage in detail, and Cáceres et al. [8] survey the web architectures behind such systems, observing that the durable designs separate the sandbox from the scaffold behind a thin HTTP interface—exactly the boundary APEX draws between its Go backend and Judge0.

The reason execution is the crux is stark: a student submission is arbitrary, untrusted code. A single line such as `os.system("rm -rf /")` or a fork bomb, if run on the application host, compromises the server. A judge must therefore run every submission inside a locked environment with strict CPU, memory, and filesystem limits and no meaningful access to the host or network. Writing such a sandbox correctly is one of the hardest and most security-sensitive tasks in systems programming, which is why the mature ecosystem has converged on a small number of shared, battle-tested implementations rather than each platform rolling its own.

2.2.2 The `isolate` sandbox

The de facto standard sandbox for this purpose is `isolate` [10], originally developed as the grading sandbox for the International Olympiad in Informatics (IOI). `isolate` builds its “locked room” from Linux kernel primitives: *namespaces* isolate a process’s view of the filesystem, process table, and network, while *control groups* (cgroups) cap its CPU time and memory. The result is a lightweight, per-submission jail that can run untrusted code, kill it when it exceeds a limit, and report the time and memory it consumed—while guaranteeing that the worst a malicious submission can do is hit a limit and be terminated. Its provenance at the IOI, where it grades the submissions of the world’s strongest high-school programmers under adversarial conditions, is exactly why it is trustworthy: it is proven technology, not an improvisation.

2.2.3 Judge0 as execution-as-a-service

`isolate` is a sandbox, not a service. Wrapping it in a language-aware, network-accessible API is the role of Judge0 [4], [5], the most prominent open-source “code execution as a service.” Došilović and Meštrović [5] describe Judge0 as a system that accepts a `{language, source_code, stdin}` payload over HTTP, compiles and runs the code in `isolate`, and returns `{stdout, stderr, status, time, memory}`. It supports dozens of language runtimes behind a uniform REST interface and enforces per-submission resource limits through the underlying sandbox.

APEX delegates all code execution to Judge0. This is a deliberate architectural choice, not a convenience: the most security-critical component in the entire system—the one that runs untrusted code—is the one the team did *not* write, precisely because writing a sandbox is how a project gets compromised. The whole of APEX’s interaction with the sandbox is confined to a single file, `internal/judge0/client.go`, and reduces to one synchronous HTTP call:

```
1 url := BaseURL +  
2   "/submissions?wait=true&base64_encoded=false" +  
3   "&fields=stdout,stderr,compile_output,status,time,memory"  
4 client := &http.Client{Timeout: 30 * time.Second}  
5 resp, err := client.Post(url, "application/json", bytes.NewReader(body))
```

Listing 2.2. APEX’s entire interaction with the sandbox: one synchronous POST to Judge0, which runs the code in `isolate`.

Several properties of this integration are worth noting because they reflect the trade-offs the literature anticipates. The call is *synchronous*: `wait=true` blocks the request until Judge0 finishes, guarded by a 30-second client timeout, avoiding the polling loops or webhook callbacks that a higher-throughput design would need. The base URL defaults to the public `https://ce.judge0.com` instance but is overridable via the `JUDGE0_URL` environment variable; the public endpoint is rate-limited and intended for evaluation, so a real deployment self-hosts Judge0. APEX maps six language names to Judge0’s numeric language identifiers and pre-validates the language before ever making the call. Notably, APEX treats Judge0 as an *executor* rather than a *judge*: it counts both Judge0’s “Accepted” (status 3) and “Wrong Answer” (status 4) as “the code ran,” then re-compares the output itself with `normalizeOutput`, because Judge0’s own diff is byte-exact and would reproduce the very trailing-newline unfairness the normalisation exists to eliminate. This separation of “run it safely” from “decide if it is correct” is the concrete embodiment of the sandbox/scaffold boundary that Cáceres et al. [8] identify as characteristic of durable designs.

2.3 Learning-Management Systems and Quiz Tooling

2.3.1 The LMS lineage

The other parent of APEX is the learning-management system. Moodle [1], first released in 2002, established the template: an open-source PHP application bundling course pages, announcements, a quiz module, and a gradebook into a single self-hostable product. Canvas [2] refined the same form factor for large institutions with a polished interface and an extensive integration ecosystem. The defining strength of the LMS lineage is *breadth*: these systems cover the entire course lifecycle, from enrolling students through delivering content to recording final marks.

That breadth is also the source of their limitation for programming courses. LMS quiz modules natively handle multiple-choice, matching, and short-answer questions, but programming-style assessment is not part of the core model. Moodle’s quiz engine, for instance, supports MCQ and numeric answers directly, but grading code requires a plugin. The consequence, documented across the assessment surveys [6], [7], is a workflow in which coding, multiple-choice, and written assessment live in separate tools with separate gradebooks, and the instructor performs the integration by hand.

2.3.2 Coding on top of the LMS

The dominant answer to “how do I grade code inside my LMS” is CodeRunner [20], a mature Moodle question-type plugin that runs student code against instructor-defined tests—typically in the unit-test style—inside a sandbox, and reports a per-test result within the Moodle quiz. CodeRunner is capable and widely deployed, and it is the closest LMS-native analogue to APEX’s coding questions. It inherits both the strengths of Moodle (rich rostering, gradebook integration, an established administrative model) and its costs (a full Moodle installation, PHP hosting, plugin and sandbox configuration, and a coding question that remains a bolt-on to a quiz engine designed around static question types).

At the other pole sits HackerRank [3], a commercial, cloud-hosted platform built judge-first. It offers a polished in-browser coding experience, a large problem bank, and strong auto-grading, and it is used both for competitive practice and for technical hiring. Its trade-offs are the mirror image of the LMS lineage: mixed-format exams that combine coding with substantial written and multiple-choice sections in a single sitting are not its centre of gravity, it is neither self-hostable nor free, and its rostering and course model are lighter than an LMS provides.

APEX sits deliberately between these poles. Rather than bolting code execution onto a quiz engine or bolting a course model onto a judge, it treats the three answer formats (coding, MCQ, written) as first-class citizens of a *single* exam attempt: one timer, one autosave path, one submission endpoint, and one grading pipeline that dispatches to `Grade`, `GradeMCQ`, or `GradeWritten` by problem type before re-aggregating a single weighted attempt score. This unification, developed in Chapter 7, is the feature that neither the LMS-plus-plugin nor the judge-first lineage offers cleanly.

2.3.3 Manual grading and mixed formats

A working classroom needs to grade prose, not only code. Written answers cannot be graded by a diff, so any honest mixed-format system needs a manual-grading path. LMS platforms have mature manual-grading UIs; pure judges generally do not. APEX carries a manual path as a first-class part of the pipeline: written answers, and multiple-choice questions for which no correct option was configured, are marked

`pending_review` and enter a queue that teachers drain via `GET /teacher/grading/pending` and `PUT /submissions/:id/grade`. Crucially, `pending_review` submissions are excluded from both the numerator and denominator of the attempt score until a human grades them, so an ungraded essay never makes a student's visible score look like a failure before the teacher has looked at it. This blending of automatic and manual grading behind one attempt score is precisely the integration the assessment surveys found missing between the two lineages.

2.4 Code-Similarity and Plagiarism Detection

Adjacent to grading, and important to its fairness, is the detection of copied submissions. The two canonical open systems are MOSS (Measure Of Software Similarity) [21] from Stanford and JPlag [22]. Both ingest a corpus of student source files and report clusters of suspiciously similar submissions, but they do so structurally rather than textually: they tokenise each program and compare token sequences, so that superficial obfuscations—renaming variables, reordering independent statements, reformatting whitespace, inserting dead comments—do not defeat detection. MOSS uses a document-fingerprinting technique (windowing) to compare token streams at scale, while JPlag performs a greedy tiling match over token strings; Prechelt et al. [22] report that JPlag reliably identifies plagiarised pairs even under such transformations.

Two properties of this work matter for APEX's scope. First, similarity detection is *corpus-relative*: it answers “which of these submissions resemble each other,” not “is this one submission correct,” and so it is orthogonal to the per-submission grading pipeline. Second, it is typically an *offline, teacher-triggered batch* action run over a completed cohort rather than an inline check at submission time. APEX does not currently ship an integrated similarity check; it is identified as future work in Chapter 13. The architectural boundary APEX already draws—storing each submission's source, language, and per-test results as durable rows—means a MOSS- or JPlag-style batch export is a natural future addition rather than a redesign. Proctoring (webcam capture, screen locking) is a separate and ethically contested body of work that APEX deliberately does not pursue; at most a future iteration would surface non-invasive signals such as tab-blur or paste events to the teacher rather than enforcing them.

2.5 Gap Analysis and Positioning

The surveys and systems above converge on a consistent picture. The assessment literature identifies, and has identified for two decades, a persistent gap between *course* tooling and *grading* tooling [6], [7]. The LMS lineage owns the course but treats code as a bolt-on; the judge lineage owns code execution but travels light on rostering, mixed formats, and manual grading. Bridging the two has been the subject of a long tail of institutional projects, but the two dominant production answers—an LMS with a coding plugin, or a commercial cloud judge—each inherit the limitations of their lineage. Four specific gaps motivate APEX's design.

1. **Mixed formats in one sitting.** No mainstream option treats coding, MCQ, and written answers as equal first-class citizens of a single timed attempt with a single score. LMS quizzes bolt code on; judges under-serve prose. APEX makes the unified attempt the centre of the design.
2. **One grading pipeline.** Where mixed grading exists, it is usually the sum of independent subsystems. APEX runs one dispatch-and-aggregate pipeline behind all three formats, with a single weighted attempt score and a single “Exam Graded” notification.

3. **Self-hostability without operational weight.** Moodle is self-hostable but heavy; Canvas and Hacker-Rank are effectively cloud services. APEX ships as one Go binary plus PostgreSQL plus an external Judge0, deployable via a four-service Docker Compose file, with the security-critical sandbox delegated to a proven external component.
4. **Cost.** The self-hostable, no-licence-fee option (Moodle plus CodeRunner) is operationally demanding; the low-friction options are commercial. APEX targets the empty quadrant: free and self-hostable and low-friction.

Table 2.1 summarises how APEX compares against four representative alternatives across the dimensions that a single-instructor or single-department deployment actually weighs. The assessment is qualitative and, in the “mixed formats” and “simplicity” rows, necessarily subjective; it is meant to make the positioning legible, not to claim measurement. The honest caveats belong in the same breath: the four comparators are mature, widely deployed, and battle-tested at institutional scale, whereas APEX is a graduation project with, by its own account, no backend unit tests, no rate limiting on authentication, and a fire-and-forget grading path with no durable work queue (all detailed in Chapters 8 and 9). The comparison claims a favourable *shape*, not superiority on every axis.

Table 2.1. APEX against representative alternatives. “Yes/No” is a capability judgement; cost reflects licensing and hosting for a small deployment. Assessments in the mixed-format and simplicity rows are subjective summaries of the discussion above.

Dimension	APEX	Moodle + CodeRunner	Canvas	Hacker-Rank	Raw Judge0
Self-hostable	Yes	Yes	No (cloud)	No (cloud)	Yes
Mixed coding + MCQ + written in one sitting	Yes, first-class	Partial (code is a plugin)	Partial (no code)	Weak	No
Unified grading pipeline	Yes, one path	Per-subsystem	Per-subsystem	Code only	Code only
Live-coding editor (Monaco, in-browser)	Yes	Basic	Basic	Yes	None (API only)
Manual grading queue for prose	Yes	Yes	Yes	Weak	None
Class rosters / invite codes	Yes	Yes	Yes	Partial	None
Sandbox for untrusted code	Delegated to Judge0/isolate	Own sandbox config	External tool	Built-in	isolate
Deployment simplicity	One Go binary + Postgres + Judge0	Full PHP + plugin + sandbox	N/A (hosted)	N/A (hosted)	Sandbox only, no scaffold
Cost	Free, self-hosted	Free, heavy ops	Paid licence	Paid	Free (self-host)

The pattern the table makes visible is the one the surveys predict. The LMS-derived options (Moodle, Canvas) are strong on classroom workflow and manual grading but treat code as a secondary format; the

judge-derived options (HackerRank, raw Judge0) are strong on code execution but thin on rostering, mixed formats, and prose grading. APEX's contribution is not a new sandbox or a new grading algorithm—it reuses `isolate` through Judge0 and uses textbook I/O diffing—but the *integration*: unifying three answer formats and a full classroom workflow behind one grading pipeline in a single self-hostable artifact, which is the empty region between the two established lineages. The chapters that follow document how that integration is engineered: the requirements it entails (Chapter 3), the architecture that realises it (Chapter 4), and the grading engine at its core (Chapter 7).

Chapter 3

Requirements Engineering

Requirements engineering is the discipline of discovering, analysing, documenting, and validating the services a system must provide and the constraints under which it must operate [23]. For APEX the exercise was neither a formal customer-elicitation programme nor a speculative wish-list: the system was built to serve a single, well-understood workflow — running mixed-format examinations in a university classroom — and the requirements below were distilled from that workflow, from the real feature set that the implementation ultimately shipped, and from the honest constraints of a five-person graduation project. This chapter records those requirements as they were actually realised, so that every functional requirement can be traced to running code and every non-functional requirement to a measurable property of the deployed system. Pressman’s distinction between what a product must *do* (functional) and the qualities it must *exhibit* (non-functional) frames the presentation [24].

The chapter is deliberately grounded rather than aspirational. Where a capability was scoped out, or shipped with a known limitation, it is stated plainly; padding the requirement set with features the code does not implement would defeat the purpose of a traceability record.

3.1 Stakeholders and Actors

A *stakeholder* is any party with a legitimate interest in the system; an *actor* is a role that interacts directly with it across the system boundary [23]. APEX has few of the former and only two of the latter, and that scarcity is itself a design decision worth stating.

3.1.1 Stakeholders

- **Teachers** own the pedagogical content — classes, exams, question banks — and are the primary paying beneficiaries of automation: the system exists to remove the manual labour of grading code.
- **Students** are the subjects of assessment. Their overriding interest is that the timed sitting be fair, resilient to network and device failures, and transparent in how a score was reached.
- **The instructor / institution** (the project’s academic supervisor and the Faculty of Engineering at Menoufia University) is a governance stakeholder: the system must be defensible as an engineering artefact and self-hostable so that no student data leaves institutional control.
- **The operator** — whoever deploys and runs the instance — has a stake in low operational cost, reproducible builds, and safe schema evolution. In a classroom deployment the operator and the teacher may well be the same person, which reinforces the requirement that operation be simple.

3.1.2 Actors: two roles, no administrator

APEX recognises exactly two interacting roles, encoded in the `role` column of the `User` table as the string literals `teacher` and `student`. Table 3.1 summarises them. There is deliberately **no administrator role**. This is a genuine architectural boundary, not an omission: the authorization middleware, `middleware.RequireRole(roles...)`, is generic and accepts any set of role strings, but the shipped build only ever instantiates it with `"teacher"` and `"student"`. The consequence is that every privileged action must be justified by *ownership of a specific resource* (a teacher may edit only exams whose `teacher_id` matches their own identity) rather than by a blanket super-user grant. Removing the administrator removes an entire class of privilege-escalation target and keeps the authorization story to two coarse roles plus a per-row ownership check.

Table 3.1. Primary actors of the APEX system.

Actor	Role literal	Goals and system interactions
Teacher	<code>teacher</code>	Create and manage classes; author coding, MCQ, and written questions; maintain a reusable question bank in folders; build, publish, and assign exams to classes; auto- and manually grade submissions; post announcements; monitor results and class statistics.
Student	<code>student</code>	Join classes by invite code; take timed exams in a browser code editor with autosave and resume; run code against sample tests before submitting; review per-test results and teacher feedback; receive notifications and reminders.

A handful of *external* systems participate in the requirements without being human actors: Judge0 (the code-execution sandbox reached over HTTP), an SMTP relay for password-reset and reminder mail, and Google Identity Services for federated sign-in. Each is optional-or-degradable — absent an SMTP host, mail is logged to standard output; absent a Google client ID, the federated endpoint returns HTTP 503 — which is itself a portability requirement captured in Section 3.5.

3.2 User Stories

Following the agile convention, requirements were first captured as user stories of the form “As a *<role>*, I want *<capability>* so that *<benefit>*” [24]. The stories below are the ones that survived into shipped functionality; each maps to one or more functional requirements in Section 3.4 and, through them, to concrete endpoints and handlers.

Teacher stories

- ST-T1** As a teacher, I want to create a class and share an invite code, so that students can enrol themselves without my manually maintaining a roster.
- ST-T2** As a teacher, I want to author coding, multiple-choice, and written questions with points, difficulty, hints, and time/memory limits, so that one exam can assess several kinds of skill in a single sitting.
- ST-T3** As a teacher, I want to keep reusable questions in a folder-organised bank, so that I can compose new exams from proven material instead of re-authoring it.
- ST-T4** As a teacher, I want exams to start as drafts and only become visible when I explicitly publish them (which requires setting a start time), so that a half-finished exam never leaks to students.

- ST-T5** As a teacher, I want to assign one exam to several classes at once, so that parallel sections sit the same paper.
- ST-T6** As a teacher, I want code answers graded automatically against my test cases, and written answers queued for me to grade with feedback and a score override, so that I spend my time only on the answers that genuinely need human judgement.
- ST-T7** As a teacher, I want to post class announcements that notify enrolled students, so that I can communicate without a separate channel.
- ST-T8** As a teacher, I want to preview an exam exactly as a student will see it and to withhold results until grading is complete, so that I control what is released and when.

Student stories

- ST-S1** As a student, I want to join a class by typing an 8-character code, so that enrolment is instant and needs no teacher intervention.
- ST-S2** As a student, I want my in-progress answers saved on the server as I work, so that a dropped connection, a dead battery, or a switched device does not cost me my work.
- ST-S3** As a student, I want to resume the same attempt after an interruption rather than starting over, so that a crash mid-exam is recoverable.
- ST-S4** As a student, I want to run my code against sample tests and see the output before I submit, so that I get fast feedback while I still have time to fix it.
- ST-S5** As a student, I want to review, after grading, exactly which tests passed and any feedback my teacher left, so that the mark is transparent and I can learn from it.

Platform stories

- ST-P1** As an operator, I want to stand up the whole system with one binary, a PostgreSQL database, and an external Judge0, so that a classroom instance is cheap and self-hostable.
- ST-P2** As an operator, I want the service to refuse to start with a weak signing secret and to evolve its schema safely, so that a misconfiguration cannot become a silent data-loss incident.
- ST-P3** As a user of either role, I want to be reminded of upcoming exams and notified when my work is graded, without duplicate messages if the server restarts.

3.3 Use-Case Model

The user stories condense into the use-case model of Fig. 3.1, which shows the principal interactions of each actor across the system boundary. The Teacher’s use cases cluster around *authoring and administration* — managing classes, building exams, curating the question bank, assigning, grading, announcing, and inspecting results — while the Student’s cluster around *participation* — joining, sitting the timed exam, running and submitting code, reviewing results, and receiving notifications. The asymmetry is expected: APEX automates the teacher’s labour and constrains the student’s environment.

Two properties of Fig. 3.1 deserve comment. First, no use case crosses from one actor to the other; the ownership model in Section 3.1.2 enforces that a teacher never “takes” an exam and a student never “grades” one. Second, several teacher use cases (*Grade Written Answers*, *View Results / Stats*) and the student’s *Review Results* are two ends of the same underlying data — the attempt aggregate — so although

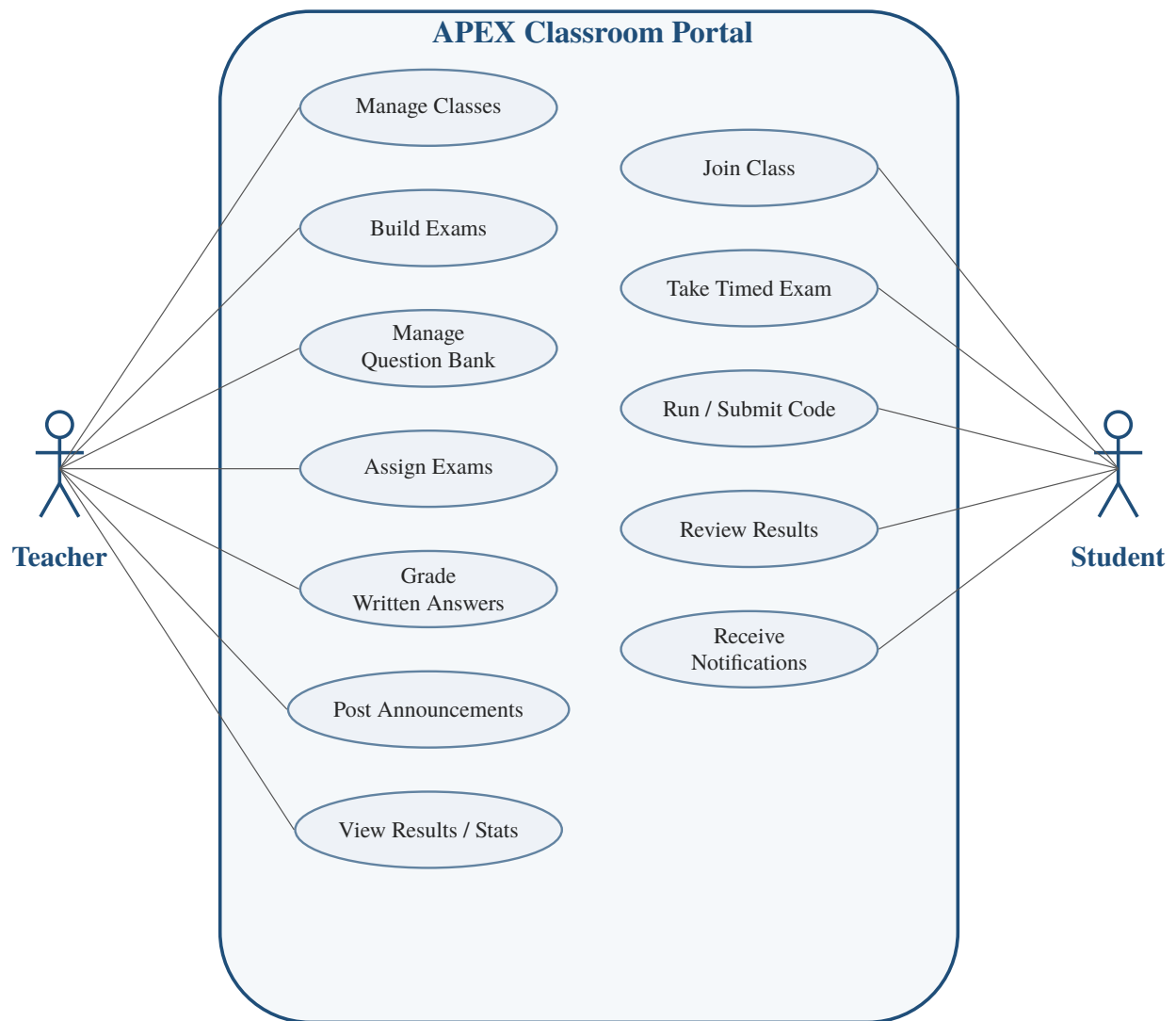


Figure 3.1. Principal use cases for the Teacher and Student actors.

they are separate use cases they are bound by a shared invariant: a student sees a result only after, and exactly as, the teacher’s grading produced it.

3.4 Functional Requirements

Functional requirements state what the system must do. They are grouped below by the party they primarily serve — Teacher, Student, and Platform — and each is given a stable identifier ($FR-Tn$, $FR-Sn$, $FR-Pn$) so that Section 3.6 can trace it to an implementing component. Every requirement listed corresponds to functionality that is present in the shipped system; the wording is drawn from the real capabilities enumerated in the project’s feature inventory.

3.4.1 Teacher requirements

Table 3.2. Functional requirements serving the Teacher actor.

ID	Requirement
FR-T1	The system shall let a teacher create, edit, and delete classes, each with a name, section, optional cover image, and an automatically generated unique 8-character invite code, and shall list the class roster and permit removal of members.
FR-T2	The system shall provide an exam builder for authoring questions of type <code>coding</code> , <code>mcq</code> , and <code>written</code> , each with points, difficulty, optional hints, image attachments, and tags; coding questions shall additionally carry a per-question time limit (default 2000 ms) and memory limit (default 262144 KB) and a set of test cases.
FR-T3	The system shall maintain a per-teacher question bank of reusable questions (<code>is_bank = true</code>), organisable into folders, so that questions can be composed into exams independently of any single exam.
FR-T4	The system shall distinguish draft from published exams and shall refuse to publish (clear <code>is_draft</code>) an exam whose <code>start_time</code> is unset, so that only complete, scheduled exams become visible to students.
FR-T5	The system shall let a teacher assign one exam to one or many classes, rejecting any assignment that references a class the teacher does not own.
FR-T6	The system shall configure per-exam options: shuffle question order, show results after submission, and a passing score.
FR-T7	The system shall automatically grade coding submissions against their test cases and MCQ submissions by set-equality of the selected against the correct options, and shall queue written submissions for manual grading.
FR-T8	The system shall provide a pending-grading queue from which a teacher grades written (and any un-configured MCQ) answers, supplying a score, a status, and optional free-text feedback, after which the affected attempt’s aggregate score is recomputed.
FR-T9	The system shall let a teacher post per-class announcements with attachments, fanning out an in-app notification to every enrolled student.
FR-T10	The system shall present a teacher dashboard, a results explorer with per-exam outcomes, and per-class statistics.
FR-T11	The system shall let a teacher preview an exam exactly as a student will see it, and shall optionally block grade announcement for a class until all manual grading for it is complete.
FR-T12	The system shall, when a teacher destructively edits a published exam, wipe the affected attempts, submissions, and test results in a single transaction, stamp <code>reset_at</code> , un-announce the class grades, and notify affected students.

3.4.2 Student requirements

Table 3.3. Functional requirements serving the Student actor.

ID	Requirement
FR-S1	The system shall let a student join a class by entering its 8-character invite code, leave a class, and list the classes and assigned exams available to them.
FR-S2	The system shall let a student start a timed attempt at a published, in-window exam, creating at most one attempt per (student, exam) pair; a repeated start shall resume the existing attempt rather than create a second one.
FR-S3	The system shall autosave a student's in-progress answers to the server (PUT /student/exams/:id/autosave) and rehydrate them on resume, so that a crash or device change does not lose work.
FR-S4	The system shall optionally present an exam's questions in shuffled order, consistently within a single attempt.
FR-S5	The system shall provide a browser code editor supporting six languages (Python, JavaScript, TypeScript, Java, C, C++) with a run-against-sample-tests action that returns output, time, and memory without persisting a graded submission.
FR-S6	The system shall let a student submit an exam, after which no further submission for that attempt is accepted, and shall grade the submission server-side.
FR-S7	The system shall let a student review each attempt's result — the aggregate score, the per-test breakdown for coding questions, and any teacher feedback — subject to the teacher's results-release settings.
FR-S8	The system shall present the student's notifications, unread count, and a help page, and shall let the student manage a profile and notification preferences.

Table 3.4. Platform-wide functional requirements (cross-cutting both roles).

ID	Requirement
FR-P1	The system shall authenticate users by email and password (bcrypt-hashed) and issue a signed session token valid for 24 hours, and shall support federated sign-in via Google Identity Services when configured.
FR-P2	The system shall support password reset by email using a single-use, SHA-256-hashed token with a one-hour expiry, and shall not reveal whether an email address is registered.
FR-P3	The system shall let a user permanently delete their own account, cascading the deletion across their dependent records in a single transaction.
FR-P4	The system shall send in-app and email reminders for upcoming exams — in-app at roughly one hour before, email at one hour and again at start — deduplicated by per-exam timestamp columns so that a process restart cannot re-send a reminder.
FR-P5	The system shall notify a student exactly once when their exam has been graded, deduplicated by a per-attempt flag so that repeated finalisation does not repeat the notification.
FR-P6	The system shall expose an unauthenticated liveness endpoint (GET /healthz) and a self-probe (--healthcheck) so that a shell-less container image can be health-checked.
FR-P7	The system shall evolve its database schema through idempotent, ordered migrations, and in production shall make a versioned migration track the sole schema authority (with GORM auto-migration disabled).

3.4.3 Platform requirements

A note on grading semantics belongs with the functional requirements because it governs observable behaviour. The aggregate score of an attempt (FR-S7, FR-T8) is not an incrementally mutated counter but a value recomputed from scratch over the exam's *full* problem list, so a skipped question counts zero in the denominator, while a `pending_review` written answer is excluded from both numerator and denominator until a human grades it, so an in-flight manual grade never transiently depresses the visible total. This behaviour is the functional contract the traceability table binds to the `maybeFinalizeAttempt` aggregator.

3.5 Non-Functional Requirements

Non-functional requirements constrain *how well* the system performs its functions. To be useful they must be measurable [23]; each requirement below is therefore stated with a concrete, checkable criterion drawn from the actual configuration and design of the system, and each is given an identifier `NFR-n`.

3.5.1 Security

- **NFR-1 (authentication strength).** Passwords shall be stored only as `bcrypt` hashes at `bcrypt.DefaultCost` (cost 10) and never returned in any API response; session tokens shall be HS256-signed JWTs with a 24-hour expiry.
- **NFR-2 (fail-fast secret).** The service shall refuse to start (via `log.Fatal`) if `JWT_SECRET` is empty, shorter than 32 characters, or the placeholder literal `dev-secret-change-in-prod`. This is a boolean, boot-time-checkable criterion.
- **NFR-3 (defence in depth).** Every protected request shall pass three independent authorization gates that each fail closed: JWT verification (401 on any doubt), role check (403 on mismatch), and per-row ownership check in the handler; a failure in any one gate shall not open the others.
- **NFR-4 (input and injection safety).** The system shall parameterise all database access through the ORM (mitigating `CWE-89` [25]), restrict cross-origin access to a configured allow-list, and never execute student code in the application process — all execution shall be delegated to the `Judge0 isolate sandbox` [4], [10].
- **NFR-5 (honest limits).** The requirement set explicitly records the known gaps as accepted risk for the thesis snapshot: authentication endpoints are not rate-limited, the password minimum is six characters, and the session token lives in browser `localStorage`. Each is a documented decision, to be hardened before public exposure, not a hidden defect.

The security requirements are aligned with the OWASP Top 10 categories for broken access control and identification failures [19]; Chapter 3 states them, Fig. 8.1 and the security chapter analyse them.

3.5.2 Performance and scalability

- **NFR-6 (responsiveness of submit).** Exam submission shall return to the student without waiting for grading: grading is dispatched to background workers so the HTTP response carries only the attempt and submission identifiers. The measurable criterion is that the submit response latency is independent of the number or runtime of test cases.
- **NFR-7 (bounded execution).** Each code execution shall run under the per-problem time limit (default 2000 ms) and memory limit (default 262144 KB) enforced by the sandbox, and the grading HTTP

client shall apply a 30-second ceiling per Judge0 call, so a pathological submission cannot occupy resources unbounded.

- **NFR-8 (connection pooling).** The database layer shall bound its connection pool — 25 max-open, 10 max-idle connections, with 5-minute idle and 30-minute maximum connection lifetimes — so that concurrent load degrades gracefully rather than exhausting the database.
- **NFR-9 (indexed hot paths).** Correctness- and performance-critical queries shall be backed by indexes: unique composite indexes on `ClassMember(class_id, user_id)`, `ExamAttempt(user_id, exam_id)`, and `ExamClass(exam_id, class_id)` enforce their one-row invariants, and `exams.reset_at` is indexed to support the take-screen cache-bust query.

The target scale is a classroom, not a MOOC: tens of concurrent students per exam, not tens of thousands. The pooling and single-binary choices are sized for that target and are stated as such rather than claimed to be web-scale.

3.5.3 Usability

- **NFR-10 (heuristic conformance).** The user interface shall conform to Nielsen’s usability heuristics [26], in particular *visibility of system status* (a submission’s state — `pending`, `running`, `graded` — is always shown; live sample-test feedback reports pass/fail before submit) and *error prevention* (a draft exam cannot be published without a start time; a submitted attempt cannot be resubmitted).
- **NFR-11 (recoverability).** The interface shall protect against user and environment error by never silently discarding work: in-progress answers are autosaved server-side and an interrupted attempt resumes where it left off, satisfying the “user control and freedom” heuristic.
- **NFR-12 (measured satisfaction).** Perceived usability shall be evaluable with the System Usability Scale [27], a ten-item instrument yielding a 0–100 score; adopting a standard instrument makes the usability claim falsifiable rather than anecdotal. The evaluation is reported in the results chapter.

3.5.4 Portability and self-hostability

- **NFR-13 (single-artefact deploy).** The backend shall compile to a single statically linked binary with no runtime dependencies beyond PostgreSQL and a reachable Judge0, and shall ship in a distroless container image running as a non-root user.
- **NFR-14 (twelve-factor configuration).** All configuration shall be read from environment variables with sensible defaults; no configuration shall be compiled in, so the same image runs in development and production unchanged.
- **NFR-15 (graceful degradation of externals).** Optional external services shall degrade rather than fail: with `SMTP_HOST` unset, mail is logged to standard output; with `GOOGLE_CLIENT_ID` unset, the federated sign-in endpoint returns HTTP 503 while password sign-in continues to work; the default Judge0 URL points at a public instance so first-run needs no local sandbox.
- **NFR-16 (managed-database compatibility).** The system shall accept a full `DATABASE_URL` (taking precedence over discrete `DB_*` variables), making it deployable against managed PostgreSQL providers without code change.

3.5.5 Maintainability and reliability

- **NFR-17 (layered structure).** The backend shall be organised into four request-processing layers (transport, handlers, domain services, persistence) with feature-scoped packages, so that a change is localised to one package.
- **NFR-18 (safe schema evolution).** Schema changes shall follow the add-nullable → backfill → constrain discipline and shall never rely on an ORM DDL default to backfill a populated table; production shall run a versioned migration track as the sole schema authority. This requirement was forced by a real incident (an ORM default silently hid every exam from students) and is now enforced by project rule.
- **NFR-19 (continuous integration).** Every change shall pass an automated pipeline — backend vet, test, and lint; frontend lint, build, and test; and a Docker smoke build — before merge, so that the mainline remains buildable.
- **NFR-20 (idempotent background work).** Reminder dispatch and grade notification shall be idempotent across process restarts, using database columns rather than in-memory state for deduplication, so that reliability does not depend on process uptime.

APEX's configuration and deployment posture follow the twelve-factor guidance [28] and the distroless-image practice [13]; the maintainability requirements reflect that lineage.

3.6 Requirements Traceability

Traceability links each requirement to the artefact that satisfies it, so that a reader — or an examiner — can verify the claim against the source rather than take it on trust [23]. Table 3.5 maps every functional requirement to the principal component and endpoint that realise it. The non-functional requirements trace to cross-cutting mechanisms already named in Section 3.5: NFR-1–5 to the authentication and authorization middleware and the Judge0 sandbox, NFR-6–9 to the grading dispatch and the database pool and indexes, NFR-10–12 to the SPA and its autosave/resume flow, NFR-13–16 to the Docker and configuration layer, and NFR-17–20 to the package structure, migration pipeline, CI workflow, and reminder scheduler.

3.7 Constraints and Assumptions

Finally, the requirements are bounded by explicit constraints and assumptions that shape what the system may assume of its environment:

- **Technology constraints.** The backend is Go with the Gin framework over PostgreSQL 16; code execution is delegated to Judge0 CE. These are fixed choices, not requirements to be re-derived, and are justified in the design and background chapters.
- **Deployment assumption.** A reachable Judge0 instance exists; the default points at the rate-limited `public ce.judge0.com`, and a real classroom deployment is assumed to run a self-hosted instance.
- **Scale assumption.** The concurrency target is classroom-sized; the system does not attempt horizontal scale-out, distributed grading queues, or multi-region deployment.
- **Trust assumption.** Students take exams on their own machines; APEX does not perform proctoring or lockdown, and treats the browser as untrusted — which is precisely why all grading and all answer state live on the server.

Table 3.5. Functional-requirement traceability to implementing components.

Req.	Component / package	Principal endpoint(s)
FR-T1	internal/class	POST/GET/PUT/DELETE /classes, DELETE /classes/:id/members/:userId
FR-T2	internal/problem, internal/testcase	POST /exams/:id/problems, POST /problems/:id/test-cases
FR-T3	internal/problem, internal/folder	POST /problems/bank, GET/POST/PUT/DELETE /folders
FR-T4	internal/exam	PUT /exams/:id (publish requires start_time)
FR-T5	internal/exam	POST /exams/:id/assign
FR-T6	internal/exam	POST/PUT /exams
FR-T7	internal/judge0	driven by POST /student/exams/:id/submit
FR-T8	internal/teacher, internal/submission	GET /teacher/grading/pending, PUT /submissions/:id/grade
FR-T9	internal/announcement, internal/notification	POST /classes/:id/announcements
FR-T10	internal/teacher, internal/class	GET /teacher/dashboard, GET /exams/:id/results, GET /classes/:id/stats
FR-T11	internal/exam, internal/class	exam preview (SPA route), grades_announced gating
FR-T12	internal/exam (reset.go)	destructive edit triggers reset_at
FR-S1	internal/student	POST /student/classes/join, DELETE /student/classes/:id
FR-S2	internal/exam	POST /student/exams/:id/start (idempotent)
FR-S3	internal/exam	PUT /student/exams/:id/autosave
FR-S4	internal/exam	shuffle_questions on the exam
FR-S5	internal/execute, internal/submission	POST /execute, POST /submissions/run
FR-S6	internal/exam, internal/judge0	POST /student/exams/:id/submit
FR-S7	internal/submission, internal/student	GET /submissions/:id, GET /attempts/mine
FR-S8	internal/notification, internal/profile	GET /notifications, GET/PUT /profile
FR-P1	internal/auth, internal/middleware	POST /auth/login, /auth/signup, /auth/google
FR-P2	internal/auth, internal/email	POST /auth/forgot-password, /auth/reset-password
FR-P3	internal/auth	DELETE /auth/account
FR-P4	internal/reminder, internal/email	background scheduler (1-min tick)
FR-P5	internal/judge0	maybeFinalizeAttempt + graded_notified
FR-P6	main.go	GET /healthz, --healthcheck
FR-P7	internal/database, cmd/migrate	migration passes / versioned SQL track

These constraints are not deficiencies to apologise for; stating them is what makes the functional and non-functional requirements above honest and achievable within the scope of a graduation project. The chapters that follow show how each requirement was designed (Chapter 3 onward), implemented, secured, and evaluated.

Chapter 4

System Architecture and Design

This chapter describes how APEX is put together. It starts from the outermost view — the running processes and the traffic that flows between them — and then works inward: the lifecycle a single HTTP request follows through the backend, the reasoning behind each technology choice, the shape of the public API, the physical layout of the source tree, and the design patterns that hold the whole thing together. The guiding principle throughout is deliberate simplicity: APEX is a graduation project intended to be self-hosted by a single instructor or a small department, so every architectural decision is weighed against the cost of operating it, not against the demands of hyperscale.

4.1 Architectural Overview

APEX is a conventional client–server web application built around a single-page application (SPA) front end and a stateless HTTP API back end that persists everything of value to a relational database. What distinguishes it from a textbook three-tier system is the deliberate refusal to multiply moving parts: there is exactly one server process to run, one database to back up, and one external dependency that actually executes untrusted code. Everything else is optional.

Figure 4.1 shows the full runtime and deployment topology. Six components appear, and it is worth being precise about which of them are first-class services and which are external dependencies, because the distinction drives both the security posture and the operational story.

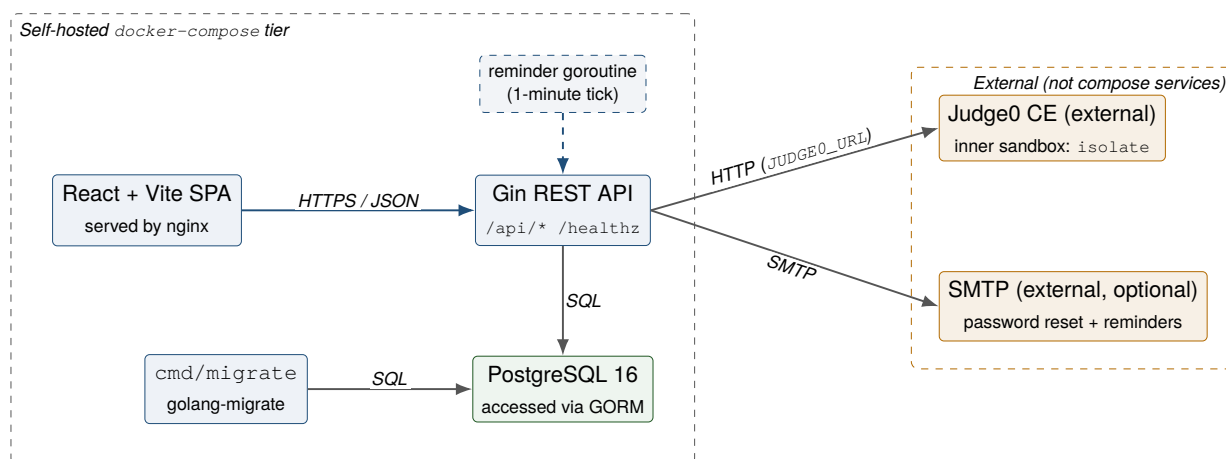


Figure 4.1. APEX runtime and deployment architecture.

- **React + Vite SPA.** The browser-side application, compiled to a static bundle and served by `nginx` in production. It holds no durable state of its own beyond a `JWT` in `localStorage`; every meaningful action is a `JSON` call to the API.
- **Gin REST API.** The single Go process (binary `apex`) that owns all business logic. It exposes routes

under `/api/*` plus an unauthenticated `/healthz` liveness endpoint. This process is the heart of the system and the subject of most of this chapter.

- **PostgreSQL 16.** The single source of durable truth, reached exclusively through the GORM object–relational mapper. No component other than the API and the migration tool ever touches it directly.
- **Judge0 CE (external).** The sandboxed code-execution service that compiles and runs student submissions inside the `isolate` jail. Crucially, Judge0 is *not* a `docker-compose` service; it is reached over HTTP at the URL named by `JUDGE0_URL` (defaulting to the public `https://ce.judge0.com`). Treating the single most security-sensitive component — the one that runs arbitrary code — as an external service reached through one URL is an intentional isolation boundary.
- **SMTP relay (external, optional).** Used for password-reset links and exam reminders. It degrades gracefully: when `SMTP_HOST` is empty, every message that would have been sent is logged to standard output instead, so the system remains fully functional without a mail provider configured.
- **cmd/migrate.** A separate one-shot binary, built from the same Go module, that owns schema evolution in production. It applies versioned SQL migrations and exits.

Two further behaviours live *inside* the API process rather than as separate services, and Fig. 4.1 draws them attached to it. The first is the **reminder goroutine**, a background loop that wakes on a one-minute ticker to dispatch exam reminders. The second, not drawn but equally important, is the fleet of **fire-and-forget grading goroutines** spawned whenever an exam is submitted. Both are examples of a recurring theme: work that a larger system would hand to a dedicated worker or cron daemon is folded into the single binary, because for a classroom-scale tool the operational saving of “one process to run and one process to reason about” outweighs the resilience a separate worker fleet would buy.

The deployment boundary in Fig. 4.1 is significant. The self-hosted tier — SPA, API, database, and the migration tool — is what a `docker-compose up` brings up on a single host. The external tier — Judge0 and SMTP — is reached over the network and is never bundled. This separation means that an operator can run the entire teaching platform on one modest machine while pointing at a shared or public Judge0 instance, and can add email only when they are ready to configure a relay.

4.2 The Four-Layer Request Lifecycle

Every request that reaches the backend passes through the same four conceptual layers, in the same order. This uniformity is what keeps a six-thousand-line Go codebase navigable: once the layering is understood, any endpoint can be read by asking which layer each line belongs to. Figure 4.2 traces the pipeline from an incoming HTTP request to a JSON response, including the two short-circuit paths where a request is rejected before it reaches the handler.

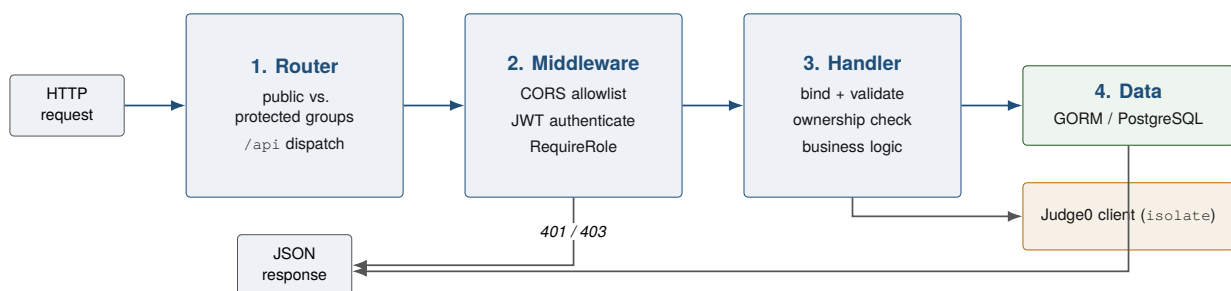


Figure 4.2. Four-layer request lifecycle through the Gin backend.

4.2.1 Layer 1 — Router and route groups

The transport layer is assembled in `main.go`. After the process initialises its configuration, database connection, and subsystems, it builds a Gin engine and installs global middleware:

```
1 r := gin.Default()
2 r.Use(middleware.CORS(cfg.AllowedOrigins))
3
4 r.GET("/healthz", func(c *gin.Context) {
5     c.JSON(http.StatusOK, gin.H{"status": "ok"})
6 })
7
8 public := r.Group("/api")
9 protected := r.Group("/api")
10 protected.Use(middleware.Auth(cfg.JWTSecret))
```

Listing 4.1. Router construction and route-group split in `main.go`.

The design pivots on the two route groups. Both share the `/api` prefix, but `public` carries no authentication middleware while `protected` applies `middleware.Auth` to *every* route registered on it. This single `.Use` call is the mechanism that separates “anyone may call this” (login, signup, password reset) from “you must present a valid token.” The `/healthz` endpoint is registered directly on the engine, outside both groups, so a load balancer or the container runtime can probe liveness without a token; it returns `{"status": "ok"}` and deliberately touches neither the database nor Judge0, making it a pure “is the process routing?” check.

4.2.2 Layer 2 — Middleware

Middleware runs before any handler and is where cross-cutting concerns are enforced. For a protected request the chain is, from outermost to innermost: the Gin `Logger` and `Recovery` middleware supplied by `gin.Default()` (the latter converts a handler panic into a 500 rather than crashing the process); the CORS filter; the JWT authentication gate; and, where a route or subgroup demands it, a role guard. Because rejection happens here, a malformed or unauthorised request is turned away with a 401 or 403 *before* it can reach business logic or the database — the left-hand short-circuit arrow in Fig. 4.2.

The CORS middleware is intentionally strict. It allows only the origins assembled at startup (`APP_URL` plus any comma-separated `ALLOWED_ORIGINS`), only the methods `GET`, `POST`, `PUT`, `DELETE`, and `OPTIONS`, and only the `Content-Type` and `Authorization` headers. It sets `AllowCredentials: false`, which is a considered decision: APEX carries its token in the `Authorization` header rather than in a cookie, so there are no ambient credentials for a cross-site request to abuse, and preflight results are cached for twelve hours to cut down on `OPTIONS` chatter.

4.2.3 Layer 3 — Handler

Once a request clears the middleware, the handler runs the actual work. Handlers across the codebase follow the same skeleton: read the verified identity from the Gin context (`c.GetUint("user_id")`), read any path parameters, load the target resource and check that the caller is entitled to it, bind and validate the JSON body with `c.ShouldBindJSON`, perform the database work through GORM, trigger any background work, and finally serialise a JSON response with an appropriate status code. Handlers never trust an identifier

taken from the request body when deciding ownership; they use only the `user_id` that the authentication middleware placed on the context.

4.2.4 Layer 4 — Data

The innermost layer is persistence. In the common case this means the shared `database.DB` handle — a `*gorm.DB` configured with a bounded connection pool (25 open connections, 10 idle, a five-minute idle timeout and a thirty-minute maximum lifetime) — which GORM translates into SQL against PostgreSQL. For coding submissions a second backing store appears at this layer: the Judge0 client, which sends code over HTTP to the sandbox. Figure 4.2 draws this as a branch off the handler, because a request that grades code touches both PostgreSQL and Judge0 before it can respond.

4.2.5 An honest note on the layering

It is worth being candid about what this layering is and is not. It is a pragmatic, package-level separation of transport, handler, service, and persistence concerns. It is *not* a ports-and-adapters or “clean” architecture: there is no repository interface abstracting the database, and handlers reach directly into the global `database.DB`. The service layer is simply the set of packages (`judge0`, `notification`, `email`, `reminder`) that handlers call into. For a single-binary classroom tool this trade favours readability and low ceremony over the testability and swappability that dependency inversion would provide, and the book states that plainly rather than dressing it up.

4.3 Three-Gate Authorization

Layering the request lifecycle is only half of the access-control story: on top of it APEX applies three independent, fail-closed authorization gates — authentication (`middleware.Auth`), a role check (`middleware.RequireRole`), and a resource-ownership check inside each handler. Because these gates belong conceptually to the request pipeline they are named here at an architectural level, but their full treatment — how each one fails closed and why a bug in one cannot open the others — lives in Chapter 8 and is depicted in Fig. 8.2.

4.4 Technology Stack and Rationale

Every stack choice in APEX was made to keep the operational surface small and the code readable by a five-person team. Table 4.1 summarises the selections; the subsections that follow justify the load-bearing ones.

4.4.1 Why Go and Gin

Go [11] was chosen for a property that shows up repeatedly in the deployment story of Chapter 10: it compiles the entire backend into a single, self-contained, statically linked binary. There is no interpreter to install, no runtime to match, and no dependency tree to reconstruct on the server — the artefact is copied and run. Go’s goroutines also make the concurrency that APEX needs (background grading, the reminder ticker) inexpensive to express: a background task is literally the keyword `go` in front of a function call.

Table 4.1. Principal technology choices and their roles.

Concern	Choice	Role in APEX
Backend language	Go 1.25.3	Compiles to one static binary; simple concurrency
HTTP framework	gin-gonic/gin 1.12	Routing, middleware, JSON binding
ORM	gorm.io/gorm 1.31	Struct-to-table mapping, query building
Database	PostgreSQL 16	Durable relational store
Sandbox	Judge0 CE	Isolated execution of untrusted code
Front end	React 18 + Vite 5 + TypeScript 5	Typed single-page application
Migrations	golang-migrate/migrate/v4	Versioned schema evolution in production

Gin [29] sits on top of Go’s standard `net/http` and supplies exactly the pieces a JSON API needs without imposing a heavyweight framework: fast radix-tree routing, a composable middleware chain, route groups, and convenience helpers for binding and emitting JSON. The route-group feature in particular is what makes the clean public-versus-protected split of Listing 4.1 possible in three lines. Gin does not dictate a project structure, which let the team impose the package layout of Section 4.6 rather than fight a framework’s conventions.

4.4.2 Why GORM

GORM [30] is the most widely used object–relational mapper in the Go ecosystem, and it earns its place here by removing the tedium of hand-writing SQL for fifteen entities while still allowing raw SQL where it matters. Handlers express intent in Go — `database.DB.Where("id = ? AND teacher_id = ?", examID, teacherID).First(&exam)` — and GORM generates the parameterised query, which as a side benefit closes the door on SQL injection for every query written this way. GORM’s `AutoMigrate` facility, which reflects over the model structs to create tables, accelerates development enormously. That same facility is also the origin of the project’s most instructive bug — the `is_draft` data-loss incident detailed in Chapter 5 — which is precisely why production disables `AutoMigrate` and hands schema authority to the versioned migration binary instead.

4.4.3 Why PostgreSQL

PostgreSQL [14] is the durable store. The data is inherently relational — users own classes, classes contain members, exams link to classes, problems belong to exams, submissions belong to attempts — and a relational database with real foreign keys and unique constraints models this faithfully. Three composite unique indexes in particular (`ClassMember(class_id, user_id)`, `ExamAttempt(user_id, exam_id)`, and `ExamClass(exam_id, class_id)`) enforce important invariants — a student joins a class at most once, and holds at most one attempt row per exam — at the database level, so that even a race between concurrent requests cannot violate them. PostgreSQL’s JSONB columns additionally give APEX a place to store semi-structured data (MCQ options, autosave payloads, announcement attachments) without abandoning the relational model for the fields that benefit from schema. Version 16 runs identically in a local Docker container and on managed providers such as Neon or Railway, reached through a single `DATABASE_URL`.

4.4.4 Why React, Vite, and TypeScript

The front end is a React [15] single-page application written in TypeScript [31] and built with Vite [16]. React’s component model suits an application that is essentially many stateful views — an exam builder, a Monaco-based coding view, a grading queue — reused across routes. TypeScript adds static types across roughly seventeen thousand lines of front-end code, catching whole classes of shape mismatches between the client and the JSON contracts described in Section 4.5 before they ever reach the browser. Vite provides a fast development server with hot module replacement and a production build that emits content-hashed, aggressively cacheable static assets. The result of the build is a folder of static files with no server-side rendering, which is why it can be served by a stock `nginx` container.

4.4.5 Why one binary that also ships `migrate` and `seed`

A single Go module produces three binaries: `apex` (the long-running API server and in-process scheduler), `migrate` (a one-shot schema tool), and `seed` (an on-demand demo-data loader). Shipping all three from one module, and baking all three into one Docker image, means there is exactly one build to trust and one set of dependencies to audit. The production container runs `apex` by default but the compose stack overrides its entrypoint to run `migrate` up for the one-shot migration service, so the same image serves two roles. This is the same economy of “one artefact, one thing to reason about” that motivated the choice of a compiled single-binary language in the first place, applied at the level of the deployment.

4.5 API Design

APEX exposes a RESTful HTTP API under the base path `/api`, following the resource-oriented style described by Fielding [32]. Nouns name resources (`/exams`, `/classes`, `/submissions`), HTTP verbs express intent (`GET` to read, `POST` to create, `PUT` to update, `DELETE` to remove), and every request and response body is JSON. Authentication is stateless: a bearer token issued at login is presented on each protected call, so the server keeps no session state and any instance can serve any request.

4.5.1 Modular route registration

Rather than declaring every route in one place, `main.go` delegates route registration to thirteen feature modules, each of which exposes a `RegisterRoutes(public, protected)` function and decides for itself which group and which role guard each of its endpoints needs:

```
1 auth.RegisterRoutes(public, protected)
2 class.RegisterRoutes(public, protected)
3 student.RegisterRoutes(public, protected)
4 exam.RegisterRoutes(public, protected)
5 problem.RegisterRoutes(public, protected)
6 testcase.RegisterRoutes(public, protected)
7 submission.RegisterRoutes(public, protected)
8 execute.RegisterRoutes(public, protected)
9 notification.RegisterRoutes(public, protected)
10 profile.RegisterRoutes(public, protected)
11 teacher.RegisterRoutes(public, protected)
12 announcement.RegisterRoutes(public, protected)
13 folder.RegisterRoutes(public, protected)
```

Listing 4.2. Modular route registration in `main.go`.

The only thing each module receives is the two router groups. There is no dependency-injection container and no service registry; the “wiring” is literally the passing of two `*gin.RouterGroup` values. This is a conscious choice in favour of explicitness — to learn what routes exist, one reads thirteen small `routes.go` files rather than tracing a configuration graph.

4.5.2 Public, protected, and role subgroups

Modules apply access control at three granularities, all built on the two top-level groups. The `auth` module registers its login, signup, and password endpoints on the `public` group and only its account-deletion route on `protected`. Feature modules that belong entirely to one role create a subgroup and apply a role guard once, as `exam` does:

```
1 func RegisterRoutes(public, protected *gin.RouterGroup) {
2     exams := protected.Group("/exams")
3     exams.Use(middleware.RequireRole("teacher"))
4     {
5         exams.POST("", CreateExam)
6         exams.GET("", GetExams)
7         exams.GET("/:id", GetExam)
8         exams.PUT("/:id", UpdateExam)
9         exams.DELETE("/:id", DeleteExam)
10        exams.POST("/:id/assign", AssignExam)
11        exams.POST("/:id/close", CloseExam)
12        exams.POST("/:id/reopen", ReopenExam)
13        exams.GET("/:id/results", GetExamResults)
14    }
15 }
```

Listing 4.3. A whole module gated to one role in `internal/exam/routes.go`.

Here `/exams` is created on the already-authenticated `protected` group and then further restricted so that *every* exam route requires the teacher role — two of the three authorization gates applied declaratively before any handler runs. The `student` module mirrors this with `RequireRole("student")`, and notably reuses the `exam` package’s attempt handlers, mounting `exam.StartAttempt`, `exam.AutosaveAttempt`, and `exam.SubmitAttempt` under `/student/exams/:id/...`: the `student` module owns those URLs while the `exam` module owns the logic, a clean reuse that avoids duplicating attempt code.

The third granularity is a per-route guard, used where endpoints in a module serve mixed audiences. The `submission` module opens `POST /run` and `GET /:id` to any authenticated user — because students run and read their own submissions and teachers read submissions on exams they own, with ownership resolved inside the handler — but pins the grade endpoint to teachers inline:

```
1 subs := protected.Group("/submissions")
2 {
3     subs.POST("/run", RunSolution)
4     subs.GET("/:id", GetSubmission)
```



```
5     subs.PUT("/:id/grade", middleware.RequireRole("teacher"), GradeSubmission)
6 }
```

Listing 4.4. A per-route role guard in `internal/submission/routes.go`.

Passing `middleware.RequireRole("teacher")` as an argument to a single route shows that Gin allows role guards to be applied group-wide or per-endpoint, and APEX uses whichever granularity fits the module.

4.5.3 JSON contracts and status codes

Request and response bodies are plain JSON objects, and the API is disciplined about status codes because the SPA relies on them. A successful read returns 200 with the resource; a successful create returns the created row. Validation failures return 400, an absent bearer token or invalid one returns 401, a role or ownership violation returns 403, and a missing resource returns 404. Semantically meaningful conflicts get their own codes — submitting an exam attempt that is already submitted returns 409 Conflict rather than a generic error, so the client can react precisely. A subtle but deliberate refinement is that owner-scoped queries return 404 rather than 403 when a teacher references another teacher’s resource: because the query filters by owner (`WHERE id = ? AND teacher_id = ?`), the row simply “does not exist” for the caller, which avoids confirming that the identifier is real. Two further contract-level choices protect users: login always returns the generic “invalid email or password” regardless of which field was wrong, and `POST /auth/forgot-password` always returns 200 — both to avoid leaking whether an account exists.

4.6 Package Layout and Separation of Concerns

The backend source lives under `internal/`, Go’s convention for packages that may not be imported outside the module. Each package owns one slice of the domain and exposes a small surface — typically a `RegisterRoutes` function and a set of handlers. The organisation is by *feature* rather than by technical layer: there is no top-level `controllers/`, `services/`, and `models/` trichotomy, but rather a package per resource that internally spans the transport and handler layers.

- **Cross-cutting infrastructure:** `config` (environment loading and connection-string construction), `database` (the GORM handle, the connection pool, and both migration tracks), `middleware` (CORS, JWT authentication, role gating), and `models` (the fifteen GORM structs that are the single source of truth for the schema).
- **Feature modules:** `auth`, `class`, `student`, `teacher`, `exam`, `problem`, `testcase`, `submission`, `execute`, `folder`, `announcement`, `notification`, and `profile`. Each maps to a coherent chunk of the teacher or student workflow.
- **Domain services:** `judge0` (the sandbox client, the coding/MCQ/written graders, and the attempt aggregator), `email` (an SMTP wrapper with a stdout fallback), and `reminder` (the one-minute background ticker). These hold logic that handlers call into but that is not itself an HTTP endpoint.

Alongside `internal/` sit two command binaries under `cmd/` — `cmd/migrate` and `cmd/seed` — and the root `main.go` that wires everything together. The separation of concerns is therefore visible in the directory tree itself: a reader looking for how exams are assigned opens `internal/exam`, and a reader looking for how code is graded opens `internal/judge0`, without needing to understand the rest of the system.

4.7 Design Patterns and Principles

Several established patterns and principles recur through the architecture, and naming them makes the design intent explicit.

Middleware / chain of responsibility. The request pipeline of Fig. 4.2 is a chain-of-responsibility [33]: each middleware either handles the request (by aborting it) or passes it to the next link with `c.Next()`. This is what lets authentication, role checking, and CORS be composed independently and reordered without touching handlers.

Twelve-factor configuration. APEX follows the twelve-factor [28] principle of strict separation of configuration from code. Every operational setting — database credentials, the JWT secret, the Judge0 URL, SMTP details — is read from the environment through the `config` package, never hard-coded, and a `.env` file is auto-loaded only as a development convenience. The one setting the process refuses to start without is `JWT_SECRET`: if it is empty, shorter than 32 characters, or the literal placeholder `dev-secret-change-in-prod`, the process calls `log.Fatal` and exits. This fail-fast validator makes “production booted with the development secret” impossible by construction, and it is the only fast-fail in the entire boot sequence — every other missing setting degrades gracefully.

Stateless services and idempotent operations. Because the API keeps no session state, it is horizontally trivial to reason about, and several operations are deliberately idempotent so that a retry or a refresh cannot cause harm. Starting an exam attempt, for instance, looks up the attempt for the (student, exam) pair and returns the existing one if present, backed by the unique index on `(user_id, exam_id)` so that even a concurrent race cannot create duplicates. The reminder scheduler records “already sent” in dedicated timestamp columns rather than in memory, which makes it idempotent across process restarts. These choices reflect the data-intensive-application discipline [34] of pushing correctness guarantees down to the database wherever a purely in-code check would be racy.

Graceful degradation. Finally, the architecture is built so that optional dependencies stay optional. Missing SMTP configuration logs email bodies to stdout; a missing `GOOGLE_CLIENT_ID` makes the Google sign-in endpoint answer 503 while every other authentication path keeps working. The system is always runnable with the minimum of one database and one secret, and each external integration can be switched on independently as an operator is ready for it.

Taken together, these decisions describe an architecture that is intentionally modest in its ambitions and precise in its execution. The remaining chapters build on this skeleton: Chapter 5 details the data model that PostgreSQL holds, Chapter 7 follows a submission through the grading engine sketched here, Chapter 8 audits the three authorization gates, and Chapter 10 shows how the single binary and its companions are packaged and deployed.

Chapter 5

Data Model and Persistence

Every exam that a teacher authors, every class a student joins, every line of code a student submits, and every verdict that Judge0 returns is, in the end, a row in a PostgreSQL 16 table [14]. This chapter is the map of that store. It presents the fifteen entities that make up the APEX schema, walks the columns and constraints that matter, explains the three composite unique indexes that carry the system’s correctness invariants, and then turns to how the schema changes over time — the migration pipeline and the one incident that taught the team why that pipeline exists at all.

The persistence layer is deliberately small. APEX is a single Go binary talking to a single database through a single object-relational mapper (GORM [30]); there is no sharding, no read replica, no separate analytics store. The design goal was a schema that a single engineer can hold in their head, on the reasoning that a schema you cannot trace by hand is a schema you will eventually corrupt. The rest of this chapter argues that this smallness is a feature: it is what makes the grading pipeline’s correctness auditable, and it is what let the team recover cleanly from a migration mistake that would have been catastrophic in a larger design.

5.1 The Entity Landscape

The canonical roster of entities is the argument list of GORM’s `AutoMigrate` call in `internal/database/database.go`. Fifteen Go structs are registered there, and each maps to exactly one table:

```
1 DB.AutoMigrate(  
2     &models.User{},  
3     &models.UserProfile{},  
4     &models.Class{},  
5     &models.ClassMember{},  
6     &models.Exam{},  
7     &models.ExamClass{},  
8     &models.Problem{},  
9     &models.TestCase{},  
10    &models.ExamAttempt{},  
11    &models.Submission{},  
12    &models.TestResult{},  
13    &models.Notification{},  
14    &models.Announcement{},  
15    &models.Folder{},  
16    &models.PasswordResetToken{},  
17 )
```

Listing 5.1. The fifteen registered models, in migration order (`internal/database/database.go`).

These fifteen tables cluster into four *aggregates* — a domain-driven-design term for a group of entities treated as one consistency unit with a single root that owns the rest [33], [34]. Table 5.1 lays out the grouping. The counts form a convenient mnemonic: identity has three tables, classes four, assessment seven, and one cross-cutting table stands alone, for a total of $3 + 4 + 7 + 1 = 15$.

Table 5.1. The four aggregates of the APEX data model.

Aggregate	Root	Members
Identity (3)	User	UserProfile (1:1), PasswordResetToken (1:N)
Classes (4)	Class	ClassMember (M:N with User), Announcement (1:N), Folder (teacher-owned)
Assessment (7)	Exam	ExamClass (M:N with Class), Problem (1:N) → TestCase (1:N); ExamAttempt (1:N) → Submission (1:N) → TestResult (1:N)
System (1)	—	Notification (belongs to a User, cross-cutting)

The assessment cluster is large enough that it is often useful to split it further into an *authoring* half (Exam, ExamClass, Problem, TestCase — the entities a teacher builds) and a *taking* half (ExamAttempt, Submission, TestResult — the entities a student produces). The taking half is the one governed by the system’s most important database constraint, and Section 5.5 gives it its own treatment. Figure 5.1 shows the complete entity-relationship diagram: the four aggregates, the foreign-key edges between them, and the join tables that model the two many-to-many relationships.

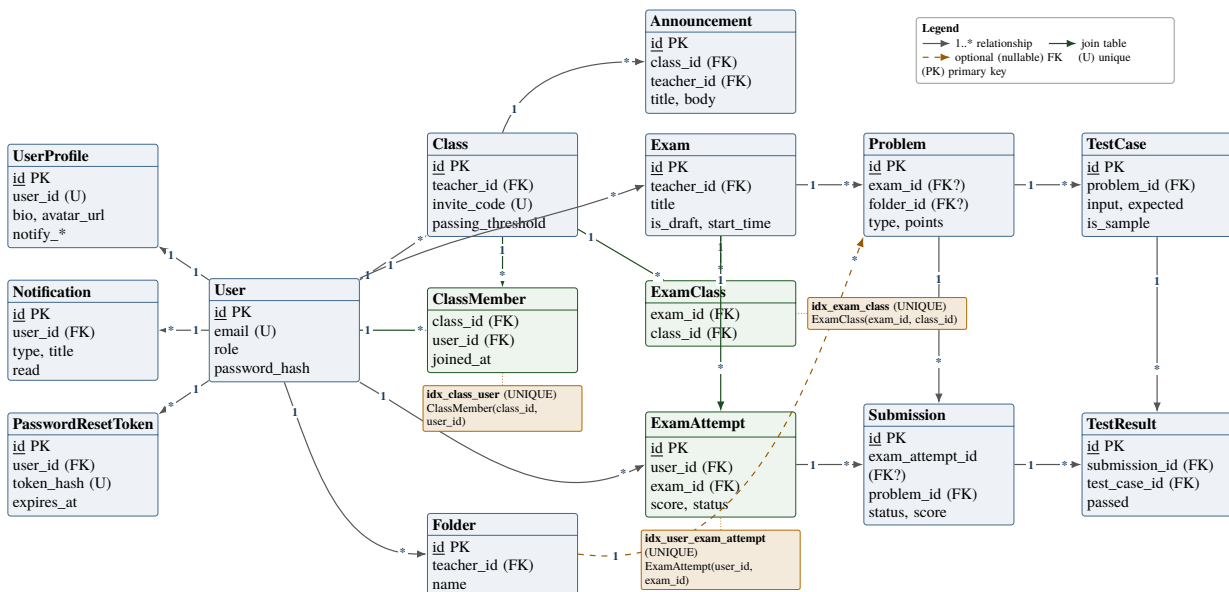


Figure 5.1. Entity-relationship diagram of the APEX data model.

5.2 Identity Tables

The identity aggregate stores who may log in and how. Its root is User:

```
1 type User struct {
2     ID          uint        `gorm:"primaryKey" json:"id"`
3     Email       string      `gorm:"size:255;uniqueIndex;not null" json:"email"`
```

```

4 PasswordHash string `gorm:"type:text;not null" json:"- "`
5 Role         string `gorm:"size:20;not null" json:"role"`
6 Name         string `gorm:"size:255;not null" json:"name"`
7 GoogleID     string `gorm:"size:64;index" json:"- "`
8 CreatedAt    time.Time `json:"created_at"`
9 }

```

Listing 5.2. The User model (internal/models/user.go).

Several conventions visible here recur across every table. The primary key is always a field named `ID` of Go type `uint` carrying `gorm:"primaryKey"`, which PostgreSQL materializes as a `BIGSERIAL` auto-incrementing column. String columns declare an explicit `size`: so they become bounded `VARCHAR(n)` rather than unbounded `text`; free-form fields such as `PasswordHash` opt into `type:text` instead. The `json:"-"` tag on `PasswordHash` and `GoogleID` instructs the marshaller never to serialize those fields into an API response, so a password hash cannot leak through a handler that carelessly returns a whole `User`. The `Role` column holds one of exactly two values, `teacher` or `student`; APEX has no administrator role.

`UserProfile` holds per-user preferences in a 1:1 relationship with `User`, enforced by `UserID` `gorm:"uniqueIndex"` — because the foreign key is unique, a user has at most one profile row. It carries the notification toggles (`notify_email`, `notify_push`, `notify_exam_reminders`, `notify_results`, `notify_exam_email`) that the reminder scheduler consults before fanning out, plus a small block of per-teacher creation defaults (`default_exam_draft`, `default_passing_threshold`, `block_announce_with_pending`). The distinction between these defaults and column defaults is central to the incident discussed in Section 5.8: `default_exam_draft` seeds a *new* exam’s draft state in application code at creation time, and is not the same thing as a DDL default backfilling existing rows.

`PasswordResetToken` completes the aggregate. It never stores the raw reset token; only its SHA-256 hash lives in `token_hash` (`size:64;uniqueIndex;not null`, hidden from JSON), so a database leak hands an attacker no usable reset links. The nullable `used_at *time.Time` makes the token single-use: a non-null value means “already spent,” and together with `expires_at` it forms a one-shot, time-limited credential.

5.3 Class Tables

The class aggregate models enrolment and classroom communication. A `Class` belongs to one teacher (`TeacherID`, with a `Teacher` `User` belongs-to association loaded on demand via `Preload`) and is joined through an eight-character `invite_code` (`size:8;uniqueIndex;not null`). The `grades_announced` boolean is the gate that releases results to students: until a teacher flips it on, graded scores stay hidden.

Enrolment is a many-to-many relationship between `User` and `Class`, and APEX models it with an explicit join *entity*, `ClassMember`, rather than GORM’s implicit `many2many` tag:

```

1 type ClassMember struct {
2     ID          uint    `gorm:"primaryKey" json:"id"`
3     ClassID     uint    `gorm:"not null;index;uniqueIndex:idx_class_user" json:"class_id"`
4     Class       Class   `gorm:"foreignKey:ClassID" json:"- "`
5     UserID      uint    `gorm:"not null;index;uniqueIndex:idx_class_user" json:"user_id"`
6     User        User    `gorm:"foreignKey:UserID" json:"user,omitempty"`
7     JoinedAt    time.Time `gorm:"autoCreateTime" json:"joined_at"`

```

```
8 }
```

Listing 5.3. The ClassMember join entity (internal/models/class.go).

The explicit join entity is chosen so the relationship can carry its own data — here a `joined_at` timestamp stamped automatically by `autoCreateTime` — and its own named constraint. That constraint is the first of the three composite unique indexes: `idx_class_user` appears on *both* `ClassID` and `UserID` under the same index name, which GORM compiles into a single unique index over the pair (`class_id`, `user_id`). Its effect is that a student physically cannot join the same class twice — the database rejects a duplicate pair, independent of any application-level check.

The aggregate is completed by `Announcement`, which belongs to a class and a teacher and stores an `attachments` list as a JSONB column (see Section 5.4.1); posting one fans out `Notification` rows to every enrolled student. `Folder` is a teacher-owned grouping for bank questions, referenced by `Problem.folder_id` with `ON DELETE SET NULL` semantics so that deleting a folder unfiles its questions rather than destroying them.

5.4 Assessment Tables and the Polymorphic Problem

The authoring half of the assessment aggregate begins with `Exam`, which owns many `Problem` rows and is assigned to classes through the `ExamClass` join entity. `ExamClass` carries the second composite unique index, `idx_exam_class` over (`exam_id`, `class_id`), forbidding the same exam from being assigned to the same class twice. Its scheduling columns (`start_time`, `end_time`, `reset_at`, and three reminder-dedup timestamps) are all `*time.Time` pointers, so they map to nullable `TIMESTAMPTZ` columns; a null `start_time` means “not scheduled yet,” and an exam cannot be published without one. The `reset_at` column is indexed because the take screen queries it on every load to invalidate stale cached sessions when a teacher destructively edits an exam.

The single most interesting table in the schema is `Problem`, which is both *polymorphic* and *dual-purpose*. It is polymorphic because one table holds all three question types — coding, MCQ, and written — discriminated by a `type` column (`size:20;not null;default:coding`). Rather than three normalized tables joined at query time, every variant-specific column lives on one wide row and is simply left null for the types that do not use it:

- **Coding** problems use `starter_code`, `time_limit_ms` (default 2000), and `memory_limit_kb` (default 262144, i.e. 256 MB), plus child `TestCase` rows.
- **MCQ** problems use `options` and `correct_option_ids` (both JSONB), `multiple_correct`, and `explanation`.
- **Written** problems use `max_word_count` (default 500), `rubric`, and `require_manual_grading`.

The trade-off is explicit and, at this scale, defensible: a wider table with more nullable columns is exchanged for a much simpler grading dispatch. The backend performs a single switch `Problem.Type` rather than joining three type-specific tables, and the exam’s “problem list” is one homogeneous query [33]. It is dual-purpose because `ExamID` is a nullable `*uint` and `IsBank` is a boolean: an ordinary problem has `ExamID` set and `is_bank=false`, while a reusable bank template has `ExamID = NULL` and `is_bank=true`, unbound to any exam and optionally filed under a `Folder`.

Each coding `Problem` owns many `TestCase` rows. The crucial column there is `is_sample`: sample cases are the ones surfaced to students through the live “Run Samples” feature, while hidden cases are

used only by the real auto-grader and are never serialized to the client. This split is the data side of a defense-in-depth boundary — hidden expected outputs never leave the server.

5.4.1 JSONB Columns

APEX uses PostgreSQL’s binary-JSON type, JSONB, in four places: `problems.options` and `problems.correct_option_ids` (MCQ option sets), `problems.tags` (default `[]`), `announcements.attachments`, and `exam_attempts.draft_answers` (discussed in Section 5.5). The common thread is that each holds a small, variable-length, client-shaped blob that the application reads and writes *whole* and never needs to query *inside* at the SQL level. Kleppmann observes that document-style storage is a good fit exactly when data has a natural tree shape that is loaded in its entirety and rarely joined against [34]; that is precisely the case here. The `draft_answers` payload in particular must round-trip the client’s exact structure, so storing a separate normalized table would add joins for no query benefit. The cost — that these fields are opaque to SQL predicates — is one APEX never pays, because it never filters on their interior.

5.5 The Attempt Aggregate

The taking half of the assessment cluster is the correctness core of the whole system. Its shape is a two-level ownership chain: `ExamAttempt` $\rightarrow$ (1:N) $\rightarrow$ `Submission` $\rightarrow$ (1:N) $\rightarrow$ `TestResult`. One sitting (`ExamAttempt`) collects one `Submission` per answered problem; each coding submission collects one `TestResult` per test case Judge0 ran against it.

```

1  type ExamAttempt struct {
2      ID          uint          `gorm:"primaryKey" json:"id"`
3      UserID      uint          `gorm:"not null;index;uniqueIndex:idx_user_exam_attempt" json:"
        ↳ user_id"`
4      ExamID      uint          `gorm:"not null;index;uniqueIndex:idx_user_exam_attempt" json:"
        ↳ exam_id"`
5      StartedAt   time.Time   `gorm:"autoCreateTime" json:"started_at"`
6      SubmittedAt *time.Time `json:"submitted_at,omitempty"`
7      Score       float64     `gorm:"not null;default:0" json:"score"`
8      Status      string      `gorm:"size:30;not null;default:in_progress" json:"status"`
9      GradedNotified bool       `gorm:"not null;default:false" json:"graded_notified"`
10     DraftAnswers *string    `gorm:"type:jsonb" json:"draft_answers,omitempty"`
11     DraftSavedAt *time.Time `json:"draft_saved_at,omitempty"`
12     Submissions []Submission `gorm:"foreignKey:ExamAttemptID" json:"submissions,omitempty"`
13 }

```

Listing 5.4. The ExamAttempt root and its unique index (internal/models/exam_attempt.go).

The load-bearing line is the third composite unique index, `idx_user_exam_attempt`, declared on both `UserID` and `ExamID`. It enforces **one attempt row per (student, exam) pair**, and that single constraint is what makes two otherwise-hard features tractable.

Resume-after-crash. Because the pair is unique, the database physically cannot hold two attempt rows for the same student and exam. The `StartAttempt` handler is therefore idempotent: it looks up the attempt by `(user_id, exam_id)`, returns the existing row if one is found, and creates a row only when

none exists. A student who refreshes the page, whose laptop dies, or who reopens the exam on a second device resolves every time to the *same* attempt. In-progress answers survive because the autosave endpoint writes the raw answer body into the `draft_answers` JSONB column of that same attempt, stamping `draft_saved_at`; on reconnect the take screen rehydrates the draft and recomputes remaining time from `started_at`. There is exactly one place the draft can live, which is what ties autosave to resume.

Safe re-scoring. The attempt’s `score` is stored on the row but *derived by recomputation*, never incrementally mutated. The finalisation routine iterates the exam’s *full* problem list — so a skipped question stays in the denominator at zero and a student who answered only the easy questions cannot appear at 100 % — sums earned points over total points, and writes back a percentage. Written answers in `pending_review` are excluded from both numerator and denominator until a teacher grades them, so an in-flight manual grade does not depress the visible total. Because there is exactly one attempt, and the score is a pure re-sum over a fixed set of children, the routine is safe to call any number of times: after each test result, after a manual grade, after a retry, the same inputs always yield the same number. If duplicate attempts were possible, this re-sum could group the wrong submissions or double-count; the unique index removes that entire failure class.

Idempotent state transitions. The attempt’s `status` moves `in_progress` → `submitted`. The `graded_notified` boolean is an idempotency guard so the “Exam Graded” notification fires exactly once even though finalisation may run many times; the finaliser sets it under the “all graded, nothing in `pending_review`” condition. This is the same philosophy as the reminder-dedup timestamp columns on `Exam` — idempotency expressed as a database column rather than in-memory state, so a process restart can never double-fire.

Below the attempt, `Submission` carries the student’s code, selected options (JSONB), or text answer, along with its own grading verdict (`status`, `passed_count`, `total_count`, `score`, and the maximum execution time and memory observed across its tests). Its `exam_attempt_id` is a nullable `*uint`: standalone playground and sample runs create no attempt, so the pointer tolerates that context while every real exam submission is grouped under its attempt. `TestResult`, the leaf, records one row per test case: whether it passed, the actual output, timing, memory, and the raw Judge0 status.

Listing 5.5 shows the `exam_attempts` table as `pg_dump` emits it, with the unique constraint spelled out in SQL.

```

1 CREATE TABLE exam_attempts (
2     id                BIGSERIAL PRIMARY KEY,
3     user_id           BIGINT NOT NULL,
4     exam_id           BIGINT NOT NULL,
5     started_at        TIMESTAMPTZ DEFAULT now(),
6     submitted_at      TIMESTAMPTZ,
7     score             DOUBLE PRECISION NOT NULL DEFAULT 0,
8     status            VARCHAR(30) NOT NULL DEFAULT 'in_progress',
9     graded_notified    BOOLEAN NOT NULL DEFAULT false,
10    draft_answers      JSONB,
11    draft_saved_at     TIMESTAMPTZ,
12    UNIQUE (user_id, exam_id)
13 );

```

Listing 5.5. The `exam_attempts` table, from the schema dump (Appendix B).

5.6 Indexes and Soft Constraints

APEX leans on the database for the invariants it cannot afford to get wrong. Three composite unique indexes, summarized in Table 5.2, each enforce a “no duplicate pair” rule at the storage layer, so the guarantee holds even under a race between two concurrent requests — the second insert simply fails with a unique-violation, and at most one row can ever exist.

Table 5.2. The three composite unique indexes and their invariants.

Index	Columns	Invariant enforced
idx_class_user	class_members- (class_id, user_id)	A student joins a given class at most once.
idx_user_exam_attempt	exam_attempts- (user_id, exam_id)	Exactly one attempt row per (student, exam) — the linchpin of resume and re-scoring.
idx_exam_class	exam_classes- (exam_id, class_id)	An exam is assigned to a given class at most once.

The reason `idx_user_exam_attempt` matters more than the other two is that it is not merely a hygiene constraint against duplicate rows; it is the foundation of a behavioural guarantee. Enforcing “one attempt per student per exam” in the database rather than in application code means the resume and re-scoring logic never has to reason about “which attempt is the real one.” There is provably only one. Had the team allowed multiple attempts and picked “the latest” in code, that correctness rule would have had to be threaded through every query touching attempts, and a refresh storm could have created two rows the finaliser would then aggregate incorrectly. Putting the rule in the schema collapses a whole category of race conditions into a single `UNIQUE` clause. The honest cost is that APEX supports exactly one attempt per exam with no native retake; adding retakes later would mean evolving the constraint to `(user_id, exam_id, attempt_number)` — a deliberate schema change, not an accident.

A fourth index deserves mention because it shows the limit of what a GORM struct tag can express. The `google_id` column is declared as a plain `index` in the struct, but true uniqueness is enforced by a *partial* unique index built in hand-written SQL:

```
1 CREATE UNIQUE INDEX IF NOT EXISTS idx_users_google_id
2   ON users(google_id)
3   WHERE google_id IS NOT NULL AND google_id <> '';
```

Listing 5.6. The partial unique index on `google_id` (internal/database/migrations.go).

Most users are password-only and have an empty `google_id`; a plain unique index would treat all those empty strings as colliding duplicates. The partial index applies uniqueness *only* to present, non-empty Google subjects, so password users never collide while Google-linked accounts stay uniquely bound. GORM’s tag vocabulary cannot express a conditional index, which is one concrete reason the project keeps a hand-written SQL layer at all.

Beyond the unique indexes, every hot-path foreign key (`user_id`, `exam_id`, `exam_attempt_id`, `submission_id`, `problem_id`, `reset_at`) carries a plain `index` tag so lookups stay logarithmic. What APEX largely does *not* do is declare database-level referential foreign-key *constraints* between most tables.

Relationships are expressed as GORM associations and enforced by application logic — “soft” constraints — rather than REFERENCES clauses with ON DELETE CASCADE. The sole hard foreign key added deliberately is `fk_problems_folder` with ON DELETE SET NULL, because folder deletion has a specific non-cascading semantics (unfile, do not destroy). The trade-off is honest: soft constraints keep AutoMigrate simple and account deletion tractable (the cascade is coded explicitly in a transaction rather than left to the database), at the price of relying on the application to keep references consistent. Account deletion is precisely where this shows — it is a single hand-written transactional cascade that clears profile, notifications, reset tokens, and role-scoped children in order before deleting the user.

5.7 The Migration Pipeline

A schema is never finished; columns get added, defaults change, data gets backfilled. APEX runs two parallel migration tracks, and which one mutates the schema is decided by a single environment flag, `SKIP_AUTOMIGRATE`, read in `internal/database/database.go`. Figure 5.2 shows the decision and the three-stage development path.

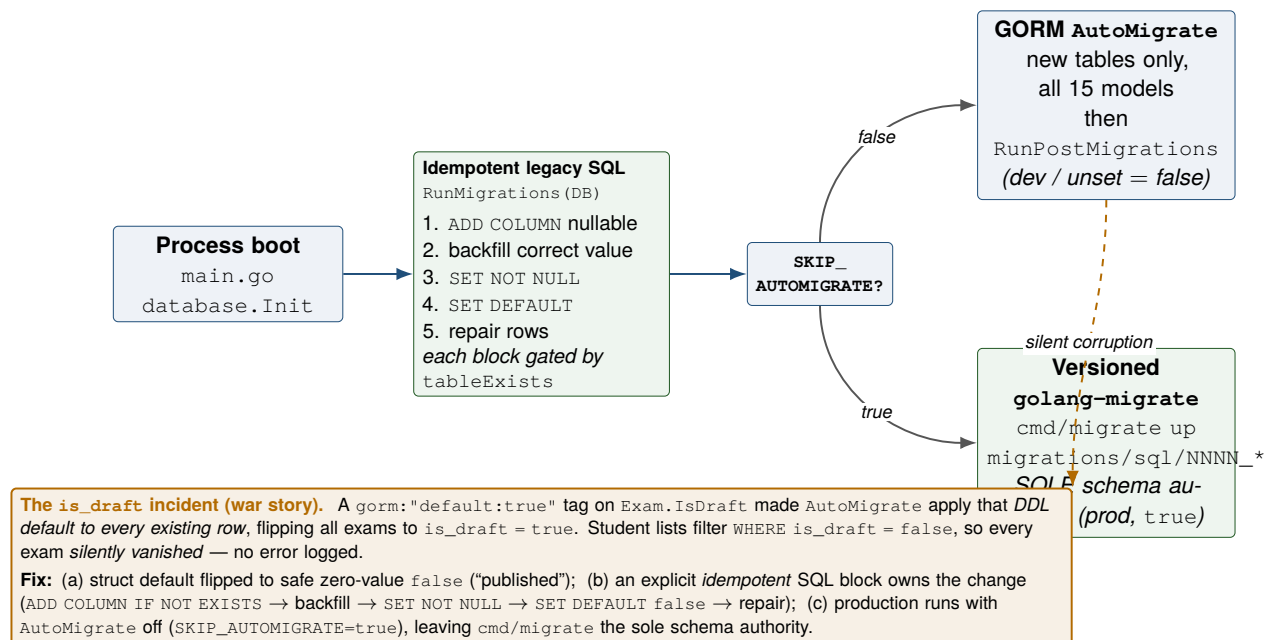


Figure 5.2. Schema-migration pipeline and the is_draft incident.

5.7.1 The Development Path: Three Stages, One Direction

When `SKIP_AUTOMIGRATE` is unset or false (the developer laptop), `Connect()` opens the pool — 25 max-open connections, 10 idle, a five-minute idle lifetime and a thirty-minute maximum lifetime — and then runs three passes in a fixed order.

Stage 1 — RunMigrations(DB), hand-written idempotent SQL, runs *before* AutoMigrate. This is where all the work that AutoMigrate cannot do safely lives: data backfills, nullability changes, default changes, the partial Google index, dropping legacy tables, and the historical score re-aggregation. Every block is gated on the target table already existing, via a helper:

```
1 func tableExists(db *gorm.DB, name string) bool {
```

```
2   var n int64
3   db.Raw("SELECT 1 FROM information_schema.tables "+
4       "WHERE table_schema = current_schema() AND table_name = ?", name).Scan(&n)
5   return n > 0
6 }
```

Listing 5.7. The table-existence gate (`internal/database/migrations.go`).

On a first boot against an empty database, every gate is false, so every block is skipped and Stage 2 creates the tables for real. On the second boot the gates are true and the SQL replays harmlessly, because every statement is written with `IF NOT EXISTS` / `IF EXISTS` guards. Idempotency here is deliberate: an operator restarting a misbehaving process at three in the morning should never have to reason about migration ordering. As the team’s own lesson puts it, idempotency is cheaper than coordination.

Stage 2 — `AutoMigrate(...)`, GORM’s automatic schema sync, creates only brand-new tables, columns, and indexes for the fifteen registered models. It does the easy, safe part; the dangerous part already happened in Stage 1.

Stage 3 — `RunPostMigrations(DB)` runs *after* `AutoMigrate` because it needs a table that `AutoMigrate` just created. It sweeps orphaned `folder_id` references to null, then adds the `fk_problems_folder` foreign key referencing folders. The ordering is forced by dependency: you cannot add a foreign key to a table that does not yet exist, and you cannot enforce the constraint while dangling rows violate it, so the orphan sweep precedes the `ADD CONSTRAINT`. The whole pipeline moves in one direction — forward only.

5.7.2 The Production Path: A Versioned Binary Owns the Schema

When `SKIP_AUTOMIGRATE=true`, `Connect()` takes an entirely different branch: it calls `RunSQLMigrations(cfg)` and returns, *never* touching `RunMigrations`, `AutoMigrate`, or `RunPostMigrations`. `RunSQLMigrations` drives the `golang-migrate` library [35] over the numbered, paired `NNNN_name.up.sql` / `.down.sql` files in `internal/database/migrations/sql/` (directory overridable via `MIGRATIONS_DIR`). It handles `ErrNoChange` and `ErrNilVersion` gracefully and logs the current version and dirty flag. The same logic is exposed as a standalone `cmd/migrate` CLI supporting `up`, `down`, `version`, and `force <n>`, which the production Docker stack runs as a one-shot `migrate` service before the backend starts.

Today the versioned track contains exactly one pair, the baseline `0001_init`, which is an intentional no-op (`SELECT 1;`) whose sole purpose is to mark the `AutoMigrated` schema as version 1 so that future `0002+` files can layer explicit, reviewed changes on top. The reason production skips `AutoMigrate` entirely is a matter of principle earned through experience: `AutoMigrate`’s schema *inference* is a convenience on a laptop, but a production database wants deterministic, version-controlled, reversible DDL that a human has reviewed — not an ORM silently reshaping a populated table [28]. That silent reshaping is exactly what caused the incident described next.

5.8 Case Study: The `is_draft` Incident

The clearest justification for the entire migration discipline is a single bug the team shipped, caught, and turned into a written rule. It is worth telling in full because it is a textbook example of how an ORM can do *more* than the programmer intended.

The mechanism. Someone added the tag `gorm:"default:true"` to the `IsDraft` field on the `Exam` struct, intending it to mean “new exams start as drafts.” On the next boot, GORM AutoMigrate translated that tag into a DDL default and applied it to the *already-populated* `exams` table, flipping every existing exam row to `is_draft = true`. The trouble is that student-facing list queries filter `WHERE is_draft = false` — students should see only published exams. With every row now marked draft, that filter matched nothing. **Every exam silently vanished from every student’s view.** Nothing crashed, nothing was logged; the data simply became invisible. This is the definition of silent corruption, and it is the failure mode that makes AutoMigrate dangerous on populated tables: GORM adds the column and applies the default, but it has no way to know that the “correct” value for existing rows is the opposite of the new default [30].

The multi-part fix. The repair had three independent parts, each addressing a different layer.

First, the struct default was flipped to the safe zero-value:

```
1 IsDraft bool `gorm:"default:false" json:"is_draft"`
```

Listing 5.8. The corrected field: the safe zero-value is “published.”

Now the default that AutoMigrate might apply is `false` — “published” — so even a stray application is harmless. Keeping the tag is legitimate: it governs the INSERT-time default when the Go field is zero-valued, which is a safe, application-level use.

Second, and more importantly, the real correctness was moved into explicit, idempotent SQL in `RunMigrations`, following the canonical add-nullable → backfill → constrain pattern, plus a repair line for rows the bad migration had already corrupted:

```
1 -- Step 1: add the column WITHOUT forcing a default onto existing rows
2 ALTER TABLE exams ADD COLUMN IF NOT EXISTS is_draft BOOLEAN;
3 -- Step 2: backfill existing rows with the CORRECT value
4 UPDATE exams SET is_draft = false WHERE is_draft IS NULL;
5 -- Step 3: only now is it safe to constrain
6 ALTER TABLE exams ALTER COLUMN is_draft SET NOT NULL;
7 ALTER TABLE exams ALTER COLUMN is_draft SET DEFAULT false;
8 -- Repair: rows the bad migration flipped, but which clearly WERE published
9 UPDATE exams SET is_draft = false
10    WHERE is_draft = true AND start_time IS NOT NULL;
```

Listing 5.9. The idempotent fix that owns the `is_draft` column (`internal/database/migrations.go`).

The repair line uses `start_time IS NOT NULL` as a reliable signal that an exam was genuinely published, because a draft cannot be published without a start time. Adding the column nullable first is the whole point: no default is forced onto existing rows, so the backfill in Step 2 can set the *correct* value before the `NOT NULL` and `DEFAULT` constraints in Step 3.

Third, production was configured to run with `SKIP_AUTOMIGRATE=true`, making the versioned migrate binary the sole authority over the production schema. GORM can never again reshape a populated production table, because in production GORM never runs DDL at all.

The resulting rule. The incident produced `ORM_RULES.md`, a short file of hard rules at the repository root. Its first and central rule is: *never use `gorm:"default:X"` on existing tables*. GORM AutoMigrate only *adds* columns; it does not manage defaults safely on populated tables, so existing rows may receive

the default unexpectedly. The `gorm:"default"` tag is reserved for application-level INSERT defaults only, never for DDL backfill. The supporting rules follow directly: audit every query when adding a filter column (the `WHERE is_draft = false` clause was the amplifier that turned a bad default into an outage); define the zero-value meaning of every new boolean explicitly (what does `false` mean, what should existing rows be); write existing-table changes as explicit idempotent SQL run *before* AutoMigrate; and verify row counts after any migration before touching application code. The fix is, in the end, exactly the shape of those rules — and the three-stage pipeline of Section 5.7 is their operational form.

5.9 Design Trade-offs and Honest Gaps

The data layer's choices were made in favour of smallness and provable correctness over normalization purity and speculative flexibility, and it is worth naming the costs honestly.

- **The polymorphic problems table is wide.** with columns that are always null for two of the three question types. This is the price of a one-line grading dispatch; a table-per-type design would be cleaner on paper but would trade a homogeneous problem-list query for three-way joins.
- **One attempt per exam, no retakes.** The unique index that makes resume and re-scoring tractable also fixes the model at a single attempt. Retakes are out of scope for this version, and the constraint makes adding them a deliberate schema evolution rather than an accident.
- **Two migration tracks coexist.** The legacy Go-SQL `RunMigrations` still owns the baseline locally, while the versioned `golang-migrate` track is the production future bridged by the no-op `0001_init`. This is transitional and documented, but it does mean two places to look when adding a change.
- **Soft constraints over hard foreign keys.** Most relationships are enforced by application logic, not database `REFERENCES`. This keeps AutoMigrate and account-deletion cascades simple at the cost of relying on the application to keep references consistent.
- **Single-instance assumptions.** The idempotent dedup columns make restarts safe, but they are not a substitute for a durable work queue; the fire-and-forget grading path and the reminder scheduler both assume a single backend process.

The through-line of this chapter is that APEX treats the database not as a passive bucket but as the place where the hardest invariants are enforced. The unique `(user_id, exam_id)` index is the single fact the entire grading-correctness story rests on; the migration pipeline exists so that the schema can evolve without ever again silently violating an invariant like it did the day every exam vanished. Small, auditable, and disciplined about change — that is the whole design.

Chapter 6

Implementation

The previous chapter fixed the shape of the system: four request layers, three authorization gates, fifteen entities, and a single Gin process fronting PostgreSQL and an external Judge0 sandbox. This chapter descends from that blueprint to the code that realizes it. It follows the two halves of APEX in turn — the Go backend and the React frontend — and traces the flows that carry the most engineering weight: the teacher’s exam builder, the timed exam-taking session with server-side autosave and resume, the exam lifecycle and attempt state machine, the notification fan-out, and the in-process reminder scheduler. The auto-grading pipeline is deliberately *not* covered here; it is substantial enough to warrant its own treatment in Chapter 7, and this chapter references it only where a flow hands off to it.

The figures throughout are drawn from the real code paths. Wherever a number, identifier, route, or column name appears, it is quoted verbatim from the repository rather than paraphrased.

6.1 Backend Handler Anatomy

Every backend package that owns an HTTP surface follows the same internal shape. A package such as `internal/exam` exposes a `RegisterRoutes` function that mounts its endpoints onto the shared `public` and `protected` route groups, and a set of handler functions that Gin invokes per request. The route file is deliberately thin — it is the single place where a reader can see the whole feature’s URL surface and its role gate at a glance:

```
1 func RegisterRoutes(public, protected *gin.RouterGroup) {
2     exams := protected.Group("/exams")
3     exams.Use(middleware.RequireRole("teacher"))
4     {
5         exams.POST("", CreateExam)
6         exams.GET("", GetExams)
7         exams.GET("/:id", GetExam)
8         exams.PUT("/:id", UpdateExam)
9         exams.DELETE("/:id", DeleteExam)
10        exams.POST("/:id/assign", AssignExam)
11        exams.POST("/:id/close", CloseExam)
12        exams.POST("/:id/reopen", ReopenExam)
13        exams.GET("/:id/results", GetExamResults)
14    }
15 }
```

Listing 6.1. The exam feature’s route registration — one place, one role gate.

The group is already on the authenticated `protected` chain, so `middleware.RequireRole("teacher")` is the only additional gate the package declares. Gin [29] is the HTTP framework throughout; its router

group mechanism is what lets the two cross-cutting concerns — authentication and role — be expressed once and inherited by every route below.

Inside the handlers, a consistent four-step spine recurs, mapping directly onto the request lifecycle established in the design chapter:

1. **Bind and validate.** The handler binds the JSON body into a request struct with `c.ShouldBindJSON`, relying on Gin's `binding:"required"` tags for the mandatory fields and returning 400 with a terse, non-leaking message otherwise.
2. **Identify the caller.** `c.GetUint("user_id")` reads the subject that the auth middleware placed on the context; the role is read the same way when a handler serves both teacher and student.
3. **Enforce resource ownership.** The handler re-queries the target row scoped to the caller — almost always as `Where("id = ? AND teacher_id = ?", id, teacherID)` — so that a missing row and a row owned by somebody else are indistinguishable and both surface as 404. This is the third authorization gate, and it lives in the handler because only the handler knows what “owns” means for its resource.
4. **Mutate, transactionally when needed.** Single-row writes go straight through GORM [30]; multi-table writes are wrapped in `database.DB.Transaction(func(tx *gorm.DB) error { ... })` so a partial failure rolls back cleanly.

Two handlers show the pattern at its extremes. `GradeSubmission` (manual grading of a written answer) is a small handler whose entire body is an ownership proof: it loads the submission, loads the exam it belongs to, and refuses with 403 unless `exam.TeacherID` equals the caller. Only then does it write the score and re-run attempt finalization:

```

1 // Verify the teacher owns the exam this submission belongs to.
2 var exam models.Exam
3 if err := database.DB.Select("id", "teacher_id").First(&exam, sub.ExamID).Error; err != nil
4     ↪ {
5     c.JSON(http.StatusNotFound, gin.H{"error": "exam not found"})
6     return
7 }
8 if exam.TeacherID != teacherID {
9     c.JSON(http.StatusForbidden, gin.H{"error": "not authorized to grade this submission"})
10    return
11 }
12 sub.Score = req.Score
13 sub.Status = req.Status
14 // ... persist, then hand off to the grading aggregator:
15 if sub.ExamAttemptID != nil {
16     judge0.FinalizeAttempt(*sub.ExamAttemptID)
17 }

```

Listing 6.2. Ownership before mutation: a teacher may grade only submissions on exams they own.

At the other end, `DeleteExam` is the largest transactional handler. An exam is the root of a small object graph — problems, test cases, attempts, submissions, test results, and class assignments — and PostgreSQL foreign keys are only partially declared, so the handler deletes children explicitly in dependency order inside one transaction: test results (selected through their parent submissions), test cases, submissions, attempts, problems, exam–class links, and finally the exam row. If any step errors, the whole delete rolls back and the client receives 500 with the underlying message; the half-deleted state never becomes visible.

A subtler variant of ownership appears in `AssignExam`, which links an exam to one or more classes. A naive implementation would trust the `class_ids` array from the request body. Instead the handler counts how many of those IDs actually belong to the calling teacher and rejects the whole request with 403 unless every one checks out:

```

1  var ownedCount int64
2  database.DB.Model(&models.Class{}).
3      Where("id IN ? AND teacher_id = ?", req.ClassIDs, teacherID).
4      Count(&ownedCount)
5  if int(ownedCount) != len(req.ClassIDs) {
6      c.JSON(http.StatusForbidden, gin.H{"error": "one or more classes do not belong to you"
7          ↪ })
8      return
9  }

```

Listing 6.3. Batch ownership check in `AssignExam` — all class IDs must belong to the caller.

This closes an insecure-direct-object-reference class of bug at the point where it would otherwise open: a teacher cannot push their exam into a colleague’s class by guessing a class ID.

6.2 The Exam Builder

The exam builder (`frontend/src/features/exams/pages/ExamBuilder.tsx`) is the most stateful screen in the product and the one that exercises the most backend endpoints in a single user action. It is a client-orchestrated composition: rather than a single “save exam” call, the builder drives the exam, problem, test-case, and assignment endpoints in sequence, treating the backend as a set of primitives and owning the orchestration itself.

The screen holds the entire draft exam in React state — title, description, duration, passing score, the shuffle and show-results toggles, the schedule (start and close date/time), the assigned course, the draft flag, and an ordered array of `Question` objects. New questions are created locally by `createQuestion(type)` and only materialize on the server at save time. The builder also imports from the teacher’s shared question bank: bank problems are filtered client-side by a search string and a type filter, selected, and mapped into the same `Question` shape via `bankProblemToQuestion`.

Validation runs before any network call. `handleSaveClick` refuses to proceed unless there is a non-empty title, a valid assigned course, at least one question, and — critically — every MCQ has at least one correct option selected. The last check is the frontend mirror of a backend guarantee from the grading chapter: an MCQ with no configured answer can only ever land in `pending_review`, so catching it at authoring time spares the teacher a queue of un-gradable questions.

When validation passes, `handleSave` executes a three-phase orchestration:

1. **Upsert the exam.** On create it POSTs to `/exams`; on edit it PUTs to `/exams/:id` and then deletes every existing problem (a deliberately simple “replace” strategy — the builder re-creates the full problem set rather than diffing it).
2. **Create each problem.** For every question it POSTs to `/exams/:id/problems`, projecting the union-typed `Question` down to the flat problem payload the backend expects. MCQ questions carry JSON-encoded options and `correct_option_ids`; written questions carry `max_word_count`, `rubric`, and `require_manual_grading`; coding questions carry `starter_code`, `time_limit_ms`,

and `memory_limit_kb`. For coding problems it then `POSTs` each non-empty test case to `/problems/:id/test-cases`, preserving order via an explicit `order_index`.

3. **Assign to the class.** Finally it `POSTs` to `/exams/:id/assign` with the single selected class ID.

The schedule is assembled from separate date and time-of-day inputs into an ISO timestamp; if the teacher sets a start but no close time, the builder derives the close as start plus the exam’s duration. The draft flag defaults to the teacher’s stored preference (`default_exam_draft`, itself defaulting to `true`), so exams are born unpublished unless the teacher opts otherwise. Because `shuffle_questions` is persisted on the exam row, it travels with the exam through create, edit, and re-hydration on the edit screen; the canonical stored order remains `order_index`, which every read path preloads in ascending order.

One further subtlety guards data integrity. If the teacher edits an exam that students have already attempted (`attempt_count > 0`), the builder does not silently save. It opens a confirmation dialog, because a structural edit is destructive: it triggers the server’s reset mechanism, which wipes attempts, submissions, and test results and stamps `exams.reset_at`. Surfacing that consequence at the moment of editing is a small piece of honesty that prevents a teacher from erasing a class’s work by accident.

6.3 Exam Lifecycle and Attempt State Machine

Two related state machines govern an exam’s life, shown together in Fig. 6.1. The upper machine is the *exam* lifecycle as seen by the teacher; the lower one is the *attempt* state machine as experienced by one student’s sitting.

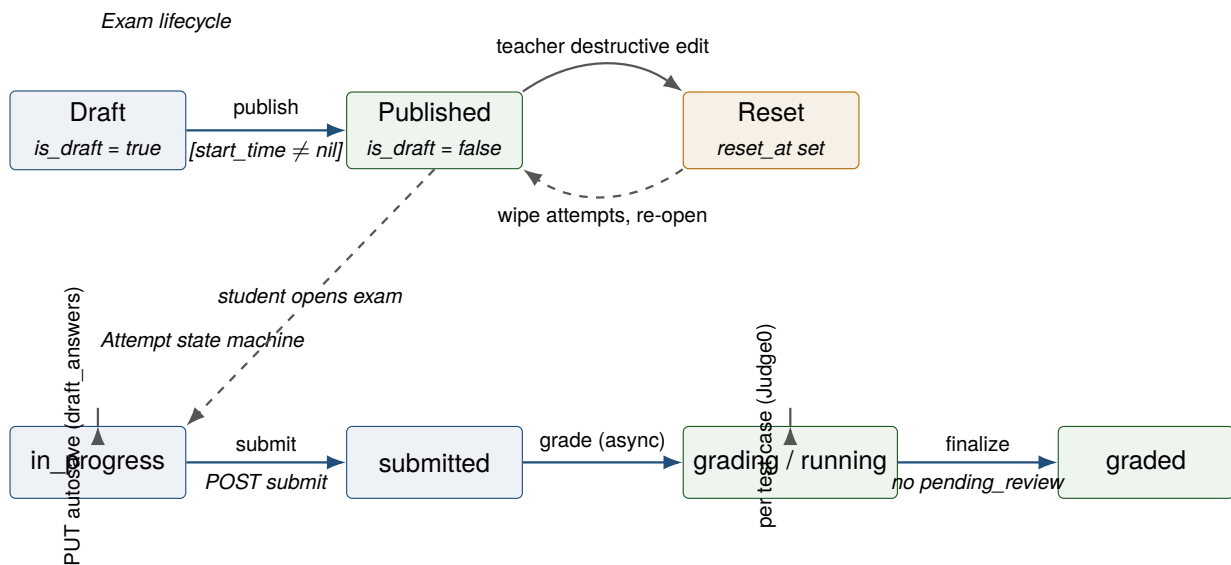


Figure 6.1. Exam lifecycle and attempt state machine.

An exam is created as a **draft** (`is_draft = true`). Publishing is the transition `is_draft = false`, and `UpdateExam` refuses it unless the exam has a `start_time`: the handler computes whether the exam “will have a start” after the update and returns 400 “set a start time before publishing” otherwise. This guard exists because an earlier version silently defaulted an unset start to “now,” which surprised teachers by opening an exam the instant they clicked Publish. The derived `status` a client sees — draft, upcoming, active, or completed — is computed by `computeExamStatus` purely from `is_draft`, `start_time`, `end_time`, and the current time, never stored, so it can never drift out of sync with the schedule.

Two teacher actions bend the timeline after publication. `CloseExam` stamps `end_time` to now, ending the window immediately. `ReopenExam` extends `end_time` by a requested number of minutes and, if the exam had not yet opened, pulls `start_time` forward to now so the window is genuinely live. The most consequential transition is the dashed loop in the figure: a destructive edit moves the exam through the **reset** state — `reset_at` is stamped, attempts and their descendants are wiped in a transaction, and class grade announcements are rolled back — before returning to a clean published state.

The attempt machine is where most of the runtime action lives. An `ExamAttempt` row is unique per `(user_id, exam_id)` via `idx_user_exam_attempt`, so a student has at most one attempt per exam. `StartAttempt` is written to be idempotent: it looks up the existing attempt and only creates one — with `status = "in_progress"` — when none exists, after checking enrollment, assignment, publication, and that the current time falls inside the exam window. Repeated “start” calls, whether from a double-click or a cross-device resume, converge on the same row rather than erroring or forking.

From `in_progress` the attempt has one forward transition, **submit**, plus a self-loop for autosave (discussed next). `SubmitAttempt` is guarded against replay: if the attempt is already submitted it returns 409 “attempt already submitted”. On a valid submit it creates one `Submission` per answered question, dispatches each to the grading engine by type, flips the attempt to submitted with a `submitted_at` stamp, and — notably — clears the autosave draft by setting `draft_answers` and `draft_saved_at` back to `NULL` in the same update. The remaining transitions, `grading/running` to `graded`, are owned entirely by the aggregation logic of Chapter 7 and are shown here only to close the loop.

6.4 The Timed Exam-Taking Flow

`ExamTaking.tsx` is the single largest component in the frontend and the one with the most demanding correctness requirements: it must run a countdown that cannot be gamed by a refresh, survive a browser crash or a device switch without losing answers, and hand a clean payload to the server exactly once. It achieves this with a two-tier persistence strategy — `localStorage` as an instant offline backup, and a server-side draft as the authoritative, cross-device source of truth.

6.4.1 Server-Side Autosave

The authoritative half of persistence is the `PUT /student/exams/:id/autosave` endpoint, backed by two columns on the attempt row: `draft_answers` (a JSONB payload) and `draft_saved_at` (a timestamp). The server treats the draft as *opaque*: the client serializes whatever shape its UI needs — the answers array, the current question index, the visited-question set, and the session start time — and the handler stores the raw bytes verbatim, so the payload round-trips unchanged and the server never needs to understand the client’s model. The handler’s only obligations are to locate the caller’s active attempt, refuse once it has been submitted, and validate that the body is well-formed JSON:

```

1 if attempt.Status == "submitted" {
2     c.JSON(http.StatusConflict, gin.H{"error": "attempt already submitted"})
3     return
4 }
5 // Read raw body so any client-shape JSON round-trips unchanged.
6 body, err := io.ReadAll(c.Request.Body)
7 if err != nil || len(body) == 0 {

```

```

8     c.JSON(http.StatusBadRequest, gin.H{"error": "invalid body"})
9     return
10 }
11 if !json.Valid(body) {
12     c.JSON(http.StatusBadRequest, gin.H{"error": "body must be valid JSON"})
13     return
14 }
15 now := time.Now()
16 database.DB.Model(&attempt).Updates(map[string]any{
17     "draft_answers": string(body),
18     "draft_saved_at": now,
19 })

```

Listing 6.4. AutosaveAttempt persists an opaque JSON draft on the active attempt.

On the client the autosave is debounced. A `useEffect` watching the answers, current index, and visited set schedules a `setTimeout` of 1500 ms; each change resets the timer, so a burst of typing produces one write rather than dozens. The call is deliberately best-effort — its `.catch` swallows failures — because `localStorage` already covers the offline case, and a transient network error during an exam must never interrupt the student. A separate, undebounced effect writes the same snapshot to `localStorage` on every change, so the local copy is always at least as fresh as the last render.

6.4.2 Resume and Reconciliation

Resume is where the two tiers meet, and the reconciliation logic is the most carefully written part of the component. On load, once the questions are known, the component reads both snapshots — the `localStorage` session and the server's `draft_answers` from the attempt included in the `GET /student/exams/:id` response — and chooses the newer one by comparing timestamps (`draft_saved_at` for the server, the session's `startedAt` locally). The winning snapshot's answers are merged by `questionId` onto a freshly initialized answer array, so questions that did not exist when the draft was saved still get clean defaults. This is what lets a student begin an exam on a laptop, close it, and resume on a phone with every answer intact.

Two guards protect the resume path. First, the remaining time is recomputed from the stored `startedAt` against the exam duration — a snapshot is only restored if time actually remains, so a refresh cannot reset the clock. Second, if the exam's `reset_at` is newer than the snapshot's start time, the teacher has destructively edited the exam since the draft was written, and the stale cache is discarded outright rather than resurrecting answers to questions that no longer exist. The submit path closes the loop by removing the `localStorage` key on success, while the server-side draft is nulled by `SubmitAttempt` itself.

The countdown timer is a one-second `setInterval` decrementing `timeLeft`; when it reaches zero, a watching effect fires `doSubmit(true)`, an automatic submission indistinguishable from a manual one except for the toast it shows. `doSubmit` is idempotent via a `submitted/isSubmitting` pair of flags so that the auto-submit and a last-second manual click cannot both fire. The payload it builds skips unanswered questions entirely: only questions with a selected option, non-empty text, or non-empty code produce an entry in the `answers` array sent to `POST /student/exams/:id/submit`.

6.5 Question Editors and the Monaco Integration

Each question type has a dedicated editor component, dispatched by the builder from a discriminated union. `MCQEditor` manages an options list (capped at six), tracks which option IDs are correct, toggles single- versus multiple-correct, and supports an optional explanation and an inline image (uploaded as a base64 data URL, rejected above 2 MB). `WrittenEditor` is the simplest — a prompt, a maximum word count defaulting to 500, an optional rubric, and a `require_manual_grading` switch — reflecting that written answers always enter the manual queue. `CodingEditor` is the richest: it manages the problem statement, starter code across the supported languages (python, javascript, c, cpp), per-problem time and memory limits, and a list of test cases each flagged sample-or-hidden with its own point value. Sample test cases are the ones surfaced to students for live feedback; hidden ones run only during real grading.

The code surfaces on both sides — the coding editor for authors and the in-exam solution pane for students — are the same Monaco editor, integrated through `@monaco-editor/react` [12]. Monaco is the editor that powers VS Code, and embedding it gives students syntax highlighting, bracket matching, and familiar keybindings inside the browser with almost no configuration. In `ExamTaking` it is mounted with a minimal, exam-appropriate option set — minimap disabled, word wrap on, `automaticLayout` enabled so it reflows with the panel, and a theme bound to the app’s light/dark mode:

```

1 <Editor
2   height="100%"
3   language={(ans.language || "python3") === "python3" ? "python"
4     : (ans.language || "python3") === "cpp" ? "cpp"
5     : (ans.language || "python3")}
6   value={ans.code || (q.starterCode?.[ans.language || "python3"] || "")}
7   onChange={(v) => updateAnswer(currentIdx, { code: v || "" })}
8   theme={theme === "dark" ? "vs-dark" : "light"}
9   options={{
10     fontSize: 13,
11     minimap: { enabled: false },
12     scrollBeyondLastLine: false,
13     automaticLayout: true,
14     wordWrap: "on",
15   }}
16 />

```

Listing 6.5. Monaco embedded in the in-exam solution pane.

Beside the editor, students can run their code two ways. “Run Code” hits the `/execute` playground with custom stdin and streams back stdout, stderr, compile output, and time/memory figures — a scratchpad that persists no submission. “Run Samples” calls `POST /submissions/run`, which executes the solution against only the problem’s *sample* test cases and returns per-test pass/fail, so a student gets immediate feedback without ever seeing the hidden tests that decide their grade. Both paths are guarded by the same ownership logic as grading: a student may run only against a problem belonging to an exam assigned to a class they are enrolled in.

6.6 Notifications and Announcements

APEX has no message broker; notifications are plain rows written synchronously in the request that causes them, with an optional email side-channel. The single choke point is `notification.Create`, called from every feature that needs to reach a user. It inserts a `Notification` row and then, in a detached goroutine, checks the recipient's `notify_email` preference and sends an email — HTML-escaped in both subject-adjacent fields to prevent the injection class that a pre-ship security review had flagged:

```

1  if err := database.DB.Create(&notif).Error; err != nil {
2      return
3  }
4  go func() { // async email if the user opted in
5      var prof models.UserProfile
6      if err := database.DB.Where("user_id = ?", userID).First(&prof).Error; err != nil {
7          return
8      }
9      if !prof.NotifyEmail {
10         return
11     }
12     // ... load user, then send with html.EscapeString on title & body
13     body := fmt.Sprintf("<p><strong>%s</strong></p><p>%s</p>",
14         html.EscapeString(title), html.EscapeString(description))
15     _ = email.Send(user.Email, subject, body, text)
16 } ()

```

Listing 6.6. `notification.Create`: one in-app row, best-effort escaped email.

Announcements are the fan-out case. When a teacher posts an announcement to a class (`announcement.Create`, gated to the class owner), the handler persists the announcement row and then loops over every enrolled student, calling `notification.Create` once per member with a deep link back to the class's announcements tab. Each recipient thus gets both an in-app row and, if opted in, an email — a genuine fan-out driven by the enrollment table rather than a broadcast primitive. The same fan-out shape recurs elsewhere: a student joining a class notifies the teacher, and a graded exam notifies the student, each through the same `Create` helper.

On the client, notifications are polled rather than pushed. The `useNotificationsQuery` and `useUnreadCount` hooks are React Query subscriptions with a 30s `refetchInterval`, enabled only when a token is present. Marking one or all as read mutates through the API and invalidates both query keys, so the badge count and the list stay consistent without a socket connection. Polling is a deliberate simplicity trade: it costs a lightweight request every half-minute in exchange for needing no persistent connection infrastructure.

6.7 The In-Process Reminder Scheduler

Exam reminders are the one piece of genuinely time-driven behavior in the system, and they run inside the same process as the API rather than as a separate cron service. `reminder.Start`, invoked once from `main.go`, launches a goroutine that sleeps 15s to let the database connection settle and then ticks every minute on a `time.Ticker`. Each tick calls `runOnce`, which is wrapped in a `recover` so a panic in one sweep — a malformed row, a transient query error — logs and is contained rather than killing the ticker.

Fig. 6.2 lays out the three column-gated sweeps that each tick performs, positioned relative to an exam’s start time T .

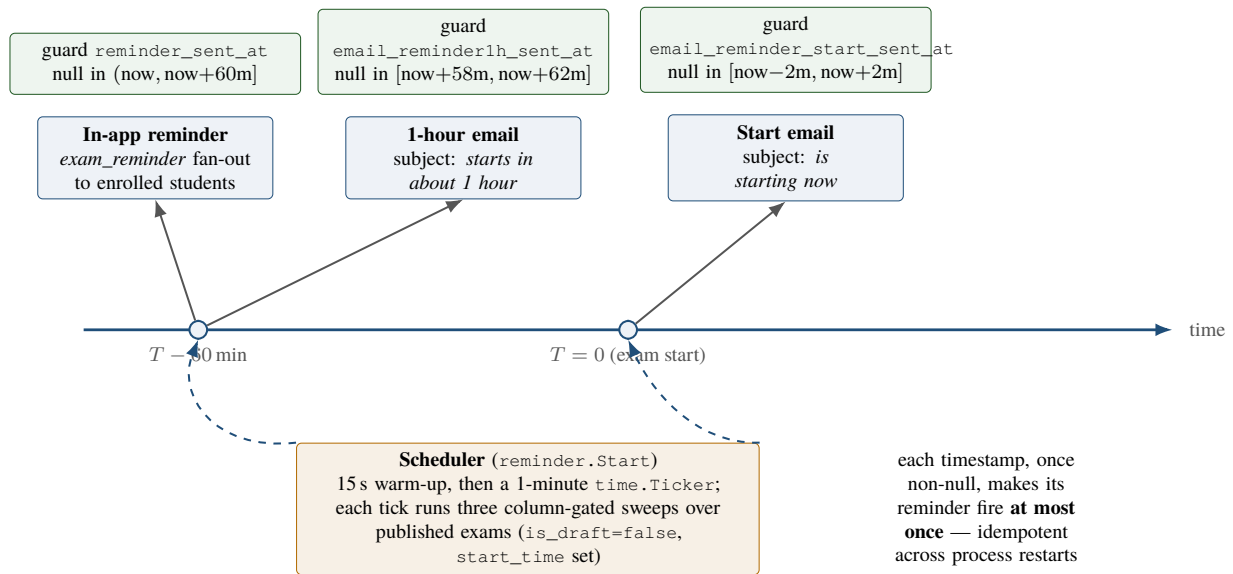


Figure 6.2. Idempotent exam-reminder schedule relative to start time T .

All three sweeps share the same idempotency mechanism, and it is the design’s central idea: each reminder is guarded by a dedicated “sent-at” column, and a reminder fires only for exams where that column is still `NULL`. After firing, the column is stamped. Because the guard is a persisted column and not in-memory state, a process restart cannot cause a re-send — the reminders are idempotent across restarts, not merely within a single process lifetime.

1. **In-app, $T-60$.** The main query in `runOnce` selects published exams (`is_draft = false`, `start_time` not null) whose start falls in `(now, now+60min]` and whose `reminder_sent_at` is `NULL`. For each, `notifyExam` resolves the enrolled students, filters to those with `notify_exam_reminders = true`, and fans out an `exam_reminder` notification linking to `/dashboard/exams`. Even an exam with no classes or members gets its column stamped, so it is not re-scanned on every subsequent tick.
2. **Email, $T-60$.** `emailSweep1Hour` selects exams whose start falls in the tight `[now+58min, now+62min]` window with `email_reminder1h_sent_at` still `NULL`, filters recipients by `notify_exam_email`, and sends the “starts in about 1 hour” email before stamping the column.
3. **Email, $T-0$.** `emailSweepStart` does the same for the `[now-2min, now+2min]` window around the start itself, guarded by `email_reminder_start_sent_at`, with the “is starting now” subject.

The narrow windows on the email sweeps (four minutes wide) combine with the one-minute tick to guarantee each email is evaluated several times but sent once. If `SMTP_HOST` is unset, `email.Send` logs the message body to `stdout` instead of dispatching it, so the whole reminder path is exercisable in local development without a mail server. Notably, the “Exam Graded” notification is *not* part of this scheduler — it belongs to the grading aggregator of Chapter 7, which is the only place that knows when every submission in an attempt has finished grading.

6.8 Frontend Architecture

The frontend is a React 18 [15] single-page application built with Vite [16] and written in TypeScript [31], totaling roughly 17,400 lines. Its organizing principle is *feature folders*: rather than grouping by technical kind (all components here, all pages there), `src/features/` holds self-contained slices — `auth`, `courses`, `dashboard`, `exams`, `grading`, `landing`, `playground`, `results`, `settings`, and `social` — each bundling its own pages, components, types, and helpers. The `exams` feature seen throughout this chapter is the largest, and its internal split into `pages/`, `components/`, `lib/`, and `types/` is representative.

6.8.1 The API Layer and Data Hooks

All network access funnels through a single `apiFetch` helper in `src/lib/api.ts`. It attaches the bearer token from `localStorage["kernel-token"]`, sets the JSON content type, and centralizes error handling. Its most important behavior is the global 401 handler: on an unauthorized response, if a token was present, it is a stale session — the helper clears the stored token and role and redirects to `/auth`; if no token was present, it simply throws so a public page can handle it. Every other non-2xx response is unwrapped into a typed `ApiError` carrying the backend’s error message and status. The rest of the file is a flat catalogue of typed, one-line functions — one per endpoint — so a caller writes `getStudentExam(id)` rather than assembling a URL.

On top of that thin layer sits TanStack React Query [36], wrapped in small feature hooks under `src/hooks/`. This is where the frontend’s server-state discipline lives. Reads are `useQuery` subscriptions keyed by a stable tuple — `["exam", id]`, `["student-exams"]`, `["my-attempts"]` — and often enabled conditionally on role so a student never fires a teacher-only query. Writes are `useMutation` hooks that invalidate the affected keys on success, letting React Query refetch and re-render rather than manually reconciling caches:

```

1  export function useStudentExam(id: number) {
2    return useQuery({
3      queryKey: ["student-exam", id],
4      queryFn: () => api.getStudentExam(id),
5      enabled: !!id,
6    });
7  }
8
9  export function useUpdateExam() {
10   const qc = useQueryClient();
11   return useMutation({
12     mutationFn: ({ id, data }: { id: number; data: any }) => api.updateExam(id, data),
13     onSuccess: (_d, vars) => {
14       qc.invalidateQueries({ queryKey: ["exams"] });
15       qc.invalidateQueries({ queryKey: ["exam", vars.id] });
16     },
17   });
18 }

```

Listing 6.7. A read hook and a write hook: React Query keys plus invalidation.

The exam-taking flow shows this coordination end to end. When a student first opens an exam, the component calls `startExamAttempt` and then invalidates `["my-attempts"]` and `["student-exams"]`, so the dashboard's "Start" button becomes "Continue" and the results view learns of the in-progress attempt — all without prop-drilling or a global store, because the query cache is the shared state.

6.8.2 UI, Styling, and Routing

The visual layer is built on `shadcn/ui` [37] components — roughly thirty Radix-based primitives (dialogs, selects, popovers, switches, scroll areas, resizable panels) copied into the repository and styled locally rather than pulled from a versioned package — over Tailwind CSS [38]. Tailwind's utility classes carry essentially all of the styling, including the conditional emphasis that matters during an exam: the countdown timer, for instance, switches to a destructive color once fewer than five minutes remain, expressed inline as a conditional class rather than a separate stylesheet. Markdown in problem statements and descriptions is rendered with `react-markdown` and `remark-gfm`, deliberately without raw-HTML passthrough so a problem statement cannot smuggle a script tag.

Routing is handled by `react-router-dom` 6. The application splits along role: a public shell serves the landing, authentication, and password-reset routes, while an authenticated dashboard shell nests the feature pages under `/dashboard`. The result is a codebase where a single feature — exams — can be read almost top to bottom: its routes on the backend, its handlers, its React Query hooks, its pages and editors, and the two-tier persistence that ties the timed session together, each layer thin enough to hold in one's head and each boundary drawn where a real responsibility changes hands.

Chapter 7

Code Execution and the Auto-Grading Engine

This chapter documents the component at the centre of APEX: the pipeline that takes a student’s source code, runs it safely, judges it fairly, and turns the result into a score. It is the crown jewel of the system and the piece that justifies the whole project’s existence, because a classroom portal that cannot grade code correctly is just a quiz tool with a text editor bolted on. Where the rest of APEX is careful CRUD—classes, exams, enrolments, notifications—the grading engine is where the genuinely hard engineering lives: running untrusted code without being compromised, comparing free-form program output without penalising correct answers on cosmetic grounds, and coordinating several concurrent grading tasks into a single, correct, once-only exam score.

The chapter traces one code submission end to end, from the moment a student presses *Submit* to the moment the “Exam Graded” notification lands, quoting the real implementation at every stage. It then broadens out to the two other answer formats—multiple-choice, which is auto-graded by set equality, and written answers, which enter a manual-grading queue—because a defining claim of APEX is that all three formats flow through *one* pipeline and aggregate into *one* attempt score. The whole flow is summarised in Fig. 7.1; the sections below expand each box of that figure into its underlying code.

Two design principles recur throughout and are worth stating up front. First, **APEX never runs untrusted code on its own host**: every execution is delegated over HTTP to Judge0 [4], [5], which runs the code inside the `isolate` sandbox [10]. Second, **APEX judges output by content, not by bytes**: a nine-line normalisation routine strips away the whitespace differences that cause the most common false negatives in output-comparison grading, a fairness concern the auto-assessment literature has flagged for two decades [6]. These two ideas—delegate execution, own the judging—shape everything that follows.

7.1 Execution Model: Delegating to a Sandbox

7.1.1 Why the sandbox is not ours

Running a student’s arbitrary program is the single most dangerous thing an assessment platform does. A submission is, by definition, untrusted code: it may loop forever, allocate until the machine swaps to death, fork-bomb the process table, try to read the server’s secrets, or open a socket to exfiltrate data. Executing such code directly on the application host—whether through `os/exec` or a hand-rolled container—makes the grader itself the attack surface, and writing a correct sandbox from scratch is one of the hardest problems in systems security. APEX declines that problem entirely.

Instead, execution is delegated to **Judge0** [4], an open-source “code execution as a service” whose design and rationale are described by da Silva Dos Reis and others [5]. Judge0 accepts a JSON document of

{language, source_code, stdin} over HTTP, compiles and runs the code, and returns {stdout, stderr, compile_output, status, time, memory}. Crucially, Judge0 runs each submission inside **isolate** [10], the sandbox originally built for the International Olympiad in Informatics to grade contestants' programs safely. *isolate* uses Linux namespaces and control groups (cgroups) to give the untrusted process an isolated view of the filesystem and network and to cap its CPU time and memory. The security posture follows directly: the APEX backend sits *outside* the sandbox and communicates with it only by an HTTP request and response. Untrusted student code never touches the application host, and the most security-critical component in the system is the one piece the team deliberately did not write.

The trade-off is deliberate and accepted: grading now depends on a network hop and on Judge0's availability. If Judge0 is unreachable, execution fails and the caller receives an explicit error rather than a wrong grade (Section 7.1.3). The team judged that correctness and safety outweigh self-sufficiency—a defensible position for a grading system, where a wrong “accepted” verdict is far worse than a temporary outage.

By default the backend targets the public endpoint `https://ce.judge0.com`, but the base URL is configurable through the `JUDGE0_URL` environment variable and wired at start-up via `judge0.Init(cfg.Judge0URL)`. The public Community Edition endpoint is rate-limited and not intended for production; a real deployment self-hosts Judge0. Note that Judge0 is reached as an *external* HTTP service and is therefore not one of the four Docker Compose services described in Chapter 10.

7.1.2 The language map

APEX supports six execution languages. The mapping from the language string sent by the frontend to Judge0's numeric language identifier is a single map in `internal/judge0/client.go`, reproduced in Listing 7.1. Several human-friendly aliases collapse onto the same identifier—`python` and `python3` both resolve to 100, and `cpp` and `c++` both resolve to 105—so the six distinct languages are exposed through ten map keys.

```
1 var LanguageMap = map[string]int{
2     "python":      100,
3     "python3":     100,
4     "javascript":  102,
5     "js":          102,
6     "typescript":  101,
7     "ts":          101,
8     "java":        91,
9     "c":           103,
10    "cpp":          105,
11    "c++":          105,
12 }
```

Listing 7.1. The language-to-Judge0-ID map (`client.go`).

The six languages and their identifiers are listed in Table 7.1. A language string absent from the map is not silently passed to Judge0: `RunCode` returns an internal sentinel result (status identifier `-1`) carrying the message “unsupported language”, and the HTTP handlers additionally pre-validate the language against `LanguageMap` before any network call is made, so an unknown language is rejected with a 400 `Bad Request` rather than a gateway error.

Table 7.1. Supported execution languages and their Judge0 language identifiers.

Language	Accepted keys	Judge0 ID
Python	python, python3	100
JavaScript	javascript, js	102
TypeScript	typescript, ts	101
Java	java	91
C	c	103
C++	cpp, c++	105

At the user-interface layer the coverage is deliberately asymmetric, which is worth stating precisely. The standalone live-coding playground exposes all six languages for free experimentation, whereas the exam-builder’s coding-question editor currently offers four of them—Python, JavaScript, C, and C++—as authoring targets. This is a front-end configuration choice, not a limitation of the grader: the execution and grading backend accepts any of the six keys in Table 7.1 regardless of which surface produced the submission, so widening the exam-builder set is a one-line change to the editor rather than any change to the pipeline described in this chapter.

7.1.3 The Judge0 client

Every execution in APEX—playground runs, sample-test feedback, and real grading—funnels through one function, `RunCode` in `internal/judge0/client.go`. It marshals the request, performs a single HTTP POST, and decodes the response into a compact `CodeResult`. The core is shown in Listing 7.2.

```

1 func RunCode(code, language, stdin string) CodeResult {
2     langID, ok := LanguageMap[language]
3     if !ok {
4         return CodeResult{StatusID: -1, Stderr: "unsupported language: " + language}
5     }
6
7     reqBody := request{LanguageID: langID, SourceCode: code, Stdin: stdin}
8     body, err := json.Marshal(reqBody)
9     if err != nil {
10        return CodeResult{StatusID: -1, Stderr: "failed to marshal request"}
11    }
12
13    url := BaseURL + "/submissions?wait=true&base64_encoded=false&fields=stdout,stderr,
        ↪ compile_output,status,time,memory"
14    client := &http.Client{Timeout: 30 * time.Second}
15    resp, err := client.Post(url, "application/json", bytes.NewReader(body))
16    if err != nil {
17        return CodeResult{StatusID: -1, Stderr: "judge0 unreachable"}
18    }
19    defer resp.Body.Close()
20    // ... read body, unmarshal, parse time (seconds -> ms) and memory (kb) ...
21 }

```

Listing 7.2. The single point of contact with the sandbox (`client.go`).

Several details of the request are deliberate and defensible:

- **Synchronous execution (`wait=true`).** The single `POST` blocks until Judge0 has finished compiling and running the code, and the verdict comes back in the same response. There is no client-side poll loop and no callback URL to manage. A **30-second** client timeout guards against a hung sandbox; if it fires, `RunCode` returns the `-1` sentinel.
- **Plain text, not base64 (`base64_encoded=false`).** Source code and standard input are sent as raw JSON strings. Judge0 supports base64 transport, but APEX opts out for simplicity; the modest cost is that exotic byte sequences could in principle be mangled, which is acceptable for classroom source code.
- **Field projection (`fields=...`).** The response is trimmed to exactly the six fields APEX consumes, keeping the payload small.
- **Metric extraction.** Judge0 reports execution time as a seconds string and memory as a number; `RunCode` parses the former into integer milliseconds (`seconds×1000`) and the latter into kilobytes, populating `CodeResult.TimeMs` and `CodeResult.MemoryKb`. These are *measured* and recorded, a point revisited in Section 7.3.6.

The status identifiers Judge0 returns drive the rest of the pipeline. Table 7.2 lists the ones APEX interprets. Two entries deserve emphasis now and are explained fully in Section 7.3.3: status 3 (Accepted) and status 4 (Wrong Answer) are *both* treated by APEX as “the program ran cleanly,” because APEX performs its own output comparison and does not defer to Judge0’s built-in diff.

Table 7.2. Judge0 status identifiers and APEX’s interpretation.

ID	Judge0 meaning	APEX action
3	Accepted (ran cleanly)	Counts as “ran” → compare normalised output
4	Wrong Answer (Judge0 diff)	Also counts as “ran” → re-compare with APEX’s own normalised diff
5	Time Limit Exceeded	Verdict <code>time_limit_exceeded</code>
6	Compilation Error	Verdict <code>compilation_error</code>
11	Runtime Error	Verdict <code>runtime_error</code>
-1	APEX sentinel (unreachable / marshal / parse failure)	Playground returns 504; other paths fail the test

7.2 Three Entry Points, One That Grades

A student’s code can reach the backend through three distinct endpoints, and keeping them separate is one of the cleaner decisions in the system. Only one of the three persists a graded `Submission`; the other two are ephemeral feedback paths that write nothing to the database. Table 7.3 summarises the split.

The practical consequence is that a student can press “Run” fifty times during an exam at a cost of zero database rows and zero grading work. Running code is feedback; submitting is the grade; they are different code paths on purpose.

Table 7.3. The three code-execution entry points.

Endpoint	Handler	Persists grade?	Purpose
POST /execute	execute.Execute	No	Playground scratchpad
POST /submissions/run	submission.RunSolution	No	Sample-test feedback
POST /student/exams/:id/submit	exam.SubmitAttempt	Yes	The real, graded submission

7.2.1 The playground path

The cheapest path is the playground handler, `execute.Execute` in `internal/execute/handler.go`. It binds a `{language, code, stdin}` request, pre-validates the language against `LanguageMap`, calls `RunCode`, and returns the raw output. The one subtlety is its error mapping: the internal `-1` sentinel is surfaced as an HTTP 504 Gateway Timeout, a precise signal that *the sandbox service* failed rather than that *the student's code* was wrong.

```

1  if _, ok := judge0.LanguageMap[req.Language]; !ok {
2      c.JSON(http.StatusBadRequest, gin.H{"error": "unsupported language: " + req.Language})
3      return
4  }
5
6  result := judge0.RunCode(req.Code, req.Language, req.Stdin)
7  if result.StatusID == -1 {
8      c.JSON(http.StatusGatewayTimeout, gin.H{"error": result.Stderr})
9      return
10 }
11 c.JSON(http.StatusOK, gin.H{
12     "stdout": result.Stdout, "stderr": result.Stderr,
13     "status_id": result.StatusID, "time_ms": result.TimeMs, "memory_kb": result.MemoryKb,
14 })

```

Listing 7.3. The playground handler maps the sandbox sentinel to 504 (`execute/handler.go`).

7.2.2 The sample-test feedback path

The second non-persisting path is `submission.RunSolution` (`internal/submission/handler.go`), which powers the “Run sample tests” button during an exam. It first enforces resource ownership—the third authorisation gate described in Chapter 6: a teacher may run only against a problem they own, and a student may run only against a problem belonging to an exam assigned to a class they are enrolled in, verified by a join across `exam_classes` and `class_members`. It then loads only the *sample* test cases (`WHERE is_sample = true`), ordered by `order_index`, and evaluates them through `RunAgainstTestCases`. Nothing is written; the student receives per-test input, expected output, and actual output so they can debug before committing to a real submission. Because `RunAgainstTestCases` applies the *same* `normalizeOutput` routine as the real grader (Section 7.3.4), the feedback a student sees while iterating matches the verdict they will later be graded by.

7.3 The Coding Grader

The graded path begins in `exam.SubmitAttempt`, which creates one `Submission` row per answer at status `pending` and dispatches a grader according to the problem type. For a coding problem, that grader is `judge0.Grade`. The full sequence—from the Monaco editor through this grader to the final notification—is depicted in Fig. 7.1.

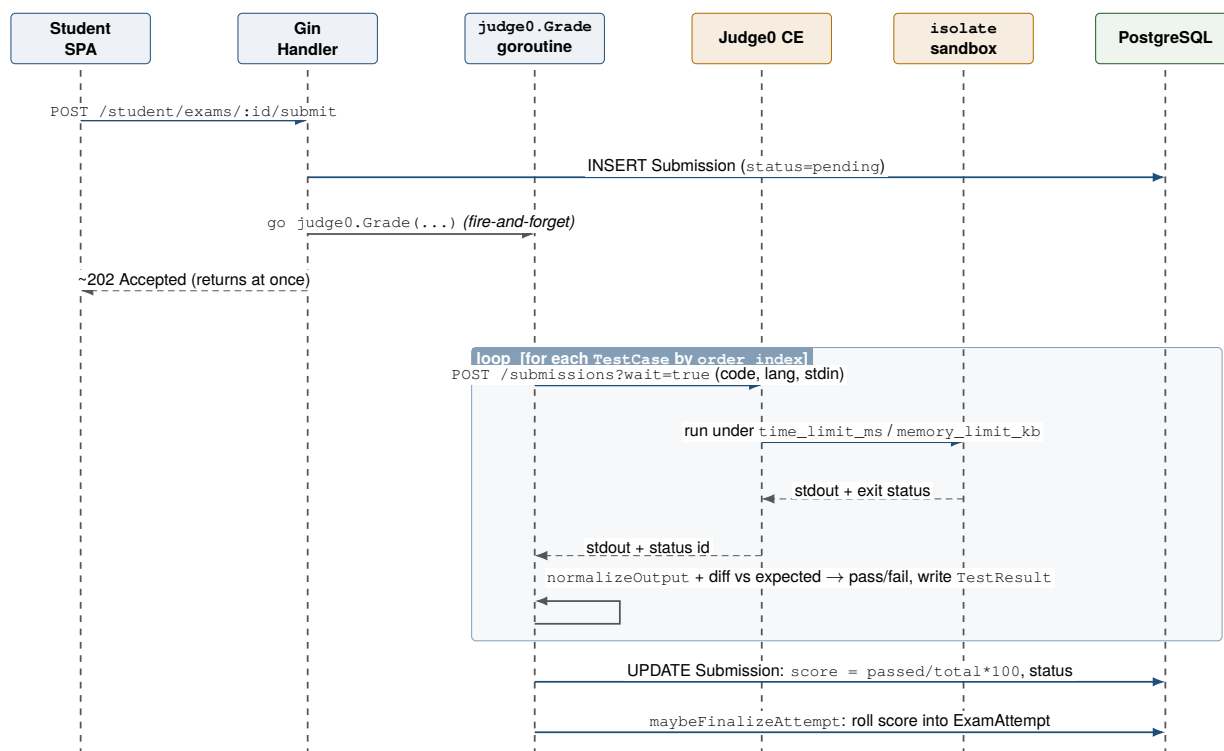


Figure 7.1. Sequence of an auto-graded code submission.

7.3.1 The fire-and-forget dispatch

Each submission is graded in its own bare goroutine. `SubmitAttempt` creates the row and then launches the appropriate grader with the `go` keyword, routing by problem type:

```

1 switch submissionType {
2 case "mcq":
3     go judge0.GradeMCQ(sub.ID)
4 case "written":
5     go judge0.GradeWritten(sub.ID)
6 default:
7     go judge0.Grade(sub.ID)
8 }

```

Listing 7.4. Per-submission grader dispatch in `SubmitAttempt`.

The `go` keyword means the HTTP handler does not wait for grading to finish: the request returns `201 Created` immediately, carrying the attempt identifier and the list of created submission identifiers, while grading proceeds in the background. The student sees an “Exam Submitted” notification at once; the “Exam Graded” notification arrives later, once every goroutine has completed.

This model is deliberately simple and it works at the concurrency of a single classroom, but it carries a durability gap that the project documents openly (`README.md` line 39). The dispatch is **fire-and-forget**: there is no work queue, no supervisor, and no retry. As the next subsection shows, `Grade` immediately flips the submission to `running`; if the backend process crashes or is redeployed mid-grade, that goroutine dies with the process and the row is left stuck in `running` forever. Because the attempt finaliser refuses to proceed while any sibling submission is still `pending` or `running` (Section 7.5), the attempt never finalises and the student never receives the “Exam Graded” notification. The failure mode is discussed in full, together with its proposed fix, in Section 7.7.

7.3.2 Loading test cases and the empty-set edge case

`Grade` begins by loading the submission, flipping its status to `running`, and loading *all* test cases for the problem ordered by `order_index`. There is deliberately no `is_sample` filter here—this is the real grade, so both sample and hidden tests run. One edge case is handled defensively: a problem with *no* test cases configured is auto-accepted at score `100.0`, on the principle that a teacher who forgot to add tests should not thereby fail the student.

```

1 database.DB.Model(&submission).Update("status", "running")
2
3 var testCases []models.TestCase
4 database.DB.Where("problem_id = ?", submission.ProblemID).Order("order_index asc").Find(&
   ↪ testCases)
5
6 if len(testCases) == 0 {
7     database.DB.Model(&submission).Updates(map[string]any{
8         "status": "accepted",
9         "score": 100.0,
10    })
11    return
12 }
```

Listing 7.5. Grader set-up and the no-test-case edge case (`grader.go`).

7.3.3 The grading loop

The heart of the coding grader is the loop that runs the student’s code against each test case and records a per-test verdict. It is quoted in full in Listing 7.6.

```

1 for _, tc := range testCases {
2     result := RunCode(submission.Code, submission.Language, tc.Input)
3
4     actualOutput := normalizeOutput(result.Stdout)
5     expectedOutput := normalizeOutput(tc.ExpectedOutput)
6     // StatusID 3 = Accepted; treat StatusID 4 (Wrong Answer) as valid-run-but-compare.
7     ran := result.StatusID == 3 || result.StatusID == 4
8     isPassed := ran && actualOutput == expectedOutput
9
10    if isPassed {
11        passed++
12    }
```

```

13
14     status := "accepted"
15     if !isPassed {
16         switch result.StatusID {
17             case 6:
18                 status = "compilation_error"
19             case 5:
20                 status = "time_limit_exceeded"
21             case 11:
22                 status = "runtime_error"
23             default:
24                 status = "wrong_answer"
25         }
26     }
27
28     tr := models.TestResult{
29         SubmissionID:    submission.ID,
30         TestCaseID:       tc.ID,
31         Passed:           isPassed,
32         ActualOutput:     actualOutput,
33         ExecutionTimeMs:  result.TimeMs,
34         MemoryKb:         result.MemoryKb,
35         Status:           status,
36     }
37     database.DB.Create(&tr)
38
39     if result.TimeMs > maxTimeMs {
40         maxTimeMs = result.TimeMs
41     }
42     if result.MemoryKb > maxMemoryKb {
43         maxMemoryKb = result.MemoryKb
44     }
45 }

```

Listing 7.6. The per-test-case grading loop (`grader.go`).

For each test case, the loop performs four conceptual steps. First, it sends the student’s code and this test’s input to Judge0 through `RunCode`. Second, it passes *both* the program’s actual standard output *and* the test’s expected output through the same `normalizeOutput` routine—applied symmetrically, which is the fairness guarantee analysed in Section 7.3.4. Third, it decides whether the test passed. Fourth, it writes one `TestResult` row recording the outcome, the actual output, and the measured time and memory, and it tracks the maximum time and memory seen across all tests.

The pass decision hides a subtle and important design choice. The predicate `ran := result.StatusID == 3 || result.StatusID == 4` treats *both* Judge0’s “Accepted” (3) and its “Wrong Answer” (4) as “the program executed cleanly.” The reason is that Judge0 performs its *own* byte-exact diff and may return status 4 purely because of a trailing newline—exactly the false negative APEX refuses to honour. So APEX ignores Judge0’s accept/reject decision and re-judges with its own normalised equality check: `isPassed := ran && actualOutput == expectedOutput`. In effect, **Judge0 is treated as an executor—run this safely and tell me what it printed—not as a judge**. The judging belongs to APEX. Only genuine execution failures (compilation error 6, time-limit 5,

runtime error 11) are surfaced as failure verdicts; anything else that ran but did not match is recorded as `wrong_answer`.

7.3.4 Output normalisation: nine lines of fairness

Output-comparison grading has one notorious weakness, identified in the literature as far back as the first systematic surveys of automated assessment [6]: a program that computes the *right* answer can be marked wrong because of invisible whitespace. A student prints `42\n` where the expected answer is `42` with no trailing newline; a student on Windows emits `\r\n` line endings where the teacher typed `\n`; a stray trailing space follows a correct number. Under naive byte-for-byte comparison every one of these is a false negative. This is the single most common unfairness on any output-diffing platform, and eliminating it is the entire job of `normalizeOutput`, reproduced in full in Listing 7.7.

```
1 // normalizeOutput strips CR, trailing whitespace on each line, and trims edges
2 // so Judge0 output (often \r\n with trailing newline) compares cleanly to
3 // teacher-entered expected output.
4 func normalizeOutput(s string) string {
5     s = strings.ReplaceAll(s, "\r\n", "\n")
6     s = strings.ReplaceAll(s, "\r", "\n")
7     lines := strings.Split(s, "\n")
8     for i, ln := range lines {
9         lines[i] = strings.TrimRight(ln, " \t")
10    }
11    return strings.TrimSpace(strings.Join(lines, "\n"))
12 }
```

Listing 7.7. `normalizeOutput`—applied symmetrically to actual and expected output (`grader.go`).

The routine applies four normalisation rules, in order, and each one closes a specific class of false negative:

1. `ReplaceAll("\r\n", "\n")` converts Windows CRLF line endings to Unix LF, so a student typing on Windows—or a teacher who pasted expected output out of a Windows editor—does not fail on invisible carriage returns.
2. `ReplaceAll("\r", "\n")` folds any lone carriage return (classic-Mac line endings) into a newline as well: belt and suspenders.
3. Splitting on `\n` and applying `TrimRight(ln, " \t")` to *each* line strips trailing spaces and tabs from every line, so a stray space after a printed value is not treated as a correctness error.
4. `TrimSpace` on the whole joined string removes leading and trailing whitespace from the entire output, absorbing the final trailing newline that `print` and `println` routinely append, as well as any leading blank line.

The fairness guarantee rests on one property: because the *same* function is applied to both the actual output and the expected output before comparison, the comparison is symmetric. It measures the *correctness of content*, not the *byte-exactness of whitespace*. Without these nine lines, a student who computed the right answer would routinely fail on a trailing newline, a CRLF line ending, or a trailing space—and each of those is a formatting artefact, not a wrong answer.

It is equally important to state plainly what this normalisation does *not* do. It is whitespace-trim-fair, not token-fair: it does not collapse *internal* runs of spaces, so `1 2` and `1 2` still differ, and it carries no

floating-point tolerance, so `3.14` and `3.140000001` still differ. It solves the most common false negative, not every conceivable one. That is a deliberate scoping decision rather than an oversight: the routine is small enough to verify by eye, and its behaviour is predictable, which for a fairness-critical function is itself a virtue.

7.3.5 Scoring and aggregation into the submission

Once every test case has been run, the grader computes a proportional score and writes the outcome back to the Submission row:

```
1 score := float64(passed) / float64(total) * 100.0
2 finalStatus := "wrong_answer"
3 if passed == total {
4     finalStatus = "accepted"
5 }
6
7 database.DB.Model(&submission).Updates(map[string]any{
8     "status":          finalStatus,
9     "passed_count":    passed,
10    "total_count":     total,
11    "score":           score,
12    "execution_time_ms": maxTimeMs,
13    "memory_kb":       maxMemoryKb,
14 })
```

Listing 7.8. Coding score and final submission update (`grader.go`).

Coding submissions therefore earn **partial credit**: passing seven of ten tests yields a score of `70.0`, and the submission’s status is `accepted` only if every test passed, otherwise `wrong_answer`. The counts `passed_count` and `total_count` are stored for display, and the *worst-case* (maximum) execution time and memory across all tests are recorded on the submission. Notably, the per-test-case `points` field that exists in the `TestCase` schema is *not* used by the coding grader—coding is scored as a flat `passed / total` fraction, and points weighting happens one level up, at the problem level, during attempt aggregation (Section 7.5).

Finally, if the submission is standalone—not part of an exam attempt, i.e. `ExamAttemptID == nil`, as with a bank-question practice run—a per-submission “result” notification fires immediately. If instead the submission belongs to an exam attempt, that per-submission notification is suppressed and control passes to `maybeFinalizeAttempt`, so the student receives *one* exam-level notification rather than one per question.

7.3.6 Limits: measured but not yet pushed

The `Problem` model carries two resource-limit fields—`time_limit_ms` (default `2000`) and `memory_limit_kb` (default `262144`, i.e. `256 MB`)—intended to let a teacher tighten the sandbox per problem. In the current implementation these fields are *not* forwarded to `Judge0`: `RunCode` sends no `cpu_time_limit` or `memory_limit`, so the effective limits are `Judge0`’s own defaults. *APEX measures* time and memory from each response and records them on every `TestResult` and `Submission`, but it does not yet *push* the per-problem ceilings down. The system remains safe—a runaway loop still becomes

a time-limit-exceeded verdict under Judge0’s default ceiling—but a teacher who sets a one-second limit does not presently get it enforced. Wiring these two fields into the request body is a small, known task rather than a design flaw.

7.4 The Other Two Formats

APEX’s defining structural claim is that coding, multiple-choice, and written answers all flow through one pipeline and aggregate into one attempt score. The two non-coding graders live alongside `Grade` in `grader.go` and share its dispatch, notification, and finalisation machinery.

7.4.1 Multiple-choice: auto-graded by set equality

`GradeMCQ` judges a multiple-choice answer by **exact set equality** between the teacher’s correct option identifiers and the student’s selected option identifiers. Both are stored as JSON arrays—`Problem.CorrectOptionIDs` and `Submission.SelectedOptions`—which the grader unmarshals into two Go sets, trimming and dropping empty entries. The answer is correct only if the two sets are exactly equal: same cardinality, and every correct identifier selected.

```
1 isCorrect := len(correctSet) == len(selectedSet)
2 if isCorrect {
3     for id := range correctSet {
4         if !selectedSet[id] {
5             isCorrect = false
6             break
7         }
8     }
9 }
```

Listing 7.9. Set-equality decision in `GradeMCQ` (`grader.go`).

MCQ grading is **all-or-nothing**: a correct answer scores `100.0` with status `accepted`, an incorrect one scores `0.0` with status `wrong_answer`, and there is no partial credit for selecting a subset of a multi-answer question. One protective special case precedes the comparison: if a teacher has configured *no* correct answers (`len(correctSet) == 0`), the submission is marked `pending_review` at score `0` rather than failed, on the principle that a student must not be failed on a question whose answer key was never set. A JSON parse failure on either field marks the submission `error`. As with coding, standalone submissions fire an immediate notification, while exam submissions defer to `maybeFinalizeAttempt`.

7.4.2 Written answers: the manual-grading queue

`GradeWritten` performs no automatic judgement at all. It records the student’s answer, sets the submission to `pending_review` at score `0` with `total_count` of `1`, and calls `maybeFinalizeAttempt`. In effect it enrolls the answer into a *manual-grading queue*: a written answer never auto-fails a student; it always waits for a human. This is the correct default for free-form prose, which no output diff can grade.

Teachers drain that queue through two endpoints in `internal/submission/handler.go`. `GetSubmission` lets an authorised reader—the student who owns the submission, or the teacher who owns the exam—retrieve a submission with its per-test results; notably, it exposes the `input` and

expected_output of a test *only* when that test is a sample (if `rv.IsSample`), so hidden test cases are never leaked to students even on the results screen. `GradeSubmission` handles the actual manual grade via `PUT /submissions/:id/grade`, gated to the exam owner:

```

1  if exam.TeacherID != teacherID {
2      c.JSON(http.StatusForbidden, gin.H{"error": "not authorized to grade this submission"})
3      return
4  }
5
6  sub.Score = req.Score
7  sub.Status = req.Status
8  sub.PassedCount = int(req.Score)
9  sub.TotalCount = 100
10 if req.TeacherFeedback != nil {
11     sub.TeacherFeedback = *req.TeacherFeedback
12 }
13 database.DB.Save(&sub)
14
15 if sub.ExamAttemptID != nil {
16     judge0.FinalizeAttempt(*sub.ExamAttemptID)
17 }

```

Listing 7.10. Manual grading applies a score override and re-finalises the attempt (submission/handler.go).

The teacher supplies a score, a status, and optional `teacher_feedback`, and the handler writes them onto the submission. Crucially it then calls `judge0.FinalizeAttempt`, the public wrapper over `maybeFinalizeAttempt`, so that grading a written answer immediately recomputes the whole attempt score and—if it was the last outstanding answer—fires the “Exam Graded” notification. This is exactly why the aggregator is written as a re-summation rather than an incremental update: a manual grade lands long after the automatic ones, and re-aggregating from the children makes that late arrival correct without any special-casing.

7.5 Attempt Aggregation

Every grader—coding, MCQ, and written—ends by calling `maybeFinalizeAttempt`. This function is the orchestrator that turns many per-question submissions into one exam score and one notification, coordinating several concurrent goroutines without a lock.

7.5.1 The barrier: only the last goroutine proceeds

The function’s first act is a guard. It counts sibling submissions still in `pending` or `running` state and returns immediately if any remain:

```

1  var pending int64
2  database.DB.Model(&models.Submission{}).
3      Where("exam_attempt_id = ? AND status IN ?", *attemptID, []string{"pending", "running"}).
4      Count(&pending)
5  if pending > 0 {
6      return
7  }

```

Listing 7.11. The concurrency barrier in `maybeFinalizeAttempt` (`grader.go`).

Several goroutines, each grading one answer, all call this same function as they finish, but only the *last* one to complete finds zero pending submissions and proceeds past the guard. This is how independent goroutines coordinate into a single finalisation without a mutex or a shared counter: the database row-count *is* the coordination primitive. It also explains the finalisation half of the fire-and-forget gap—a submission stuck in `running` because its goroutine died keeps this count above zero forever, and the attempt never finalises.

7.5.2 Re-summing over the exam’s full problem list

Past the barrier, the aggregator recomputes the attempt score from scratch. It loads the *exam’s full problem list*—not the submissions—and folds each problem’s contribution into an earned/total ratio:

```

1  for _, p := range problems {
2      s, hasSub := subByProblem[p.ID]
3      if hasSub && s.Status == "pending_review" {
4          continue
5      }
6      pts := float64(p.Points)
7      if pts <= 0 {
8          pts = 10
9      }
10     totalPoints += pts
11     if hasSub {
12         earnedPoints += s.Score / 100.0 * pts
13     }
14 }
15 pct := 0.0
16 if totalPoints > 0 {
17     pct = earnedPoints / totalPoints * 100.0
18 }

```

Listing 7.12. Points-weighted re-aggregation over the problem list (`grader.go`).

Three behaviours in this loop are subtle and each is deliberate:

- **Skipped questions stay in the denominator at zero.** Because the loop iterates the exam’s problem list rather than the student’s submissions, a problem the student never answered still adds its points to `totalPoints` while contributing nothing to `earnedPoints`. A student cannot inflate their percentage by leaving questions blank.
- **Pending-review answers are excluded from both numerator and denominator.** A written answer awaiting manual grading is `continued` over entirely, so an ungraded essay does not depress the visible score to look like a failure before the teacher has read it. Once the teacher grades it and re-runs the aggregator, it re-enters both sums.
- **Points weighting happens here, at the problem level.** Each problem contributes in proportion to its `Points` (defaulting to 10 when unset or non-positive), so a 20-point hard problem and a 5-point easy one weigh proportionally. The submission’s own `score` is a 0–100 percentage; scaling it by `pts / 100` converts it into earned points.

The computed percentage is written to `ExamAttempt.score`. Because the whole score is recomputed from the children every time, the function is idempotent and safe to call any number of times—there is no running total to double-count, and manual grading simply re-runs it.

7.5.3 The single, deduplicated notification

The final stage fires exactly one “Exam Graded” notification, and only under the right conditions. The aggregator counts any submissions still in `pending_review`; if written answers remain ungraded, it updates the score but withholds the notification. Only when no `pending_review` submissions remain, the attempt’s status is submitted, and the attempt has not already been notified does it fire—using the boolean column `exam_attempts.graded_notified` as an idempotency flag, set true before the notification is created so it can never be sent twice.

```
1 if attempt.Status != "submitted" || attempt.GradedNotified {
2     return
3 }
4 database.DB.Model(&attempt).Update("graded_notified", true)
5 notification.Create(attempt.UserID, "result",
6     "Exam Graded",
7     fmt.Sprintf("Your submission for \"%s\" has been graded. Score: %.0f%%", attempt.Exam.
8         ↪ Title, pct),
9     fmt.Sprintf("/dashboard/exam/%d/review", attempt.ExamID),
10 )
```

Listing 7.13. Once-only “Exam Graded” notification, deduplicated by `graded_notified` (`grader.go`).

The result is a clean invariant: no matter how many questions an exam contains or in what order their goroutines finish, the student receives one submission notification when they submit and one grading notification when the last answer—automatic or manual—is graded.

7.6 Sample versus Hidden Test Cases

The distinction between sample and hidden test cases runs through the whole pipeline and is what prevents students from reverse-engineering the answer key. Each `TestCase` carries an `is_sample` boolean, and the three code paths treat it differently:

- **Sample tests are visible.** The “Run sample tests” path (`RunSolution`) filters `is_sample = true` and returns input, expected output, and actual output, giving students pre-submission feedback.
- **Hidden tests are opaque.** The real grader (`Grade`) applies no `is_sample` filter and runs every test, so hidden tests count toward the score but are never surfaced during the exam.
- **Results reveal only samples.** On the results screen, `GetSubmission` attaches input and expected_output to a result only when its test case is a sample; for hidden tests the student sees pass/fail and timing but never the input or the expected output.

Test-case management is teacher-owned and side-effecting. `AddTestCase`, `UpdateTestCase`, and `DeleteTestCase` in `internal/testcase/handler.go` each verify ownership—the teacher must own the bank problem, or own the exam the problem belongs to—before mutating a test. Critically, any change that could alter grading (adding a test, deleting a test, or editing a test’s input or expected output on a problem attached to an exam) triggers `examsvc.ResetExamAttempts(*problem.ExamID)`, so that

prior attempts graded against the old test set are invalidated rather than left silently inconsistent. This connects the grading engine to the destructive-edit reset mechanism described in Chapter 4.

7.7 The Honest Gap: Fire-and-Forget Durability

The most significant limitation of the grading engine is the one already foreshadowed: grading is dispatched as a bare goroutine (`go judge0.Grade(sub.ID)`) with no durable queue, no supervisor, and no retry. The failure mode is precise and worth stating exactly. `Grade` flips the submission to `running` before contacting Judge0; if the backend process crashes or is redeployed while a grade is in flight, the goroutine dies with the process and the row is left permanently in `running`. Because `maybeFinalizeAttempt` refuses to proceed while any sibling is `pending` or `running`, the whole attempt never finalises and the student never receives their grade. There is also no concurrency cap: a large exam could spawn hundreds of simultaneous Judge0 requests and exhaust the public endpoint's rate limit. This gap is documented openly in the project's README.

The mitigating context is scale. At the concurrency of a single classroom—tens of students, not thousands—the model works, and the simplicity of returning the HTTP response instantly while grading proceeds in the background is a real benefit. The gap is a considered trade-off, not an accident.

The fix is well understood and follows directly from a property the current design already has: **grading is idempotent**. `Grade` recomputes every result from the submission's stored `code` and the problem's test cases and overwrites the previous results, so re-running it is always safe. The durable version replaces the bare goroutine with the enqueue of a job into a persistent work queue—a database-backed jobs table, or a broker such as Redis—consumed by a bounded pool of workers. On start-up, a recovery sweep re-enqueues anything stuck in `running`, and because re-grading is idempotent, the replay is safe. The bounded pool simultaneously caps concurrency against Judge0. None of this changes the grading logic in this chapter; it changes only how that logic is scheduled and made durable, which is why it sits on the roadmap rather than blocking the current classroom-scale deployment.

7.8 Summary

This chapter traced a code submission from the Monaco editor to a graded, notified exam attempt. The design rests on a clean separation of concerns: execution is delegated to Judge0 [4], [5] and its `isolate` sandbox [10], so untrusted code never touches the application host, while the *judging*—output normalisation, pass/fail determination, proportional scoring, and points-weighted aggregation—is owned entirely by APEX. The nine-line `normalizeOutput` routine is the fairness keystone, applied symmetrically to actual and expected output so that grading measures content rather than whitespace [6]. Around the coding grader, MCQ answers are auto-judged by set equality and written answers enter a manual-grading queue, and all three formats aggregate through `maybeFinalizeAttempt` into a single attempt score and a single, deduplicated notification. The one material weakness—fire-and-forget dispatch without a durable queue—is disclosed plainly, along with an idempotency-based fix that the existing design already makes straightforward. Together these pieces deliver the project's central promise: three answer formats, one grading pipeline, one score.

Chapter 8

Security

Security in APEX was treated as a scheduled engineering activity rather than an afterthought. A dedicated review was carried out *before* the system left development, and every issue it surfaced was reachable in the deployed code path even though none was ever discovered in production. This chapter documents the result: the threat model that scopes what APEX defends against, the authentication stack, the layered authorization model, the input-handling posture, the sandbox boundary for untrusted code, a mapping of the whole system onto the OWASP Top 10, and the gaps that remain open on the roadmap.

The guiding principle throughout is *fail closed*: on any error, ambiguity, or missing precondition, the system denies the action rather than granting it. A missing header, a malformed token, an unknown role, a resource the caller does not own, or a critical environment variable left unset all resolve to a refusal, never to a silent “allow.” This is the single most important adjective in the chapter, and it recurs at every layer.

8.1 Threat Model

APEX is a self-hostable classroom portal whose actors—teachers and students of a single institution—are known to one another. This shapes the threat model: the realistic adversary is not a nation-state but an insider probing for data that is not theirs, hostile content flowing through user-authored fields, a forged or stolen bearer token, and untrusted program code submitted for grading. Figure 8.1 names the four adversary classes APEX defends against and pairs each with the concrete control that mitigates it. Naming the four classes is also an act of scoping: physical server theft, denial-of-service at the network edge, and supply-chain compromise of third-party dependencies are explicitly *out* of scope for a graduation project and are expected to be handled by the hosting environment.

The four classes read as follows. The **unauthenticated network attacker** holds no token and can only probe the public surface—the login, signup, password-reset, and Google sign-in endpoints, plus the unauthenticated `GET /healthz` liveness probe. The **authenticated student attempting privilege escalation** holds a valid token but tries to act with teacher intent, for example by calling a teacher-only route such as `POST /exams`. The **curious student** holds a valid token and the correct role but probes *other* users’ resources by guessing object identifiers—the classic Insecure Direct Object Reference (IDOR, CWE-639 [39]), which is the thematically central risk for a portal whose whole value is keeping one student’s work private from another. Finally, **malicious submitted code** treats the grading pipeline as an execution primitive: a student’s program is, by definition, hostile input, and running it carelessly is remote code execution by design.

A fifth concern cuts across the model: the **operator who misconfigures a deploy**. A self-hosting instructor cannot be relied on to set every environment variable correctly, so the system must fail closed when a critical variable is missing or weak. The trust boundary is drawn at the process edge—the browser is untrusted, the server is trusted, and per-resource ownership re-checks happen *inside* handlers rather than

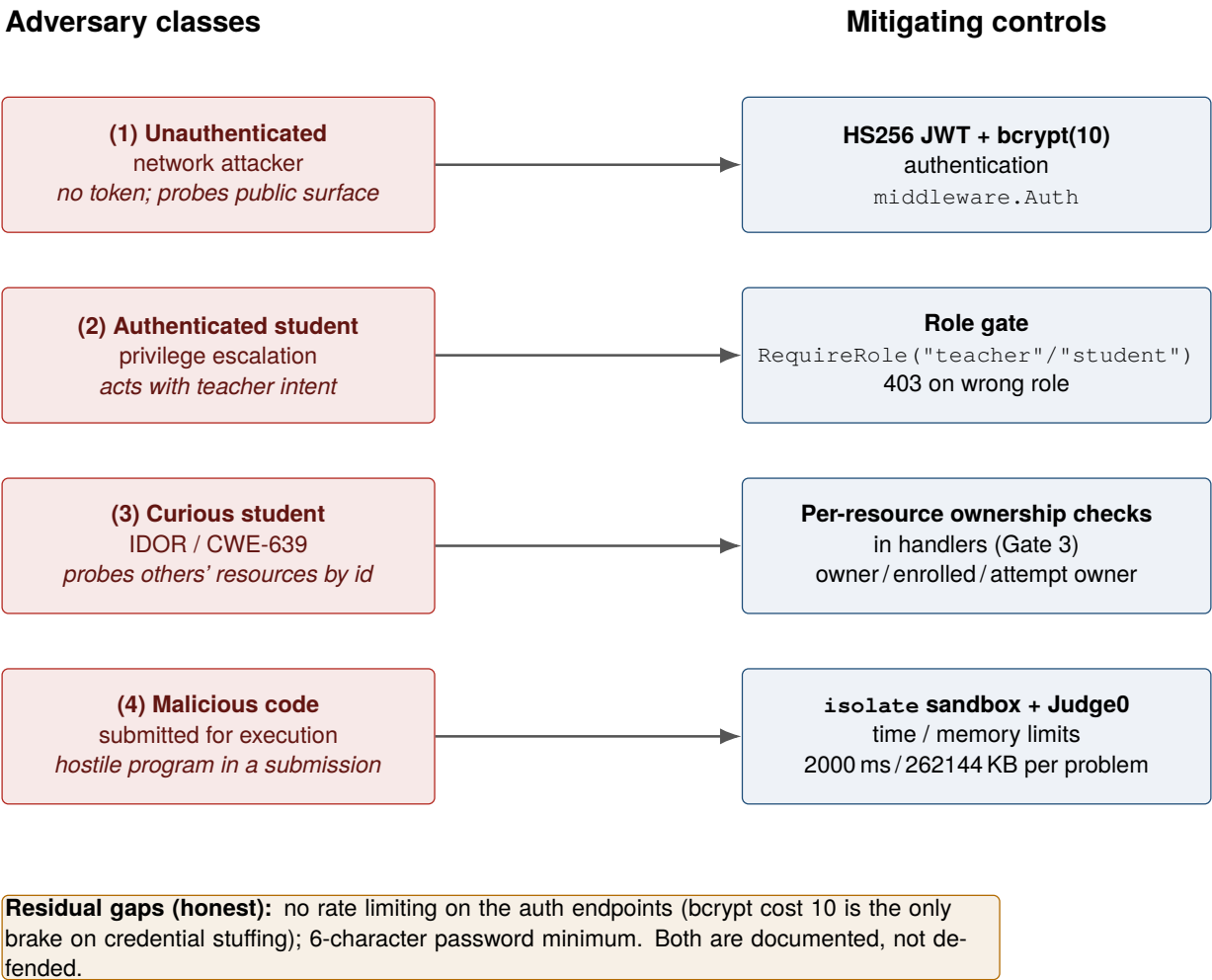


Figure 8.1. Threat model: four adversary classes and their controls.

at the middleware boundary, for reasons developed in Section 8.3.

8.2 Authentication

Authentication answers “who are you?”. APEX proves identity with a signed JSON Web Token issued after a successful credential check, and it protects the long-lived secrets—passwords and reset tokens—with the appropriate one-way primitives.

8.2.1 The fail-closed JWT secret

The signing secret is the linchpin of the entire scheme: anyone who knows it can forge a token for any user. APEX therefore refuses to boot without a strong one. The configuration loader validates `JWT_SECRET` at startup and calls `log.Fatal` (which exits with a non-zero status) if it is absent, is the historical placeholder, or is shorter than 32 characters.

```
1 jwtSecret := os.Getenv("JWT_SECRET")
2 if jwtSecret == "" ||
3     jwtSecret == "dev-secret-change-in-prod" ||
4     len(jwtSecret) < 32 {
5     log.Fatal("JWT_SECRET must be set to a strong random value (>=32 chars)")
6 }
```

Listing 8.1. Fail-closed validation of the signing secret at startup (`internal/config/config.go`).

Three foot-guns are rejected: an unset variable, the exact placeholder string `dev-secret-change-in-prod` that once served as a silent default, and any value under 32 characters—a deliberately conservative bar for HMAC-SHA-256. Because `log.Fatal` exits non-zero, no process supervisor such as `systemd` or `Docker` can mask the failure; a deploy with a weak secret stops dead rather than limping on insecurely. This is the mitigation for the misconfiguring operator, and it embodies the twelve-factor discipline of drawing configuration from the environment rather than from source [28]. Secrets never enter version control: the repository ships `.env.example`, never `.env`.

8.2.2 Token issuance and verification

Tokens are HS256 (HMAC with SHA-256), a symmetric scheme in which the same secret both signs and verifies—appropriate here because a single server issues and validates its own tokens, with no third party needing a public verification key [18]. On successful signup, login, or Google sign-in the server issues a token whose claims are `{user_id, email, role, name, exp}`, with `exp` set 24 hours from issue.

```
1 func generateToken(user models.User) (string, error) {
2     claims := jwt.MapClaims{
3         "user_id": user.ID,
4         "email":   user.Email,
5         "role":    user.Role,
6         "name":    user.Name,
7         "exp":     time.Now().Add(24 * time.Hour).Unix(),
8     }
9     token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
10    return token.SignedString([]byte(cfg.JWTSecret))
}
```

```
11 }
```

Listing 8.2. Token issuance in `internal/auth/handler.go`.

The 24-hour `exp` claim is the only automatic kill-switch for a leaked token, since there is no server-side revocation list (a limitation revisited in Section 8.7). The payload is signed but not encrypted, so no secret is placed in it; the claims merely mirror what the verification middleware reads back.

Verification runs on every protected request in `middleware.Auth`. It enforces five fail-closed checks in sequence, the most security-critical of which is *algorithm pinning*.

```
1 tokenStr := strings.TrimPrefix(header, "Bearer ")
2 token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (any, error) {
3     if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
4         return nil, jwt.ErrSignatureInvalid
5     }
6     return []byte(secret), nil
7 })
8 if err != nil || !token.Valid {
9     c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "invalid token"})
10    return
11 }
12
13 claims, ok := token.Claims.(jwt.MapClaims)
14 if !ok { /* 401 */ }
15
16 userIDFloat, ok := claims["user_id"].(float64)
17 if !ok { /* 401: invalid user_id in token */ }
```

Listing 8.3. JWT verification middleware with algorithm pinning and safe claim extraction (`internal/middleware/auth.go`).

The checks are: (1) the `Authorization` header must be present and begin with `Bearer` ; (2) the token's declared signing method must be an `*jwt.SigningMethodHMAC`—if it is not, the key callback returns `jwt.ErrSignatureInvalid` and the token is rejected, which closes the well-known `alg=none` forgery and the `RS256→HS256` confusion attack [40]; (3) `jwt.Parse` verifies the signature against the secret and validates `exp` (expiry checking is built into the v5 parser); (4) the claims must decode to a `jwt.MapClaims`; and (5) each claim is extracted with a comma-ok type assertion, so a malformed claim yields a clean 401 rather than a panic in the request goroutine. Because JSON numbers decode to `float64` in Go, `user_id` is read as a `float64` and converted to `uint`. Only after all five pass is the verified identity stashed on the Gin context with `c.Set`. A request that fails this check never reaches a handler; this is Gate 1 of the authorization model.

8.2.3 Password storage with `bcrypt`

Passwords are never stored in plaintext and never returned to clients—the `User.PasswordHash` field carries the struct tag `json:"-"`, so it is never serialized into any response. Hashing uses `bcrypt` at `bcrypt.DefaultCost`, which is cost 10, meaning $2^{10} = 1024$ internal iterations per hash [17]. `bcrypt` is deliberately slow and incorporates a per-hash salt automatically, so two users who chose the same password receive different stored hashes, defeating precomputed rainbow-table attacks. The same call hashes

on signup and on password reset, and login verifies with `bcrypt.CompareHashAndPassword`, which is constant-time within the `bcrypt` scheme and therefore does not leak how many characters matched through timing.

A subtle but deliberate defense is *account-enumeration resistance*: the login handler returns the identical message—`"invalid email or password"`—whether the email is unknown, the stored hash is empty (a Google-only account with no password set), or the password is simply wrong. An attacker cannot distinguish “no such account” from “wrong password,” so the error text cannot be used to harvest valid email addresses.

8.2.4 Google sign-in

APEX supports federated sign-in through Google Identity Services [41], which layers OpenID Connect [42] over OAuth 2.0 [43]. The frontend obtains an ID token from Google and POSTs it to `/auth/google` as `{id_token, role?}`. The backend verifies the token server-side with `google.golang.org/api/idtoken` against `GOOGLE_CLIENT_ID` (returning 503 if the client ID is unconfigured, so the feature fails closed when disabled) and requires both `email_verified` to be true and a non-empty subject before trusting any claim. Account resolution then proceeds in three steps: match by `google_id`; otherwise match by email and *link* the existing local account by stamping its `google_id`; otherwise create a new account. When the role is omitted for a brand-new account, the server returns `{needs_role: true, ...}` and the client re-submits with the chosen role, so a role is never silently defaulted. A Google-provisioned account has an empty `PasswordHash`, which is exactly why password login on such an account returns the same generic failure rather than an internal error.

8.2.5 Password reset

The password-reset flow is a self-contained story in fail-closed design, with six properties worth naming explicitly. First, the raw token is 32 bytes read from `crypto/rand`—a cryptographically secure source—and hex-encoded to 64 characters, making it unguessable. Second, the database never sees the raw token: only its SHA-256 hash is stored in the `token_hash` column (itself marked `json: "-"`). Plain SHA-256 is sufficient here, unlike for passwords, precisely because the token is already 256 bits of uniform randomness—there is nothing to dictionary-attack—whereas a human-chosen password needs `bcrypt`’s deliberate slowness. Third, the token expires one hour after issue. Fourth, requesting a new link invalidates all prior unused tokens for that user, so only the newest link works. Fifth, a token is single-use: its `used_at` timestamp is stamped on success and checked on every attempt, so it cannot be replayed even inside its one-hour window. Sixth, the `ForgotPassword` endpoint always returns the same 200 response—“If an account exists for that email, a reset link has been sent”—regardless of whether the email exists, extending the enumeration resistance of the login path to the reset path. If `SMTP_HOST` is unset, the reset link is logged to stdout instead of emailed, which keeps local development functional without a mail server.

8.3 Authorization: The Three-Gate Model

Authorization answers “are you allowed to do *this*?”. APEX enforces it with three independent gates, each of which fails closed, illustrated in Fig. 8.2. A request must clear all three before a handler mutates any state.

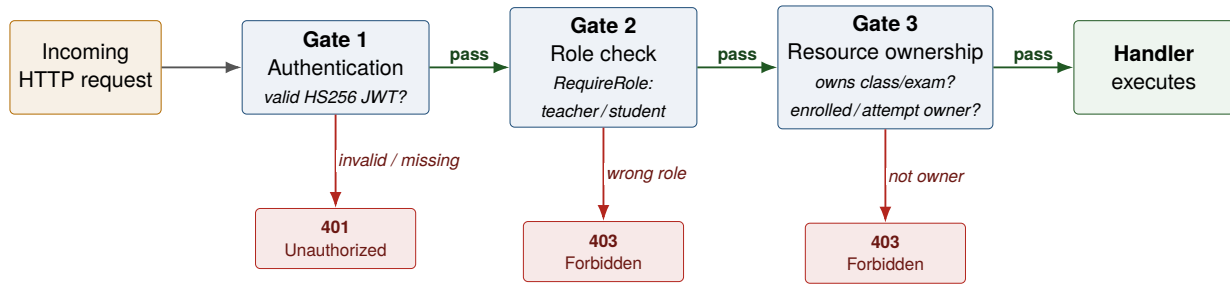


Figure 8.2. Three-gate authorization model.

Gate 1—Authentication. `middleware.Auth` (Listing 8.3) validates the HS256 token and sets `user_id`, `email`, and `role` on the context. A missing or invalid token is a 401, and the request never reaches a handler. This gate establishes *identity* only.

Gate 2—Role. `middleware.RequireRole(roles...)` reads the role from context and admits the request only if it matches one of the allowed roles; a missing role is a 401 and a wrong role is a 403 “insufficient permissions.” Feature subgroups apply it as an allow-list—`/exams` is `teacher-only`, `/student/*` is `student-only`, and `PUT /submissions/:id/grade` is `teacher-only`. APEX has exactly two roles, `teacher` and `student`; there is no `admin` role. A student calling a teacher-only route is stopped here at Gate 2.

Gate 3—Resource ownership. Gates 1 and 2 know *that* the caller is a real teacher, but neither knows *which* exam this is or *who owns it*. Only the handler—which has just loaded the row—knows. Gate 3 is therefore an in-handler check that the authenticated user owns or may access the specific resource, and it is the gate that cannot be removed. It is also the direct mitigation for the curious-student adversary and the fix for the IDOR finding (CWE-639 [39]) described in Section 8.6.

```

1  var problem models.Problem
2  if err := database.DB.Select("id", "exam_id", "teacher_id").
3      First(&problem, req.ProblemID).Error; err != nil {
4      c.JSON(http.StatusNotFound, gin.H{"error": "problem not found"})
5      return
6  }
7  userID := c.GetUint("user_id")
8  role, _ := c.Get("role")
9  if role == "teacher" {
10     if problem.TeacherID != userID {
11         c.JSON(http.StatusForbidden, gin.H{"error": "not authorized"})
12         return
13     }
14 } else {
15     if problem.ExamID == nil { /* 403 */ }
16     var assigned int64
17     database.DB.Model(&models.ExamClass{}).
18         Joins("JOIN class_members ON class_members.class_id = exam_classes.class_id").
19         Where("exam_classes.exam_id = ? AND class_members.user_id = ?",
20             *problem.ExamID, userID).
21         Count(&assigned)
22     if assigned == 0 {
23         c.JSON(http.StatusForbidden, gin.H{"error": "not authorized"})
24         return

```

```
25     }  
26 }
```

Listing 8.4. Gate 3 ownership check on the run-solution endpoint: teachers must own the problem; students must be enrolled in an assigned class (`internal/submission/handler.go`).

The problem is loaded by the request-supplied identifier, then the caller’s own identity—not any request field—drives the decision. A teacher may only run solutions against problems whose `teacher_id` equals their own; a student may only reach a problem that belongs to an exam assigned to a class in which they are enrolled, verified by a join across `exam_classes` and `class_members`. Zero matches is a 403. The same shape recurs across the codebase: `GradeSubmission` checks the parent exam’s `teacher_id`; `GetSubmission` lets a teacher read only submissions on exams they own and a student read only their own submission; `AssignExam` validates that every supplied `class_id` belongs to the calling teacher; and `DeleteAccount` scopes every cascade delete by `user_id` or `teacher_id` inside a single transaction that rolls back on any failure.

8.3.1 Why ownership checks are not centralized

A natural instinct is to factor every ownership check into one generic `RequireOwnership` middleware. APEX deliberately does *not*, because the rule genuinely differs per endpoint: an exam handler checks against `exam.teacher_id`, a submission handler against the parent exam’s `teacher_id`, and an announcement handler against `class.teacher_id` or class membership. Hiding these behind a generic helper makes the security-relevant rule less visible at the call site, which is the opposite of what such a check wants. The chosen trade-off is readable repetition over an abstraction that would be harder to audit—repetition one can read beats abstraction one cannot verify.

8.4 Input Handling

8.4.1 SQL injection

Every database access goes through GORM’s parameterized helpers [30], which bind values as query parameters rather than interpolating them into SQL text—structurally closing SQL injection (CWE-89 [25]). The clause `Where("email = ?", req.Email)` sends `req.Email` as a bound parameter, so a payload such as `' OR 1=1--` is treated as a literal string, never as SQL. The only raw `db.Exec` calls in the codebase live in `internal/database/migrations.go`, where the SQL is static and developer-authored, with no user input reaching it.

8.4.2 Cross-site scripting

The API returns JSON only—responses are produced by `c.JSON`, and there is no path that assembles an HTML document server-side, so reflected XSS through the API has no vector. On the client, React escapes text by default, and APEX never applies `dangerouslySetInnerHTML` to user-authored content such as announcement bodies, problem descriptions, MCQ option text, or exam titles. The one place raw markup could once execute was the Markdown renderer: the `rehype-raw` plugin re-parsed inline HTML inside a Markdown source, turning a problem description into a stored-XSS vector (CWE-79 [44]). That plugin was removed; Markdown still renders through `react-markdown` with `remark-gfm` for tables and task

lists, but raw HTML is now stripped at parse time. The trade-off—teachers can no longer hand-write raw HTML in a description—is acceptable, as Markdown is expressive enough for the intended content.

8.4.3 CSRF and the `localStorage` trade-off

Cross-Site Request Forgery (CWE-352 [45]) exploits the browser’s automatic attachment of ambient credentials—chiefly cookies—to cross-origin requests. APEX carries no such ambient credential: the JWT is stored in `localStorage` under the key `kernel-token` and is attached *explicitly* by application code as an `Authorization: Bearer` header on each request. A cross-site form or image cannot read `localStorage` or set an `Authorization` header, so there is no classic CSRF surface, and CORS (with `AllowCredentials: false`) further signals that no credentialed cross-origin access is expected.

This choice is a deliberate trade-off rather than a free win. Storing the token in `localStorage` trades CSRF resistance for XSS exposure: a successful XSS would let injected script read `localStorage` and exfiltrate the token, whereas an `httpOnly` cookie would be unreadable to script but would reintroduce CSRF concerns and demand anti-CSRF tokens plus same-site controls. The design was chosen for a simpler self-hosted deployment—a bearer header is trivial to reason about behind a same-origin proxy—and the residual XSS risk is narrowed by the `rehype-raw` removal above, by the absence of `dangerouslySetInnerHTML`, and by the 24-hour token expiry that bounds the blast radius of any stolen token. Hardening the design with `httpOnly`, same-site cookies and a refresh-token model is named explicitly in Section 8.7.

8.5 Sandbox Isolation for Untrusted Code

Student submissions are arbitrary programs in one of six languages—Python, JavaScript, TypeScript, Java, C, and C++—and are, by definition, hostile input. Executing them in the APEX process would be remote code execution by design: a submission could delete files, read other students’ data, open network connections, spin an infinite loop, or exhaust memory. APEX avoids this entirely by never executing student code itself. The `internal/judge0` package is purely an HTTP client: it marshals `{language_id, source_code, stdin}` to JSON, POSTs it to Judge0 [4], and reads a JSON verdict back, with a 30-second client timeout. There is no `os/exec`, no `eval`, and no shell anywhere in the package; an unsupported language short-circuits to a sentinel status and never reaches Judge0 at all.

Judge0 in turn runs each submission inside `isolate` [10], a sandbox built on Linux namespaces and cgroups—the same technology used to grade the International Olympiad in Informatics. Each run receives a throwaway filesystem that cannot read the host, no network access, a hard CPU-time limit, and a memory cap. APEX passes the per-problem `time_limit_ms` (default 2000 ms) and `memory_limit_kb` (default 262144 KB, i.e. 256 MB) as those bounds, so an infinite loop becomes a “time limit exceeded” verdict rather than a hung server, and a memory bomb is killed rather than allowed to crash the host. The most security-sensitive component in the system is thus one that APEX delegates entirely rather than builds—the safest code to run is code one does not run. The operational cost of this delegation is a production dependency on a self-hosted Judge0, since the default public CE instance at <https://ce.judge0.com> is rate-limited and unsuitable for real use.

8.6 Findings and OWASP Mapping

The scheduled pre-ship review found four issues, all reachable in the deployed code path and all fixed in a single commit before the system left development. Table 8.1 summarizes them. The unifying observation is that impact is not proportional to line count: each fix is a few lines, and each closes a real, reachable exploit path.

Table 8.1. Security findings from the pre-ship review, with severity, weakness class, and fix.

ID	Severity	Class	Finding and fix
F1	HIGH	IDOR (CWE-639)	<code>POST /submissions/run</code> trusted the request-supplied <code>problem_id</code> and leaked any problem’s sample tests. Fixed by the Gate 3 ownership check (Listing 8.4).
F2	HIGH	Stored XSS (CWE-79)	Markdown rendered with <code>rehype-raw</code> executed embedded raw HTML. Fixed by removing the plugin so raw HTML is stripped at parse time.
F3	MED	Weak secret	<code>config.Load</code> defaulted <code>JWT_SECRET</code> to a publicly-known placeholder. Fixed by the fail-closed validator (Listing 8.1).
F4	MED	HTML injection	Notification-email bodies interpolated unescaped user fields. Fixed by passing them through <code>html.EscapeString</code> .

Mapped onto the OWASP Top 10 [19] and cross-checked against the Application Security Verification Standard [46], the posture is summarized in Table 8.2. The two categories that dominate a multi-tenant classroom portal—broken access control and injection—are the ones the design invests most heavily in.

Table 8.2. APEX posture against the OWASP Top 10 categories.

Category	APEX posture
A01 Broken Access Control	Three-gate model; Gate 3 per-resource ownership checks in every handler (fixed F1/IDOR).
A02 Cryptographic Failures	<code>bcrypt</code> cost 10 for passwords; SHA-256-hashed, single-use reset tokens; HS256 tokens signed with a validated ≥ 32 -char secret.
A03 Injection	GORM parameterized queries (SQLi); JSON-only API and raw-HTML stripping (XSS, fixed F2).
A05 Security Misconfiguration	Fail-closed <code>JWT_SECRET</code> validation (fixed F3); secrets kept out of source control.
A07 Identification & Auth Failures	Algorithm-pinned JWT verification (closes <code>alg=none</code>); enumeration-resistant login and reset.
A08 Software & Data Integrity	<code>html.EscapeString</code> on email bodies (fixed F4); transactional, idempotent destructive operations.

Categories such as A04 (Insecure Design) and A09 (Logging Failures) map onto the open gaps of the next section rather than to closed controls.

8.7 Known Limitations

An honest report names what it has not closed. Four gaps are on the roadmap, each stated as risk, current mitigation, and intended fix.

No rate limiting on the auth endpoints. Login, signup, password-reset requests, and Google sign-in are not rate-limited, so a determined adversary could attempt credential stuffing with bcrypt's cost 10 as the only brake on each guess. The enumeration-resistant identical responses make targeting harder, but the correct fix is a per-IP and per-account rate limit at the auth endpoints—the most cost-effective next mitigation—or fronting the API with a WAF in production.

Six-character password minimum. The only complexity rule is `len(password) >= 6`, enforced at both signup and reset, with no character-class or breached-password check. This falls short of contemporary guidance, which favors longer minimums and screening against known-breached secrets rather than composition rules [47], and it compounds the rate-limiting gap. The minimum should be raised before the instance is exposed to the public internet.

JWT in `localStorage` with no revocation. As discussed in Section 8.4, a leaked token remains valid until its `exp` claim because there is no server-side revocation list, and `localStorage` storage means a successful XSS could lift the token. The 24-hour expiry bounds the blast radius today; the roadmap fix is a refresh-token model with a server-side revocation table and a move to `httpOnly`, same-site cookies.

Fire-and-forget grading. Each submission is graded in a bare goroutine—`go judge0.Grade(...)`—with no worker queue and no retry. If the process crashes mid-grade, the submission is left stuck in `running` forever, and because attempt finalization blocks while any sibling submission is pending or running, the whole attempt never finalizes. This is an availability and data-integrity gap rather than a confidentiality breach. The aggregator is idempotent, so the fix is a durable work queue with a bounded worker pool and a recovery sweep that re-grades anything left in `running`.

Beyond these, APEX keeps no audit log of logins and resets (fix: an append-only audit table), performs no plagiarism detection, and emits no proctoring signals. Each is a bounded, prioritized roadmap item with a known remedy—a documented gap is preferable to a hidden one.

Chapter 9

Testing and Quality Assurance

Quality assurance for APEX rests on two pillars that must be described plainly and separately. The first is the *automated quality gate*: a GitHub Actions pipeline that compiles, statically analyses, lints, builds, and container-smoke-tests every commit before it is allowed to reach the `main` branch. This gate is mature, fast, and enforced. The second is the *test suite itself*: at the thesis snapshot the backend ships no `*_test.go` files and the frontend ships a single sanity test. These two facts sit in tension — a rigorous pipeline running over an almost-empty suite — and this chapter confronts that tension rather than papering over it. Following Sommerville’s guidance that verification and validation are distinct activities that together establish fitness for purpose [23], we first describe the strategy and the gate, then give an unflinching assessment of coverage and a concrete plan for the tests that the architecture most needs.

9.1 Test Strategy

The testing philosophy for APEX is organised around the classical test pyramid [33]: a broad base of fast, isolated unit tests, a narrower band of integration tests that exercise real collaborators (the HTTP router and a live PostgreSQL instance), and a thin cap of end-to-end and manual exploratory testing. Figure 9.1 contrasts this aspirational shape with what the repository actually contains today, and the gap between the two is itself a finding this chapter reports.

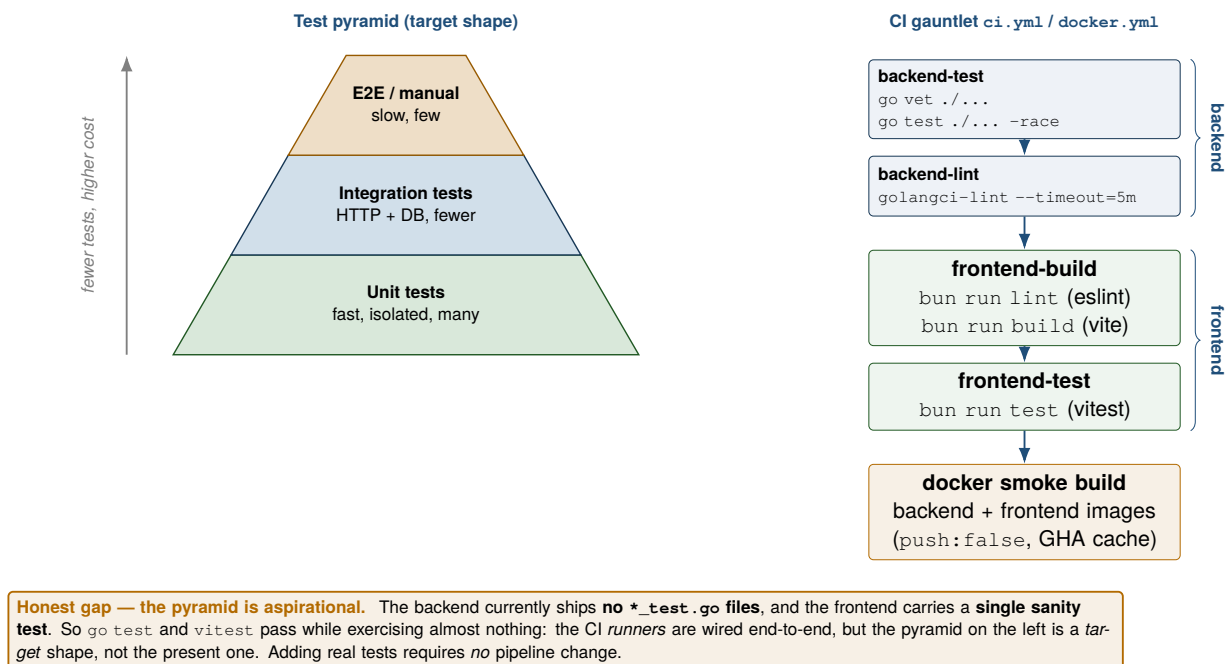


Figure 9.1. Quality gates: CI pipeline and the test pyramid.

Pressman frames test planning as the deliberate allocation of effort to where defects are both most likely and most costly [24]. For a system like APEX, that allocation is not uniform. Large parts of the codebase are thin CRUD handlers whose failure modes are shallow and would be caught immediately in manual use; other parts — the grading determinism logic, the three authorization gates, output normalization, and the migration pipeline — carry correctness obligations whose violations are silent, data-corrupting, and hard to detect by inspection. The `is_draft` incident (Chapter 5) is the canonical example: a schema-default migration silently hid every existing exam from students with no error logged, and a single migration-idempotency assertion on row counts would have caught it. The strategy this chapter advocates therefore weights unit and integration effort toward exactly those silent-failure hotspots.

9.1.1 The Test Pyramid Applied to APEX

Mapping the three pyramid tiers onto APEX’s concrete surfaces gives the following picture.

- **Unit tier (base).** Pure functions with no I/O: `normalizeOutput` (the symmetric whitespace normalizer applied to both actual and expected program output), the Judge0 status-code mapper, the MCQ set-equality comparison in `GradeMCQ`, the attempt-aggregation arithmetic in `maybeFinalizeAttempt`, the JWT claim extraction, and the frontend’s form validators and score formatters. These are cheap to test, deterministic, and where the subtle correctness bugs live.
- **Integration tier (middle).** Handlers exercised through the real Gin router against a real PostgreSQL database — precisely the shape the `CI backend-test` job already provisions via a `postgres:16-alpine` service container. This tier is where the three authorization gates, resource-ownership checks, and the migration passes should be verified end to end within the process.
- **End-to-end / manual tier (cap).** Full browser-driven flows through the React SPA, the Monaco editor, and live Judge0 execution. Because Judge0 is an external HTTP dependency and the editor is heavyweight, this tier is deliberately thin and, in the current snapshot, entirely manual (Section 9.5).

The deliberate inversion visible in Fig. 9.1 — almost no unit or integration tests, with confidence resting instead on static analysis and manual use — is a known anti-pattern (the “ice-cream cone”). Section 9.4 treats it as the central finding of the chapter.

9.2 The Continuous Integration Pipeline

Every push to `main` and every pull request triggers two GitHub Actions workflows [48]: `ci.yml` (compilation, static analysis, lint, build, and test) and `docker.yml` (image smoke builds). CI is configured with a concurrency group keyed on the git ref and `cancel-in-progress: true`, so a rapid sequence of pushes to the same branch cancels superseded runs and only the newest commit consumes runner minutes. Together the two workflows constitute a six-leg gauntlet that a change must clear before merge. Table 9.1 summarises the jobs.

9.2.1 The Backend Jobs

The `backend-test` job runs on `ubuntu-latest` with Go 1.25 and provisions a `postgres:16-alpine` service container (database `apex_test`, `user/password` `postgres/postgres`). The service is health-gated with `pg_isready` at a five-second interval, a three-second timeout, and ten retries, so the test steps only begin once the database accepts connections. Two environment

Table 9.1. Continuous-integration jobs and their gates.

Job	Workflow	What it runs
backend-test	ci.yml	go mod download, go vet ./..., go test ./... -race -count=1 against a live Postgres service.
backend-lint	ci.yml	golangci-lint via golangci-lint-action@v6, --timeout=5m.
frontend-build	ci.yml	bun install --frozen-lockfile, bun run lint (ES- Lint), bun run build (Vite).
frontend-test	ci.yml	bun install --frozen-lockfile, bun run test (Vitest single run).
build-backend	docker.yml	Docker smoke build of the distroless backend image (push:false, GHA layer cache).
build-frontend	docker.yml	Docker smoke build of the nginx frontend image (push:false, GHA layer cache).

values matter: `DATABASE_URL` points the application and migration code at the service container (`postgres://postgres:postgres@localhost:5432/apex_test?sslmode=disable`), and `JWT_SECRET` is set to the literal `ci-secret-thirty-two-chars-min-length-ok`. That secret is not cosmetic: APEX refuses to start if `JWT_SECRET` is empty, shorter than 32 characters, or equal to the placeholder `dev-secret-change-in-prod`, so the CI value is deliberately crafted to be exactly 32-plus characters of non-placeholder text. Any future integration test that boots the app inherits a valid signing key and a real database for free.

The three backend steps run in order and each is a hard gate:

1. `go mod download` — resolves and caches modules; `setup-go@v5` is configured with `cache: true` so the module and build caches persist across runs.
2. `go vet ./...` — Go’s built-in suspicious-construct analyser. It catches `printf-format` mismatches, unreachable code, struct-tag typos, and lost-cancel context bugs before they reach review.
3. `go test ./... -race -count=1` — runs the full test suite with the race detector enabled and caching disabled. The `-race` flag is particularly relevant to APEX because grading runs in fire-and-forget goroutines (`go judge0.Grade(...)`); once real grading tests exist, the race detector will flag any unsynchronised access between a grading goroutine and the aggregation path. `-count=1` disables Go’s test-result cache so the suite genuinely re-executes on every run.

The `backend-lint` job is a separate parallel job (so a lint failure does not mask a test failure or vice-versa) running `golangci-lint` through the official `golangci-lint-action@v6` with `--timeout=5m`. Its behaviour is pinned by the repository’s `.golangci.yml`, discussed next.

9.2.2 Static Analysis as a Quality Gate

The project takes a deliberate, curated stance on linting rather than enabling every available check. The `.golangci.yml` file sets `disable-all: true` and then enables exactly six linters, so the gate is explicit and stable rather than dependent on whatever the linter’s default set happens to be in a given release:

- `govet` — the same vet analysers as the test job, run again in the lint context (belt and suspenders).
- `staticcheck` — the flagship Go static analyser; finds dead code, incorrect `sync` usage, impossible type assertions, redundant conversions, and a large catalogue of correctness smells.

- `errcheck` — flags unchecked error returns, with `check-type-assertions: false` and `check-blank: false` so that intentionally discarded comma-ok assertions and blank-assigned returns do not create noise.
- `ineffassign` — detects assignments to variables that are never subsequently read (a common source of silent logic bugs).
- `unused` — reports unused constants, functions, types, and struct fields.
- `gosimple` — suggests simpler equivalent constructs, keeping the codebase idiomatic.

Two more settings shape the gate. `run.tests: true` means the linters also analyse `_test.go` files, so test code is held to the same standard as production code — a forward-looking choice given that the backend suite is yet to be written. And `issues.exclude-dirs` removes `frontend`, `book`, `photos`, and `seed` from analysis, so the Go linter never wastes time on the TypeScript SPA, the thesis LaTeX sources, screenshot assets, or the committed seed binary. Curating the linter set this way trades a slightly smaller catalogue of warnings for a gate that is comprehensible, reproducible, and free of the false-positive churn that leads teams to ignore lint output entirely.

9.2.3 The Frontend Jobs

The two frontend jobs both run inside the `frontend/` working directory and both pin Bun to 1.1.38 via `oven-sh/setup-bun@v2`. The pin is intentional and documented: CI and the Netlify build are locked to the same Bun version so that the lockfile resolves and the build behaves identically in both places, eliminating a class of “works in CI, breaks on deploy” drift. Both jobs begin with `bun install --frozen-lockfile`, which fails the build if `bun.lockb` is out of sync with `package.json` rather than silently resolving new versions — a supply-chain and reproducibility safeguard.

The `frontend-build` job then runs `bun run lint` followed by `bun run build`. The lint step invokes ESLint through the flat config in `eslint.config.js`, which composes the recommended JavaScript rules with `typescript-eslint`’s recommended set and adds the `react-hooks` plugin (enforcing the rules of hooks) and `react-refresh` (warning on non-component exports that would break fast refresh). The build step runs `vite build`, a full production TypeScript compile and bundle; because TypeScript is a static type system [31], a successful `vite build` is itself a substantial correctness check — type errors, missing imports, and undefined identifiers all fail the build. In a repository of roughly 17,400 lines of TypeScript, that compile-time guarantee does a large share of the verification work that a dynamically typed frontend would need runtime tests to approximate.

The `frontend-test` job runs `bun run test`, mapped in `package.json` to `vitest run` — a single, non-watch execution suitable for CI. The Vitest configuration (`vitest.config.ts`) registers the SWC React plugin, sets the `jsdom` environment (so component tests get a simulated DOM), enables `globals` (so `describe/it/expect` need no import), loads `./src/test/setup.ts` before the suite, and includes any `src/**/*.{test,spec}.{ts,tsx}` file. The setup file imports `@testing-library/jest-dom` for its extended DOM matchers and stubs `window.matchMedia` — which `jsdom` does not implement but which the Tailwind/responsive components query at render time — so component tests do not crash on a missing browser API. The infrastructure is therefore fully wired for real component and hook tests; only the tests themselves are missing.

9.2.4 The Docker Smoke Build

The `docker.yml` workflow runs on the same push/PR triggers and smoke-builds both container images with `docker/build-push-action@v6`: `build-backend` builds the root `Dockerfile` (the multi-stage distroless backend) and `build-frontend` builds `frontend/Dockerfile` (the `oven/bun` build stage into an `nginx` runtime). Both set `push: false` — the images are built and discarded, never published — and both use the GitHub Actions layer cache (`cache-from/cache-to: type=gha`) to keep rebuilds fast. The purpose is narrow but valuable: it proves that the production images still *build* on every change. As the workflow’s own comment notes, PR-level Docker builds catch lockfile drift and Go-module changes that a source-only test job would miss, because the image build performs a clean `--frozen-lockfile` install and a from-scratch compile in an environment with none of a developer’s local caches. This is a build gate, not a runtime gate: it does not start the containers or exercise `/healthz`, so a change that builds but fails at runtime would pass this leg (see Section 9.4).

9.3 Validation Against Requirements

Verification asks “are we building the product right?”; validation asks “are we building the right product?” [23]. The automated gate above is almost entirely verification. Validation — that the system satisfies the functional and non-functional requirements of Chapter 3 — is currently established by manual and exploratory testing mapped back to the requirement set. Each major requirement traces to an observable system behaviour that was exercised by hand during development:

- *Mixed-format exams in one sitting* — validated by building an exam containing coding, MCQ, and written questions, assigning it to a class, and taking it as a student; the single grading pipeline produces one aggregated attempt score.
- *Auto-grading correctness* — validated by submitting known passing and failing solutions and confirming the per-test breakdown, the `passed_count / total_count * 100` score, and the terminal submission status match expectation.
- *Authorization* — validated by attempting teacher-only routes as a student and cross-tenant access to another teacher’s resources, confirming 401/403 responses (Fig. 8.2).
- *Reminder scheduling* — validated by setting an exam start time inside each sweep window and confirming exactly one in-app notification and one email (logged to stdout when SMTP is off) fire, with the column stamps preventing re-send across a restart.

This requirement-to-behaviour mapping is genuine validation, but it is manual and non-repeatable. The clear next step, described below, is to promote the highest-value items in this list into the automated integration tier so that validation runs on every commit rather than only when a developer remembers to perform it.

9.4 Honest Assessment of Current Coverage

The candid position, stated in the project README and worth repeating here without softening, is that **test coverage is minimal**. The Go backend — roughly 6,100 lines across thirteen route modules — contains *no* `*_test.go` files, so `go test ./...` in CI currently compiles the packages and reports “no test files” rather than exercising behaviour. The frontend ships exactly one test, `src/test/example.test.ts`, whose entire body asserts `expect(true).toBe(true)`. It is a genuine sanity check — it proves the

Vitest runner, the jsdom environment, and the setup file all load and execute — but it verifies nothing about the application.

It is important to be precise about what this does and does not mean. The CI pipeline is *not* theatre: `go vet`, the six-linter `golangci-lint` gate, the TypeScript compile inside `vite build`, ESLint, and the two Docker builds all perform real, enforced checks on every commit, and static typing plus static analysis catch a substantial class of defects that unit tests would otherwise be needed to find [24]. What is missing is the *behavioural* layer — the assertions that a given input produces a given output — and its absence is felt most acutely in exactly the silent-failure hotspots identified in the strategy. The remainder of this section names those hotspots and specifies how each should be tested, so that the gap is documented as an actionable backlog rather than a vague admission.

9.4.1 What Should Be Tested, and How

Grading determinism (unit). The coding-grade path must be deterministic: identical code, language, and test cases must always yield the same status, score, and per-test results. The status-code mapper (Judge0 3/4 counted as “ran and compared”; 6→`compilation_error`, 5→`time_limit_exceeded`, 11→`runtime_error`, else `wrong_answer`) is a pure function of the Judge0 response and should be table-tested against every documented status ID, including the unknown-status default. The scoring arithmetic and the “no test cases → auto-accepted, score 100.0” edge case should each be asserted directly. Because these tests need no network, they can hit the mapper and scorer functions in isolation while a fake Judge0 client returns canned responses.

Output normalization (unit). `normalizeOutput` is the single most leverage-rich unit-test target in the codebase: a bug here silently marks correct submissions wrong or vice-versa across every coding question. Its four documented transformations — `\r\n`→`\n`, lone `\r`→`\n`, right-trim of each line, and a final whole-string trim — must each be asserted, along with the property that normalization is *symmetric* (applied identically to actual and expected output) and *idempotent* (`normalize(normalize(x)) == normalize(x)`). A representative table appears in Listing 9.1.

```

1 func TestNormalizeOutput(t *testing.T) {
2     cases := []struct{ name, in, want string }{
3         {"crlf",          "a\r\nb",      "a\nb"},
4         {"lone cr",        "a\rb",        "a\nb"},
5         {"trailing ws",    "a \nbt\n",    "a\nb"},
6         {"outer trim",     "\n a\nb \n",  "a\nb"},
7         {"idempotent",     "a\nb",        "a\nb"},
8     }
9     for _, c := range cases {
10        t.Run(c.name, func(t *testing.T) {
11            if got := normalizeOutput(c.in); got != c.want {
12                t.Fatalf("normalizeOutput(%q) = %q, want %q", c.in, got, c.want)
13            }
14            // symmetry/idempotence: normalizing twice is a fixed point
15            if got := normalizeOutput(c.want); got != c.want {
16                t.Fatalf("not idempotent: %q -> %q", c.want, got)
17            }
18        })
19    }

```

20 }

Listing 9.1. Sketch of the table-driven unit tests `normalizeOutput` should carry.

Authorization gates (integration). The three-gate model — authentication, role, and resource ownership — is a security invariant and belongs in the integration tier, exercised through the real Gin router against the CI Postgres service. The matrix of cases is small and enumerable: a missing or malformed bearer token yields 401; a token signed with the wrong algorithm (non-HMAC) is rejected; a valid student token on a teacher-only route yields 403; a valid teacher token accessing *another* teacher’s exam or submission is denied by the ownership check. Each of these maps to a request the test can issue and an HTTP status it can assert, and each guards against a regression whose real-world consequence is a data-exposure or privilege-escalation bug.

Attempt aggregation (unit/integration). The `maybeFinalizeAttempt` logic carries several subtle rules that are easy to break: it must return early while any sibling submission is still `pending/running`; it must re-aggregate over the exam’s *full* problem list so skipped questions count zero in the denominator; and it must *exclude* `pending_review` submissions from both numerator and denominator so an in-flight written grade does not depress the visible total. Each rule is a distinct test with a hand-built set of submission rows and an asserted final `ExamAttempt.score`. The “Exam Graded” notification’s once-only, `graded_notified-deduped` firing is a companion assertion.

Migration idempotency (integration). This is the test whose absence caused the `is_draft` incident (Chapter 5). Against a fresh Postgres database, the migration passes should run *twice* in succession with the second run a no-op, and — critically — a test should seed exam rows, run the migration, and assert that *row counts and column values are unchanged* for pre-existing data (Rule 4 of the project’s `ORM_RULES.md`: verify row counts after migrating). Because the CI job already provisions a real Postgres service, this integration test needs no new infrastructure — only the test code.

Frontend components and hooks (unit/component). With the Vitest, jsdom, and Testing Library stack already configured, the frontend backlog is component and hook tests for the high-risk surfaces: the Zod form schemas in the exam builder, the score and duration formatters, the React Query hooks’ loading/error states, and the auth store’s 401-handling (clear state and redirect to `/auth`). These replace the current single tautological assertion with tests that would actually fail if the behaviour regressed.

Because CI already *runs* both test commands, none of this backlog requires a pipeline change — the runners are wired and waiting, which is the whole point of standing up the gate before the suite. Adding a `grade_test.go` or a `*.test.tsx` file is sufficient for it to execute on the next commit.

9.5 Manual and Exploratory Testing

Given the state of the automated suite, manual and exploratory testing carries much of the day-to-day validation load, and it was conducted throughout development in a structured way. Two harnesses support it. The `./start.sh` script brings up the full dev topology with one command — Postgres in Docker, the Go backend via `go run` (so the latest source is always exercised), and the Vite dev server proxying `/api`

to the backend — and drops the developer at the login page, making it cheap to reproduce a scenario end to end. The distroless production image is validated separately through `docker compose up`, which brings up the four-service stack (`postgres` → `one-shot migrate` → `backend` → `nginx`) in dependency order and lets the built binary's `--healthcheck` self-probe confirm liveness without a shell.

Exploratory sessions followed recognisable heuristics: role-swapping (perform every teacher action, then attempt it as a student), boundary probing (empty exams, exams with no test cases, MCQ questions with no correct answers configured, written questions routed to the manual queue), and failure injection (submitting code that times out or throws, to confirm the status mapping and the per-test breakdown). Several of the project's documented gaps were *found* this way rather than reasoned about in the abstract — most notably that a process crash mid-grade leaves a submission stuck in `running` and the attempt never finalizes (the fire-and-forget grading model of Chapter 7). Discovering that failure mode through exploratory use, and documenting it rather than hiding it, is itself a quality-assurance outcome, and it directly motivates the durable-work-queue item on the backlog.

Manual testing is not a substitute for the automated behavioural suite that Section 9.4 specifies — it is not repeatable, it does not run on every commit, and it scales with developer time rather than with CI minutes. But paired with a strong static gate and disciplined gap-tracking, it provided sufficient validation confidence for a graduation-project snapshot, while leaving a clearly enumerated path from the current ice-cream-cone shape toward the pyramid that Fig. 9.1 depicts.

9.6 Summary

APEX's quality assurance is a story of an asymmetry stated plainly: a mature, enforced, six-leg CI gauntlet — `go vet`, a curated six-linter `golangci-lint` gate, the race-enabled test runner against a live Postgres service, ESLint, a full TypeScript compile via `vite build`, Vitest, and two Docker smoke builds, all with Bun pinned to 1.1.38 for reproducibility — running over a behavioural suite that is, at the thesis snapshot, essentially empty. The static and type-level gates catch a real and substantial class of defects, and structured manual and exploratory testing validated the system against its requirements and surfaced its documented failure modes. The real deficit is the behavioural tier, and this chapter has converted that deficit into a concrete, prioritised backlog — grading determinism, output normalization, the authorization gates, attempt aggregation, and migration idempotency first — that the already-wired pipeline will execute the moment the tests are written.

Chapter 10

Deployment and DevOps

This chapter documents how APEX runs: how a developer brings the system up on a laptop, how the same source is packaged into container images for a production-shaped deployment, how those images are wired together into a four-service Docker Compose stack, how the system is configured entirely through the environment, and how new code reaches an image safely through continuous integration. The guiding principle throughout is deliberate boredom. APEX is a graduation project that a single instructor should be able to self-host, so the deployment target is one Go binary plus a managed PostgreSQL — not a Kubernetes cluster, not a service mesh, not a message broker. Every operational decision in this chapter is made in service of that constraint, and where a shortcut has been taken the trade-off is named rather than hidden.

10.1 One Codebase, Three Binaries, Two Run-Modes

The backend is a single Go module (`module apex, Go 1.25.3`). From that one module the build produces three small, statically linked binaries, each serving a distinct operational role:

- `apex` — the long-running HTTP API server, which also hosts the in-process reminder scheduler. This is the `backend` service.
- `migrate` — a one-shot schema tool that applies the versioned SQL migration track and exits. This is the `migrate` service.
- `seed` — an on-demand demo-data loader, run manually and not wired into the Compose stack at all.

That same source runs two ways. In *local development* it runs from source under `go run` for fast iteration, alongside the Vite dev server and a containerized PostgreSQL. In a *production-shaped deployment* it runs as the compiled distroless image, behind an nginx image serving the built single-page application (SPA), with the one-shot `migrate` container guaranteeing the schema is current before the server ever accepts a request. Keeping a single codebase behind both modes is what makes the local experience representative: the same configuration loader, the same migration SQL, and the same health endpoint are exercised in both.

10.2 Local Development

The repository ships a single bring-up script, `start.sh`, whose job is to make a clean checkout runnable with one command. Its responsibilities, in order, are: copy `.env.example` to `.env` on first boot; bring up the Compose PostgreSQL service and wait for it to become healthy; start the Go backend from source in the background; poll `http://localhost:8080/api/auth/login` up to thirty times (once per second) until the port answers; and finally start the frontend dev server, preferring Bun and falling back to npm. A single `trap` on `EXIT`, `INT`, and `TERM` tears the whole thing down — the two background processes are killed and the Compose stack is brought down — so one `Ctrl+C` cleans up after itself.

Two details are worth calling out. First, the `.env` bootstrap copies the example file verbatim, and the example `JWT_SECRET` (`replace-me-with-a-strong-random-secret-32+chars`) is deliberately long enough to clear the 32-character validator described in Section 10.6, so a fresh checkout boots without hand-editing. Second, the backend is started with `go run main.go` rather than a prebuilt binary, so every restart picks up the latest edits — the entire point of a development mode.

The resulting development topology is three host processes plus one container: the Vite dev server on `:5173` proxies `/api` requests to the Go binary on `:8080`, which speaks to PostgreSQL on host port `:5433` (the container still listens on `:5432`; the host is remapped because `5432` is frequently already occupied by a locally installed PostgreSQL). Judge0 is the public hosted instance at `ce.judge0.com`, and SMTP is off, so emails are logged to stdout.

The script is intentionally readable rather than idempotent. It is a way to bring the system up once, not a service supervisor: running it twice without stopping the first run clashes on the fixed ports. Developers who prefer split terminals can reproduce the same effect manually — `docker compose up -d postgres`, `go run main.go` in one shell, and `bun run dev` (or `npm run dev`) in another — and this is the recommended workflow when actively iterating on either half of the stack.

10.3 Container Images

Both halves of the system are packaged as multi-stage Docker images. The multi-stage pattern separates a heavy *build* stage — which carries the compiler and toolchain — from a minimal *runtime* stage that receives only the finished artifacts. The toolchain never ships, which is what keeps the final images small and their attack surface narrow.

10.3.1 Backend: Multi-Stage to Distroless

The backend Dockerfile compiles all three binaries in a `golang:1.25-alpine` stage, then copies them into a `gcr.io/distroless/static-debian12:nonroot` runtime stage. Listing 10.1 reproduces it in full.

```
1 # syntax=docker/dockerfile:1.7
2
3 FROM golang:1.25-alpine AS build
4 WORKDIR /src
5
6 RUN apk add --no-cache git ca-certificates
7
8 COPY go.mod go.sum ./
9 RUN go mod download
10
11 COPY . .
12
13 ENV CGO_ENABLED=0 GOOS=linux
14
15 RUN go build -trimpath -ldflags="-s -w" -o /out/apex ./
16 RUN go build -trimpath -ldflags="-s -w" -o /out/migrate ./cmd/migrate
17 RUN go build -trimpath -ldflags="-s -w" -o /out/seed ./cmd/seed
18
19 FROM gcr.io/distroless/static-debian12:nonroot AS runtime
```

```

20 WORKDIR /app
21
22 COPY --from=build /out/apex /app/apex
23 COPY --from=build /out/migrate /app/migrate
24 COPY --from=build /out/seed /app/seed
25 COPY --from=build /src/internal/database/migrations /app/internal/database/migrations
26
27 ENV PORT=8080
28 EXPOSE 8080
29
30 USER nonroot:nonroot
31 ENTRYPOINT ["/app/apex"]

```

Listing 10.1. The backend multi-stage Dockerfile: three static binaries copied into a distroless runtime.

Several deliberate choices are packed into these thirty lines. Copying `go.mod` and `go.sum` and running `go mod download` *before* `COPY . .` is a layer-caching optimization: the dependency download is a cached layer that only re-runs when the module manifests change, so ordinary source edits reuse it and rebuild quickly. The pair `CGO_ENABLED=0 GOOS=linux` produces a fully static binary with no dependency on the system C library, which is precisely what makes a `distroless/static` base — a base that ships no `libc` — viable. The build flags `-trimpath` (strip absolute filesystem paths for reproducibility and to avoid leaking the builder’s home directory) and `-ldflags="-s -w"` (strip the symbol table and DWARF debug information) shrink each binary at the acceptable cost of less informative crash traces.

The runtime stage is the security-relevant half. A distroless image contains essentially only the application binary and the CA certificates needed to make outbound HTTPS calls; there is no shell, no package manager, and no `curl`. If an attacker were to achieve code execution inside the container, none of the usual pivoting tools would be present. The `:nonroot` tag together with `USER nonroot:nonroot` runs the process as an unprivileged user, so a container breakout is far less dangerous — a straightforward application of least privilege [13], [49]. The four `COPY --from=build` lines carry out only the three binaries plus the `internal/database/migrations` directory, so the `migrate` binary can find its versioned SQL at `/app/internal/database/migrations/sql` at runtime. The default `ENTRYPOINT` runs `/app/apex`, the server; the Compose `migrate` service overrides it to `/app/migrate up`, reusing the exact same image for a different role.

The shell-less runtime has one important consequence: the conventional container health check — `curl http://localhost/healthz` run inside the container — is impossible, because neither `curl` nor a shell exists. Section 10.5 describes the self-probing mechanism that resolves this.

10.3.2 Frontend: Bun Build to Nginx

The frontend Dockerfile follows the same multi-stage shape. An `oven/bun:1-alpine` stage installs dependencies with `bun install --frozen-lockfile` — which installs exactly what the committed `bun.lock` specifies and fails if resolution would change, guaranteeing reproducible installs — and runs `bun run build` to produce the static `dist/` bundle. The runtime stage is a thin `nginx:1.27-alpine` that copies `dist/` into the web root and drops in the project’s `nginx.conf`. No Node, no Bun, and no source travel into the final image; it is static files plus `nginx`.

A subtlety here is that the four `VITE_*` values are consumed as Docker build arguments and baked into the JavaScript bundle at build time via `ARG/ENV`, not read at runtime. Changing any of them — for

instance repointing `VITE_API_URL` or `VITE_GOOGLE_CLIENT_ID` — therefore requires rebuilding the frontend image. The default `VITE_API_URL` of `/api` is what allows a single built bundle to work both behind the dev proxy and behind the production single-origin reverse proxy, because in both cases the SPA simply issues same-origin requests to `/api`.

The `nginx.conf` enforces three behaviours. Content-hashed assets under `/assets/` are served with a one-year immutable cache header, which is safe precisely because Vite gives each build unique filenames. Requests to `/api/` are reverse-proxied to `http://backend:8080` on the internal Compose network, with `X-Forwarded-*` headers preserving client information; this makes the deployed app single-origin, so the browser only ever talks to the frontend host. Finally, the catch-all location `/` falls back to `index.html` via `try_files`, so a browser reload on a client-side route such as `/dashboard/exams` serves the SPA shell and lets React Router resolve the path rather than returning a 404.

10.4 The Four-Service Compose Stack

Figure 10.1 draws the production-shaped Docker Compose topology. There are exactly four services — `postgres`, `migrate`, `backend`, and `frontend` — plus one named volume for database persistence. Judge0 is deliberately *not* a Compose service: it is the most security-sensitive component of the system, so it is treated as an external HTTP dependency reached through the `JUDGE0_URL` environment variable, whether that points at the public `ce.judge0.com` or a self-hosted instance. SMTP is likewise external.

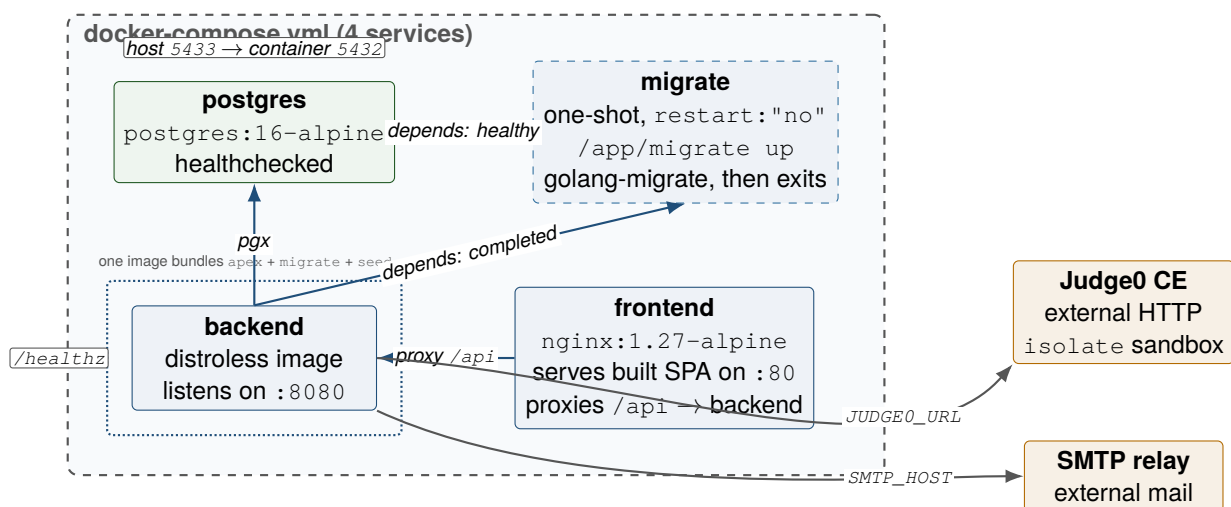


Figure 10.1. Container topology of the four-service Docker Compose stack.

The single most important property of the stack, visible in Fig. 10.1, is its startup ordering, expressed through Compose's `depends_on` conditions:

1. `postgres` (`postgres:16-alpine`) starts first. It carries a `pg_isready` health check on a five-second interval with up to ten retries, and persists its data in the named `pgdata` volume so the database survives `docker compose` down. The host-to-container port mapping is `5433→5432`; other services still reach it internally at `postgres:5432` regardless of the host remap.
2. `migrate` starts only once PostgreSQL reports `service_healthy`. It is built from the same backend image but overrides the entrypoint to `["/app/migrate", "up"]`, applies the versioned SQL migrations, and exits. It is marked `restart: "no"` so Compose does not treat its clean exit as a crash to be restarted — it is a genuine one-shot job.

3. backend starts only after *both* postgres is service_healthy *and* migrate has reported service_completed_successfully. This is the ordering guarantee that matters: the schema is provably current before the application ever serves a request, eliminating any race against a half-migrated database. The backend publishes port 8080 and carries its own health check.
4. frontend (nginx:1.27-alpine) starts after the backend and publishes 8081→80. In the deployed stack the user-facing origin is therefore `http://localhost:8081`, and the API is at `http://localhost:8080`.

Listing 10.2 shows the two services that carry this ordering, abbreviated to the load-bearing keys.

```

1  migrate:
2    build:
3      context: .
4    entrypoint: ["/app/migrate", "up"]
5    environment:
6      DB_HOST: postgres
7      DB_PORT: "5432"
8      JWT_SECRET: ${JWT_SECRET}
9      MIGRATIONS_DIR: file:///app/internal/database/migrations/sql
10   depends_on:
11     postgres:
12       condition: service_healthy
13   restart: "no"
14
15  backend:
16    build:
17      context: .
18    environment:
19      DB_HOST: postgres
20      DB_PORT: "5432"
21      PORT: "8080"
22      JWT_SECRET: ${JWT_SECRET}
23      JUDGE0_URL: ${JUDGE0_URL:-https://ce.judge0.com}
24      APP_URL: ${APP_URL:-http://localhost:8081}
25    ports:
26      - "8080:8080"
27    depends_on:
28      postgres:
29        condition: service_healthy
30      migrate:
31        condition: service_completed_successfully
32    healthcheck:
33      # Distroless image has no shell/curl -- self-probe via the binary.
34      test: ["CMD", "/app/apex", "--healthcheck"]
35      interval: 10s
36      timeout: 3s
37      retries: 5
38      start_period: 10s

```

Listing 10.2. The migrate and backend services, showing the one-shot migration gate and the shell-less health check.

Two points from Listing 10.2 reward attention. First, the migrate service is given a JWT_SECRET

even though it never signs a token. This is because `cmd/migrate` shares the same `config.Load()` loader as the server, and that loader fast-fails on a weak secret (Section 10.6); the migration tool therefore inherits the same boot requirement. Second, `APP_URL` defaults to `http://localhost:8081` here, whereas the application's own default is `http://localhost:5173`. This is intentional: in the deployed stack the user-facing origin is the nginx frontend on `:8081`, so both email links and the CORS allow-list must reflect it.

A production-shaped local run is therefore a single command that also generates a strong secret on the spot: `JWT_SECRET=$(openssl rand -hex 32) docker compose up --build`. This yields a 64-hex-character secret and brings up the entire stack in the correct order.

10.5 Health Checks in a Shell-Less Image

The backend exposes `GET /healthz` outside the authenticated `/api` groups; it returns `{"status":"ok"}` with HTTP 200 and exercises the routing and CORS middleware without touching the database or Judge0. It is a *liveness* probe — is the process up and serving? — rather than a deep *readiness* probe that would report dependency reachability.

The challenge is invoking that endpoint from inside a distroless container that has no shell and no `curl`. The solution is that the `apex` binary is dual-role. Before it starts the server, `main.go` inspects its first argument: when invoked as `/app/apex --healthcheck` it does *not* start a server but instead acts as a tiny HTTP client, issuing `GET http://127.0.0.1:$PORT/healthz` with a two-second time-out and exiting 0 on HTTP 200 or 1 otherwise. The only program guaranteed to exist in the image is the binary itself, so the binary probes itself. The Compose backend.healthcheck of `["CMD", "/app/apex", "--healthcheck"]` and any production HEALTHCHECK directive both drive this branch, with `start_period: 10s` granting the server ten seconds to boot before failing probes count against it. This self-probe pattern is the smallest possible way to health-check a shell-less image without reintroducing the very tooling that distroless was chosen to remove.

10.6 Twelve-Factor Configuration

APEX follows the twelve-factor discipline of storing all configuration in the environment [28]: nothing operator-specific is compiled into the binary, and the same image runs in development, staging, and production distinguished only by its environment. Configuration is read once at startup through `internal/config/config.go`, whose `init()` best-effort loads a `.env` file via `godotenv` (ignoring the “no file” case, which is normal in production where real environment variables are supplied by the orchestrator).

The one hard invariant is the `JWT_SECRET` validator, which is the only variable whose absence or weakness crashes the process on purpose:

```
1 jwtSecret := os.Getenv("JWT_SECRET")
2 if jwtSecret == "" || jwtSecret == "dev-secret-change-in-prod" || len(jwtSecret) < 32 {
3     log.Fatal("JWT_SECRET must be set to a strong random value (>=32 chars)")
4 }
```

Listing 10.3. The single fast-fail in the configuration loader.

This rejects three cases — empty, the literal placeholder `dev-secret-change-in-prod`, and anything shorter than 32 characters — and makes “production booted with the development secret” structurally

impossible. It is the most important operational invariant in the system, and because `cmd/migrate` shares the loader, it protects the migration path too.

Every other variable degrades a feature gracefully rather than aborting. Table 10.1 inventories the backend variables, their defaults, and their behaviour when unset.

Table 10.1. Environment variables read by the backend (`internal/config/config.go`) and their graceful-degradation behaviour.

Variable	Default / requirement	Behaviour
<code>DATABASE_URL</code>	empty (prod-only)	full Postgres URL; <i>wins</i> over <code>DB_*</code> for both app and <code>cmd/migrate</code>
<code>DB_HOST</code>	<code>localhost</code>	Postgres host
<code>DB_PORT</code>	<code>5432</code>	Postgres port
<code>DB_USER</code>	<code>postgres</code>	Postgres user
<code>DB_PASSWORD</code>	<code>postgres</code>	Postgres password
<code>DB_NAME</code>	<code>apex</code>	Postgres database
<code>DB_SSLMODE</code>	<code>disable</code>	set to <code>require</code> for managed providers
<code>PORT</code>	<code>8080</code>	HTTP listen port; also probed by <code>--healthcheck</code>
<code>SKIP_AUTOMIGRATE</code>	empty	<code>true</code> disables GORM <code>AutoMigrate</code> ; <code>cmd/migrate</code> becomes sole schema authority
<code>MIGRATIONS_DIR</code>	file URL to the SQL dir	where <code>cmd/migrate</code> reads versioned SQL
<code>JWT_SECRET</code>	required, ≥ 32 chars	<i>fast-fails</i> on empty, weak, or the placeholder
<code>JUDGE0_URL</code>	<code>https://ce.judge0.com</code>	Judge0 base URL
<code>APP_URL</code>	<code>:5173</code> (compose: <code>:8081</code>)	password-reset links + first CORS origin
<code>ALLOWED_ORIGINS</code>	empty	comma-separated extra CORS origins
<code>GOOGLE_CLIENT_ID</code>	empty	unset $\rightarrow$ <code>/auth/google</code> returns HTTP 503
<code>SMTP_HOST</code>	empty	unset $\rightarrow$ emails logged to stdout
<code>SMTP_PORT</code>	<code>587</code>	ignored when <code>SMTP_HOST</code> empty
<code>SMTP_USER</code>	empty	SMTP username
<code>SMTP_PASS</code>	empty	SMTP password
<code>SMTP_FROM</code>	empty	envelope-from address

The degradation rule is worth stating in one sentence: only `JWT_SECRET` fast-fails; an unset `SMTP_HOST` logs password-reset and reminder emails to stdout while the in-app side keeps working, and an unset `GOOGLE_CLIENT_ID` makes the backend answer Google sign-in with HTTP 503 while the password flows continue unaffected. This keeps a partially configured deployment usable rather than dead.

The CORS allow-list is assembled from `APP_URL` (always the first entry) plus each comma-separated origin in `ALLOWED_ORIGINS`, trimmed of whitespace. The connection string is resolved by two builders — `DSN()` for the GORM/pgx application path and `MigrateURL()` for `golang-migrate` — both of which prefer `DATABASE_URL` when set and otherwise assemble a string from the discrete `DB_*` parts. This is exactly why a managed PostgreSQL such as Neon or Railway “just works”: the operator sets the single `DATABASE_URL` string the provider hands them (typically including `sslmode=require`), leaves the `DB_*` variables empty, and both the app and the migration tool pick it up.

10.7 Schema as Deployment Artifact

APEX carries two migration tracks, and which one owns the schema is decided by `SKIP_AUTOMIGRATE`. In development (the flag unset), each boot runs an idempotent legacy-SQL pass, then GORM's `AutoMigrate` for new tables, then a post-migration pass — convenient for fast local iteration. In production (`SKIP_AUTOMIGRATE=true`) `AutoMigrate` is disabled entirely and the versioned `golang-migrate` track becomes the sole authority [35]. That versioned track is exactly what the one-shot migrate container in Section 10.4 applies: paired `NNNN_name.up.sql/.down.sql` files under `internal/database/migrations/sql/`, applied as a pre-deploy step by the `cmd/migrate` binary bundled in the image.

Treating the versioned binary as the schema authority in production is a direct response to a real incident. During development, a `gorm:"default:true"` tag was added to a boolean exam field; GORM's `AutoMigrate` applied that as a DDL default across every existing row, silently flipping published exams into a draft state and making them vanish from student queries with no error logged. The fix was threefold: the struct default was flipped to the safe zero value, an explicit idempotent SQL sequence (add nullable, backfill, constrain) took ownership of the change, and — most relevant to deployment — production was moved to `SKIP_AUTOMIGRATE=true` with the versioned `migrate` binary as the single source of truth. The operational lesson is that in production, schema evolution must be an explicit, reviewed, version-controlled artifact applied by a dedicated step, never an implicit side effect of the ORM booting against a populated database. Re-running the migration binary on an up-to-date database is a no-op, and rolling back the production path is a single `migrate down`.

10.8 In-Process Background Jobs

APEX has no separate cron daemon and no separate worker process. The exam-reminder scheduler runs *inside* the same apex binary: `main.go` calls `reminder.Start()` before the server begins serving, which spawns a single goroutine that waits fifteen seconds for the database connection to settle, then ticks once per minute for the life of the process. Each tick runs three sweeps — an in-app reminder at roughly sixty minutes before an exam, an email reminder one hour out, and an email reminder at exam start — and the whole tick is wrapped in a `recover()` guard so a panic in one sweep is logged and the loop survives to the next minute.

The property that makes this safe to co-locate with the request-serving process, and safe across restarts, is that all deduplication is column-based: each reminder stamps a timestamp column on the exam row (for example `reminder_sent_at`) and each sweep filters on that column being null. Because "already sent" lives in the database rather than in memory, a process restart — a deploy, a crash, a scale event — can never cause a double-send; the scheduler is idempotent across restarts. Figure 6.2 details the sweep windows.

This co-location is an honest engineering trade-off. It keeps the deployment to a single binary with no broker or worker fleet to operate, which is exactly the "boring" target this chapter aims for. The cost is that reminders are bound to the lifecycle of the API process and do not scale horizontally: running two backend replicas would run two schedulers, and while the column-based dedup prevents double-sends, the design assumes a single backend instance. For a classroom-scale deployment that assumption holds comfortably; a larger deployment would extract the scheduler into its own process or gate it behind an advisory lock.

10.9 Continuous Integration

Every pull request and every push to `main` runs through GitHub Actions [48], with `concurrency` configured to cancel superseded runs. Two workflows make up the gauntlet. `ci.yml` runs four jobs: a backend test job that spins up a `postgres:16-alpine` service container and runs `go mod download`, `go vet ./...`, then `go test ./... -race -count=1` under a CI-only `JWT_SECRET`; a backend lint job running `golangci-lint` with a five-minute timeout; a frontend build job on Bun pinned to 1.1.38 running `bun install --frozen-lockfile`, `bun run lint`, and `bun run build`; and a frontend test job running the Vitest suite. The second workflow, `docker.yml`, smoke-builds both container images — the backend distroless image and the frontend nginx image — with `push: false` and GitHub Actions build caching, on every pull request and every push to `main`. Building the images in CI catches lockfile drift and Go-module changes that would break a deploy but would not necessarily fail a unit test.

Two honesty notes belong here. First, backend test coverage is currently minimal — there are no `*_test.go` files, and the frontend ships a single sanity test — so the CI test legs primarily exercise the build, vet, lint, and race-detector paths rather than deep behavioural assertions. Because CI already stands up a PostgreSQL service and runs the runners, adding real tests requires no pipeline change. Second, APEX has continuous *integration* but not full continuous *deployment*: the images are built and validated on every change, but the registry push and the remote restart that would make this a true push-to-deploy pipeline remain manual. This is a deliberate scope boundary for a graduation project, and naming it is preferable to over-claiming an automation that does not yet exist.

10.10 Operability and Summary

A few remaining operational facts round out the picture. The server logs to stdout in Gin's default format, so a production host redirects stdout to journald or a log file under rotation. The current model is a single-instance restart with a five-to-ten-second window; because the exam-taking SPA autosaves drafts and retries the autosave call when the backend returns, an in-flight exam survives a restart without draft loss. Backups are a nightly `pg_dump`: the schema is fully captured by the versioned migrations in source control, so a restore is "create empty database, apply the SQL track, load the dump." Deeper readiness probes and Prometheus metrics — per-tick scheduler duration first — are identified as sensible next steps but are out of scope for this iteration.

The deployment story is intentionally boring, and that is the design goal rather than a limitation. One Go binary carrying three roles, one PostgreSQL, one external Judge0, one optional SMTP relay, and an nginx reverse proxy compose into a four-service stack whose startup ordering guarantees a current schema before the first request. Configuration lives entirely in the environment with a single deliberate fast-fail; the distroless runtime keeps the image small and its attack surface narrow; and continuous integration gates every change through vet, lint, race-tested builds, and image smoke-builds. The rough edges — minimal test coverage, a single-instance scheduler, and the still-manual final deploy hop — are named rather than hidden, in keeping with the project's stance that an honestly documented limitation is worth more than a concealed one.

Chapter 11

Results and Evaluation

The preceding chapters described what APEX *is* and how each subsystem was built. This chapter asks a different, harder question: how good is the delivered artefact, and how do we know? Evaluating a graduation project fairly means resisting two temptations — inflating the result with invented benchmarks, and deflating it out of false modesty. The stance taken here is that every claim must either trace to a fact already established in the source (a line count, an index, a configuration constant, a shipped endpoint) or be labelled explicitly as illustrative or as a future measurement. Nothing in this chapter is a fabricated user study or a synthetic benchmark presented as if it were measured.

The evaluation proceeds along four axes. Section 11.1 quantifies the *scale and scope* of what was actually built. Section 11.2 audits *functional completeness* against the requirements catalogued in Chapter 3, giving a full traceability of requirement to delivery status. Section 11.3 evaluates *usability* against Nielsen’s heuristics and positions the System Usability Scale as the instrument for the empirical study that a classroom deployment would run next. Section 11.4 characterises *performance* qualitatively, because APEX’s dominant latency term lives inside an external service and was not instrumented under controlled load. Section 11.5 places the work on its calendar, and Section 11.6 closes with a candid “what works / what is limited” reckoning that ties back to the gaps owned throughout the book.

11.1 Scale and Scope of the Delivered System

The most objective measure of a software project’s output is the size and shape of its codebase. APEX is not a prototype or a proof of concept: it is a working, two-role, mixed-format examination portal with a real database schema, a real sandboxed grading pipeline, and a real single-page front end. Figure 11.1 reports its source size by component.

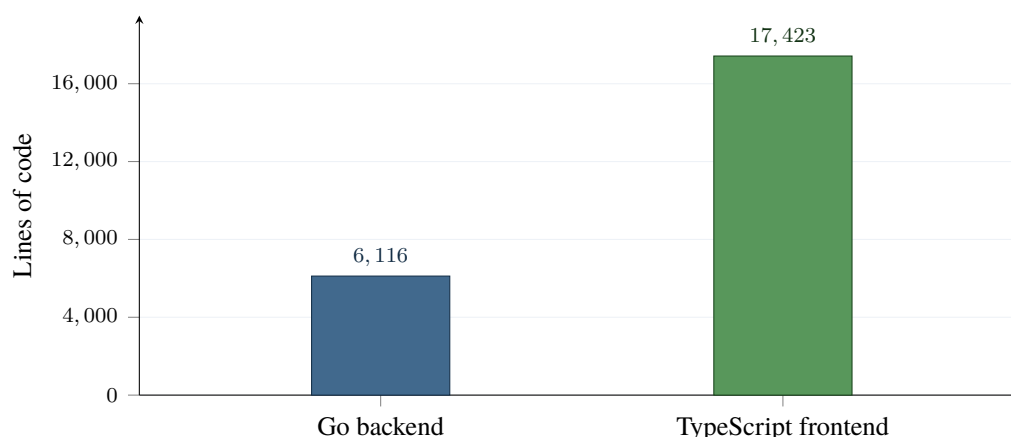


Figure 11.1. Source-code size by language and component.

The backend comprises approximately **6,116** lines of Go across the feature-scoped `internal/` pack-

ages, and the front end approximately **17,423** lines of TypeScript and TSX under `frontend/src`. The roughly 2.8:1 ratio of front-end to back-end code is characteristic of the architecture rather than incidental: APEX deliberately pushes every security- and correctness-critical decision to the server — authentication, the three authorization gates, grading, and all answer state — and keeps the client a presentation and orchestration layer. A compact, auditable Go core carrying the logic that “fails an exam”, wrapped in a larger but lower-risk React surface that renders 47 pages of it, is exactly the distribution one wants for an assessment system where the browser is treated as untrusted.

Line counts alone under-describe a system, so Fig. 11.2 summarises APEX along four structural dimensions that better convey its breadth.

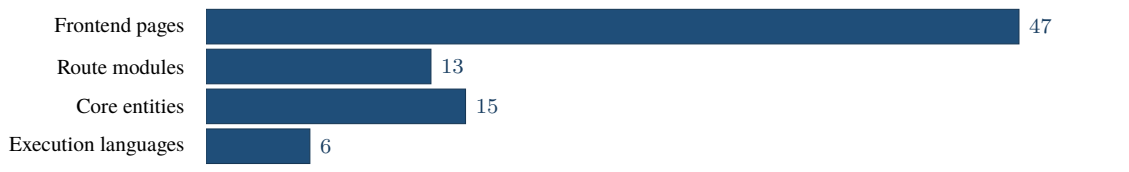


Figure 11.2. APEX by the numbers.

Each of these numbers is a verified structural fact, not a marketing figure:

- **15 core entities** — the GORM models registered in `database.AutoMigrate`, from `User` and `Class` through `Exam`, `Problem`, `TestCase`, `ExamAttempt`, `Submission`, and `TestResult` to the supporting `Notification`, `Announcement`, `Folder`, and `PasswordResetToken`. These are not toy tables: three of them carry unique composite indexes (`idx_class_user`, `idx_user_exam_attempt`, `idx_exam_class`) that encode genuine business invariants at the storage layer.
- **13 route modules** — the `RegisterRoutes` calls wired in `main.go`: `auth`, `class`, `student`, `exam`, `problem`, `testcase`, `submission`, `execute`, `notification`, `profile`, `teacher`, `announcement`, and `folder`. Each is a self-contained feature package following the same four-layer request lifecycle.
- **47 feature pages** — the front-end surface a user can actually reach, spread across the `auth`, `courses`, `dashboard`, `exams`, `grading`, `landing`, `playground`, `results`, `settings`, and `social` feature directories.
- **6 execution languages** — Python, JavaScript, TypeScript, Java, C, and C++, mapped to Judge0 language IDs in the `LanguageMap` of `internal/judge0/client.go`; an unknown language is rejected as `unsupported language` rather than silently mis-run.

Taken together, these figures answer the “is it real?” question quantitatively. APEX is a mid-sized full-stack application whose complexity is concentrated where it belongs: fifteen persisted entities, thirteen server modules enforcing a uniform authorization discipline, and a client broad enough to cover the entire teacher-and-student workflow.

11.2 Functional Completeness Against Requirements

Chapter 3 catalogued the requirements as they were actually realised, with each functional requirement (FR) traced to an implementing component. The evaluation question here is the inverse of that traceability: taking the requirement set as the yardstick, how much of it was delivered, and with what caveats? Table 11.1 answers this for the twelve teacher, eight student, and seven platform functional requirements. The status column uses three levels: *Delivered* means the capability ships and is reachable; *Delivered, with limitation*

means the capability ships but a named gap qualifies it; there are no *Not delivered* rows, because the requirements chapter was written to record only functionality that reached the shipped system rather than an aspirational wish-list.

Table 11.1. Functional-requirement coverage of the delivered system.

Req.	Status	Evidence / caveat
FR-T1–T3	Delivered	Class CRUD, the three-type exam builder, and the folder-organised question bank all ship as the <code>class</code> , <code>problem</code> , <code>testcase</code> , and <code>folder</code> modules.
FR-T4	Delivered	Draft/publish gating is enforced; publishing requires a <code>start_time</code> . This is the requirement whose earlier mishandling caused the <code>is_draft</code> incident (Chapter 5), now fixed and rule-bound.
FR-T5–T6	Delivered	Multi-class assignment validates class ownership; per-exam shuffle, results-visibility, and passing-score options persist on the <code>Exam</code> .
FR-T7	Delivered	Coding, MCQ (set-equality), and written grading all run through <code>internal/judge0</code> ; unconfigured MCQ correctly falls to <code>pending_review</code> rather than failing a student.
FR-T8	Delivered	The pending-grading queue and <code>PUT /submissions/:id/grade</code> recompute the attempt aggregate on each manual grade.
FR-T9–T11	Delivered	Announcements fan out notifications; the teacher dashboard, results explorer, per-class statistics, exam preview, and pending-work grade-gating all ship.
FR-T12	Delivered	Destructive edits trigger the transactional <code>reset_at</code> wipe-and-notify path in <code>internal/exam/reset.go</code> .
FR-S1–S4	Delivered	Invite-code join, the idempotent one-attempt start, server-side autosave/resume, and per-attempt shuffle all ship. Autosave payloads are opaque JSON in <code>draft_answers</code> , rehydrated on resume.
FR-S5	Delivered	The Monaco editor drives six languages with a <code>run-against-sample-tests</code> action (<code>POST /submissions/run</code>) that persists no graded submission.
FR-S6	Delivered, with limitation	Submission closes the attempt and grades server-side, but grading is a fire-and-forget goroutine with no queue or retry — the durable-queue gap owned in Chapter 7.
FR-S7–S8	Delivered	Per-test result review, teacher feedback, notifications, unread counts, and profile/preference management all ship, subject to the teacher’s results-release settings.
FR-P1	Delivered, with limitation	Email/bcrypt login and 24-hour HS256 JWTs ship; Google sign-in ships when <code>GOOGLE_CLIENT_ID</code> is configured. Limitations: no rate limiting on auth, a six-character password minimum, and the token in <code>localStorage</code> .
FR-P2–P3	Delivered	Enumeration-safe password reset (single-use SHA-256 token, one-hour expiry) and transactional account deletion both ship.
FR-P4–P5	Delivered	The in-process reminder scheduler and the once-only “Exam Graded” notification are deduplicated by database columns, idempotent across restarts.
FR-P6	Delivered	<code>GET /healthz</code> and the <code>--healthcheck</code> self-probe support health-checking the shell-less image.
FR-P7	Delivered	Idempotent <code>dev</code> migrations plus a production-only versioned <code>golang-migrate</code> track (<code>SKIP_AUTOMIGRATE=true</code>) as sole schema authority.

The picture is one of *high functional completeness with a small, well-understood set of qualified rows*. Of the 27 functional requirements, 24 are delivered without caveat and 3 are delivered with an explicitly named limitation. Crucially, the two limitations that carry real risk — fire-and-forget grading (FR-S6) and the un-hardened authentication surface (FR-P1) — are not surprises uncovered by this audit; they are the same gaps owned in the security and architecture chapters and listed in the project’s own README. That consistency is itself an evaluation result: the system’s documented behaviour matches its actual behaviour.

The non-functional requirements (NFR-1 through NFR-20) trace to cross-cutting mechanisms rather

than to endpoints, and their delivery was argued in the chapters that own them: the fail-fast `JWT_SECRET` check and three-gate authorization (security), the bounded connection pool and indexed hot paths (data model), the single-artefact deploy and twelve-factor configuration [13], [28] (deployment), and the idempotent migration and scheduler discipline (deployment/DevOps). The one NFR that is deliberately deferred to future empirical work is NFR-12 (measured satisfaction via SUS), discussed next.

11.3 Usability Evaluation

APEX was not subjected to a formal, statistically powered usability study within the project timeline; claiming otherwise would overstate the evidence. What can be reported rigorously is an *expert heuristic evaluation* against Nielsen’s ten usability heuristics [26], tracing each relevant heuristic to a concrete, shipped interface mechanism rather than to a subjective impression. Table 11.2 presents that mapping for the heuristics APEX most directly engages.

Table 11.2. Heuristic evaluation: Nielsen’s heuristics traced to shipped mechanisms.

Heuristic	APEX mechanism
Visibility of system status	A submission’s lifecycle state (<code>pending</code> → <code>running</code> → <code>graded</code>) is surfaced in the UI; live sample-test runs report pass/fail, output, time, and memory before submit; the autosave indicator shows when work was last persisted.
User control and freedom	Server-side autosave with resume: an interrupted attempt continues where it left off rather than restarting, and a dropped connection or switched device does not cost the student their work (NFR-11).
Error prevention	A draft exam cannot be published without a <code>start_time</code> ; a submitted attempt cannot be resubmitted; the one-attempt-per-(<code>student</code> , <code>exam</code>) unique index makes a duplicate attempt impossible at the storage layer.
Match between system and the real world	The vocabulary mirrors the classroom: classes, invite codes, exams, questions, grading, announcements — the teacher’s mental model, not a database schema.
Consistency and standards	A single component library (<code>shadcn/ui</code> over <code>Radix</code> primitives) and one design system across all 47 pages; the same editor (<code>Monaco</code>) for playground and exam.
Recognition rather than recall	The question bank and folders let a teacher compose exams from previously authored material instead of re-remembering and re-typing it.
Help users recognise and recover from errors	Login returns a single generic “invalid email or password” (which is also a security choice); per-test result review shows exactly which hidden tests failed and the actual versus expected behaviour, so a student can learn from a wrong answer.

Two usability decisions deserve particular emphasis because they resolve tensions rather than merely satisfying a heuristic. First, output normalisation in the grader (`normalizeOutput`) is a usability decision disguised as an engineering one: by stripping trailing whitespace and normalising line endings symmetrically on the student’s output and the expected output, it prevents the single most demoralising failure mode of naive auto-graders — a correct answer failed for an invisible trailing newline. Second, the exclusion of `pending_review` answers from the attempt aggregate means a student never sees their visible score transiently dip while a written answer awaits human grading; the number they see is accurate at every moment.

11.3.1 Toward an Empirical Study: the SUS Instrument

Heuristic evaluation finds structural problems but cannot produce a defensible satisfaction number. The natural next step, and the one NFR-12 commits to, is a System Usability Scale study [27]: a ten-item questionnaire whose alternating positive and negative items are each normalised to a 0–4 contribution, summed, and multiplied by 2.5 to yield a single 0–100 score, benchmarkable against a well-established industry mean. The protocol APEX is designed to support is a short scripted task list exercising both roles — *build an exam, assign it, take it, grade a written answer, review results* — followed by the SUS questionnaire, with results reported separately by role because the teacher’s authoring workflow and the student’s timed-sitting workflow stress different parts of the interface.

This study is deliberately *not* reported here with invented numbers. Nielsen’s own guidance is that a handful of participants surfaces the majority of an interface’s usability problems [26], so even a modest cohort would be a meaningful first measurement; but a modest cohort drawn from the author’s own institution would also carry a self-selection bias that must be acknowledged when the study is eventually run. The instrument, the protocol, and the analysis recipe are specified so that the measurement is reproducible when a real classroom deployment makes a representative sample available.

11.4 Performance Characteristics

Performance is where candour matters most, because APEX’s dominant cost is not in APEX. This section characterises performance qualitatively and labels clearly what was measured versus what is reasoned from the design.

11.4.1 Grading Latency Is Dominated by Judge0

The end-to-end latency of grading a coding submission is the sum of three terms: the Go handler’s own work (a database write of the `Submission` row and, later, `TestResult` rows), the HTTP round-trip to Judge0, and — dominating both — Judge0’s own compile-and-run time inside the `isolate` sandbox [4], [10]. Because student code is compiled and executed under per-problem limits (default 2000 ms wall time, 262144 KB memory), a single test case can legitimately consume up to two seconds of sandbox time, and a problem with several test cases multiplies that. The Go side, by contrast, adds only the milliseconds of a local database write and an HTTP call, bounded by a 30-second client timeout per Judge0 request. The architectural consequence, argued in NFR-6, is deliberate: exam *submission* does not block on grading at all. Each submission is graded in a background goroutine, so the HTTP response to `POST /student/exams/:id/submit` returns immediately with identifiers, and the student is never left staring at a spinner whose duration is set by how slow their own code is. The trade-off is the durable-queue gap already named (Chapter 7).

The clearest way to state the performance model is this: *APEX’s grading throughput is Judge0’s throughput*. Doubling the sandbox’s worker capacity doubles the number of submissions that can be graded per unit time; the Go server is not the bottleneck for any classroom-sized load, because it does no CPU-heavy work itself. A controlled load benchmark against a self-hosted Judge0 — stepping concurrency and measuring the latency distribution and the throughput plateau at the worker count — is the correct experiment to quantify this, and it is registered as future work rather than reported here with numbers that were not measured.

11.4.2 Image Size, Startup, and API Latency

Several qualitative performance properties follow directly from design choices documented earlier and can be stated without measurement:

- **Container image size.** The backend is a statically linked (`CGO_ENABLED=0`) Go binary, stripped with `-ldflags="-s -w"` and built `-trimpath`, copied onto a `gcr.io/distroless/static-debian12` base that ships essentially only the binary and CA certificates — no shell, no package manager, no `libc`. This is about as small and as narrow-attack-surface as a Go service image gets. The one avoidable weight is off-image: a 17 MB pre-built `seed` binary committed at the repository root, convenient for first boot but bloating the clone — an owned gap, not a runtime cost.
- **Startup.** There is no application-server warm-up: the process reads configuration once, opens a bounded connection pool (25 max-open, 10 max-idle), and begins serving. The reminder scheduler waits a 15-second warm-up before its first sweep, so background work never contends with the boot path. In Compose, the one-shot `migrate` service completing successfully gates the backend, so the server never accepts a request against a stale schema.
- **Non-grading API latency.** Endpoints that do not call `Judge0` — starting an attempt, autosaving answers, listing exams, reading a result — are ordinary indexed PostgreSQL reads and writes through GORM, backed by the unique composite indexes on the hot paths (NFR-9). These are expected to respond in the low tens of milliseconds against a co-located database; the precise percentiles are a measurement left for the load study, and the figure just given is presented as an *illustrative* design expectation, not a recorded result.

The overarching performance verdict is that APEX is correctly *sized* for its stated target — a classroom of tens of concurrent students, not a MOOC of tens of thousands — and that its one heavy operation is deliberately isolated behind an external service whose capacity the operator controls independently.

11.5 Project Timeline

APEX was executed over the 2025–2026 graduation year. Figure 11.3 shows the schedule as it actually unfolded, with the phases overlapping the way real engineering does rather than proceeding as a tidy waterfall.

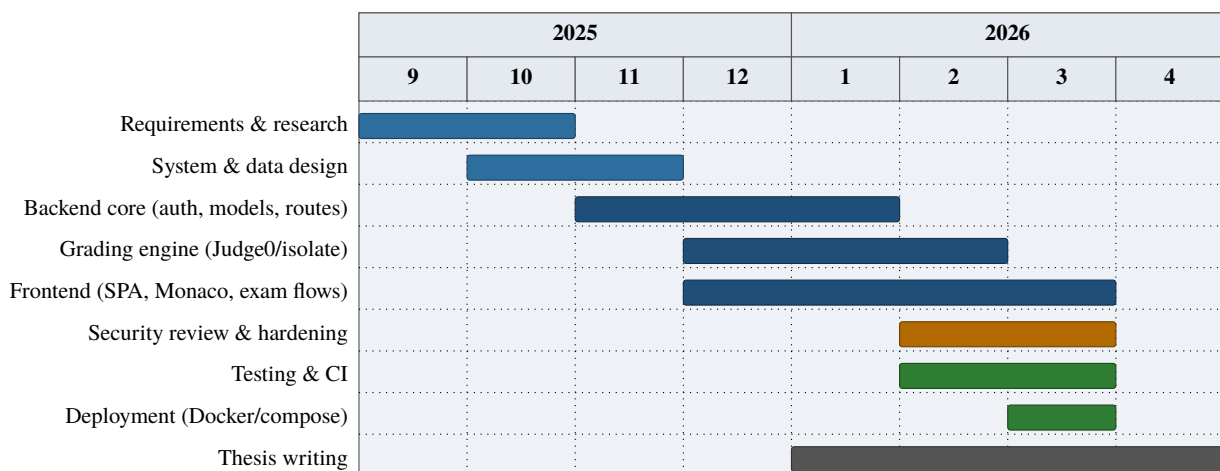


Figure 11.3. Graduation-project timeline (Gantt).

Three features of the timeline are worth reading off the chart. First, requirements and design front-load the schedule but do not finish before implementation begins; the backend core (auth, models, routes) starts while design is still settling, which is why the requirements chapter is written as a record of what shipped rather than a speculative contract. Second, the grading engine and the front end run in parallel across the middle of the year — the two largest and most independent workstreams — reflecting the front-end-heavy code distribution seen in Fig. 11.1. Third, and most importantly for the project’s engineering identity, the *security review and hardening* phase (in amber) runs *before* deployment, not after. The four findings — the IDOR on `/submissions/run`, the stored-XSS via Markdown, the weak fallback JWT secret, and the HTML injection in emails — were all found and fixed pre-ship, by design. Testing, CI, and Dockerised deployment cluster at the end, and thesis writing runs as a continuous band across the back half of the year.

11.6 What Works and What Is Limited

An honest evaluation ends with a plain reckoning. The following two lists are the distilled verdict; neither hides behind the other.

What works

- **One pipeline, three formats.** Coding, MCQ, and written questions are assessed in a single timed sitting behind one grading path, with a single attempt aggregate that recomputes correctly over the exam’s full problem list and excludes in-flight manual grades from the visible total.
- **Fair auto-grading.** Sandboxed execution under enforced time and memory limits, symmetric output normalisation that refuses to fail a correct answer on whitespace, and a per-test breakdown that makes every mark explainable to the student.
- **Defence in depth that fails closed.** Three independent authorization gates (Chapter 8) plus a fail-fast secret check that refuses to boot with a weak `JWT_SECRET`, and a security review conducted before shipping rather than after an incident.
- **Resilient exam-taking.** Server-side autosave and single-attempt resume mean a crash, a dead battery, or a switched device does not cost a student their work — the student environment is treated as untrusted and unreliable, and the server holds the truth.
- **Boring, reproducible deployment.** One Go binary carrying three roles, a four-service Compose stack with schema-migration ordering, distroless images running as non-root, twelve-factor configuration, and a CI gauntlet that gates every change.
- **A schema that learned from its own failure.** The `is_draft` incident (Chapter 5) produced not just a fix but a rule set (`ORM_RULES.md`) and a production migration discipline in which a versioned track, not the ORM, is the sole schema authority.

What is limited

- **Grading has no durable queue.** `go judge0.Grade(...)` is a bare goroutine with no persistence and no retry; a crash mid-grade can strand a submission in `running` and leave an attempt unfinalised (Chapter 7). Grading is idempotent, so the fix is a durable work queue — the single highest-value piece of future work.

- **The authentication surface is not hardened for public exposure.** No rate limiting on auth (bcrypt cost 10 is the only brake on credential stuffing), a six-character password minimum, and a JWT in `localStorage` that XSS could lift. Each is a deliberate, documented decision for the thesis snapshot, to be hardened before any public deployment.
- **Test coverage is thin.** The backend ships no `*_test.go` files and the front end a single sanity test; CI already exercises the runners, so adding tests requires no pipeline change, but the current automated confidence is low relative to the manual and heuristic evaluation.
- **Usability is heuristically, not empirically, evaluated.** The SUS study is specified but not yet run on a representative sample; the numbers must come from a real deployment, not from the author's institution alone.
- **Performance is reasoned, not benchmarked.** The Judge0-dominated latency model is sound but the controlled load experiment — concurrency sweep, latency distribution, throughput plateau — remains future work.
- **Operational single points.** The reminder scheduler runs in-process on a single instance, and the final production deploy hop is still manual; both are acceptable at classroom scale and named as such.

11.7 Summary

Measured against its own requirements, APEX is a substantially complete system: roughly 6,100 lines of Go and 17,400 of TypeScript realise 15 entities, 13 route modules, and 47 pages, delivering 24 of 27 functional requirements without caveat and the remaining three with limitations that were owned rather than discovered. Its usability conforms to the heuristics that matter for an assessment tool, with the empirical SUS study specified for a future classroom cohort. Its performance model is bounded by Judge0's sandbox capacity, with the load benchmark registered as future work rather than fabricated here. The project's engineering identity — security before shipping, a schema that learned from a real data-loss incident, and a deployment that is deliberately boring — shows up as much in *how* the gaps are handled as in the features that work. The remaining limitations, each concrete and each with a known fix, form the agenda carried into Chapter 13.

Chapter 12

User Manual

This chapter walks through APEX from the operator’s chair rather than the engineer’s. Where earlier chapters explained *how* the four-layer Go backend, the Judge0 grading pipeline, and the React single-page application are built, this one explains *how to use* the finished system to run a real classroom exam. It is organised around the two — and only two — roles the platform recognises: **teacher** and **student**. There is no administrator role and no separate configuration console; every operational action is performed through the same web interface documented here.

The screen captures were taken from a local instance brought up with `./start.sh`, which starts PostgreSQL in Docker, the Go binary on `:8080`, and the Vite dev server on `:5173` (Fig. 10.1 shows the corresponding container topology). The seed data set provides one teacher account, one student account, one demonstration class, and one exam of each supported question type (coding, MCQ, and written) so that every screen below has representative content. Browser chrome has been cropped from the captures; the routes named in the text are the real `react-router-dom` paths served by the SPA.

12.1 Getting Started: Accounts and Sign-in

12.1.1 The landing page

An unauthenticated visitor arrives at the marketing landing page served at `/` (Fig. 12.1). It summarises the product — mixed-format exams, automatic code grading, and per-test feedback — and offers a single call to action, *Get started*, which routes to the authentication screen at `/auth`. No functionality beyond sign-up and sign-in is reachable without an account; every `/dashboard/*` route is guarded on the client by an auth check and on the server by the JWT authentication gate (Fig. 8.2).

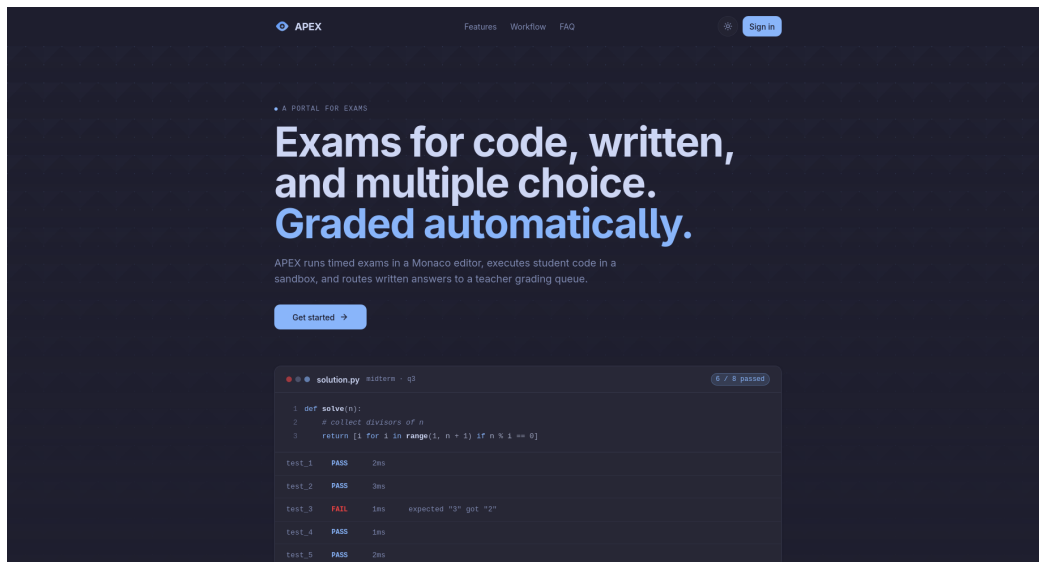


Figure 12.1. The public landing page at /.

12.1.2 Signing up and signing in

The authentication screen (Fig. 12.2) exposes three paths. A returning user signs in with email and password; the credentials are posted to the backend, which verifies the bcrypt hash (cost 10) and, on success, returns a signed HS256 JSON Web Token [18], [40] carrying the user's identifier, email, role, and name with a 24-hour expiry. A new user chooses *Sign up*, which additionally prompts for a role — *teacher* or *student* — because that single choice determines every subsequent screen the account will ever see. A third button, *Continue with Google*, delegates to Google Identity Services [41], [42]: the browser obtains a Google credential and the backend verifies it against the configured client ID. If the Google account is new to APEX and no role was supplied, the backend replies that a role is still needed and the client re-prompts before completing registration.

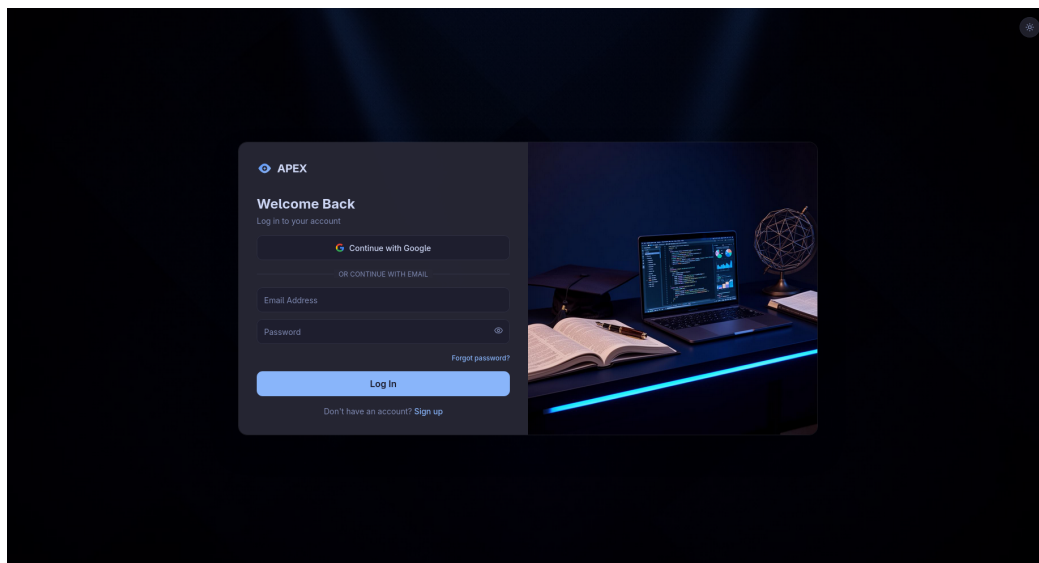


Figure 12.2. The sign-in and sign-up screen at /auth. The role picker is shown for new sign-ups and for first-time Google accounts.

Passwords must be at least six characters; the email field is validated against a standard address pattern

before the request is sent. A failed login returns a deliberately generic “invalid email or password” message so that the response does not reveal whether an address is registered. If a user forgets their password, the *Forgot password* link triggers a reset flow that always reports success (to avoid disclosing which addresses exist) and, when an SMTP server is configured, emails a single-use, one-hour reset link; on a development instance with no mail server the link is instead written to the server log.

12.1.3 The two dashboards

A successful sign-in stores the token in the browser and redirects to `/dashboard`, whose layout is chosen from the role encoded in the token. The **teacher dashboard** (Fig. 12.3) foregrounds the teacher’s own classes and recent activity and provides entry points to the class list, the exam list, the grading queue, and the question bank. The **student dashboard** (Fig. 12.4) instead foregrounds the exams that matter to a learner right now: upcoming exams, exams currently within their time window, and recently completed ones. The two roles never share a screen; a student has no route to the exam builder, and a teacher has no route to the take-exam surface except through the read-only preview described later.

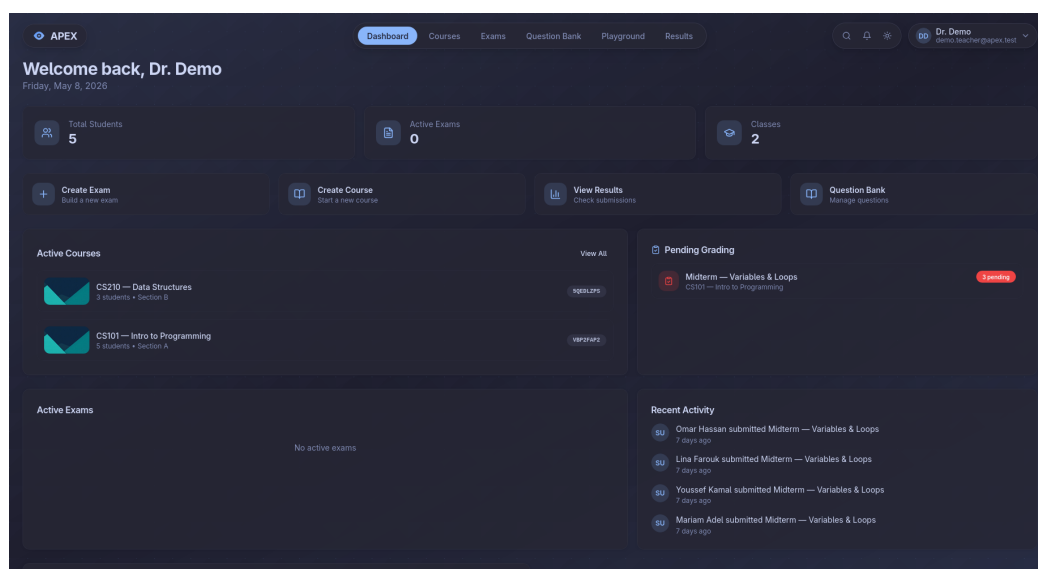


Figure 12.3. Teacher dashboard: owned classes and recent activity.

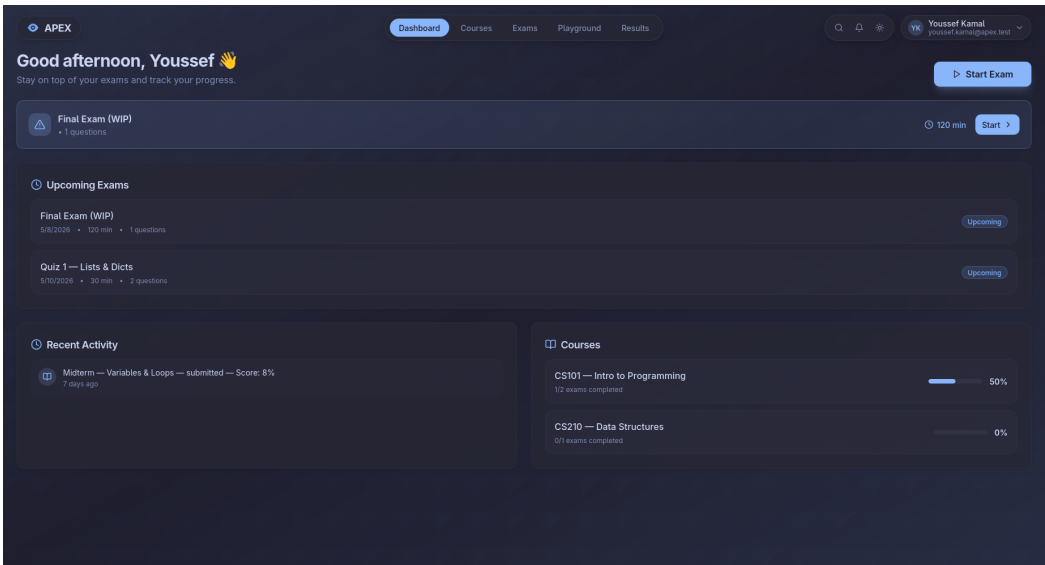


Figure 12.4. Student dashboard: upcoming, active, and completed exams.

12.2 Teacher Guide

The teacher’s workflow forms a single pipeline: create a class, build an exam, assign it to the class, publish it, and — once students have submitted — grade any manual answers and announce results. The following subsections follow that order.

12.2.1 Creating and managing classes

The *Courses* area (`/dashboard/courses`) lists the teacher’s owned classes. *New class* opens a dialog that collects a name and an optional section; on save the class is created with an automatically generated, eight-character **invite code** that is unique across the platform (Fig. 12.5). This code is the sole enrolment mechanism: there is no roster upload and no manual add-student form. The teacher shares the code (for example on a slide or in a chat), and students join with it themselves.

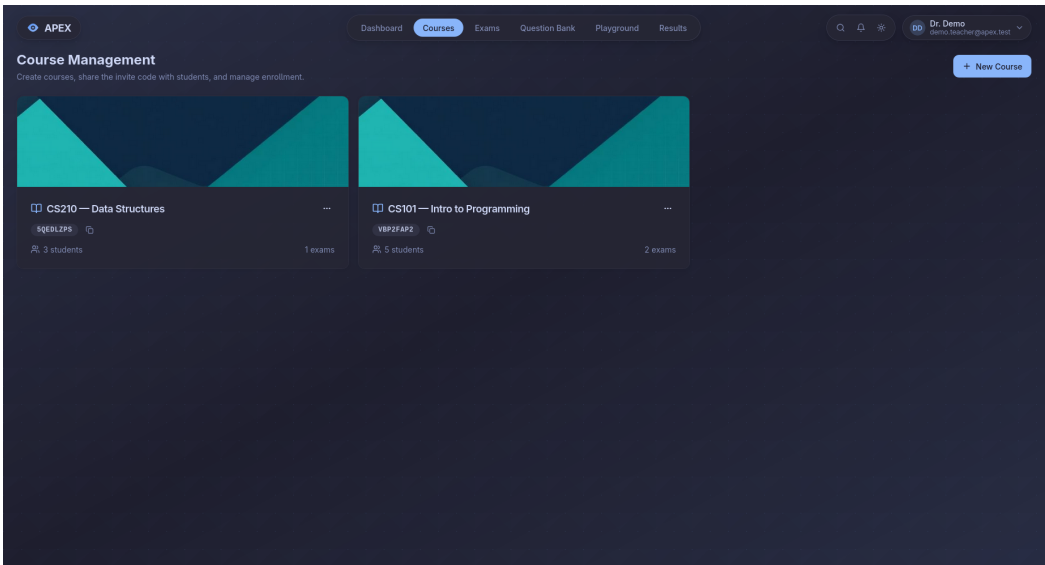


Figure 12.5. The teacher’s class list, each card showing its eight-character invite code.

Opening a class shows its detail screen (Fig. 12.6), which gathers three things in one place: the enrolled roster, the exams assigned to the class, and the announcements feed. From here the teacher can rename the class, change its cover image, adjust the passing threshold, remove a member, or post an announcement — which fans out an in-app notification to every enrolled student. The *Grades announced* toggle is the switch that later reveals results to students; it is examined in Section 12.4. A safeguard prevents accidentally announcing while manual grading is still outstanding, so that no student sees a total that is about to change.

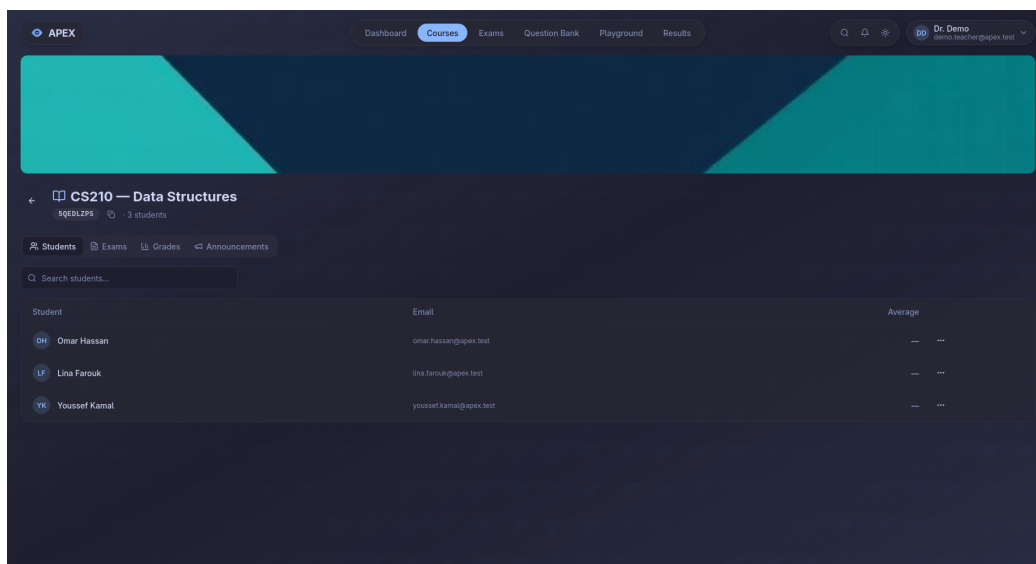


Figure 12.6. Class detail (teacher view): roster, assigned exams, and announcements.

12.2.2 Building an exam

Exams (/dashboard/exams) lists the teacher’s exams with status badges — Draft, Upcoming, Active, or Completed — derived from the draft flag and the start and end times. *New exam* opens the **exam builder** (Fig. 12.7). The builder is a three-part surface: a header collects exam-level metadata (title, description, duration in minutes, start and end times, and behavioural toggles such as *shuffle questions* and *show results after*); a left rail lists the questions in order; and the right pane edits whichever question is selected. The same builder assembles all three question types into a single sitting, which is the platform’s central idea — a student answers coding, multiple-choice, and written questions in one timed attempt, and one grading pipeline stands behind all three.

Figure 12.7. The exam builder: metadata header, question rail, and editing pane.

Coding questions

A coding question (Fig. 12.8) carries a title, a Markdown problem statement, optional starter code, and — most importantly — a list of **test cases**. Each test case pairs an input with an expected output and is flagged as either *sample* or *hidden*. Sample test cases are shown to the student and can be run during the attempt for immediate feedback; hidden test cases are withheld and are executed only when the exam is graded, which is what prevents a student from reverse-engineering the full grader. Each question also declares an execution time limit (default 2000 ms) and a memory limit (default 262144 KB, i.e. 256 MB) that are passed to the Judge0 sandbox [4], [10] when the code runs. The scoring is proportional: the earned score is the fraction of test cases that pass, so a solution that clears three of four tests scores 75%.

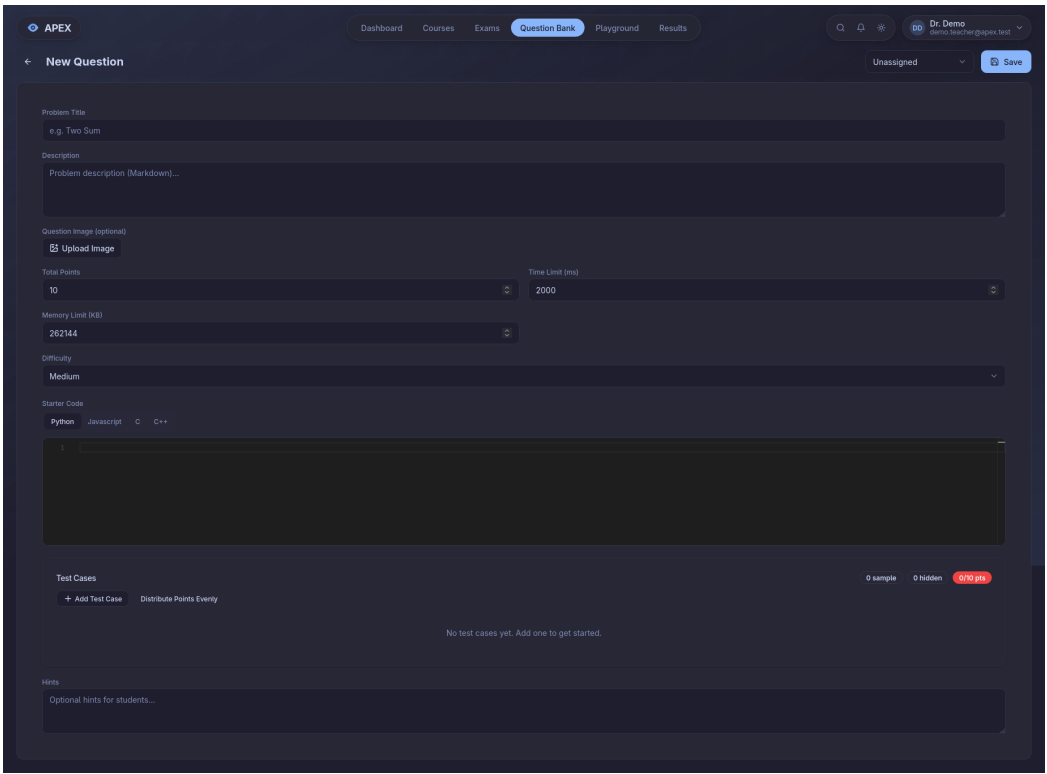


Figure 12.8. Coding-question editor: statement, starter code, limits, and sample/hidden test cases.

Multiple-choice questions

A multiple-choice question (Fig. 12.9) is a list of options, each with a *Correct* checkbox. Marking more than one option correct turns the question into a multiple-answer question, graded by exact set equality: the student earns full marks only if their selected set matches the correct set exactly, otherwise zero. The editor requires at least one option to be marked correct — if a question is published with no correct answer configured, the grader cannot fairly fail anyone on it and instead routes that submission to the manual-review queue rather than awarding a wrong-answer zero.

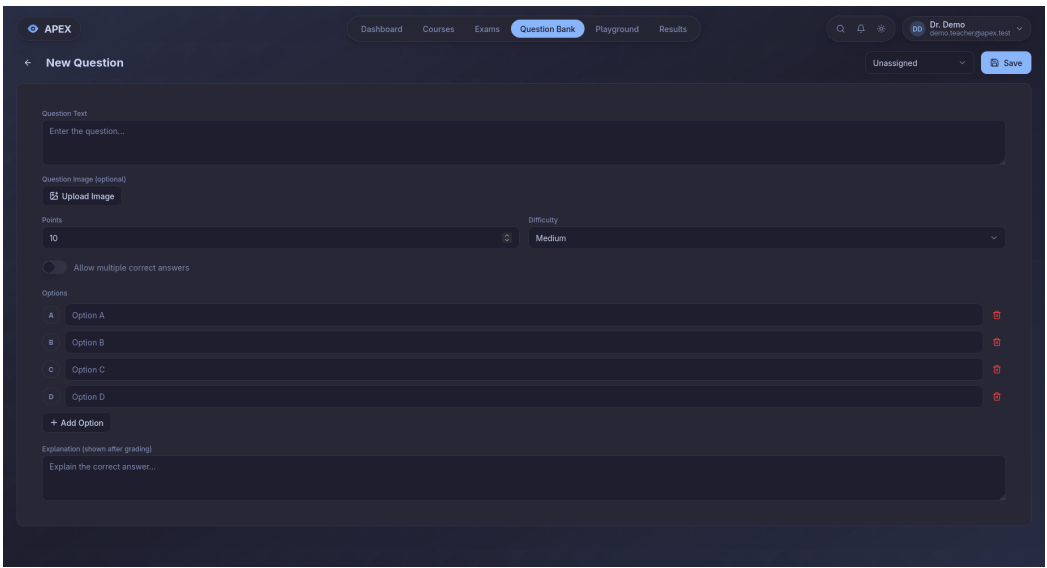


Figure 12.9. Multiple-choice editor: options with per-option *Correct* checkboxes.

Written questions

A written question (Fig. 12.10) collects a prompt, an optional grading rubric, and a maximum word count (default 500). Written answers are never graded automatically — every written submission begins life in the *pending review* state and enters the manual-grading queue, where the teacher reads it and assigns a score. Crucially, a written answer that is still awaiting a manual grade is excluded from the student’s running total, so an in-flight essay grade never artificially depresses the score the student sees.

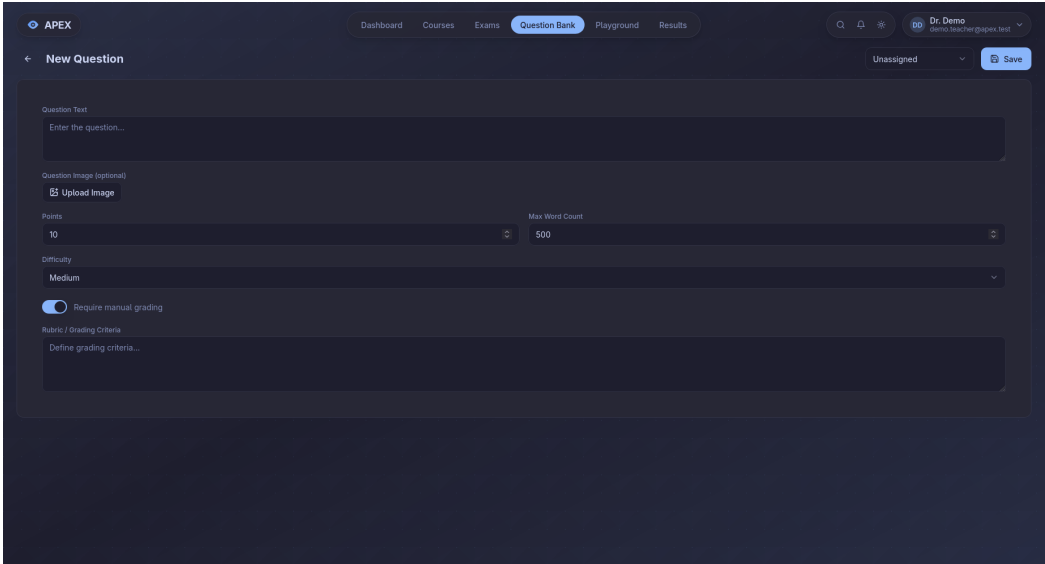


Figure 12.10. Written-question editor: prompt, rubric, and word-count limit.

Table 12.1 summarises how the three question types differ from the teacher’s point of view.

Table 12.1. The three question types and how each is graded.

Type	Author supplies	How it is scored
Coding	Statement, starter code, time/memory limits, sample and hidden test cases	Automatically: code runs in the Judge0 sandbox against every test case; score is the fraction of tests passed.
Multiple choice	Options with one or more correct answers, optional explanation	Automatically: exact set-equality on selected vs. correct options; all or nothing. Unconfigured answers go to manual review.
Written	Prompt, optional rubric, maximum word count	Manually: always starts in <i>pending review</i> ; the teacher assigns a score from the grading queue.

12.2.3 The question bank

Authoring good questions is expensive, so a teacher rarely wants to write each one from scratch for every exam. The **question bank** (/dashboard/question-bank, teacher only) lets the teacher save questions independently of any exam, organise them into folders, and later import them into a new exam. Bank questions and exam-attached questions share the same underlying storage but are distinguished by a bank flag, so importing a question copies it into the exam without disturbing the reusable original.

12.2.4 Assigning, publishing, and previewing

Building a draft exam does not expose it to anyone. Two further steps are required. First, the exam must be **assigned** to one or more classes; the assignment step validates that every target class actually belongs to the teacher performing it, so a teacher cannot push an exam into a colleague’s class. Second, the exam must be **published**. The *Publish* toggle is enabled only once the exam has at least one question and a non-empty start time (Fig. 12.11); an exam with no questions or no schedule stays a draft. This gate matters because student-facing queries deliberately hide draft exams — the same filter whose accidental corruption is the “`is_draft`” war story recounted in Fig. 5.2.

The screenshot shows the 'Edit Exam' page in the APEX system. The top navigation bar includes 'Dashboard', 'Courses', 'Exams', 'Question Bank', 'Playground', and 'Results'. The user is logged in as 'Dr. Demo'. The 'Edit Exam' form has the following fields and values:

- Title: Final Exam (WIP)
- Assign to Course: CS210 — Data Structures — Section B
- Duration (min): 120
- Passing Score (%): 50
- Start Date: May 8th, 2026
- Start Time: 01:43 PM
- Closes Date: Pick a date
- Closes Time: 11:59 PM

Below the form, there are two toggle switches: 'Shuffle' (disabled) and 'Show Results' (disabled). The 'Published' toggle is enabled. The description field contains the text 'Final exam draft — not yet published.'

At the bottom, there is a section for 'Questions (1)'. A question titled 'Q1: Binary Search' is listed. The question details are as follows:

- Problem Title: Binary Search
- Description: Implement iterative binary search on a sorted list.
- Question Image (optional): Upload Image
- Total Points: 20
- Time Limit (ms): 2000
- Memory Limit (KB):

A 'Save to Bank' button is located at the top right of the question details section.

Figure 12.11. The publish toggle, enabled once the exam has questions and a start time.

Before publishing, the teacher can *Preview* the exam (Fig. 12.12), which renders it exactly as a student will see it — question rail, Monaco editor, and all — but read-only and without creating an attempt. Pre-viewing is the recommended last check that starter code compiles, sample tests behave, and the wording is unambiguous.

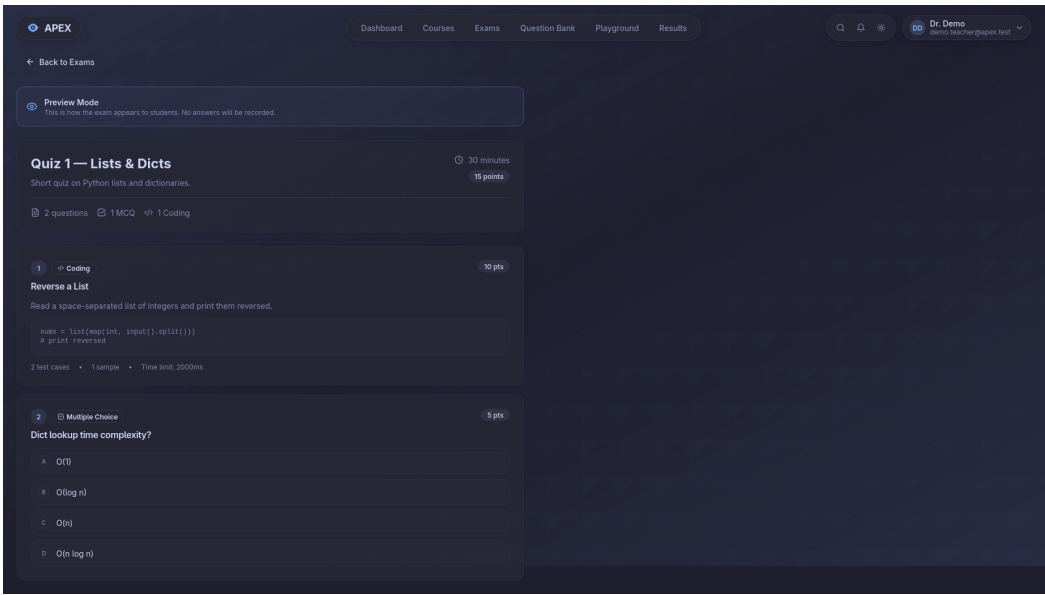


Figure 12.12. Exam preview: the student-facing view, rendered read-only for the teacher.

12.3 Student Guide

The student’s workflow is the mirror image of the teacher’s: join a class, wait for an exam to open, take it within its window, and later review the result.

12.3.1 Joining a class

A student opens *Courses* and chooses *Join with code* (Fig. 12.13). The eight-character invite code is the only input. Enrolment is idempotent — a student can be a member of a given class at most once — so re-entering the same code is harmless. Once joined, the class appears on the dashboard, and any exam already assigned to that class surfaces under *Upcoming*.

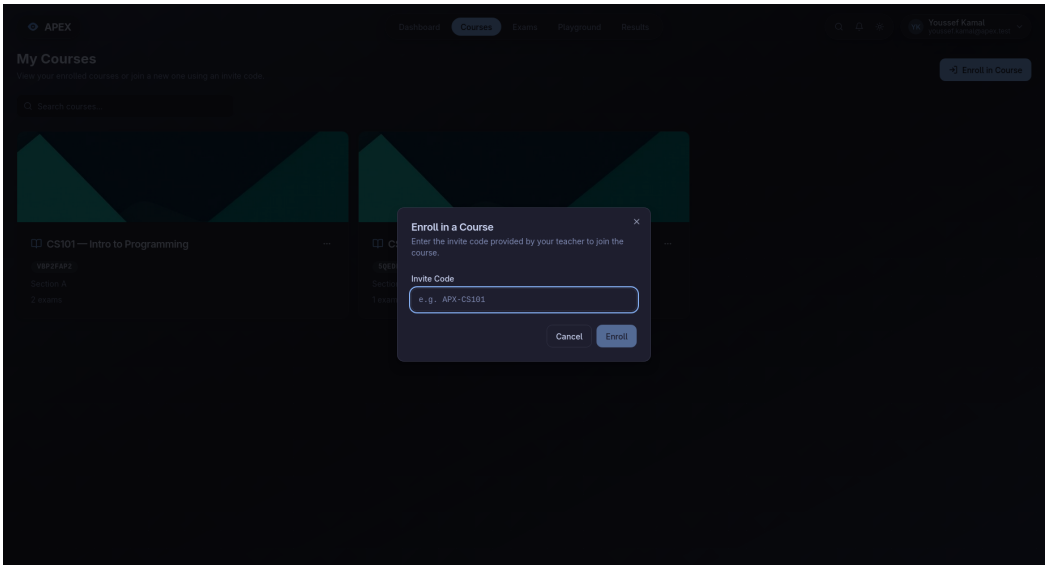


Figure 12.13. Joining a class with its invite code.

12.3.2 Taking a timed exam

When an exam is within its window, *Start exam* opens the take screen at `/dashboard/exam/:id` (Fig. 12.14). The layout is consistent across question types: a left rail numbers the questions and shows which are answered, the remaining time counts down in the top corner, and a Submit button sits at the end. Coding questions render in the Monaco editor [12] — the same engine that powers Visual Studio Code — with syntax highlighting and a language selector covering the six supported languages (Python, JavaScript, TypeScript, Java, C, and C++). Multiple-choice questions render as a checkbox list, and written questions render as a textarea with a live word counter that enforces the author’s limit.

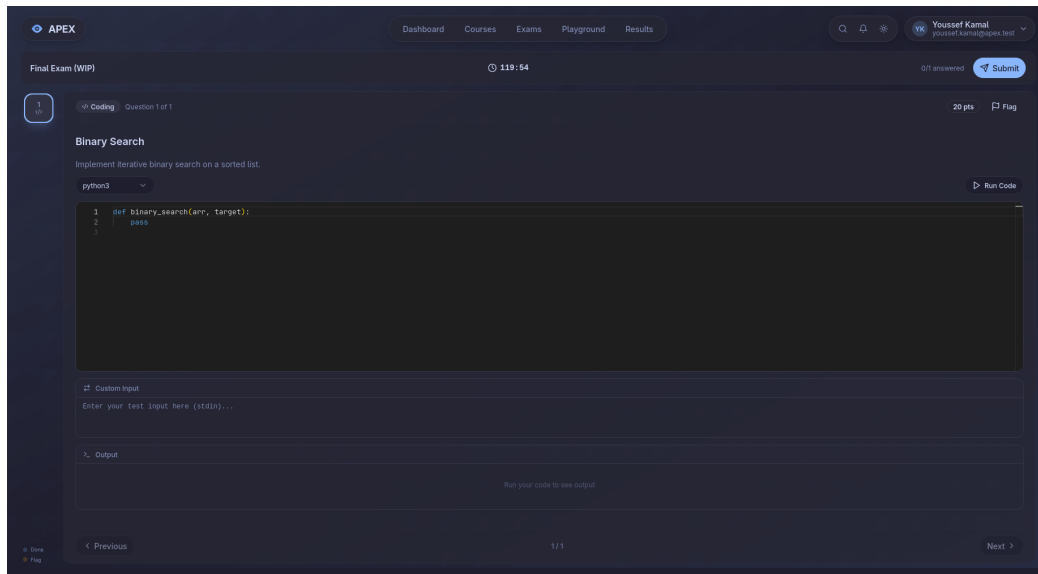


Figure 12.14. Taking a coding question in the Monaco editor, with the question rail and countdown timer.

Running sample tests. On a coding question the student can run their code against the *sample* test cases at any time before submitting. This sends the current code and chosen language to the backend, which executes it in the Judge0 sandbox and returns per-test results — expected versus actual output — rendered inline. Only sample tests run on this path; the hidden tests that ultimately decide the grade are never revealed here, and a sample run does not count as a submission.

Autosave. The client periodically saves a draft of every answer to the server while the attempt is open. Because the draft lives on the server and not merely in the browser, a crash, an accidental tab close, or a switch to another device does not lose work: reopening the exam restores the saved answers. Output comparison during grading is whitespace-tolerant — trailing spaces and Windows versus Unix line endings are normalised on both sides — so a correct answer is not marked wrong over an invisible newline difference.

Submitting. *Submit* raises a confirmation dialog (Fig. 12.15) — submission is final and ends the attempt, so the extra click is deliberate. On confirmation the client sends every answer to the server, which records the submissions atomically and immediately begins grading them. The student then sees the post-submission screen (Fig. 12.16), which acknowledges receipt and explains that automatically graded questions score right away while any written answers await the teacher.

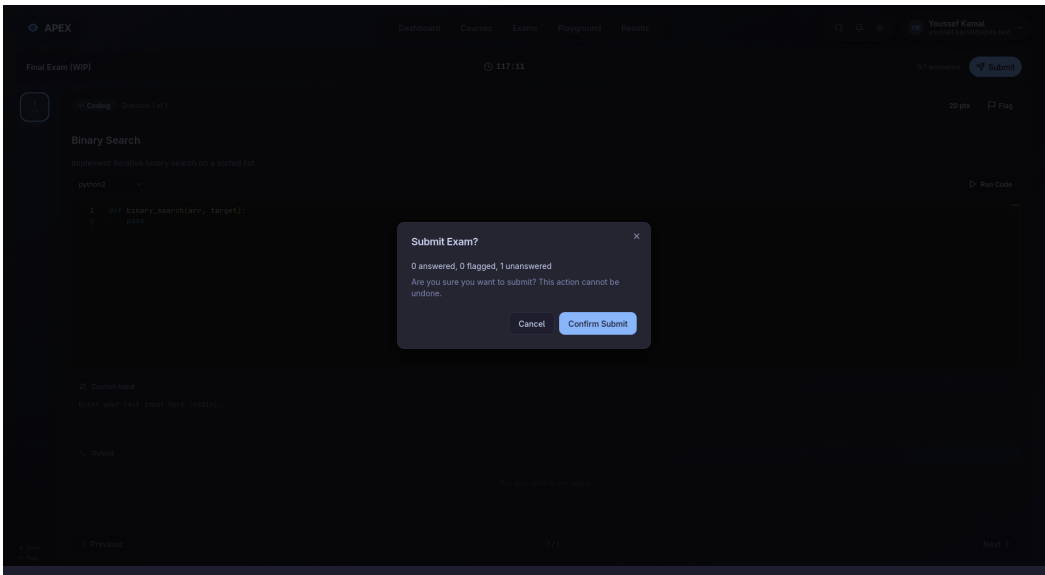


Figure 12.15. The submit confirmation dialog.

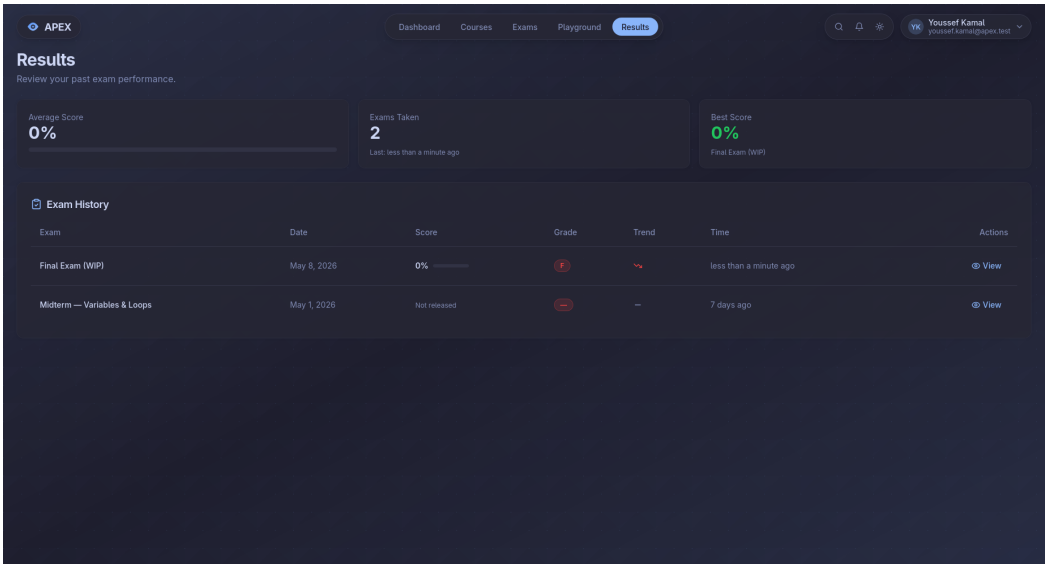


Figure 12.16. The post-submission acknowledgement screen.

12.4 Grading and Results

12.4.1 The manual-grading queue

Coding and multiple-choice answers are scored automatically within seconds of submission, but written answers wait for a human. The teacher’s *Grading* area (`/dashboard/grading`, Fig. 12.17) lists every submission still in *pending review*. Opening one shows the student’s answer alongside the rubric and inputs for a numeric score and free-text feedback (Fig. 12.18). Saving a grade both records the score and re-aggregates the student’s overall exam total; the sequence of steps from submission through automatic grading to this manual step is diagrammed in Fig. 7.1. Only the teacher who owns the exam may grade its submissions — the ownership check is the third authorization gate in Fig. 8.2.

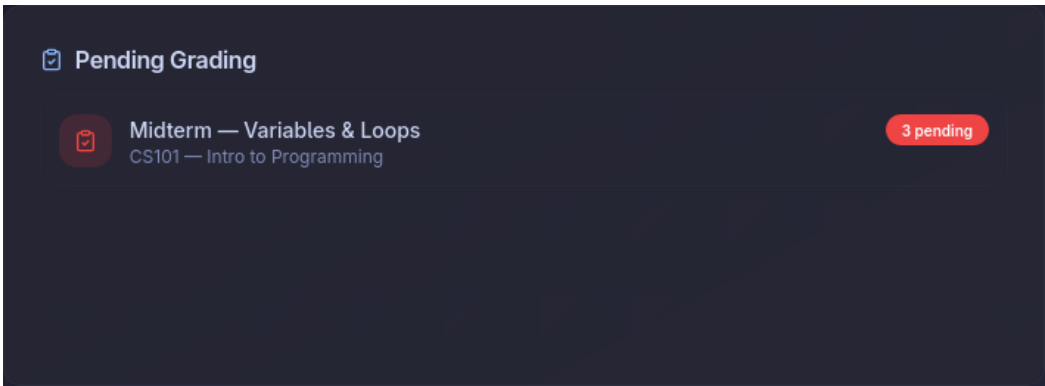


Figure 12.17. The pending-grading queue of written answers awaiting a score.

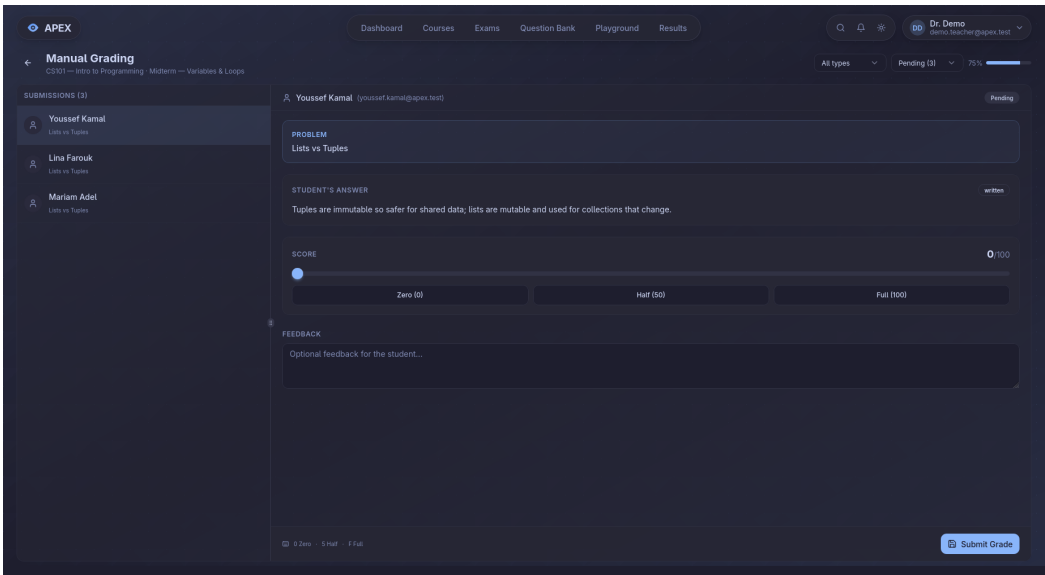


Figure 12.18. Grading a written answer against its rubric, with score and feedback inputs.

12.4.2 Reviewing results as a teacher

The per-exam results screen (Fig. 12.19) presents one row per student — total score, percentage, and time taken — giving the teacher an at-a-glance view of how a class performed. Selecting a row drills into that student’s per-question breakdown (Fig. 12.20), which shows how each answer scored, including per-test outcomes for coding questions. This is the teacher’s audit trail: it is where a disputed grade can be traced back to the exact test case that failed.

Student Results					
Student	ID	Score	Status	Time	Actions
Omar Hassan	18	8.2%	Failed	75 min 0s	View
Mariam Adel	21	20%	Failed	75 min 0s	View
Youssef Kamal	28	45%	Failed	75 min 0s	View
Lina Farouk	19	41%	Failed	75 min 0s	View

Figure 12.19. Per-student results for one exam: score, percentage, and time taken.

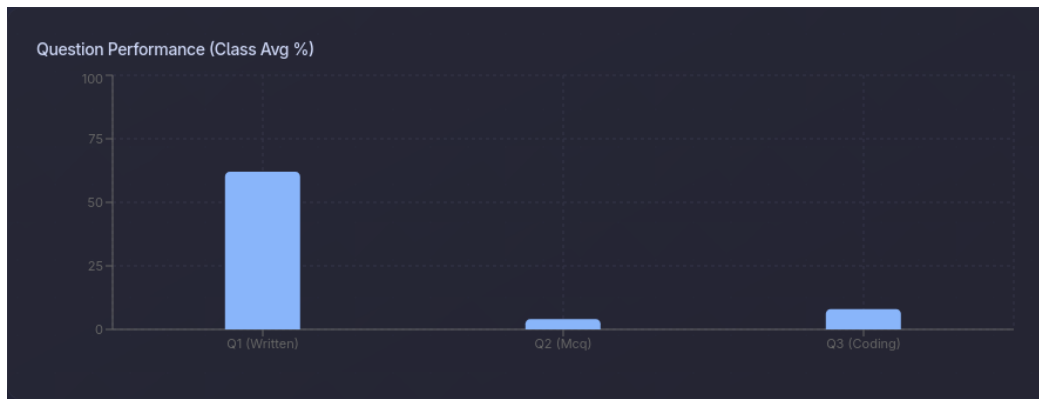


Figure 12.20. Per-question breakdown for a single student’s attempt.

12.4.3 Announcing and reviewing results as a student

Results are not visible to students the instant grading finishes. Two conditions must hold: the exam’s *show results after* behaviour must be on, and the teacher must flip *Grades announced* for the class. This two-step gate lets a teacher finish manual grading, sanity-check the distribution, and then release every student’s result together. Once released, the student’s *Results* area (Fig. 12.21) lists their attempts; opening one reveals the per-question result, the teacher’s written feedback, and — for coding questions — the outcomes of the sample tests only, keeping the hidden tests confidential for reuse in future exams.

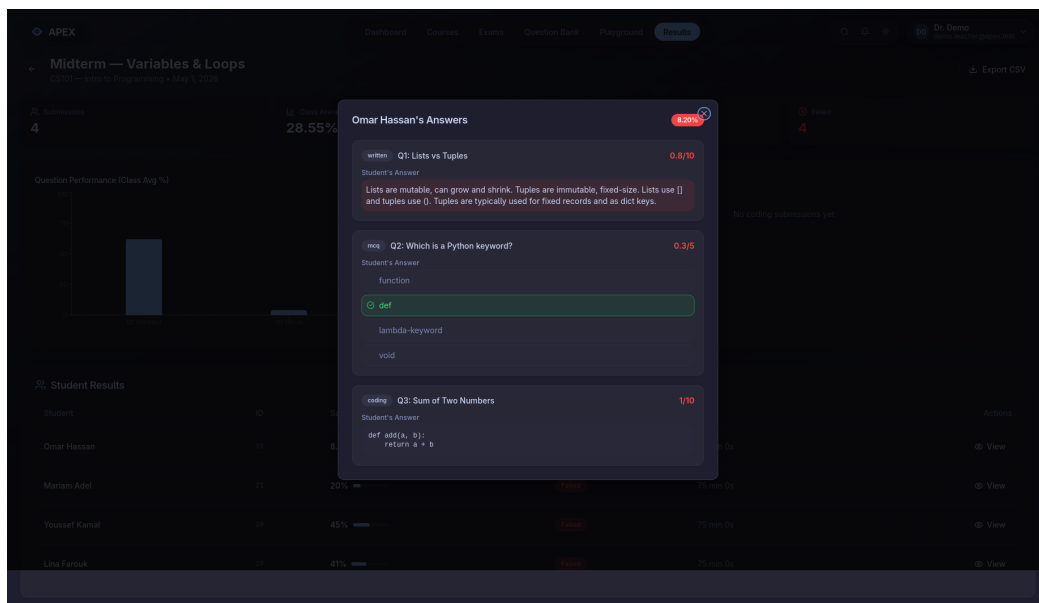


Figure 12.21. The student’s review screen: per-question results and teacher feedback.

12.5 Notifications, Profile, and Settings

Beyond the exam pipeline, three supporting surfaces are shared by both roles.

Notifications. A bell icon in the top bar opens a popover of recent notifications, and a dedicated notifications page lists the full history. Each notification carries a deep link to the thing it concerns — a new assignment, a posted announcement, an imminent exam, or the “Exam Graded” event that fires once a

student's whole attempt has finished grading. Exam reminders are also delivered by email: the in-process scheduler sweeps for published exams and sends both an in-app and an email reminder roughly an hour before an exam starts, and a second email as it begins, honouring each user's notification preferences.

Profile. The profile page lets a user edit their display name, a short biography, and an avatar image.

Settings. The settings page governs how the platform contacts the user. Separate toggles control in-app versus email notifications for each category — announcements, results, and the exam-reminder emails that the scheduler respects. Teachers additionally see per-account defaults, such as the default passing threshold applied to new exams and whether new exams start as drafts. This page is also where a user can permanently delete their account, which cascades through their owned or enrolled data in a single transaction.

12.6 Summary

APEX presents two coherent, role-shaped paths through one shared data model. The teacher path runs *create class* → *build mixed-format exam* → *assign and publish* → *grade* → *announce*; the student path runs *join by code* → *take timed exam in Monaco* → *submit* → *review*. The screenshots in this chapter are the load-bearing user-facing artefacts of the project, and together they show that the architecture described in the preceding chapters resolves, at the surface, into a small number of focused screens that a working teacher and a working student can operate without training.

Chapter 13

Conclusion and Future Work

This final chapter closes the thesis. It restates what APEX set out to do and what was actually built, distils the engineering lessons that were paid for in real incidents rather than borrowed from textbooks, and lays out a concrete roadmap that turns each known limitation into a scoped piece of future work. The intent throughout is to be specific and plain-spoken: the project names its rough edges and explains exactly how it would fix each one.

13.1 Summary of Contributions

The central claim of this thesis is that mixed-format programming assessment does not need a heavyweight platform to be taken seriously. APEX demonstrates this by unifying three assessment formats that have historically lived in three separate families of tools — online judges, quiz engines, and document hand-ins — into a *single* timed sitting with a *single* grading pipeline behind all three, packaged as a *single* self-hostable artifact. Concretely, the work delivered the following.

A unified mixed-format examination portal. Teachers author coding, multiple-choice (MCQ), and written (free-text) questions inside one exam, assign it to one or more classes, and set one timer. Students join a class by an eight-character invite code, sit the exam in a Monaco editor with live sample-test feedback, and review per-test results afterward. The system models fifteen first-class entities and exposes forty-seven feature pages across ten frontend feature areas, all behind exactly two roles — `teacher` and `student` — with no administrator role to complicate the trust model. The scale of the delivered surface is summarised in Fig. 11.1 and Fig. 11.2.

One grading pipeline behind all three formats. Rather than three graders bolted together, a single dispatch in the exam-submit handler routes each answer to `judge0.Grade` (coding), `judge0.GradeMCQ`, or `judge0.GradeWritten`, and every path ends by calling the same attempt aggregator, `maybeFinalizeAttempt`. Coding answers earn proportional partial credit ($\text{passed}/\text{total} \times 100$); MCQ answers are graded by exact set-equality; written answers enter a manual-grading queue. The full path from the editor to a final score and a single notification is traced in Fig. 7.1.

A sandboxed, fairness-corrected code execution engine. Untrusted student code never runs on the application host. It is delegated over HTTP to Judge0 [4], [5], which executes it inside the `isolate` sandbox — the same sandbox used at the International Olympiad in Informatics. APEX treats Judge0 as an *executor*, not a *judge*: it accepts both Judge0 status 3 (Accepted) and 4 (Wrong Answer) as “the program ran” and then re-compares the output itself through a nine-line `normalizeOutput` function applied symmetrically to the student’s output and the expected output. Those nine lines eliminate the single most common false negative in I/O-diff grading: a correct answer failing on a trailing newline, a Windows CRLF line ending, or a stray trailing space.

A self-hostable one-artifact deployment. The backend compiles to one statically linked Go binary (`CGO_ENABLED=0`, built with `-trimpath -ldflags="-s -w"`) shipped in a `gcr.io/distroless/static-debian12:nonroot` image that runs as an unprivileged user with no shell. A production instance is therefore just that binary, a PostgreSQL 16 database, and an external Judge0 endpoint; the `docker-compose.yml` stack composes exactly four services — `postgres`, a one-shot migrate job, `backend`, and an `nginx`-served frontend — with Judge0 reached externally via `JUDGE0_URL`. The deployment topology is shown in Fig. 10.1.

Defence-in-depth authorization and a pre-ship security review. Every request passes a four-layer life-cycle (CORS, authentication, role, handler) and a three-gate authorization model: HS256 JWT authentication, role checks, and per-handler resource-ownership checks. A security review run *before* shipping surfaced four findings — an IDOR on `/submissions/run`, a stored-XSS vector through Markdown `rehype-raw`, a weak fallback JWT secret, and HTML injection in outbound e-mail — all fixed in a single commit prior to release. The three-gate model and threat model are detailed in Chapter 8 and depicted in Fig. 8.1.

Table 13.1 maps the original project objectives onto these delivered outcomes.

Table 13.1. Project objectives and the outcome delivered against each.

Objective	Delivered outcome
Run coding, MCQ, and written questions in one timed exam	Achieved — one <code>ExamAttempt</code> per (student, exam), enforced by the <code>idx_user_exam_attempt</code> unique index; one submit endpoint creates all answer rows atomically.
One grading pipeline behind all formats	Achieved — a single dispatch and a shared aggregator (<code>maybeFinalizeAttempt</code>) with idempotent re-aggregation.
Safe execution of untrusted code	Achieved — delegated to Judge0/ <code>isolate</code> ; the host never executes student code [4].
Fair automatic grading	Achieved — symmetric output normalisation eliminates the trailing-whitespace false negative.
Self-hostable with minimal moving parts	Achieved — one distroless binary, PostgreSQL, and an external Judge0; a four-service compose stack.
Secure by construction	Largely achieved — three-gate authorization and a pre-ship review; several hardening gaps remain and are named in Section 13.3.

13.2 Lessons Learned

The most durable lessons of the project were not the ones that went smoothly. Three in particular changed how the team writes code, and each is worth recording because each was paid for with a real failure or a deliberate trade-off rather than absorbed from a manual.

13.2.1 The `is_draft` Incident and Schema-Change Discipline

The signature war story of the project — a single `gorm: "default:true"` struct tag that `AutoMigrate` applied as a *DDL* default to the already-populated `exams` table, silently flipping every published exam out of every student's view with no error and no stack trace — is told in full in Chapter 5. Its lesson is a standing schema-change discipline: a boolean's zero value must be the safe, non-destructive state; ownership of a populated-table change belongs in explicit, idempotent SQL rather than a struct tag; and production runs

with `AutoMigrate` disabled, leaving a versioned `golang-migrate` track [35] as the sole authority over the schema.

That discipline — audit every query when you add a filter column, define the zero-value behaviour of every boolean before you add it, and verify row counts after every migration — was distilled into the `ORM_RULES.md` rules file, whose first rule is blunt: **never rely on gorm: "default:x" for a DDL backfill on a populated table**. The general principle is old wisdom about the gap between an ORM's convenient abstraction and the DDL it silently emits [30], [33], [34]; APEX simply learned it the expensive way. The two-track migration pipeline that resulted is shown in Fig. 5.2.

13.2.2 The Value of Idempotent Migrations

The second lesson is a positive one that the first made unavoidable: every migration in APEX is written to be safely re-runnable. The dev-mode legacy SQL pass (`RunMigrations`) gates each block behind a `tableExists(...)` check so that a first boot against an empty database does not spew “relation does not exist” errors, and every column addition uses `ADD COLUMN IF NOT EXISTS` before backfilling. The reminder scheduler carries the same property into runtime: every reminder sweep dedups against a dedicated timestamp column (`reminder_sent_at`, `email_reminderlh_sent_at`, `email_reminder_start_sent_at`), so a process restart mid-sweep can never re-send a reminder. Idempotency turned out to be the cheapest possible form of operational safety: a migration or a scheduled job that can be run twice without harm removes an entire category of “did that already apply?” anxiety from every deploy and every crash recovery. It is the same principle the durable-queue future work below leans on — because grading recomputes results from the stored submission and test cases, it too is idempotent, and idempotent work is safe to retry.

13.2.3 Fire-and-Forget Versus Durable Work

The third lesson is a trade-off the team made with its eyes open, and it defines the largest single item of future work. Grading is dispatched as a bare goroutine — `go judge0.Grade(sub.ID)` — so the HTTP handler can return `201 Created` immediately; at the concurrency of a single classroom it works, but a bare goroutine has no supervisor, no queue, and no retry, so a crash mid-grade can strand a submission in `running` and leave the whole attempt permanently unfinalised. The mechanism and its failure mode are traced in full in Chapter 7 and documented openly in `README.md`. The lesson is not that fire-and-forget was wrong for the scale it was built for; it is that *durability is a property you must design in deliberately*, and that the right time to add a work queue is before you need to scale past one room, not after a mid-exam crash. The design of that queue is the first item below, and it is made tractable precisely by the idempotency lesson above.

13.3 Future Work

The remaining limitations of APEX are known, named, and prioritised. This section presents each as a scoped roadmap item: the gap, its current mitigation, and the concrete engineering that would close it. Table 13.2 gives the priority ordering; the subsections that follow give the detail.

Table 13.2. Future-work roadmap: each remaining gap with its current mitigation and planned fix.

Gap	Current mitigation	Planned fix
No auth rate limiting	bcrypt cost 10 only	Per-IP and per-account limiter on auth routes; front with a WAF.
Fire-and-forget grading	Works at classroom scale	Durable job queue, bounded worker pool, restart recovery sweep.
JWT in localStorage	No cookies, so no CSRF	httpOnly + SameSite cookies; refresh-token model.
Minimal test coverage	CI runs vet, lint, build	Handler-level tests per protected route; grader unit tests.
Six execution languages	Covers common cases	Add languages via <code>LanguageMap</code> ; push per-problem limits.
No plagiarism detection	Hidden tests unshown	MOSS / JPlag structural similarity over submissions.
Single-instance backend	Sufficient for one class	Stateless horizontal scaling plus metrics/tracing/logging.

13.3.1 Hardening Authentication: Rate Limiting and a WAF

The authentication endpoints have *no* rate limiting. The only brake on credential stuffing today is the deliberate cost of bcrypt at cost 10, and while that meaningfully slows an offline or online guessing campaign, it does not stop one. The cheapest and highest-value next mitigation is a per-IP *and* per-account rate limiter on `/auth/login`, `/auth/google`, and `/auth/forgot-password`, implemented as a small Gin middleware backed by an in-memory or Redis token bucket, coupled with exponential backoff on repeated failures for a given account. For a public deployment, the README’s standing recommendation is to front the entire API with a web application firewall (WAF) or an edge rate limiter so that abusive traffic is shed before it ever reaches the application. The companion password-policy change — raising the six-character minimum in line with modern password-strength guidance [47] — is a one-line validation change that should ship alongside the limiter. Both items sit at the top of the roadmap because, together, they are the weakest part of the current authentication story, and both align with the relevant guidance in the OWASP Top Ten and the Application Security Verification Standard [19], [46].

13.3.2 A Durable Grading Work Queue with Retries

The largest architectural item is replacing the fire-and-forget grading of Chapter 7 with a durable work queue. Concretely, `go judge0.Grade(sub.ID)` would be replaced by enqueueing a job — a row in a database-backed `grading_jobs` table, or a message on a Redis-backed broker such as `asynq` — consumed by a bounded pool of workers. Three properties follow from this design and directly close the fire-and-forget gap:

- **Crash recovery.** On startup, a recovery sweep re-picks any submission left in `running` (or any job left in-flight) and re-grades it. This is safe precisely because grading is *idempotent*: `Grade` recomputes every `TestResult` from the stored `submission.Code` plus the problem’s test cases and overwrites the previous results, so re-running it produces the same verdict. A stuck submission is no longer stuck forever; it is picked up on the next boot.
- **Bounded concurrency.** A worker pool caps the number of simultaneous Judge0 requests. Today a

200-student exam could spawn hundreds of concurrent POSTs and trip the public Judge0 CE rate limit; a pool with, say, a configurable maximum turns that burst into an orderly queue.

- **Retries with backpressure.** Transient Judge0 failures (the `StatusID = -1` sentinel today surfaces as a stuck or failed grade) become retrieable jobs with a bounded retry count and a dead-letter path for permanent failures.

Because the attempt aggregator already re-sums the children on every call and guards on “no siblings pending or running,” it needs no change to work with a queue — the last worker to finish still finds zero pending and finalises the attempt exactly once, deduped by the `graded_notified` flag. The queue is an additive change, not a rewrite.

13.3.3 httpOnly + SameSite Cookie Authentication

The JWT is currently stored in the browser’s `localStorage` under the key `kernel-token` and sent as an `Authorization: Bearer` header. Because no cookie is involved, the design has *no* CSRF exposure by construction — but any successful cross-site scripting (XSS) would be able to read `localStorage` and lift the token. The chosen trade was a simpler deployment; the hardening path is to move the session token into an `httpOnly, Secure, SameSite=Lax` (or `Strict`) cookie that JavaScript cannot read, which neutralises the token-theft-via-XSS vector at the cost of reintroducing the need for CSRF defences (a double-submit token or an origin check on state-changing requests). The same change is the natural moment to introduce a proper refresh-token model with a server-side revocation table, so that the current 24-hour non-revocable HS256 token can be invalidated on logout or compromise — an improvement squarely in line with established guidance on the safe use of JWTs and OAuth-style session handling [18], [40], [43], [47].

13.3.4 Real Test Coverage

Test coverage is the most straightforward gap to close and, arguably, the most valuable given how much of the system is security-sensitive. The backend currently ships *no* `*_test.go` files, and the frontend ships a single sanity test. Crucially, no pipeline work is required to add tests: the CI already spins up a `postgres:16-alpine` service and runs `go test ./... -race -count=1`, so the runners are in place and simply have nothing to exercise. The priority targets are the ones where a regression would be invisible and dangerous: a handler-level test per protected route asserting that the three authorization gates reject the wrong role and the wrong owner (an ownership-check regression is exactly the class of bug that the pre-ship IDOR finding represents); unit tests over `normalizeOutput` covering CRLF, lone CR, trailing whitespace, and trailing-newline cases; and tests over `maybeFinalizeAttempt` asserting that skipped questions stay in the denominator, that `pending_review` answers are excluded from both numerator and denominator, and that the “Exam Graded” notification fires exactly once. Building this suite would move the project meaningfully up the test pyramid described in Fig. 9.1, following standard software-engineering testing practice [23], [24].

13.3.5 More Execution Languages and Enforced Per-Problem Limits

APEX supports six execution languages — Python, JavaScript, TypeScript, Java, C, and C++ — resolved through the `LanguageMap` that maps a language string to a Judge0 numeric identifier. Adding a language is a small, localised change: a new entry in `LanguageMap` pointing at the corresponding Judge0 language id, and the rest of the pipeline is language-agnostic. Two adjacent improvements belong here. First,

the per-problem limits that already exist in the schema — `Problem.TimeLimitMs` (default 2000) and `Problem.MemoryLimitKb` (default 262144, i.e. 256 MB) — are *not* currently pushed down to Judge0; execution is bounded by Judge0's own defaults instead. Wiring `cpu_time_limit` and `memory_limit` into the `RunCode` request body is a small change that would let a teacher who sets a one-second limit actually have it enforced. Second, the grader currently sends and receives plain text (`base64_encoded=false`); moving to base64 transport would make the pipeline robust to exotic byte sequences in inputs and outputs.

13.3.6 Plagiarism Detection

APEX has no plagiarism detection today. Because the platform already stores every submission's source code alongside its language and the exam it belongs to, it is well positioned to run structural-similarity analysis across a cohort's submissions after an exam closes. The obvious approach is to integrate a document-fingerprinting detector in the style of MOSS [21] or a token-based structural comparator such as JPlag [22], both of which compare programs by structure rather than surface text and are therefore robust to renamed variables and reordered statements. The feature would surface a ranked list of suspiciously similar submission pairs to the teacher as a review aid — explicitly a signal for human judgement, not an automatic accusation. This pairs naturally with lighter-weight proctoring signals (tab-blur, fullscreen-exit, and paste events captured client-side) as a future integrity layer.

13.3.7 Horizontal Scaling and Observability

The backend runs today as a single instance, which is entirely sufficient for one classroom but is a ceiling for wider deployment. The good news is that the application is already close to horizontally scalable: it is a stateless Go process (all state lives in PostgreSQL) that follows the twelve-factor principles of configuration-through-environment and disposable processes [28]. The two things standing between the current design and running N replicas behind a load balancer are both addressed by the work above — the in-process reminder scheduler would need a leader-election lock so that only one replica runs the sweeps (or it would move into the same durable job system as grading), and grading would move from in-process goroutines to the shared work queue so that any worker can grade any submission. Alongside scaling, the project needs real observability: structured request logging, Prometheus-style metrics on grading latency and queue depth, and distributed tracing across the SPA → backend → Judge0 path. Today the system has liveness (`GET /healthz`) and a self-probing `--healthcheck` flag but no metrics or tracing; adding them is what turns “it seems to work” into “we can see it working, and we get paged before the student does.”

13.4 Closing Statement

APEX set out to build the smallest system that still takes mixed-format classroom assessment seriously, and it succeeded on its own terms: three assessment formats in one timed sitting, one grading pipeline behind them, and one self-hostable artifact of a Go binary, PostgreSQL, and an external Judge0 sandbox. Along the way the project learned — expensively, from a single struct tag that hid every exam from every student — that the boundary between an ORM's convenience and the DDL it silently emits is exactly where data integrity is won or lost, and it built a schema-change discipline and an idempotent, crash-resilient runtime around that lesson. The limitations that remain are not hidden: no auth rate limiting, fire-and-forget grading, a token in `localStorage`, thin test coverage, and no plagiarism detection. Each is named in this thesis,

each has a concrete fix on the roadmap above, and each was a deliberate trade for a system that a single instructor can actually stand up and run. That honesty is the point. The most useful thing a graduation project can leave behind is not the illusion of a finished product but a working system whose author knows exactly where its edges are and exactly how to extend them — and that is what APEX is.

Appendix A

API Reference

This appendix enumerates every HTTP route registered by the APEX backend. The tables are transcribed directly from the thirteen `RegisterRoutes` functions wired up in `main.go` (packages `auth`, `class`, `student`, `exam`, `problem`, `testcase`, `submission`, `execute`, `notification`, `profile`, `teacher`, `announcement`, `folder`); the `student` module additionally mounts three handlers from the `exam` package. Endpoints are grouped by the access level enforced through the layered middleware described in Fig. 8.2.

A.1 Access model

Every route except the liveness probe is mounted under one of two Gin groups declared in `main.go`:

- `public := r.Group("/api")` — no authentication; open to anonymous callers.
- `protected := r.Group("/api")` with `middleware.Auth(cfg.JWTSecret)` — requires a valid `Authorization: Bearer <jwt>` header (HS256, 24-hour expiry). A failed or missing token yields **401**.

Within the protected group, feature subgroups add `middleware.RequireRole("teacher")` or `("student")` as a second gate (**403** on mismatch). A third gate — resource ownership — is enforced inside the handlers themselves and is not visible in the route table. The base URL in development is `http://localhost:8080/api`.

Four protected modules (`notification`, `profile`, `announcement`, `folder`) register their routes without an explicit `RequireRole` call. For `notification` and `profile` this is deliberate — both roles own notifications and a profile. For `announcement` and `folder` the operations are teacher-facing and rely on handler-level ownership checks (`teacher_id` scoping) rather than a role gate; they are listed under the teacher section below for clarity, with that caveat noted.

A.2 Liveness

Method	Path	Description
GET	<code>/healthz</code>	Liveness probe; returns <code>{"status": "ok"}</code> . Registered outside the <code>/api</code> groups, so it takes no authentication. The container <code>HEALTHCHECK</code> self-probes this route via the <code>--healthcheck</code> flag.

A.3 Public

No authentication. All paths live under the `public.Group("/auth")` subgroup registered by the `auth` module.

Method	Path	Description
POST	<code>/api/auth/signup</code>	Create a user (email, password ≥ 6 chars, name, role); returns a JWT.
POST	<code>/api/auth/login</code>	Email + password login; returns a JWT or the generic “invalid email or password”.
POST	<code>/api/auth/forgot-password</code>	Request a password reset. Always 200 (no email enumeration); logs/sends a single-use token with a 1-hour expiry.
POST	<code>/api/auth/reset-password</code>	Consume the reset token and set a new password.
POST	<code>/api/auth/google</code>	Exchange a Google Identity Services credential (<code>id_token</code> , optional <code>role</code>) for a JWT; 503 if <code>GOOGLE_CLIENT_ID</code> is unset.

A.4 Protected: shared

Any authenticated user (either role). Auth required; no role gate.

Method	Path	Description
DELETE	<code>/api/auth/account</code>	Permanently delete the calling user’s account via a role-aware cascade in one DB transaction.
GET	<code>/api/profile</code>	Read the calling user’s profile.
PUT	<code>/api/profile</code>	Update profile fields (bio, avatar, notification toggles, teacher defaults).
PUT	<code>/api/profile/password</code>	Change the calling user’s password.
GET	<code>/api/notifications</code>	List the caller’s notifications.
PUT	<code>/api/notifications/:id/read</code>	Mark one notification read.
PUT	<code>/api/notifications/read-all</code>	Mark all of the caller’s notifications read.
GET	<code>/api/notifications/unread-count</code>	Unread count for the notification bell.
POST	<code>/api/submissions/run</code>	Run a candidate solution against the problem’s sample test cases; no <code>Submission</code> is persisted.
GET	<code>/api/submissions/:id</code>	Read a submission; ownership re-checked (student owner or exam-owning teacher).
POST	<code>/api/execute</code>	Standalone code execution (playground); no persistence.

The two `run/execute` endpoints and the submission read are the shared surface between the coding UI paths for both roles.

A.5 Protected: teacher

Auth + RequireRole("teacher"), unless noted. Handlers additionally enforce ownership (e.g. a teacher may only touch their own classes, exams, and problems).

A.5.1 Classes

Method	Path	Description
POST	/api/classes	Create a class; generates an 8-character invite code.
GET	/api/classes	List the teacher's classes.
GET	/api/classes/:id	Read one class with members.
PUT	/api/classes/:id	Update class fields (name, section, cover, thresholds).
DELETE	/api/classes/:id	Delete a class (cascades enrollment).
GET	/api/classes/:id/stats	Aggregate class statistics.
DELETE	/api/classes/:id/members/:userId	Remove a student from the class.

A.5.2 Exams

Method	Path	Description
POST	/api/exams	Create an exam (defaults to draft).
GET	/api/exams	List the teacher's exams.
GET	/api/exams/:id	Read one exam with problems and class assignments.
PUT	/api/exams/:id	Update an exam; destructive edits may trigger the reset_at wipe.
DELETE	/api/exams/:id	Delete an exam (cascades problems and attempts).
POST	/api/exams/:id/assign	Assign the exam to one or more classes; validates every class_id belongs to the caller.
POST	/api/exams/:id/close	Close the exam window immediately.
POST	/api/exams/:id/reopen	Reopen a closed exam.
GET	/api/exams/:id/results	Per-student results for the exam.

A.5.3 Problems and question bank

Method	Path	Description
POST	/api/exams/:id/problems	Add a problem to an exam.
GET	/api/problems	List all of the teacher's problems (exam-bound and bank).
POST	/api/problems/bank	Create a standalone question-bank problem (no exam).

GET	/api/problems/:id	Read one problem.
PUT	/api/problems/:id	Update a problem.
DELETE	/api/problems/:id	Delete a problem.

A.5.4 Test cases

Method	Path	Description
POST	/api/problems/:id/test-cases	Add a test case to a coding problem.
PUT	/api/test-cases/:id	Update a test case (input, expected output, sample flag, points).
DELETE	/api/test-cases/:id	Delete a test case.

A.5.5 Grading and dashboard

Method	Path	Description
GET	/api/teacher/dashboard	Teacher dashboard summary.
GET	/api/teacher/grading/pending	Pending manual-grading queue (written answers, un-configured MCQs).
PUT	/api/submissions/:id/grade	Manually grade a submission (score, status, teacher_feedback); gated to the exam owner and re-runs attempt finalization.

The grade route is registered by the `submission` module with an inline `RequireRole("teacher")` on that single verb, while its sibling `run/:id` routes stay shared.

A.5.6 Announcements and folders

Registered under `protected` without an explicit role gate; teacher ownership is enforced in the handlers via `teacher_id` scoping.

Method	Path	Description
GET	/api/classes/:id/announcements	List announcements for a class.
POST	/api/classes/:id/announcements	Create an announcement; fans out notifications to enrolled students.
PUT	/api/announcements/:id	Edit an announcement.
DELETE	/api/announcements/:id	Delete an announcement.
GET	/api/folders	List the teacher's bank-question folders.
POST	/api/folders	Create a folder.
PUT	/api/folders/:id	Rename a folder.
DELETE	/api/folders/:id	Delete a folder (problem references set to <code>NULL</code>).

A.6 Protected: student

`Auth + RequireRole("student")`. Registered by the `student` module; the three exam-attempt verbs mount handlers from the `exam` package.

Method	Path	Description
POST	<code>/api/student/classes/join</code>	Join a class by invite code (unique per student via <code>idx_class_user</code>).
DELETE	<code>/api/student/classes/:id</code>	Leave a class.
GET	<code>/api/student/classes</code>	List the student's enrolled classes.
GET	<code>/api/student/classes/:id</code>	Read one enrolled class.
GET	<code>/api/student/exams</code>	List published exams visible to the student.
GET	<code>/api/student/exams/:id</code>	Read one exam for taking.
GET	<code>/api/student/submissions</code>	List the student's submissions.
GET	<code>/api/student/stats</code>	Student statistics summary.
GET	<code>/api/student/performance</code>	Performance history/metrics.
POST	<code>/api/student/exams/:id/start</code>	Start an attempt (one row per student+exam via <code>idx_user_exam_attempt</code>).
PUT	<code>/api/student/exams/:id/autosave</code>	Autosave the opaque draft-answers payload mid-exam.
POST	<code>/api/student/exams/:id/submit</code>	Submit the attempt; creates submissions and triggers auto-grading.
GET	<code>/api/attempt/mine</code>	Fetch the caller's attempts (reused on results pages).

Appendix B

Database Schema (DDL)

This appendix records the durable relational schema behind APEX. The canonical source is the set of GORM model structs under `internal/models/`; the data-definition language (DDL) reproduced here is the schema those structs project onto PostgreSQL 16, transcribed field-for-field from the `gorm` struct tags (column types, sizes, `NOT NULL` constraints, defaults, and indexes). All fifteen models are registered, in dependency order, in `database.AutoMigrate(...)`. Where a tag such as `gorm:"default:X"` appears, it is reproduced as a column `DEFAULT`; the honest caveat — that DDL defaults must never be introduced on a populated table — is the subject of the `is_draft` incident recounted in the migration chapter, and is why production disables `AutoMigrate` in favour of the versioned `golang-migrate` track.

A few conventions apply throughout. Every model embeds an `ID uint` with `gorm:"primaryKey"`, which GORM maps to `BIGSERIAL PRIMARY KEY` (an auto-incrementing bigint). Go `uint` foreign-key fields become `BIGINT`; a pointer field such as `*uint` or `*time.Time` is nullable, whereas a value field is `NOT NULL` only where the tag says so. `time.Time` maps to `TIMESTAMPTZ`; `autoCreateTime` supplies a creation timestamp. Fields tagged `type:jsonb` are stored as PostgreSQL `JSONB`. The reference DDL below omits the foreign-key relationship structs (e.g. the embedded `User Teacher`), which carry no columns of their own.

B.1 Identity and Accounts

The `users` table is the root of every ownership chain; both `teacher` and `student` rows live here, distinguished only by the `role` column (there is no separate admin table). The password hash and the Google subject are hidden from JSON responses (`json:"-"`) but are ordinary columns at rest. `user_profiles` is a one-to-one satellite keyed by a unique `user_id`, holding notification toggles and the per-teacher authoring defaults.

```
1 CREATE TABLE users (  
2     id                BIGSERIAL PRIMARY KEY,  
3     email             VARCHAR(255) NOT NULL,  
4     password_hash     TEXT          NOT NULL,    -- bcrypt, JSON-hidden  
5     role              VARCHAR(20)   NOT NULL,    -- 'teacher' | 'student'  
6     name              VARCHAR(255) NOT NULL,  
7     google_id         VARCHAR(64),    -- JSON-hidden  
8     created_at        TIMESTAMPTZ  
9 );  
10 CREATE UNIQUE INDEX idx_users_email ON users (email);  
11 CREATE INDEX      idx_users_google_id ON users (google_id);  
12  
13 CREATE TABLE user_profiles (  
14     id                BIGSERIAL PRIMARY KEY,
```

```

15     user_id          BIGINT,
16     bio             TEXT,
17     avatar_url      TEXT,          -- base64 dataURL or remote URL
18     notify_email    BOOLEAN DEFAULT true,
19     notify_push     BOOLEAN DEFAULT true,
20     notify_exam_reminders  BOOLEAN DEFAULT true,
21     notify_results  BOOLEAN DEFAULT false,
22     notify_exam_email  BOOLEAN DEFAULT true,
23     block_announce_with_pending  BOOLEAN DEFAULT true,
24     default_exam_draft  BOOLEAN DEFAULT true,
25     default_passing_threshold  INTEGER DEFAULT 60
26 );
27 CREATE UNIQUE INDEX idx_user_profiles_user_id ON user_profiles (user_id);

```

Listing B.1. users and user_profiles

The `password_reset_tokens` table stores only the SHA-256 hash of a reset token (the plaintext is never persisted), with a single-use marker (`used_at`) and a short expiry.

```

1 CREATE TABLE password_reset_tokens (
2     id          BIGSERIAL PRIMARY KEY,
3     user_id    BIGINT NOT NULL,
4     token_hash VARCHAR(64) NOT NULL,    -- SHA-256 hex, JSON-hidden
5     expires_at TIMESTAMPTZ,             -- 1 hour after issue
6     used_at    TIMESTAMPTZ,             -- NULL until redeemed
7     created_at TIMESTAMPTZ
8 );
9 CREATE INDEX      idx_password_reset_tokens_user_id ON password_reset_tokens (user_id)
10 ↪ ;
11 CREATE UNIQUE INDEX idx_password_reset_tokens_token_hash ON password_reset_tokens (
12 ↪ token_hash);

```

Listing B.2. password_reset_tokens

B.2 Classes and Enrolment

A class belongs to one teacher and carries an eight-character unique `invite_code` that students redeem to enrol. `class_members` is the enrolment join table; its composite unique index `idx_class_user` (`class_id`, `user_id`) guarantees a student joins any class at most once.

```

1 CREATE TABLE classes (
2     id          BIGSERIAL PRIMARY KEY,
3     teacher_id  BIGINT NOT NULL,
4     name        VARCHAR(255) NOT NULL,
5     section     VARCHAR(100),
6     invite_code  VARCHAR(8) NOT NULL,
7     cover_image TEXT,
8     grades_announced  BOOLEAN NOT NULL DEFAULT false,
9     passing_threshold  INTEGER NOT NULL DEFAULT 60,
10    block_announce_with_pending  BOOLEAN NOT NULL DEFAULT true,
11    created_at    TIMESTAMPTZ
12 );

```

```

13 CREATE INDEX          idx_classes_teacher_id ON classes (teacher_id);
14 CREATE UNIQUE INDEX idx_classes_invite_code ON classes (invite_code);
15
16 CREATE TABLE class_members (
17     id          BIGSERIAL PRIMARY KEY,
18     class_id    BIGINT NOT NULL,
19     user_id     BIGINT NOT NULL,
20     joined_at   TIMESTAMPTZ          -- autoCreateTime
21 );
22 CREATE INDEX          idx_class_members_class_id ON class_members (class_id);
23 CREATE INDEX          idx_class_members_user_id  ON class_members (user_id);
24 CREATE UNIQUE INDEX idx_class_user              ON class_members (class_id, user_id);

```

Listing B.3. classes and class_members

B.3 Exams and Assignment

An exam is owned by a teacher and assigned to zero or more classes through the `exam_classes` join table, whose composite unique index `idx_exam_class (exam_id, class_id)` makes each assignment idempotent. The three timestamp columns `reminder_sent_at`, `email_reminderlh_sent_at`, and `email_reminder_start_sent_at` are the column-based deduplication stamps consumed by the reminder scheduler, so that a process restart cannot re-send. `reset_at` is indexed because the take screen filters on it to invalidate stale attempts. Note the safe default on `is_draft`: `false` (published) is the zero value, the corrective outcome of the `is_draft` incident.

```

1 CREATE TABLE exams (
2     id          BIGSERIAL PRIMARY KEY,
3     teacher_id  BIGINT      NOT NULL,
4     title       VARCHAR(255) NOT NULL,
5     description  TEXT,
6     duration_minutes INTEGER NOT NULL DEFAULT 60,
7     start_time   TIMESTAMPTZ,          -- nullable (*time.Time)
8     end_time     TIMESTAMPTZ,          -- nullable
9     reset_at     TIMESTAMPTZ,          -- nullable, indexed
10    reminder_sent_at TIMESTAMPTZ,       -- in-app T-60 dedup
11    email_reminderlh_sent_at TIMESTAMPTZ, -- email T-60 dedup
12    email_reminder_start_sent_at TIMESTAMPTZ, -- email T-0 dedup
13    created_at     TIMESTAMPTZ,
14    shuffle_questions BOOLEAN DEFAULT false,
15    show_results_after BOOLEAN DEFAULT true,
16    passing_score   INTEGER DEFAULT 50,
17    is_practice     BOOLEAN DEFAULT false,
18    is_draft        BOOLEAN DEFAULT false -- zero value = published
19 );
20 CREATE INDEX idx_exams_teacher_id ON exams (teacher_id);
21 CREATE INDEX idx_exams_reset_at   ON exams (reset_at);
22
23 CREATE TABLE exam_classes (
24     id          BIGSERIAL PRIMARY KEY,
25     exam_id     BIGINT NOT NULL,
26     class_id    BIGINT NOT NULL

```

```

27 );
28 CREATE INDEX      idx_exam_classes_exam_id ON exam_classes (exam_id);
29 CREATE INDEX      idx_exam_classes_class_id ON exam_classes (class_id);
30 CREATE UNIQUE INDEX idx_exam_class          ON exam_classes (exam_id, class_id);

```

Listing B.4. exams and exam_classes

B.4 Problems and Test Cases

The problems table is deliberately wide: a single row can model a coding, mcq, or written question, with format-specific columns left null when unused. A problem may belong to an exam (exam_id, nullable) or live in the reusable question bank (is_bank = true, in which case it may be grouped under a folder). The folder_id foreign key uses ON DELETE SET NULL so deleting a folder orphans rather than destroys its questions. Defaults encode the execution budget: time_limit_ms of 2000 ms and memory_limit_kb of 262144 KiB (256 MB) per test case.

```

1 CREATE TABLE problems (
2     id                BIGSERIAL PRIMARY KEY,
3     exam_id           BIGINT,                -- nullable: bank Qs have none
4     is_bank           BOOLEAN NOT NULL DEFAULT false,
5     teacher_id        BIGINT,
6     class_id          BIGINT,                -- nullable
7     folder_id         BIGINT REFERENCES folders(id) ON DELETE SET NULL,
8     title             VARCHAR(255) NOT NULL,
9     description        TEXT NOT NULL,
10    type              VARCHAR(20) NOT NULL DEFAULT 'coding',
11    points             INTEGER NOT NULL DEFAULT 10,
12    difficulty         VARCHAR(20) NOT NULL DEFAULT 'medium',
13    starter_code       TEXT,
14    hints              TEXT,
15    time_limit_ms      INTEGER NOT NULL DEFAULT 2000,
16    memory_limit_kb    INTEGER NOT NULL DEFAULT 262144,
17    order_index        INTEGER NOT NULL DEFAULT 0,
18    created_at         TIMESTAMPTZ,
19    -- MCQ fields
20    options             JSONB,
21    correct_option_ids  JSONB,
22    multiple_correct    BOOLEAN DEFAULT false,
23    explanation        TEXT,
24    -- Written fields
25    max_word_count     INTEGER DEFAULT 500,
26    rubric             TEXT,
27    require_manual_grading BOOLEAN DEFAULT false,
28    -- Media / organization
29    image_url          TEXT,
30    tags               JSONB DEFAULT '[]'
31 );
32 CREATE INDEX idx_problems_exam_id ON problems (exam_id);
33 CREATE INDEX idx_problems_is_bank ON problems (is_bank);
34 CREATE INDEX idx_problems_teacher_id ON problems (teacher_id);
35 CREATE INDEX idx_problems_class_id ON problems (class_id);

```



```
36 CREATE INDEX idx_problems_folder_id ON problems (folder_id);
```

Listing B.5. problems

Each coding problem owns a set of `test_cases`. Both `input` and `expected_output` are non-null text; `is_sample` distinguishes the visible sample tests (which drive live feedback) from the hidden tests that run only during real grading.

```
1 CREATE TABLE test_cases (
2     id                BIGSERIAL PRIMARY KEY,
3     problem_id        BIGINT  NOT NULL,
4     input             TEXT    NOT NULL,
5     expected_output    TEXT    NOT NULL,
6     is_sample         BOOLEAN NOT NULL DEFAULT false,
7     points            INTEGER NOT NULL DEFAULT 0,
8     order_index        INTEGER NOT NULL DEFAULT 0
9 );
10 CREATE INDEX idx_test_cases_problem_id ON test_cases (problem_id);
```

Listing B.6. test_cases

B.5 Attempts, Submissions, and Results

An `exam_attempt` is the per-student sitting of an exam; the composite unique index `idx_user_exam_attempt (user_id, exam_id)` enforces exactly one attempt row per student per exam. `draft_answers` is an opaque JSON autosave payload (nullable) that lets a student resume on a fresh device, cleared once the attempt is submitted. `score` is a `DOUBLE PRECISION` percentage produced by the aggregation pass.

```
1 CREATE TABLE exam_attempts (
2     id                BIGSERIAL PRIMARY KEY,
3     user_id           BIGINT      NOT NULL,
4     exam_id           BIGINT      NOT NULL,
5     started_at        TIMESTAMPTZ,           -- autoCreateTime
6     submitted_at      TIMESTAMPTZ,           -- nullable
7     score             DOUBLE PRECISION NOT NULL DEFAULT 0,
8     status            VARCHAR(30)  NOT NULL DEFAULT 'in_progress',
9     graded_notified    BOOLEAN      NOT NULL DEFAULT false,
10    draft_answers      JSONB,              -- nullable autosave blob
11    draft_saved_at     TIMESTAMPTZ          -- nullable
12 );
13 CREATE INDEX          idx_exam_attempts_user_id ON exam_attempts (user_id);
14 CREATE INDEX          idx_exam_attempts_exam_id ON exam_attempts (exam_id);
15 CREATE UNIQUE INDEX   idx_user_exam_attempt     ON exam_attempts (user_id, exam_id);
```

Listing B.7. exam_attempts

A submission records one answer to one problem. Its `exam_attempt_id` is nullable because playground and sample runs never persist an attempt link. The `status` column follows the lifecycle pending → running → a terminal value (accepted, wrong_answer, compilation_error, time_limit_exceeded, runtime_error, or pending_review).

```

1 CREATE TABLE submissions (
2     id                BIGSERIAL PRIMARY KEY,
3     user_id           BIGINT      NOT NULL,
4     problem_id        BIGINT      NOT NULL,
5     exam_id           BIGINT      NOT NULL,
6     exam_attempt_id   BIGINT,      -- nullable: playground runs
7     type              VARCHAR(20)  NOT NULL DEFAULT 'coding',
8     language          VARCHAR(20),
9     code              TEXT,
10    selected_options   JSONB,      -- MCQ choices
11    text_answer        TEXT,      -- written answer
12    status             VARCHAR(30)  NOT NULL DEFAULT 'pending',
13    teacher_feedback   TEXT,
14    passed_count       INTEGER      NOT NULL DEFAULT 0,
15    total_count        INTEGER      NOT NULL DEFAULT 0,
16    score              DOUBLE PRECISION NOT NULL DEFAULT 0,
17    execution_time_ms  INTEGER      DEFAULT 0,
18    memory_kb          INTEGER      DEFAULT 0,
19    submitted_at       TIMESTAMPTZ  -- autoCreateTime
20 );
21 CREATE INDEX idx_submissions_user_id      ON submissions (user_id);
22 CREATE INDEX idx_submissions_problem_id   ON submissions (problem_id);
23 CREATE INDEX idx_submissions_exam_id     ON submissions (exam_id);
24 CREATE INDEX idx_submissions_exam_attempt_id ON submissions (exam_attempt_id);

```

Listing B.8. submissions

Each coding submission fans out to one `test_result` per test case executed, capturing the pass/fail outcome, the captured `actual_output`, resource usage, and the raw Judge0 status label.

```

1 CREATE TABLE test_results (
2     id                BIGSERIAL PRIMARY KEY,
3     submission_id     BIGINT      NOT NULL,
4     test_case_id      BIGINT      NOT NULL,
5     passed            BOOLEAN     NOT NULL DEFAULT false,
6     actual_output      TEXT,
7     execution_time_ms  INTEGER     DEFAULT 0,
8     memory_kb         INTEGER     DEFAULT 0,
9     status            VARCHAR(30)
10 );
11 CREATE INDEX idx_test_results_submission_id ON test_results (submission_id);
12 CREATE INDEX idx_test_results_test_case_id  ON test_results (test_case_id);

```

Listing B.9. test_results

B.6 Communication and Organisation

The remaining tables carry classroom communication and teacher-side organisation. An announcement is posted by a teacher to a class and fans out notification rows to enrolled students; attachments is a JSONB array. A notification is the per-user inbox item. folders provide per-teacher grouping for bank questions (referenced by `problems.folder_id`).

```

1 CREATE TABLE announcements (
2     id          BIGSERIAL PRIMARY KEY,
3     class_id    BIGINT      NOT NULL,
4     teacher_id BIGINT      NOT NULL,
5     title       VARCHAR(255) NOT NULL,
6     body        TEXT,
7     attachments JSONB,
8     created_at  TIMESTAMPTZ          -- autoCreateTime
9 );
10 CREATE INDEX idx_announcements_class_id ON announcements (class_id);
11 CREATE INDEX idx_announcements_teacher_id ON announcements (teacher_id);
12
13 CREATE TABLE notifications (
14     id          BIGSERIAL PRIMARY KEY,
15     user_id     BIGINT      NOT NULL,
16     type        VARCHAR(20) NOT NULL,          -- e.g. 'exam_reminder'
17     title       VARCHAR(255) NOT NULL,
18     description TEXT,
19     link_to     VARCHAR(255),
20     read        BOOLEAN DEFAULT false,
21     created_at  TIMESTAMPTZ
22 );
23 CREATE INDEX idx_notifications_user_id ON notifications (user_id);
24
25 CREATE TABLE folders (
26     id          BIGSERIAL PRIMARY KEY,
27     teacher_id BIGINT      NOT NULL,
28     name        VARCHAR(255) NOT NULL,
29     created_at  TIMESTAMPTZ
30 );
31 CREATE INDEX idx_folders_teacher_id ON folders (teacher_id);

```

Listing B.10. announcements, notifications, folders

B.7 Constraint Summary

Three composite unique indexes carry the schema’s most important integrity invariants, and each maps directly to a domain rule enforced without additional application code:

- `idx_class_user` on `class_members` (`class_id`, `user_id`) — a student may join a given class at most once.
- `idx_user_exam_attempt` on `exam_attempts` (`user_id`, `exam_id`) — exactly one attempt row per (student, exam) pair.
- `idx_exam_class` on `exam_classes` (`exam_id`, `class_id`) — an exam is linked to a class at most once, making re-assignment idempotent.

Beyond these, single-column unique indexes protect `users.email`, `classes.invite_code`, `password_reset_tokens.token_hash`, and `user_profiles.user_id`. The lone declared foreign key with an explicit referential action is `problems.folder_id` (`ON DELETE SET NULL`); the remaining ownership relationships are modelled as indexed `BIGINT` columns and enforced at the resource-ownership

gate in the handlers rather than by database-level `FOREIGN KEY` constraints. The entity–relationship view of these tables is given in Fig. 5.1, and the evolution of the schema through the migration pipeline is traced in Fig. 5.2.

Appendix C

Environment Variables

APEX follows the twelve-factor discipline of keeping all deployment-specific configuration in the process environment rather than in code [28]. Every backend setting is read once at boot by `config.Load()` in `internal/config/config.go`; a companion `config.init()` calls `godotenv.Load()` so a local `.env` file is transparently promoted into the environment before `Load()` runs. This appendix documents every variable the backend consults, the four build-time variables the frontend bakes in, and the one Compose-only convenience variable, each faithful to the source rather than to convention.

C.1 How configuration is read

With a single exception, every backend variable flows through the helper `getEnv(key, fallback)`, which returns the environment value when non-empty and the compiled-in fallback otherwise. Consequently an *empty* value and an *absent* value are treated identically: exporting `DB_HOST=` is the same as not exporting it at all, and both yield `localhost`. The exception is `JWT_SECRET`, which has no fallback and is validated eagerly.

The validation of `JWT_SECRET` is the only place `Load()` can terminate the process. The guard rejects three conditions in one predicate—an empty value, the literal placeholder `dev-secret-change-in-prod`, and any value shorter than 32 characters—and on any of them calls `log.Fatal`, so the server never starts with a weak or default HMAC key:

```
1 jwtSecret := os.Getenv("JWT_SECRET")
2 if jwtSecret == "" || jwtSecret == "dev-secret-change-in-prod" || len(jwtSecret) < 32 {
3     log.Fatal("JWT_SECRET must be set to a strong random value (>=32 chars)")
4 }
```

Listing C.1. The fail-fast secret guard in `config.Load()`

Two variables interact rather than stand alone. `DATABASE_URL`, when set, is returned verbatim by both `Config.DSN()` (for the GORM/pgx connection) and `Config.MigrateURL()` (for `golang-migrate`), short-circuiting all six discrete `DB_*` variables; this is the production path for managed providers such as Neon or Railway. `ALLOWED_ORIGINS` is split on commas, trimmed, and merged with `APP_URL` into the single CORS allow-list slice consumed by `middleware.CORS`, so `APP_URL` is always an allowed origin even when `ALLOWED_ORIGINS` is empty.

C.2 Backend variables

Table C.1 lists every variable read by the backend. The *Required?* column distinguishes the one hard requirement (`JWT_SECRET`) from variables that are optional only because a compiled-in default exists or because an empty value degrades gracefully.

Table C.1. Backend environment variables (from `internal/config/config.go` and `.env.example`).

Name	Required?	Default	Purpose
JWT_SECRET	Yes	<i>none</i>	HS256 signing key for the 24-hour session token [18]. Must be ≥ 32 characters and must not be the placeholder <code>dev-secret-change-in-prod</code> ; any violation aborts startup via <code>log.Fatal</code> (Listing C.1).
DATABASE_URL	No	<i>empty</i>	Full PostgreSQL connection URL. When non-empty it takes precedence over every <code>DB_*</code> variable for both the application and <code>cmd/migrate</code> . The production path for Neon/Railway; use <code>sslmode=require</code> in the query string.
DB_HOST	No	<code>localhost</code>	PostgreSQL host, used only when <code>DATABASE_URL</code> is empty.
DB_PORT	No	<code>5432</code>	PostgreSQL port (discrete-vars path).
DB_USER	No	<code>postgres</code>	PostgreSQL user (discrete-vars path).
DB_PASSWORD	No	<code>postgres</code>	PostgreSQL password; omitted from the DSN when empty.
DB_NAME	No	<code>apex</code>	Database name (discrete-vars path).
DB_SSLMODE	No	<code>disable</code>	<code>sslmode</code> for the DSN; set to <code>require</code> for managed providers.
PORT	No	<code>8080</code>	HTTP listen port. Also probed by the <code>--healthcheck</code> self-probe and reported at <code>GET /healthz</code> .
SKIP_AUTOMIGRATE	No	<i>empty (false)</i>	Parsed case-insensitively; <code>true</code> disables GORM <code>AutoMigrate</code> so the versioned <code>golang-migrate track</code> (<code>cmd/migrate</code>) becomes the sole schema authority. Set <code>true</code> in production.
MIGRATIONS_DIR	No	<code>file://internal/databases/migrations/sql</code>	Source directory for the versioned SQL migrations, consumed by the migration runner and <code>cmd/migrate</code> ; override for non-default deployment layouts.
JUDGE0_URL	No	<code>https://ce.judge0.com</code>	Base URL of the Judge0 code-execution service [4]. The default targets the public, rate-limited instance; self-hosting is strongly recommended for real use.

continued on next page

Name	Required?	Default	Purpose
APP_URL	No	<code>http://localhost:5173</code>	Primary frontend origin. Embedded in password-reset email links (<code><APP_URL>/auth/reset?token=...</code>) and always merged into the CORS allow-list.
ALLOWED_ORIGINS	No	<i>empty</i>	Comma-separated additional CORS origins (e.g. preview deployments), trimmed and appended to APP_URL.
GOOGLE_CLIENT_ID	No	<i>empty</i>	Google Identity Services web client ID used to verify sign-in tokens [42]. When empty, <code>POST /auth/google</code> returns 503. Must match the frontend's <code>VITE_GOOGLE_CLIENT_ID</code> .
SMTP_HOST	No	<i>empty</i>	Outbound mail host. When empty, all email bodies (reminders, reset links) are logged to stdout instead of sent; in-app notifications are unaffected.
SMTP_PORT	No	587	Mail submission port; ignored when SMTP_HOST is empty.
SMTP_USER	No	<i>empty</i>	SMTP username; ignored when SMTP_HOST is empty.
SMTP_PASS	No	<i>empty</i>	SMTP password / app-password; ignored when SMTP_HOST is empty.
SMTP_FROM	No	<i>empty</i>	Envelope/From address; ignored when SMTP_HOST is empty.

C.2.1 Notes on individual variables

Secrets. `JWT_SECRET` is the only value whose absence is fatal; treat it as a rotation-worthy secret and generate it with, for example, `openssl rand -hex 32`. `DB_PASSWORD` and the `SMTP_*` credentials should likewise come from a secret manager rather than a committed file.

Graceful degradation. Three subsystems are deliberately optional and fail soft rather than hard: an empty `GOOGLE_CLIENT_ID` disables only the Google sign-in endpoint; an empty `SMTP_HOST` redirects email to the log; and an empty `ALLOWED_ORIGINS` simply leaves `APP_URL` as the sole cross-origin entry. This lets a developer clone the repository and run a fully functional local instance with nothing but a valid `JWT_SECRET` and a reachable database.

Schema authority. `SKIP_AUTOMIGRATE` and `MIGRATIONS_DIR` exist because of the migration war story recounted in Chapter C of the main text: in production, disabling `AutoMigrate` and delegating to the versioned SQL track removes GORM's ability to silently rewrite existing rows.

C.3 Frontend variables

The React client is a static bundle, so its configuration is resolved at *build* time by Vite and inlined into the emitted JavaScript; changing a `VITE_*` value therefore requires a rebuild, not merely a restart. Table C.2 lists them, drawn from `frontend/.env.example` and the fact sheet.

Table C.2. Frontend build-time environment variables.

Name	Required?	Default	Purpose
<code>VITE_API_URL</code>	No	<code>/api</code>	Base path/URL the client uses for API calls. The default <code>/api</code> works with the Vite dev proxy and a co-located reverse proxy in production.
<code>VITE_GOOGLE_CLIENT_ID</code>	No	<i>empty</i>	Google Identity Services web client ID for the sign-in button; must equal the backend's <code>GOOGLE_CLIENT_ID</code> or token verification fails.
<code>VITE_GITHUB_URL</code>	No	<i>empty</i>	Optional repository link rendered in the landing-page footer.
<code>VITE_CONTACT_EMAIL</code>	No	<i>empty</i>	Optional contact address rendered in the landing-page footer.

Under Docker, the `VITE_*` values are passed as build arguments to `frontend/Dockerfile` and baked into the `dist/` assets served by nginx; they cannot be changed by setting container environment variables at run time.

C.4 Compose-only variable

One further variable is not read by application code at all but by `docker-compose.yml`: `POSTGRES_HOST_PORT` (default 5433) maps the host-side port onto the container's internal 5432, avoiding a clash with a PostgreSQL server already running on the developer's machine. It never reaches the Go process.

C.5 Minimal working configuration

The smallest valid backend configuration is a single required secret plus a reachable database; every other value can inherit its default for local development. Listing C.2 shows a representative `.env` distilled from `.env.example`.

```

1 # Required: >=32 chars, not the placeholder. openssl rand -hex 32
2 JWT_SECRET=replace-me-with-a-strong-random-secret-32+chars
3
4 # Database (discrete vars; or set DATABASE_URL to override all of these)
```



```
5 DB_HOST=localhost
6 DB_PORT=5432
7 DB_USER=postgres
8 DB_PASSWORD=postgres
9 DB_NAME=apex
10 DB_SSLMODE=disable
11
12 # Server. Set SKIP_AUTOMIGRATE=true in production.
13 PORT=8080
14 SKIP_AUTOMIGRATE=
15
16 # Code execution (self-host for production use)
17 JUDGE0_URL=https://ce.judge0.com
18
19 # Frontend origin / CORS
20 APP_URL=http://localhost:5173
21 ALLOWED_ORIGINS=
22
23 # Optional: SMTP (empty host -> emails logged to stdout)
24 SMTP_HOST=
25 SMTP_PORT=587
26
27 # Optional: Google sign-in (empty -> /auth/google returns 503)
28 GOOGLE_CLIENT_ID=
```

Listing C.2. A minimal development `.env`.

Appendix D

Selected Source Listings

This appendix reproduces the handful of Go functions that carry the most conceptual weight in the APEX backend. Each is quoted faithfully from the real source tree under `internal/`; the only edits are the occasional trimming of a long sibling function so the reader's attention stays on the mechanism under discussion. The listings are the concrete counterparts of the pipelines drawn abstractly in Fig. 7.1, Fig. 8.2, and Fig. 6.2. Reading them together should make it clear how little exotic machinery the system relies on: the correctness of APEX rests on a small number of carefully ordered database writes and a few honest normalisation and aggregation rules, not on a framework doing the thinking for us.

D.1 The Grading Engine

The grading engine in `internal/judge0/grader.go` is the heart of the project. It is deliberately plain Go: it loads rows, calls out to Judge0 over HTTP, compares strings, and writes results back. Three parts are worth reading closely — the output-normalisation helper, the per-test-case grading loop, and the attempt-level aggregator.

D.1.1 Output Normalisation

Comparing program output to a teacher's expected answer is where naive auto-graders quietly fail thousands of students. Judge0 returns whatever the sandboxed process wrote to `stdout`, which on many toolchains carries `\r\n` line endings and a trailing newline, while the expected output was typed into a web form and may carry stray trailing spaces. `normalizeOutput` in Listing D.1 applies exactly four transformations — CRLF to LF, bare CR to LF, right-trim of spaces and tabs on every line, then a whole-string trim — and is applied *symmetrically* to both sides before the equality check. What to notice is the restraint: it does not collapse interior whitespace or lowercase anything, so a genuinely wrong answer still fails. It only forgives the differences that are artefacts of transport and data entry, never differences of substance.

```
1 // normalizeOutput strips CR, trailing whitespace on each line, and trims edges
2 // so Judge0 output (often \r\n with trailing newline) compares cleanly to
3 // teacher-entered expected output.
4 func normalizeOutput(s string) string {
5     s = strings.ReplaceAll(s, "\r\n", "\n")
6     s = strings.ReplaceAll(s, "\r", "\n")
7     lines := strings.Split(s, "\n")
8     for i, ln := range lines {
9         lines[i] = strings.TrimRight(ln, " \t")
10    }
11    return strings.TrimSpace(strings.Join(lines, "\n"))
}
```

```
12 }
```

Listing D.1. Symmetric output normalisation applied to both actual and expected output before comparison.

D.1.2 The Coding Grading Loop

Listing D.2 is the core coding path, `Grade`. Several design decisions are visible in it. First, the empty-test-case guard: a coding problem with no configured test cases is scored `100.0` and marked `accepted` rather than crashing or scoring zero — a defensive choice that avoids penalising students for a teacher’s incomplete setup. Second, the status mapping treats Judge0 status 3 (Accepted) and 4 (Wrong Answer) identically as “the program ran, now compare the strings”; only a genuine execution failure (6 compilation error, 5 time-limit, 11 runtime error) short-circuits the comparison. Third, one `TestResult` row is persisted per test case with its own timing and memory figures, while the maximum time and memory across all tests are rolled up onto the `Submission`. Finally, the score is the simple fraction `passed / total * 100`, and control passes to `maybeFinalizeAttempt` to decide whether the whole exam attempt can now be finalised. The notification is only fired for standalone submissions (`ExamAttemptID == nil`); exam submissions defer their single notification to the aggregator.

```
1 func Grade(submissionID uint) {
2     var submission models.Submission
3     if err := database.DB.First(&submission, submissionID).Error; err != nil {
4         log.Printf("grading: submission %d not found: %v", submissionID, err)
5         return
6     }
7
8     database.DB.Model(&submission).Update("status", "running")
9
10    var testCases []models.TestCase
11    database.DB.Where("problem_id = ?", submission.ProblemID).Order("order_index asc").Find(&
12        ↪ testCases)
13
14    if len(testCases) == 0 {
15        database.DB.Model(&submission).Updates(map[string]any{
16            "status": "accepted",
17            "score": 100.0,
18        })
19        return
20    }
21
22    passed := 0
23    total := len(testCases)
24    var maxTimeMs int
25    var maxMemoryKb int
26
27    for _, tc := range testCases {
28        result := RunCode(submission.Code, submission.Language, tc.Input)
29
30        actualOutput := normalizeOutput(result.Stdout)
31        expectedOutput := normalizeOutput(tc.ExpectedOutput)
```

// StatusID 3 = Accepted; treat StatusID 4 (Wrong Answer) as valid-run-but-compare.

```
32     ran := result.StatusID == 3 || result.StatusID == 4
33     isPassed := ran && actualOutput == expectedOutput
34
35     if isPassed {
36         passed++
37     }
38
39     status := "accepted"
40     if !isPassed {
41         switch result.StatusID {
42             case 6:
43                 status = "compilation_error"
44             case 5:
45                 status = "time_limit_exceeded"
46             case 11:
47                 status = "runtime_error"
48             default:
49                 status = "wrong_answer"
50         }
51     }
52
53     tr := models.TestResult{
54         SubmissionID:    submission.ID,
55         TestCaseID:      tc.ID,
56         Passed:           isPassed,
57         ActualOutput:    actualOutput,
58         ExecutionTimeMs: result.TimeMs,
59         MemoryKb:         result.MemoryKb,
60         Status:          status,
61     }
62     database.DB.Create(&tr)
63
64     if result.TimeMs > maxTimeMs {
65         maxTimeMs = result.TimeMs
66     }
67     if result.MemoryKb > maxMemoryKb {
68         maxMemoryKb = result.MemoryKb
69     }
70 }
71
72 score := float64(passed) / float64(total) * 100.0
73 finalStatus := "wrong_answer"
74 if passed == total {
75     finalStatus = "accepted"
76 }
77
78 database.DB.Model(&submission).Updates(map[string]any{
79     "status":          finalStatus,
80     "passed_count":    passed,
81     "total_count":     total,
82     "score":           score,
83     "execution_time_ms": maxTimeMs,
84     "memory_kb":       maxMemoryKb,
```

```

85     })
86
87     if submission.ExamAttemptID == nil {
88         notification.Create(submission.UserID, "result",
89             "Submission Graded",
90             fmt.Sprintf("Your coding submission scored %.0f%% (%d/%d test cases passed)", score,
91                 ↪ passed, total),
92             "/dashboard/results",
93         )
94     }
95     maybeFinalizeAttempt(submission.ExamAttemptID)
96 }

```

Listing D.2. The coding grading loop: run each test case through Judge0, compare normalised output, persist per-test results, and roll up the score.

D.1.3 Attempt Aggregation

A single exam attempt spans many submissions of mixed type (coding, MCQ, written), each graded independently in its own goroutine. `maybeFinalizeAttempt` in Listing D.3 is the join point that turns those scattered results into one attempt score, and it is written to be safe to call after *every* submission finishes. Three subtleties deserve attention. First, it returns early if any sibling submission is still `pending` or `running`, so the last grader to finish is the one that actually aggregates. Second, it iterates the exam’s *full problem list* rather than the submissions, so a question the student skipped correctly contributes its points to the denominator at a score of zero. Third, submissions in `pending_review` — written answers awaiting manual grading — are excluded from both numerator and denominator, so an ungraded essay does not artificially depress the visible total before the teacher gets to it. The “Exam Graded” notification is fired exactly once, guarded by the `graded_notified` column, and only after no `pending_review` submissions remain.

```

1 // maybeFinalizeAttempt aggregates score across all submissions once none
2 // remain in pending/running state. Safe to call multiple times.
3 func maybeFinalizeAttempt(attemptID *uint) {
4     if attemptID == nil {
5         return
6     }
7     var pending int64
8     database.DB.Model(&models.Submission{}).
9         Where("exam_attempt_id = ? AND status IN ?", *attemptID, []string{"pending", "running"
10             ↪ }).
11         Count(&pending)
12     if pending > 0 {
13         return
14     }
15     var attempt models.ExamAttempt
16     if err := database.DB.First(&attempt, *attemptID).Error; err != nil {
17         return
18     }
19 }

```

```

20  var problems []models.Problem
21  database.DB.Where("exam_id = ?", attempt.ExamID).Find(&problems)
22  if len(problems) == 0 {
23      return
24  }
25
26  var subs []models.Submission
27  database.DB.Where("exam_attempt_id = ?", *attemptID).Find(&subs)
28  subByProblem := make(map[uint]models.Submission, len(subs))
29  for _, s := range subs {
30      subByProblem[s.ProblemID] = s
31  }
32
33  // Iterate the exam's problem list so skipped questions (no submission)
34  // stay in the denominator at score=0. Pending-review submissions
35  // (written answers awaiting manual grading) are still excluded from
36  // both numerator and denominator until a teacher grades them.
37  var earnedPoints float64
38  var totalPoints float64
39  for _, p := range problems {
40      s, hasSub := subByProblem[p.ID]
41      if hasSub && s.Status == "pending_review" {
42          continue
43      }
44      pts := float64(p.Points)
45      if pts <= 0 {
46          pts = 10
47      }
48      totalPoints += pts
49      if hasSub {
50          earnedPoints += s.Score / 100.0 * pts
51      }
52  }
53  pct := 0.0
54  if totalPoints > 0 {
55      pct = earnedPoints / totalPoints * 100.0
56  }
57
58  database.DB.Model(&models.ExamAttempt{}).
59      Where("id = ?", *attemptID).
60      Update("score", pct)
61
62  // Fire a single "Exam Graded" notification once all submissions are
63  // fully graded (no pending_review remaining). Idempotent via
64  // exam_attempts.graded_notified.
65  var anyPendingReview int64
66  database.DB.Model(&models.Submission{}).
67      Where("exam_attempt_id = ? AND status = ?", *attemptID, "pending_review").
68      Count(&anyPendingReview)
69  if anyPendingReview > 0 {
70      return
71  }
72

```

```

73  if err := database.DB.Preload("Exam").First(&attempt, *attemptID).Error; err != nil {
74      return
75  }
76  if attempt.Status != "submitted" || attempt.GradedNotified {
77      return
78  }
79  database.DB.Model(&attempt).Update("graded_notified", true)
80  notification.Create(attempt.UserID, "result",
81      "Exam Graded",
82      fmt.Sprintf("Your submission for \"%s\" has been graded. Score: %.0f%%", attempt.Exam.
83          ↪ Title, pct),
84      fmt.Sprintf("/dashboard/exam/%d/review", attempt.ExamID),
85  )

```

Listing D.3. Attempt-level aggregation: re-score over the exam’s problem list once no submission is still in flight, excluding pending manual grades.

D.2 Authentication and Authorisation Middleware

The first two of APEX’s three authorisation gates (Fig. 8.2) live in `internal/middleware/auth.go`, shown in Listing D.4. `Auth` validates the bearer token and is defensive at every step that involves untrusted input. The key-lookup callback rejects any token whose signing method is not `*jwt.SigningMethodHMAC`, closing the classic “alg:none” and algorithm-confusion attacks in which an attacker asks the server to verify an HS256 token using an RS256 public key as the HMAC secret [40]. Every claim is then read with a comma-ok type assertion — `user_id` as `float64` (JSON’s only number type, cast back to `uint`), `email` and `role` as `string` — so a malformed or partial token produces a clean 401 rather than a panic. Only after all checks pass are the three values written onto the Gin context for downstream handlers to trust. `RequireRole` is the second gate: it reads the `role` the first gate deposited, returns 401 if it is absent (an unauthenticated request), and 403 “insufficient permissions” if it is present but not among the allowed roles [18], [43].

```

1  func Auth(secret string) gin.HandlerFunc {
2      return func(c *gin.Context) {
3          header := c.GetHeader("Authorization")
4          if header == "" || !strings.HasPrefix(header, "Bearer ") {
5              c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "missing or invalid
6                  ↪ token"})
7              return
8          }
9          tokenStr := strings.TrimPrefix(header, "Bearer ")
10         token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (any, error) {
11             if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
12                 return nil, jwt.ErrSignatureInvalid
13             }
14             return []byte(secret), nil
15         })
16         if err != nil || !token.Valid {
17             c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "invalid token"})

```

```

18     return
19 }
20
21 claims, ok := token.Claims.(jwt.MapClaims)
22 if !ok {
23     c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "invalid token claims"
24         ↪ })
25     return
26 }
27
28 // Safe type assertions with comma-ok pattern.
29 userIDFloat, ok := claims["user_id"].(float64)
30 if !ok {
31     c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "invalid user_id in
32         ↪ token"})
33     return
34 }
35 email, ok := claims["email"].(string)
36 if !ok {
37     c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "invalid email in token
38         ↪ "})
39     return
40 }
41 role, ok := claims["role"].(string)
42 if !ok {
43     c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "invalid role in token"
44         ↪ })
45     return
46 }
47
48 c.Set("user_id", uint(userIDFloat))
49 c.Set("email", email)
50 c.Set("role", role)
51 c.Next()
52 }
53 }
54
55 func RequireRole(roles ...string) gin.HandlerFunc {
56     return func(c *gin.Context) {
57         role, exists := c.Get("role")
58         if !exists {
59             c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
60             return
61         }
62         roleStr, ok := role.(string)
63         if !ok {
64             c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "invalid role"})
65             return
66         }
67         for _, r := range roles {
68             if roleStr == r {
69                 c.Next()
70                 return
71             }
72         }
73     }
74 }

```



```

67     }
68 }
69 c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "insufficient permissions"})
70 }
71 }

```

Listing D.4. JWT authentication middleware and role gate: signing-method enforcement, safe claim extraction, and the 401-vs-403 distinction.

The third gate — resource ownership — deliberately does not live in middleware, because whether a user “owns” a resource depends entirely on the resource. It is enforced inside each handler against the `user_id` the middleware deposited, as Fig. 8.2 illustrates.

D.3 The Reminder Scheduler

APEX runs its exam reminders in-process rather than depending on an external cron system, keeping the deployment to a single Go binary. Listing D.5 shows the driver in `internal/reminder/scheduler.go`. `Start` spawns one goroutine that waits fifteen seconds for the database connection pool to settle, then ticks once a minute forever. The `runOnce` tick is wrapped in a deferred `recover` so that a panic in one sweep — a malformed row, a transient query error — logs and is swallowed rather than killing the scheduler goroutine and silently stopping all future reminders. Each tick performs the in-app T-60 sweep inline and then delegates the two email sweeps. The critical property, visible in the `WHERE` clauses of Fig. 6.2, is that every sweep filters on a dedicated “sent-at” timestamp column being `NULL` and stamps it after sending. Because the dedup state lives in the database rather than in memory, a process restart cannot cause a reminder to be re-sent: the scheduler is idempotent across restarts.

```

1  // Start launches a goroutine that periodically scans for upcoming exams and
2  // notifies enrolled students who opted in to exam reminders. Reminder fires
3  // once per exam (deduped via exams.reminder_sent_at).
4  func Start() {
5      go func() {
6          // Initial short delay to let DB settle.
7          time.Sleep(15 * time.Second)
8          tick := time.NewTicker(1 * time.Minute)
9          defer tick.Stop()
10         runOnce()
11         for range tick.C {
12             runOnce()
13         }
14     }()
15 }
16
17 func runOnce() {
18     defer func() {
19         if r := recover(); r != nil {
20             log.Printf("[reminder] panic: %v", r)
21         }
22     }()
23
24     now := time.Now()

```

```

25     cutoff := now.Add(60 * time.Minute)
26
27     var exams []models.Exam
28     if err := database.DB.
29         Where("is_draft = ? AND reminder_sent_at IS NULL AND start_time IS NOT NULL AND
30             ↳ start_time > ? AND start_time <= ?", false, now, cutoff).
31         Find(&exams).Error; err != nil {
32             log.Printf("[reminder] query failed: %v", err)
33             return
34         }
35
36     for _, exam := range exams {
37         notifyExam(exam)
38     }
39
40     emailSweep1Hour(now)
41     emailSweepStart(now)
42 }

```

Listing D.5. The reminder scheduler tick loop: a fifteen-second warm-up, a one-minute ticker, and a panic-guarded sweep over published exams.

The fan-out itself, in `notifyExam`, resolves an exam to its assigned classes, those classes to their enrolled members, and those members to the subset whose profile has `notify_exam_reminders = true`. One nuance worth noting is that an exam with no assigned classes or no enrolled members is still stamped `reminder_sent_at` so that it is not rescanned on every subsequent tick for the rest of its life — a small guard that keeps the sweep’s working set bounded to genuinely pending exams.

D.4 Destructive-Edit Reset

When a teacher makes a destructive edit to a live exam — changing a test case, editing an MCQ’s correct answers, or deleting a problem — every attempt already scored against the old configuration is invalid. `ResetExamAttempts` in `internal/exam/reset.go`, shown in Listing D.6, cleans this up atomically. The whole operation runs inside a single `database.DB.Transaction`, so either all of the student progress is wiped and the exam re-stamped, or nothing changes. The deletion order respects foreign-key dependencies: `TestResult` rows are removed first (looked up via their parent submission IDs), then `Submission` rows, then the `ExamAttempt` rows themselves. The transaction then stamps `exams.reset_at`, un-announces grades at the class level so stale scores stop showing, and — only if there were real attempts to reset — notifies enrolled students that they may re-enter. That last guard is what keeps the “replace-all problems” save path from fanning out one spurious notification per deleted problem.

```

1 func ResetExamAttempts(examID uint) error {
2     return database.DB.Transaction(func(tx *gorm.DB) error {
3         var exam models.Exam
4         if err := tx.First(&exam, examID).Error; err != nil {
5             return err
6         }
7
8         var attemptCount int64

```

```

9      tx.Model(&models.ExamAttempt{}).Where("exam_id = ?", examID).Count(&attemptCount)
10     hadAttempts := attemptCount > 0
11
12     var subIDs []uint
13     tx.Model(&models.Submission{}).Where("exam_id = ?", examID).Pluck("id", &subIDs)
14     if len(subIDs) > 0 {
15         if err := tx.Where("submission_id IN ?", subIDs).Delete(&models.TestResult{}).Error;
16             ↪ err != nil {
17             return err
18         }
19     }
20     if err := tx.Where("exam_id = ?", examID).Delete(&models.Submission{}).Error; err !=
21         ↪ nil {
22         return err
23     }
24     if err := tx.Where("exam_id = ?", examID).Delete(&models.ExamAttempt{}).Error; err !=
25         ↪ nil {
26         return err
27     }
28
29     now := time.Now()
30     if err := tx.Model(&models.Exam{}).Where("id = ?", examID).Update("reset_at", now).
31         ↪ Error; err != nil {
32         return err
33     }
34     exam.ResetAt = &now
35
36     // Un-announce grades for the classes this exam belongs to - old
37     // announced scores are now invalid.
38     var classIDs []uint
39     tx.Model(&models.ExamClass{}).Where("exam_id = ?", examID).Pluck("class_id", &classIDs)
40     if len(classIDs) > 0 {
41         if err := tx.Model(&models.Class{}).Where("id IN ?", classIDs).Update("
42             ↪ grades_announced", false).Error; err != nil {
43             return err
44         }
45     }
46
47     // Only notify if there were actual attempts to reset.
48     if !hadAttempts {
49         return nil
50     }
51     if len(classIDs) > 0 {
52         var userIDs []uint
53         tx.Model(&models.ClassMember{}).Where("class_id IN ?", classIDs).Pluck("user_id", &
54             ↪ userIDs)
55
56         link := fmt.Sprintf("/dashboard/exam/%d", examID)
57         desc := fmt.Sprintf("Exam '%s' was updated by your teacher. Your attempt has been
58             ↪ reset - you can re-enter.", exam.Title)
59
60         for _, uid := range userIDs {
61             notification.Create(uid, "exam_reset", "Exam updated", desc, link)
62         }
63     }

```

```
55     return nil
56   })
57 }
```

Listing D.6. Transactional reset of student progress after a destructive teacher edit, with FK-ordered deletes and a guarded notification.

Taken together, these five listings capture the engineering personality of APEX: correctness enforced by explicit, ordered database writes; safety enforced by early returns and comma-ok assertions rather than exceptions; and idempotence enforced by persisted state columns rather than in-memory flags. There is no hidden magic — which is precisely what makes the system auditable.

References

- [1] M. Dougiamas and P. C. Taylor, “Moodle: Using learning communities to create an open source course management system,” in *Proc. EdMedia*, 2003.
- [2] Instructure, *Canvas LMS*, 2011–2026. [Online]. Available: <https://www.instructure.com/canvas>
- [3] HackerRank, *HackerRank for Schools and Work*, 2009–2026. [Online]. Available: <https://www.hackerrank.com/>
- [4] H. Z. Došilović, *Judge0: A Robust, Scalable, Open-Source Online Judge System*, 2017–2026. [Online]. Available: <https://judge0.com/>
- [5] H. Z. Došilović, “Judge0: a robust, scalable, open-source online judge system,” in *Proc. CECIS*, 2018.
- [6] C. Douce, D. Livingstone, and J. Orwell, “Automatic test-based assessment of programming: A review,” *ACM Journal on Educational Resources in Computing*, vol. 5, no. 3, 2005.
- [7] P. Ihantola, T. Ahoniemi, V. Karavirta, and O. Seppälä, “Review of recent systems for automatic assessment of programming assignments,” 2010.
- [8] C. Mateos, A. Zunino, and M. Campo, “Web technologies and frameworks for online judges: a survey,” in *Proc. ICCSA*, 2014.
- [9] S. Wasik, M. Antczak, J. Błazewicz, and P. Lukasiak, “A Survey on Online Judge Systems and Their Applications,” *ACM Computing Surveys*, vol. 51, no. 1, 3:1–3:34, 2018.
- [10] M. Mareš, *Isolate: A sandbox for the secure execution of programs*, 2012–2026. [Online]. Available: <https://github.com/ioi/isolate>
- [11] Google, *The Go Programming Language*, 2009–2026. [Online]. Available: <https://go.dev/>
- [12] Microsoft, *Monaco Editor: the code editor that powers VS Code*, 2016–2026. [Online]. Available: <https://microsoft.github.io/monaco-editor/>
- [13] Google, *Distroless Container Images*, 2017–2026. [Online]. Available: <https://github.com/GoogleContainerTools/distroless>
- [14] PostgreSQL Global Development Group, *PostgreSQL: The World’s Most Advanced Open Source Database*, 1996–2026. [Online]. Available: <https://www.postgresql.org/>
- [15] Meta Platforms, *React: The library for web and native user interfaces*, 2013–2026. [Online]. Available: <https://react.dev/>
- [16] E. You and contributors, *Vite: Next generation frontend tooling*, 2020–2026. [Online]. Available: <https://vitejs.dev/>
- [17] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in *Proc. of the 1999 USENIX Annual Technical Conference (FREENIX Track)*, 1999, pp. 81–91.

- [18] M. Jones, J. Bradley, and N. Sakimura, *JSON Web Token (JWT)*, HMAC SHA-256 (HS256) signing used by APEX., 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [19] OWASP Foundation, *OWASP Top 10: 2021*, 2021. [Online]. Available: <https://owasp.org/Top10/>
- [20] R. Lobb and J. Harlow, *CodeRunner: a tool for assessing computer programming skills*, 2014–2026. [Online]. Available: <https://coderunner.org.nz/>
- [21] A. Aiken, *MOSS: A System for Detecting Software Plagiarism*, 1994–2026. [Online]. Available: <https://theory.stanford.edu/~aiken/moss/>
- [22] L. Prechelt, G. Malpohl, and M. Philippsen, “Finding plagiarisms among a set of programs with JPlag,” in *Journal of Universal Computer Science*, 2002.
- [23] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.
- [24] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. McGraw-Hill, 2019.
- [25] MITRE, *CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)*, 2024. [Online]. Available: <https://cwe.mitre.org/data/definitions/89.html>
- [26] J. Nielsen, *10 Usability Heuristics for User Interface Design*, 1994. [Online]. Available: <https://www.nngroup.com/articles/ten-usability-heuristics/>
- [27] J. Brooke, “SUS - A quick and dirty usability scale,” in *Usability Evaluation in Industry*, Taylor & Francis, 1996.
- [28] A. Wiggins, *The Twelve-Factor App*, 2017. [Online]. Available: <https://12factor.net/>
- [29] Gin contributors, *Gin: HTTP web framework written in Go*, 2014–2026. [Online]. Available: <https://github.com/gin-gonic/gin>
- [30] GORM contributors, *GORM: The fantastic ORM library for Golang*, 2013–2026. [Online]. Available: <https://gorm.io/>
- [31] Microsoft, *TypeScript: JavaScript with syntax for types*, 2012–2026. [Online]. Available: <https://www.typescriptlang.org/>
- [32] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [33] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [34] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [35] golang-migrate contributors, *golang-migrate: Database migrations written in Go*, 2016–2026. [Online]. Available: <https://github.com/golang-migrate/migrate>
- [36] TanStack, *TanStack Query: Powerful asynchronous state management*, 2021–2026. [Online]. Available: <https://tanstack.com/query>
- [37] shadcn, *shadcn/ui: Beautifully designed components built with Radix UI and Tailwind CSS*, 2023–2026. [Online]. Available: <https://ui.shadcn.com/>

- [38] Tailwind Labs, *Tailwind CSS: A utility-first CSS framework*, 2017–2026. [Online]. Available: <https://tailwindcss.com/>
- [39] MITRE, *CWE-639: Authorization Bypass Through User-Controlled Key (IDOR)*, 2024. [Online]. Available: <https://cwe.mitre.org/data/definitions/639.html>
- [40] OWASP Foundation, *JSON Web Token for Java Cheat Sheet*, 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web-Token_for_Java_Cheat_Sheet.html
- [41] Google, *Google Identity Services for the web*, 2022–2026. [Online]. Available: <https://developers.google.com/identity/gsi/web>
- [42] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, *OpenID Connect Core 1.0*, 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html
- [43] D. Hardt, *The OAuth 2.0 Authorization Framework*, 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [44] MITRE, *CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)*, 2024. [Online]. Available: <https://cwe.mitre.org/data/definitions/79.html>
- [45] MITRE, *CWE-352: Cross-Site Request Forgery (CSRF)*, 2024. [Online]. Available: <https://cwe.mitre.org/data/definitions/352.html>
- [46] OWASP Foundation, *OWASP Application Security Verification Standard (ASVS) 4.0*, 2021. [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/>
- [47] P. A. Grassi et al., *NIST SP 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management*, 2017. [Online]. Available: <https://pages.nist.gov/800-63-3/sp800-63b.html>
- [48] GitHub, *GitHub Actions: Automate your workflow from idea to production*, 2019–2026. [Online]. Available: <https://github.com/features/actions>
- [49] Docker Inc., *Docker: Accelerated container application development*, 2013–2026. [Online]. Available: <https://www.docker.com/>